

FORWARD DEPLOYED ENGINEERING

Forward Deployed Engineering

From Ambiguous Workflow to Production Outcome

WORKFLOW	EVIDENCE	PRODUCTION	OWNERSHIP
----------	----------	------------	-----------

Komal Nakrani

First edition - Version 1.0.0

Copyright and professional boundary

Professional boundary and disclaimer

This book is technical and professional education, not legal, security, privacy, safety, accessibility, audit, medical, financial, regulatory, or compliance advice. Framework mappings are not certification. Readers must use qualified specialists and explicitly designated organizational authorities for formal decisions.

The Forward Deployed Engineer prepares evidence, implements within delegated scope, recommends options, and escalates. The role does not acquire formal risk authority merely by owning delivery.

Preface

Forward deployed engineering is difficult because the work crosses code, workflow, customer systems, users, operations, and authority. A convincing prototype can still fail if it solves the wrong problem, hides exceptions, mishandles identity, cannot recover, lacks qualified approval, or depends indefinitely on its builder.

This book treats a deployment as a decision system. Every chapter creates evidence that can reject a weak assumption and improve the next decision. The running case, Orchid Assist, begins as a request for an AI copilot and becomes a bounded workflow whose data, interfaces, controls, evaluation, release, recovery, adoption, ownership, and reuse claims remain explicit.

How to use this book

- Read Chapters 1-5 before committing scope.
- Use Chapters 6-10 to create the design dossier.
- Run the companion while studying Chapters 11-14.
- Use Chapters 15-17 for readiness, stabilization, and ownership.
- Use Chapters 18-19 only after bounded field evidence exists.
- Complete the learning packs; they are marked NOT FOR LIVE CERTIFICATION BANK and are not a certification exam.
- Treat final figures F01 . 1 through F19 . 2 as decision artifacts: read their visible structure with the caption and use the embedded long description when visual detail is inaccessible.

The book assumes production-engineering basics. Appendix A contains reusable artifact structures; Appendix B contains gates; Appendix C explains the executable companion; Appendix D defines terminology/source use; Appendix E indexes the completed fictional dossier.

The five part introductions are maintained in `part-introductions.md` and belong in the web/PDF sequence before Chapters 1, 6, 11, 15, and 18. They state the reader transformation and dossier handoff for each stage without repeating chapter instruction.

Running case and companion

Orchid Equipment Services is fictional. Orchid Assist integrates a synthetic ticket, equipment candidates, approved evidence, inventory state, bounded model suggestion, deterministic policy, qualified approval, audit, telemetry, release/recovery, readiness, stabilization, ownership, reuse, and portfolio closure.

Run from the repository root:

```
npm run test:companion
npm run demo:companion
npm run rehearse:companion
```

These commands provide deterministic local evidence. They do not prove real customer integrations, identities, performance, accessibility, UAT, security, safety, recovery time, or production outcomes.

Contents

Copyright and professional boundary	2
Contents	4
Part I - Own the Outcome	6
1. The Deployed Outcome	7
2. Enter the Customer System	25
3. Map the Workflow That Actually Exists	42
4. Contract for an Outcome	59
5. Scope the First Safe Production Path	76
Part II - Design the Deployment	95
6. Draw the Real System Boundary	96
7. Make Interfaces and Data Explicit	113
8. Fit the Customer Environment	132
9. Put AI Inside a Bounded Workflow	149
10. Build Security and Governance Into Delivery	167
Part III - Build Evidence Into the System	182
11. Build a Production Vertical Slice	183
12. Prove Behavior Before Production	200
13. Make the System Observable and Operable	217
14. Engineer Release and Recovery	232
Part IV - Launch Into Reality	245
15. Cross the Production Threshold	246

16. Stabilize Under Real Conditions	258
17. Make the Solution Usable and Ownable	270
Part V - Turn Delivery Into Leverage	283
18. Turn Field Evidence Into Product Leverage	284
19. Lead Beyond One Deployment	298
Appendices	313
Appendix A - Deployment Dossier Templates	314
Appendix B - Gate and Review Checklists	319
Appendix C - Provider-Neutral Companion Guide	322
Appendix D - Glossary and Source Use	324
Appendix E - Completed Dossier Index	325
Figure registry	327
Sources	333
Edition record	338

Part I - Own the Outcome

The book begins before architecture or code because the first production risk is often a mistaken unit of work. A requested feature can be delivered accurately while the real workflow stays slow, unsafe, unsupported, or unused. Part I teaches the reader to replace feature certainty with an evidence-bearing deployment contract.

Chapter 1 establishes the FDE's unit of accountability and authority boundary. Chapter 2 enters the customer system through purposeful evidence and legitimate access. Chapter 3 maps real decisions, states, handoffs, exceptions, incentives, and failure cost. Chapter 4 turns that evidence into outcomes, guardrails, adoption signals, decision rights, and stop/exit criteria. Chapter 5 selects the least exposed representative production path and sequences uncertainty before irreversible commitment.

The Orchid dossier advances from OA-01 role/discovery through OA-04 scope/delivery. At the end of the part, the reader should be able to reject a broad AI `copilot` request, a metric without a denominator, a scope that hides unsafe dependency, or an engagement whose authority and exit are undefined.

1. The Deployed Outcome

Define the Forward Deployed Engineer's unit of accountability, lifecycle, role boundary, and evidence-based meaning of done.

A production deployment can fail while every software task is marked complete.

The repository may contain reviewed code. The release pipeline may be green. The model may perform well on a benchmark. The customer may even have attended a polished demonstration. Yet the people who do the work may avoid the system, the critical exception may still require an undocumented workaround, the support team may not know how to diagnose failure, or the organization may be unable to operate the change after the delivery team leaves.

That gap--between shipped output and an owned production outcome--is the central problem of forward deployed engineering.

Current employer descriptions use the title differently, but they repeatedly place Forward Deployed Engineers across more of the lifecycle than implementation alone: discovery, design, production engineering, rollout, adoption, measurable workflow results, and the return of field learning into product work. Some organizations use the narrower title Forward Deployed Software Engineer for an engineering-heavy version of the work. These are observations about present employers, not a universal standard for the profession. The operating model in this book is a synthesis designed to make the work explicit, testable, and teachable. It does not grant authority merely because someone carries the title. [CLM-001] [CLM-002] [CLM-007]

The synthesis begins with a demanding definition:

A deployed outcome is measurable, sustained workflow value achieved in the live operating environment, inside explicit guardrails, with evidence that the system is used, supportable, and owned.

Each word rules out a familiar way to declare victory too early.

- **Measurable** means that a decision can be informed by evidence. It does not require false precision.
- **Sustained** means that the result survives beyond a demonstration or launch-day observation window.
- **Workflow value** means an improvement in the work and its consequences, not merely more software activity.
- **Live operating environment** means real dependencies, identities, policies, users, queues, failure modes, and ownership are present.
- **Guardrails** mean value is not obtained through unacceptable safety, privacy, security, reliability, accessibility, cost, or human consequences.
- **Used** means the intended participants actually incorporate the capability into representative work, not merely that accounts exist.
- **Supportable** means degradation can be detected, diagnosed, contained, corrected, and communicated by named people with the necessary access and evidence.

- **Owned** means ongoing operation, change, risk decisions, and support do not depend on an unrecorded personal relationship with the FDE.

This definition is not a slogan. It is a test for project decisions. It determines what must be discovered, which evidence belongs in scope, where architecture ends, what “production-ready” must mean, and when the FDE should stop rather than ship.

The unit of accountability

Traditional engineering plans often make the component or feature the unit of work. “Build the inventory adapter” is a clear implementation task. “Add retrieval” can be a clear technical task. “Deploy version 1.3” is a clear release task. Each can be necessary. None says why the customer should accept the change or what evidence would show that it helped.

Forward deployed work changes the unit of accountability from a component to a bounded operational result.

Consider Orchid Equipment Services, the fictional company used throughout this book. Orchid maintains industrial equipment for customers in several regions. Service requests arrive through inconsistent channels. A technician may search manuals, old tickets, an equipment registry, inventory records, and personal notes before deciding what to inspect, which part to carry, or when to escalate. Leaders ask for “an AI copilot trained on our manuals and service history.”

If the team accepts the request as its unit of work, it can produce a chatbot and still learn almost nothing about the deployed outcome. The interface could respond fluently while equipment identifiers do not reconcile, current inventory is unavailable, unsafe actions are presented confidently, low-connectivity sites time out, or technicians ignore suggestions that hide their evidence.

The outcome-oriented unit is different:

Improve evidence-supported field-service resolution for a bounded equipment family, region, and technician cohort, while preserving qualified approval for safety-relevant decisions, regional data constraints, service reliability, auditable evidence, and an operable support path.

That statement is not yet a finished outcome contract. It contains no baseline, target, observation window, or decision rights; Chapter 4 will add them. But it already changes the engineering conversation.

It asks whether the equipment registry and ticket identifiers agree. It makes inventory freshness and failure semantics material. It exposes the difference between summarizing evidence and authorizing action. It requires the team to identify who is qualified to approve safety-relevant recommendations. It makes low-connectivity behavior part of the path, not a later optimization. It treats the support model as a design input. A production outcome pulls the whole socio-technical path into view.

Public service guidance offers a useful durable analogy: a service is understood through the outcome it enables across a user’s path, not solely through its digital interface. This book transfers that idea carefully to customer deployments. It does not claim that every commercial deployment should copy a government delivery process. It does claim that an interface is only one participant in a service or workflow. [CLM-003]

Release, deployment, and outcome are different events

The words *release* and *deployment* are often used interchangeably. That ambiguity causes trouble in this role, so this book uses them precisely.

A **release** is a controlled change to code, configuration, data schema, model, prompt, policy, or another system artifact. A release has a version, evidence, owner, and recovery path. It may enter production gradually. It may remain disabled behind a feature control. A release is an event inside a larger system of change.

A **deployment**, in this book, is the complete socio-technical change from scoped problem through production adoption and ownership. It includes releases, but also workflow discovery, decision rights, user and operator preparation, controls, rollout, stabilization, support, handoff, and measured learning.

A **production outcome** is the result the deployment is intended to create and sustain.

The distinctions matter because the evidence differs:

Claim	Necessary evidence examples	What the evidence does not prove
"The release completed."	artifact/version, automated checks, environment record, release log	representative workflow value, adoption, long-term reliability
"The deployment is in production."	enabled production path, real dependencies and identities, bounded user exposure	success, safety, supportability, or customer ownership
"The path is being used."	representative task events, cohort and frequency, observed workflow integration	outcome improvement, correctness, voluntary trust, guardrail compliance
"The system is supportable."	signals, runbooks, access, on-call/support owner, rehearsed diagnosis/recovery	that every failure is covered or the owner can evolve the system
"The outcome is achieved."	outcome and guardrail evidence over an agreed window, adoption and operating evidence, limitations	perfect causal attribution or permanent value

Software release or user activity is therefore evidence, but neither is complete outcome evidence. [CLM-006]

This prevents two opposite errors. The first is technical reduction: "We deployed it, so the customer must manage the rest." The second is unlimited expansion: "Because we own the outcome, every organizational problem belongs to us." A disciplined FDE avoids both.

The deployed-outcome lifecycle

The lifecycle used in this book has ten movements:

1. **Discover the work.** Learn how the workflow actually operates, including decisions, data, exceptions, incentives, handoffs, and failure cost.

2. **Contract the outcome.** Define evidence of value, adoption, guardrails, non-goals, assumptions, decision rights, and stop/exit criteria.
3. **Scope the first safe production path.** Choose the smallest path that crosses representative constraints and can produce decision-quality evidence.
4. **Design the deployment.** Make system, interface, data, environment, identity, security, governance, operating, and ownership boundaries explicit.
5. **Build representative behavior.** Implement an end-to-end vertical slice under production constraints, including controlled failure behavior.
6. **Verify claims.** Connect workflow risk and acceptance criteria to tests, evaluations, user acceptance, controls, and known limitations.
7. **Launch deliberately.** Review readiness, choose a rollout pattern, bound blast radius, name decision authority, and precommit to stop/recovery triggers.
8. **Stabilize under real conditions.** Use production evidence to contain failure, correct the system, and confirm support readiness.
9. **Transfer ownership.** Demonstrate that users and operators can perform, diagnose, release, recover, support, and evolve the bounded system.
10. **Turn field evidence into leverage.** Preserve the customer boundary while converting repeated evidence into product feedback, playbooks, configurable capability, or reusable components.

F01.2 · FORWARD DEPLOYED ENGINEERING

The deployed-outcome lifecycle



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F01.2 - The deployed-outcome lifecycle. Release is one event inside a larger evidence loop from discovery through ownership and field learning.

Text alternative: Circular lifecycle from discovery and outcome contract through build, verification, launch, stabilization, ownership, and product learning, with evidence returning to earlier decisions.

This is not a waterfall. Every later movement can invalidate an earlier assumption. A contract test can reveal that the supposed interface semantics were wrong. A security review can make the selected scope impossible. A canary can show that an aggregate evaluation hid a critical cohort failure. Stabilization can reveal that the support owner lacks the required access. Adoption work can prove that the technically correct interface does not fit the actual workflow.

Returning to an earlier decision is not automatically rework or failure. It is the expected response to stronger evidence. The failure is erasing the decision history and pretending the new state was always known.

The lifecycle is an original synthesis, not a formal external standard. Its purpose is to connect the practices repeatedly observed in current deployment-oriented roles with durable service, engineering, and risk principles. [CLM-005]

Outcome leadership without authority inflation

“Own the outcome” is dangerous when used carelessly. It can become permission for an engineer to collect unnecessary data, make domain decisions without qualification, accept risk for someone else, or pressure users into a launch. It can also become a cultural demand for permanent availability and personal heroics.

This book uses **outcome leadership** instead of unlimited outcome ownership. Outcome leadership means maintaining the chain of evidence and decisions required to reach a bounded result. It includes doing hard technical work, making gaps visible, bringing the right owner to a decision, recommending a course, and escalating when consequence exceeds authority.

It does not mean that the FDE personally holds every decision right.

Current deployment descriptions themselves make access, policy, approval, security, and escalation material. Durable security and privacy guidance likewise treats trust and risk as explicit organizational concerns rather than privileges inherited from location or job title. Outcome leadership does not remove those boundaries. [CLM-004]

Use five separate questions whenever ownership language becomes vague:

1. **Who is affected by the decision?** This identifies users and stakeholders, not authority.
2. **Who has relevant evidence or expertise?** This identifies contributors, not necessarily approvers.
3. **Who can execute the change?** This identifies an implementation owner, not necessarily a risk owner.
4. **Who is accountable for accepting the consequence?** This identifies the decision authority.
5. **Who must assemble evidence and drive the decision to closure?** This is often where the FDE leads.

The same person can answer several questions, but the questions should not be collapsed.

Suppose Orchid Assist proposes a recommendation that affects a safety-relevant inspection sequence. The FDE can discover how evidence reaches the technician, implement deterministic eligibility rules,

design an approval gate, produce evaluation results, and recommend a bounded rollout. A qualified Orchid owner must define the operational safety rule. Security and privacy owners must decide whether the access and data-handling design meets the organization's requirements. The business owner must accept the consequences of the rollout choice. The FDE drives the technical evidence chain and escalates missing decisions; the title does not confer the other authorities.

This is not a retreat from accountability. "That is not my decision" is incomplete when the decision is necessary for the deployed outcome. A professional response is:

"This decision belongs to the regional safety owner. I have documented the affected workflow, alternatives, test evidence, residual uncertainty, and the date by which a decision is required. Until that approval exists, the production path remains blocked."

The sentence holds the deployment while respecting the boundary.

A practical responsibility classifier

At the start of an engagement, classify work into four categories.

FDE core

Core work directly maintains the chain from workflow evidence to production outcome. Examples include:

- discovering the real workflow and its exceptions;
- translating evidence into outcome, guardrail, scope, and exit decisions;
- designing and building the representative production path;
- making interfaces, data, identity, controls, failure, and ownership explicit;
- producing verification, rollout, recovery, stabilization, adoption, and handoff evidence;
- converting field evidence into product learning.

Core does not mean exclusive. A product manager, designer, data engineer, security engineer, SRE, implementation lead, or customer specialist may share the work. "Core" means the FDE cannot ignore the missing link and still claim the deployed outcome is owned.

Shared

Shared work requires joint judgment or execution. Architecture review, threat modeling, user research, production readiness, incident response, and adoption diagnosis commonly fit here. The local operating model must name who recommends, approves, executes, and is informed.

Supporting

Supporting work helps the engagement but is not normally the FDE's defining accountability. Preparing a demonstration, helping a customer engineer understand an API, improving a deployment template, or writing a field note can support the outcome. Supporting work becomes suspect when it consumes the engagement without advancing evidence or removing a material blocker.

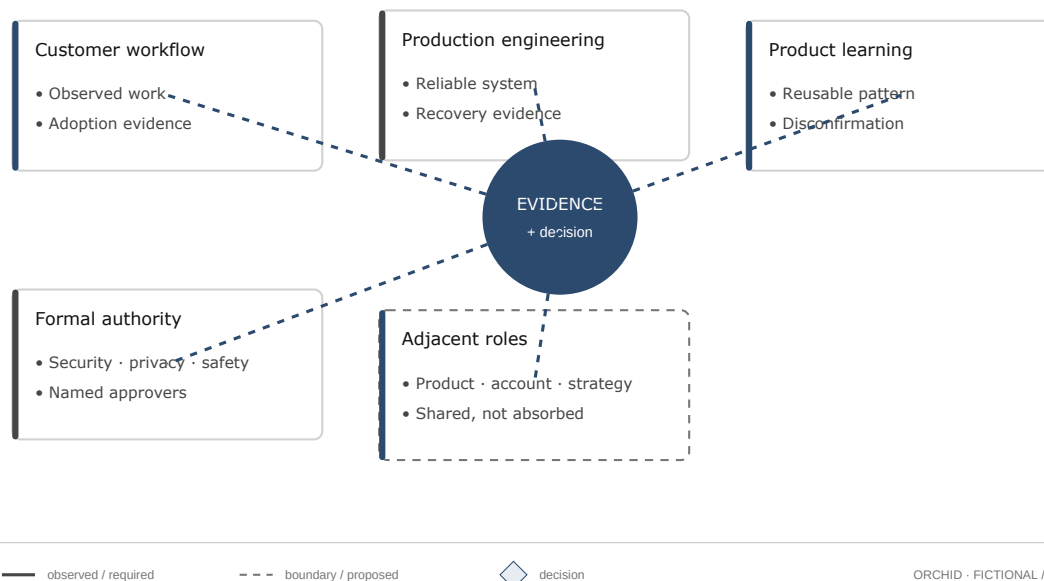
Specialist-owned or outside scope

Some work belongs to a qualified or formally authorized role: legal interpretation, independent audit, formal risk acceptance, clinical or safety authorization, employment decisions, commercial pricing, or enterprise-wide transformation governance. The FDE may supply technical evidence and implement an approved decision. The FDE does not impersonate the owner.

Outside scope also includes work with no defensible relationship to the bounded outcome. Saying no is not abandonment when the request would distract the team or expand risk without evidence.

F01.1 · FORWARD DEPLOYED ENGINEERING

The FDE responsibility boundary



F01.1 - The FDE responsibility boundary. The FDE integrates workflow, engineering, and product learning while formal authority remains explicitly assigned.

Text alternative: Boundary map placing the FDE between customer workflow, production engineering, and product learning, with formal authorities outside the shared delivery area.

The classifier should be applied to tasks, not to people. A customer engineer can perform FDE-core work. An FDE can perform supporting implementation. The goal is to clarify the responsibility chain, not defend organizational territory.

The responsibility charter

The first capstone artifact, OA-01 Responsibility Charter and Discovery Plan, begins with a short responsibility charter. It is intentionally created before deep discovery and revised when evidence changes.

A useful charter contains the following.

Engagement purpose

One paragraph states the operational hypothesis, target boundary, and why the work matters. It avoids promising a solution before discovery.

For Orchid:

The Orchid Assist engagement will determine whether a bounded evidence-assisted path can improve field-service resolution for a selected equipment family and region without weakening qualified safety decisions, regional data handling, reliability, cost control, technician judgment, or operational ownership.

The wording says “determine whether” because the outcome is not yet proven. It describes a bounded workflow, not a feature commitment.

FDE responsibilities

List responsibilities as observable work: maintain evidence and decision records; lead technical discovery; design and implement the bounded vertical slice; coordinate verification and rollout evidence; expose material gaps; prepare decisions for accountable owners; support stabilization and handoff; return validated patterns to product owners.

Avoid personality claims such as “be scrappy,” “do whatever it takes,” or “ensure customer happiness.” They cannot be reviewed and can hide unsafe expectations.

Decision and authority limits

Name decisions the FDE may make, decisions the FDE recommends, and decisions requiring other approval. Include data access, production change, security/privacy exception, safety rule, go/no-go, external communication, and engagement exit where relevant.

Evidence obligations

State how assumptions, contradictions, decisions, tests, limitations, incidents, and changes will be recorded. The charter should require that a missing approval, unknown owner, or unsupported claim remains visible.

Escalation triggers

Examples include:

- requested access or data cannot be tied to a legitimate discovery question;
- a formal decision has no accountable owner;
- a safety, privacy, security, legal, or compliance consequence exceeds the team's authority;
- the selected scope cannot produce representative evidence;
- a deadline would require bypassing a material control or recovery test;
- the customer cannot staff operation or handoff;
- new evidence invalidates the outcome hypothesis;
- repeated customer-specific work suggests product leverage, but isolation or ownership is unclear.

Exit conditions

The engagement is not designed to continue forever. Initial exit language might require that the bounded system has an accepted outcome review, explicit limitations, a customer-owned operating/support model, demonstrated release and recovery, resolved privileged access, and a disposition for reusable work. Later chapters will make these conditions testable.

Named non-scope

For Orchid, early non-scope can include automatic execution of safety-relevant actions, all equipment families, all regions, enterprise data remediation, and replacement of the customer's safety, security, privacy, legal, or support authority. Non-scope can change through documented decision; it cannot change through quiet expectation.

Evidence changes the meaning of “done”

Teams often use *done* as a local state: code merged, ticket closed, demo accepted, production flag enabled. Local completion remains useful. The FDE must connect it to the next evidence obligation.

Use this pattern:

The deliverable is complete when [local evidence]. **The deployment may advance when** [gate evidence and authority]. **The outcome may be claimed when** [operational evidence over a defined boundary and window].

For example:

The inventory adapter is complete when its versioned contract, success and error behavior, authentication path, retry/idempotency behavior, reconciliation process, tests, observability, and owner review satisfy the selected criteria. The deployment may advance to a production cohort when integrated evidence, security/operations approval, capacity assumptions, and recovery triggers pass the readiness gate. Improved first-visit resolution may be claimed only after the agreed cohort and observation window show the outcome direction without violating safety, escalation, latency, reliability, privacy, cost, or adoption guardrails, subject to the stated attribution limitations.

The longer statement is not bureaucracy. It prevents one kind of evidence from impersonating another.

Failure modes of outcome language

The hero contract

“You own the outcome” becomes “you are personally responsible for everything.” The FDE accumulates privileged access, makes undocumented exceptions, becomes the only operator, and compensates for organizational gaps through availability. Short-term progress masks an untransferable system.

Repair: replace personal heroics with named decision rights, operating owners, escalation triggers, and handoff evidence. Treat bus-factor reduction as part of the outcome.

The feature contract

The team delivers the requested feature accurately but never tests whether the feature changes representative work. The customer’s stated solution becomes the problem definition.

Repair: preserve the request as evidence, then verify the current workflow, outcome, constraints, and failure cost before committing the design.

The activity contract

Logins, prompts, generated documents, or recommendation counts become success. Activity can rise because users are confused, repeating failed attempts, or complying with policy.

Repair: connect activity to segmented workflow outcomes, guardrails, support signals, and qualitative evidence. Activity is a diagnostic measure, not the outcome by default.

The authority shortcut

The FDE makes a safety, privacy, security, or business decision because the accountable owner is slow or absent.

Repair: make the missing owner and consequence visible. Reduce scope, delay, or stop. Escalation is a production skill.

The permanent pilot

The system remains “pilot” indefinitely. It has real users and consequences but avoids production standards, operating ownership, or a stop decision.

Repair: define pilot entrance, observation, guardrail, support, exit, and shutdown criteria. Small exposure changes blast radius, not the need for responsibility.

The productization reflex

One customer-specific component is renamed a platform. Its embedded assumptions become invisible dependencies for future deployments.

Repair: maintain a pattern ledger. Separate observation from invariant. Require an isolation and validation plan before abstraction. Some current FDE role descriptions explicitly include reusable patterns and product feedback; that makes evidence discipline more important, not less. [CLM-008]

Tradeoffs belong inside the outcome

An outcome statement that mentions only benefit is incomplete. Real deployments trade among speed, scope, reliability, security, privacy, safety, performance, cost, maintainability, accessibility, user effort, and future leverage. The task is not to maximize every attribute. It is to make the important tradeoffs explicit enough for the right owner to decide.

Suppose the Orchid team can improve recommendation latency by caching manuals and inventory results at regional sites. The change could improve low-connectivity performance, but it introduces freshness, storage, access, revocation, and data-residency questions. The FDE's job is not to say “latency wins” or “security says no.” The job is to model the affected workflow, quantify or bound the latency need, identify the data and threat consequences, propose alternatives, test feasible controls, and route the decision to the owners who can accept the remaining consequence.

A decision record should show:

- the outcome or risk that triggered the decision;

- the evidence and its confidence;
- alternatives considered, including no change;
- consequences and affected parties;
- assumptions and expiration conditions;
- who recommends, approves, and executes;
- the decision and date;
- validation and rollback/revisit triggers.

This structure lets later evidence change the decision without rewriting history.

Field learning is part of delivery

A forward deployed engagement produces two potential forms of value. The first is the bounded customer outcome. The second is validated learning that improves later delivery or the product itself.

The second is not guaranteed. It must not justify extracting customer data, generalizing confidential assumptions, or building a platform from one anecdote. But ignoring repeated field evidence wastes an important property of the role.

Current employer descriptions explicitly mention codifying patterns, tools, playbooks, building blocks, and product feedback. This supports the existence of the practice, not a claim that every organization structures it identically. [CLM-008]

Field learning can become:

- a clearer discovery question;
- an integration or evaluation playbook;
- a reusable test harness;
- a configurable product capability;
- a safer default;
- a missing diagnostic signal;
- documentation that prevents a recurrent misunderstanding;
- a product requirement backed by repeated consequence;
- or a decision not to generalize because the invariants are weak.

Chapter 18 will define evidence thresholds and isolation rules. For now, the charter should establish that learning is recorded and routed, while customer boundaries remain intact.

The Orchid responsibility charter, first pass

The following condensed record shows the level expected after this chapter. It is a draft to be revised as discovery proceeds.

Purpose. Determine whether a bounded Orchid Assist path can improve evidence-supported field-service resolution for one equipment family and region within safety, data, reliability, cost, adoption, and ownership guardrails.

FDE leads. Technical discovery; evidence and decision records; bounded design and implementation; interface/data/control evidence; verification; readiness recommendation; rollout technical leadership; stabilization; technical adoption blockers; handoff evidence; field-learning packet.

Shared. Workflow research with technicians and service operations; architecture with customer IT/data/platform owners; threat and control work with security/privacy; evaluation and approval design with qualified domain owners; rollout and incident work within Orchid's operating model.

Specialist/authority owned. Safety rule and approval; privacy/legal interpretation; security exception/risk acceptance; regional data decision; business outcome target; production go/no-go acceptance; external customer communication; employment/incentive policy.

Initial non-scope. Autonomous safety-relevant action; automatic parts ordering; multilingual voice; all equipment families/regions; enterprise identifier remediation; replacement of customer systems or governance functions.

Escalate when. Access lacks a named purpose; accountable owner is missing; representative data cannot be used safely; guardrail evidence cannot be produced; deadline requires bypassing a material control; customer cannot own operation; outcome hypothesis is contradicted.

Initial exit hypothesis. Outcome and guardrail review completed; limitations accepted by named owners; representative users can perform the workflow; support owns signals/runbooks; customer engineers demonstrate release/recovery; privileged FDE access is revoked or explicitly time-bounded; reusable work has an isolation-aware disposition.

The charter is intentionally uncomfortable. It exposes decisions that a feature plan could hide. It gives the team permission to delay, reduce, or stop work when the evidence chain cannot be completed.

What the FDE should be able to say

At the end of an engagement, a mature FDE should be able to make a statement like this:

“For the named technician cohort, equipment family, region, and observation window, the deployment changed the agreed workflow measures in the reported direction. The safety, escalation, reliability, privacy, cost, and adoption guardrails remained within their accepted bounds except for these stated limitations. The customer team has demonstrated the defined operating, support, release, and recovery responsibilities. These claims rely on the recorded evidence and have these attribution limits. Two patterns may justify reusable investment; one remains customer-specific. The following owner will review the outcome and open risks on the stated date.”

It is a stronger statement than “we shipped.” It is also more modest than “we transformed the customer.” It names the boundary, evidence, limitations, ownership, and next decision.

That combination--technical depth, operational continuity, evidence discipline, and authority restraint--is the foundation of the role.

Working the boundary through concrete decisions

Role boundaries become useful only when they change a decision. The following situations show how the classifier operates under pressure.

A customer asks for a production export

The sponsor says that a full export of service tickets will make discovery faster. The export contains customer identifiers, technician notes, location data, and attachments. The FDE has credentials that could retrieve it.

The technical ability to obtain data is not the authority to collect it. The core FDE responsibility is to name the discovery question and find the minimum evidence that can answer it. Data access and handling are shared with the customer's data, security, and privacy owners. Formal permission and risk decisions belong to the authorized customer roles.

A weak response is to accept the export and promise to delete it later. Another weak response is to refuse all evidence and declare discovery impossible. The outcome-led response decomposes the request:

- Which workflow hypotheses require ticket evidence?
- Which fields, time window, equipment family, and region are material?
- Can counts, redacted samples, synthetic fixtures, supervised queries, or customer-run analysis answer the early questions?
- Who approves access and handling, and when does access expire?
- Where will the evidence reside, who can see it, and how will deletion be verified?
- What decision becomes impossible if the minimized evidence is insufficient?

The FDE owns bringing this decision to closure because discovery cannot proceed responsibly without it. The FDE does not own the customer's privacy or legal interpretation.

The customer wants an automatic action

Orchid leadership wants the system to order the most likely replacement part as soon as a recommendation is generated. Automatic ordering could reduce delay, but an incorrect order can create cost, stock imbalance, or a missed safety-relevant inspection.

The request spans all four categories. Understanding the operational decision and designing a representative path are FDE core. Integration and control design are shared with inventory, service operations, and customer engineering. Spending authority, safety policy, and acceptable error consequences are specialist or business-owner decisions. A demonstration of automatic ordering might be supporting work; it must not silently become a commitment.

The initial charter can place automatic ordering outside scope while retaining an evidence path. The bounded deployment may show a recommended part, provenance, stock state, and a qualified approval action without executing the order. If later evidence and authority support automation, the scope changes through a recorded decision.

This is not timid engineering. It is sequencing reversibility. The first path produces evidence about data quality, recommendation behavior, user trust, and inventory semantics before the system creates a financially or operationally consequential side effect.

A dependency team misses a date

The identity team cannot deliver the planned workload identity before the pilot window. The FDE is asked to use a shared long-lived credential “temporarily.”

Coordinating the dependency and proposing alternatives are core to the deployment. Implementing the identity mechanism is shared or customer-owned, depending on the operating model. Accepting the risk of the temporary credential is not implied by the FDE title.

The decision record should compare at least these choices: delay; reduce the path to a non-production environment; use a properly scoped temporary mechanism with expiration, storage, rotation, monitoring, and named acceptance; or change the design. The record must show which outcome evidence is lost by delay and which risk is introduced by the workaround. If no authorized owner accepts an adequate option, the path remains blocked.

The common failure is not that dependencies slip. It is that schedule pressure converts an exception into an invisible architecture decision.

Users do not adopt the system

After rollout, use remains low. A leader asks the FDE to “drive adoption.”

Adoption diagnosis is FDE core because the deployed outcome depends on representative use. Training, communications, workflow policy, product design, performance, access, evidence visibility, incentives, and support may each be shared with different owners. General workforce performance management is outside the FDE boundary.

The FDE should not begin by assuming resistance. The system may be slow at field sites, require duplicate entry, omit the evidence technicians need, fail on the most common exception, or conflict with local responsibility. The outcome-led response collects segmented evidence, observes representative work, and identifies the smallest test that distinguishes these explanations. Chapter 17 will develop this diagnosis fully.

The same integration appears at a second customer

A similar ticket-system adapter appears in another engagement. A product leader asks the FDE to make it a shared service.

Recording the pattern and supplying evidence are core. Designing a reusable component may be shared with product and platform engineering. Roadmap priority and long-term service ownership do not transfer automatically to the FDE.

Two adapters with similar endpoints may contain different identity, data, error, tenancy, version, and support semantics. The professional response is not “never abstract until the third occurrence,” because severe repeated cost or product risk can justify earlier work. It is also not “build a platform now.” The response is a pattern record: what repeated, what varied, which consequence matters, what evidence exists, what customer boundaries apply, what maintenance commitment follows, and which validation would disconfirm the proposed invariant.

These examples share a structure. The FDE remains responsible for the continuity of the outcome even when another person holds the decision authority. Continuity means that the gap has an owner, evidence, consequence, option set, due date, and escalation path. It does not mean the FDE quietly makes the decision.

How an outcome claim matures

The outcome statement should mature as the evidence matures. Treat it as a controlled claim, not a marketing sentence.

Stage 1: request

“Give every technician an AI copilot.”

This is evidence of stakeholder intent. It is not yet a problem statement or outcome.

Stage 2: discovery hypothesis

“Technicians may be losing time and first-visit effectiveness because evidence needed for inspection and parts decisions is fragmented across systems and informal knowledge.”

This is a testable hypothesis. It remains uncertain until the workflow and exceptions are observed.

Stage 3: bounded outcome contract

“For the selected equipment family, region, and technician cohort, reduce the agreed measure of evidence-gathering and resolution delay while improving or preserving first-visit success, with no unacceptable change in safety escalation, incorrect-part consequence, privacy, reliability, cost, or support burden.”

This version requires definitions, baselines, targets, segments, windows, and decision rights. Chapter 4 supplies them.

Stage 4: readiness claim

“The bounded path has produced the required verification, UAT, control, operability, recovery, user/support preparation, and approval evidence for the stated cohort rollout; these gaps remain and have this disposition.”

This supports a decision to expose the system. It does not claim the production outcome.

Stage 5: production outcome claim

“During the named observation window, the bounded cohort showed the stated outcome direction and guardrail state, with these adoption, reliability, support, and attribution limitations.”

This is the claim the deployment exists to support. It remains bounded in time and context.

Stage 6: ownership and leverage claim

“The customer has demonstrated the accepted operating and change responsibilities; these patterns are supported for reuse, these remain customer-specific, and these require further validation.”

The maturity sequence prevents language from outrunning evidence. A request is not a hypothesis; a hypothesis is not a contract; verification is not a production result; a production result is not permanent value; one result is not a platform.

At every stage, store the claim beside its evidence, limitations, owner, and next decision. If a later stage fails, retain the earlier history. That history is how the next design becomes better without inventing hindsight.

Conduct the responsibility exercise

Take a fictional request to build an AI assistant for field service. Classify at least twelve decisions as FDE core, shared, supporting, or specialist/outside scope. Include workflow discovery, data access, architecture, security exception, safety threshold, release, adoption, incident command, commercial promise, product roadmap, customer ownership, and reusable pattern review.

For each decision record the FDE action, named authority, required evidence, escalation trigger, and exit. Then write a one-page responsibility charter with purpose, outcome boundary, non-scope, evidence obligations, access limits, and handoff conditions.

Pass when the charter makes the FDE accountable for integrating the deployed outcome without claiming authority over adjacent formal decisions. A reviewer should be able to point to what done means, what evidence can reject it, and how the engagement stops or transfers.

Chapter close: the first gate

Before entering the customer system, the FDE should have a provisional answer to five questions:

1. What bounded operational result are we trying to learn or achieve?
2. Which parts of the evidence-and-decision chain will the FDE lead?
3. Which decisions belong to customer or specialist authorities?
4. What conditions require escalation, reduced scope, delay, or stop?
5. What would make the deployment operable and independent of the FDE?

The answers will change. Their purpose is not to predict the engagement perfectly. Their purpose is to prevent ambiguity from silently becoming unlimited authority or unlimited obligation.

Chapter 2, *Enter the Customer System*, turns the provisional charter into a legitimate discovery plan. It separates impact, knowledge, access, responsibility, and decision authority; ties each access request to a question; and establishes an evidence ladder that keeps reported opinion from becoming production fact.

2. Enter the Customer System

Plan legitimate discovery, distinguish stakeholder dimensions, minimize access and data, and keep evidence confidence explicit.

The first technical failure in a deployment often occurs before anyone writes code. It happens when the team enters the customer system with the wrong permission, the wrong witnesses, or the wrong standard of evidence.

A sponsor describes the problem. A dashboard supplies a number. A policy document shows the official path. An experienced operator remembers the exceptions. A production trace reveals behavior no one mentioned. Each may be useful. None is the whole workflow.

Forward deployed engineers work close to customers because distance hides consequential detail. Proximity, however, does not create domain expertise, consent, authority, or unlimited access. It creates an obligation to learn carefully.

The goal of entry is not to “learn everything about the customer.” It is to acquire enough legitimate, representative evidence to make the next bounded decision. That requires a discovery plan, a stakeholder model, an access-purpose map, and an evidence log that can preserve contradiction without turning it into false certainty.

Official service guidance provides durable support for beginning with the user's whole problem and context rather than a chosen solution, and for using interviews, observation, and existing evidence to learn how real users work. This chapter transfers those principles to FDE engagements without claiming that a government service process is mandatory for every customer. [CLM-009] [CLM-010]

Enter with a question, not a data appetite

Chapter 1 left Orchid Assist with a provisional responsibility charter. The charter did not authorize a solution. It established a bounded hypothesis: Orchid may be able to improve evidence-supported field-service resolution for one equipment family, region, and technician cohort without weakening safety, privacy, reliability, cost, judgment, or ownership.

That hypothesis creates questions:

- Where does a service request begin, and when is it considered resolved?
- Which people make which decisions?
- Which evidence is available when each decision is made?
- What happens when an equipment identifier is missing, duplicated, or wrong?
- When does a technician need qualified approval?
- Which systems contain manuals, inventory, prior service, and telemetry?
- Which path is official, which path is common, and which path succeeds?

- What failure is merely inconvenient, and what failure carries safety, customer, financial, or operational consequence?
- Who owns the decision to change the workflow?

Questions are not decoration before access. They are the basis for access.

Use an access-purpose chain:

discovery question -> minimum credible evidence -> minimum data or access -> handling constraint
-> owner and approval -> expiry -> decision enabled

Suppose the team asks for all production tickets. "We need to understand the workflow" is too vague. A better chain is:

Determine how often selected equipment records fail to match the ticket identifier -> customer-run aggregate and a small redacted sample of mismatch classes for the selected family and recent period -> no names, free-text notes, unrelated regions, or attachments -> reviewed in the customer-controlled environment -> approved by the data owner and security/privacy contacts -> access ends after taxonomy validation -> decide whether identifier reconciliation must enter the first vertical slice.

The exact minimization depends on context. The durable rule is that access should be tied to a named purpose, made no broader or longer than necessary, and governed by explicit owners and handling. Current deployment descriptions and security/privacy guidance make least-required access and risk ownership material; the FDE title does not override them. [CLM-011]

This rule improves engineering as well as governance. Large uncontrolled exports create noise, unclear provenance, accidental dependencies, irreproducible analysis, and handling burden. Purpose-bound evidence makes it easier to record what a finding does and does not support.

Stakeholders are not one-dimensional

A stakeholder list often contains names and job titles. That is insufficient for discovery because several different relationships are collapsed into one word.

For each person or group, consider at least six dimensions:

1. **Impact.** How can the deployment affect this person's work, rights, risk, performance, or customers?
2. **Knowledge.** What workflow, exception, system, policy, or history can this person explain or demonstrate?
3. **Access.** Which people, environments, records, tools, or sites can this person legitimately make available?
4. **Responsibility.** Which operation, artifact, control, support path, or result is this person expected to maintain?
5. **Influence.** How can this person accelerate, resist, interpret, or reshape the deployment?
6. **Decision authority.** Which named choice can this person approve, reject, accept, or escalate?

F02.1 · FORWARD DEPLOYED ENGINEERING

Stakeholders are not interchangeable



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F02.1 - Stakeholders are not interchangeable. Impact, knowledge, access, influence, responsibility, and decision authority are recorded separately.

Text alternative: Stakeholder matrix showing sponsor, technician, regional owner, security owner, and FDE across impact, workflow knowledge, access, influence, responsibility, and authority.

The dimensions may point in different directions. A sponsor may have high influence and business authority but limited knowledge of field exceptions. A technician may have deep workflow knowledge and high impact but no authority to accept privacy risk. A platform engineer may control access to an API but not own the service operation that depends on it. A support analyst may see failure patterns that leadership dashboards omit.

Separating the dimensions prevents three errors.

First, access is not authority. Someone who can grant a credential may not be permitted to authorize the proposed use.

Second, knowledge is not representativeness. The most experienced operator may have rare expertise and an unusual workflow. Their account is valuable but should not silently describe every user.

Third, influence is not impact. A leader can dominate the project while people with little organizational power absorb most of its consequence.

The stakeholder map is a hypothesis. Mark it as such. Revise it as interviews, observation, records, and operating behavior reveal missing people or misunderstood decisions.

Build an evidence ladder

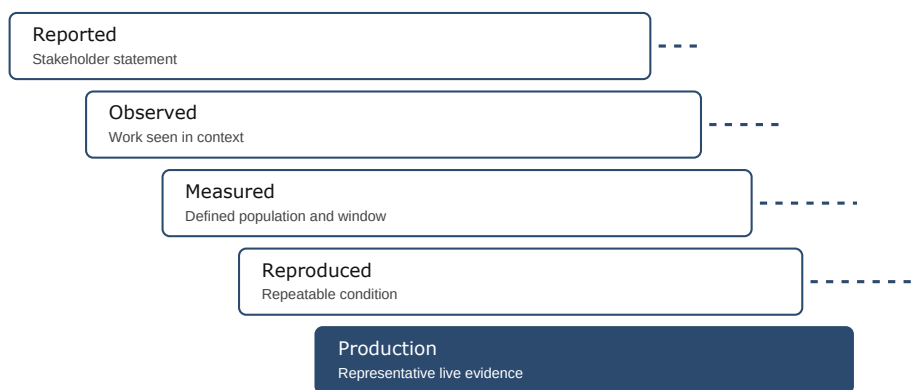
Discovery produces statements with different strengths. Store the strength with the statement.

This book uses five practical levels:

- **Reported.** A person, document, or system says something is true.
- **Observed.** The team witnesses an instance under stated conditions.
- **Measured.** A defined method produces a value or distribution over a stated population and window.
- **Reproduced.** The team can trigger or reconstruct the behavior under controlled, documented conditions.
- **Production evidence.** The behavior is supported by live operating evidence across the bounded context and observation window.

F02.2 · FORWARD DEPLOYED ENGINEERING

Evidence ladder with retained limits



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F02.2 - Evidence ladder with retained limits. Confidence can rise from reported to production evidence without making any observation universal.

Text alternative: Five-step evidence ladder from reported to observed, measured, reproduced, and production evidence, with source, context, sample, confidence, contradiction, and limitation retained at every rung.

The ladder is not a ranking of human worth. Reported evidence can reveal a safety consequence no event log contains. Production metrics can be misleading when event semantics changed. A reproduced failure may be more useful for engineering than a weak production correlation. The ladder helps the team state what kind of support it has.

An entry in the evidence log should include:

- evidence ID and date;
- exact statement or observation;
- level;
- source and collection method;
- affected user/workflow/system/context;
- sample or time boundary where relevant;
- provenance and handling constraint;
- confidence and reason;
- corroborating or contradictory evidence IDs;
- decision or hypothesis affected;
- owner and next validation step.

Consider these entries:

E-017, reported: Regional service lead states that inventory is “usually current.” No definition of current; applies to Region West; validation pending against update timestamps and technician experience.

E-023, observed: In two supervised Region West cases, technicians opened personal notes before the official manual portal because equipment search returned multiple near-matches. Small convenience sample; affects identifier and trust hypotheses; do not infer prevalence.

E-031, measured: For the customer-run query definition v2 over selected-family tickets in the stated 30-day window, 14 percent lacked the canonical equipment ID. Excludes phone-only requests and corrected tickets; query and data owner recorded.

The precision in the third statement is defensible only because the definition, query version, population, exclusion, and owner are recorded. Without them, “14 percent” is false confidence.

An opinion is evidence of a report

Stakeholders frequently say:

- “Technicians will never use another tool.”
- “The API is reliable.”
- “Security will not allow that.”
- “This exception almost never happens.”
- “The model needs all historical tickets.”

Do not dismiss these statements. Do not promote them to workflow fact either.

An opinion is direct evidence that the person reports, expects, fears, or prefers something. It may also be an informed hypothesis about the workflow. Appropriate testing depends on consequence.

“Technicians will never use it” can prompt observation of current tools, access constraints, prior deployments, evidence needs, and incentive conditions. “The API is reliable” can prompt service objectives, incident history, timeout/error behavior, and a controlled integration test. “Security will not allow that” can prompt the exact control objective, policy owner, affected data/action, alternatives, and formal decision path.

Guidance to start by learning user needs explicitly warns against treating stakeholder opinions as established user needs. The same discipline applies here: preserve the report, record the speaker's context, and test the inference. [CLM-012]

Contradictions are not a nuisance to smooth over. They often reveal segmentation, incentives, hidden state, or different meanings.

If technicians say equipment search is unreliable while the data team reports 99.5 percent field completeness, both may be correct. The required field may contain a syntactically valid but operationally ambiguous identifier. The data metric may cover records while technicians encounter tickets. One region may differ. A recent migration may have changed behavior. The contradiction directs discovery toward semantics and population, not toward deciding who is “right.”

Choose complementary discovery methods

No single method reveals the whole customer system.

Interviews

Interviews expose language, intent, perceived problems, exceptions, decision reasoning, and organizational history. They are especially useful for asking why a person made a choice and what evidence was unavailable.

Limitations: memory is selective; official process can replace actual process in the retelling; interview conditions affect candor; confident accounts can be unrepresentative.

Good practice:

- ask for a recent concrete case before asking for general frequency;
- ask what happened next, what was checked, who decided, and what failed;
- separate “should” from “did”;
- request artifacts or demonstrations with legitimate handling;
- record exact uncertainty instead of filling it with assumptions.

Contextual observation

Observation exposes tools, switching, waiting, workaround, collaboration, environment, and embodied knowledge that may not appear in procedure documents. It can show where the visible interface is a small part of the actual path.

Limitations: observation can alter behavior; a few cases are not a prevalence estimate; sensitive contexts may prohibit observation; the observer can misunderstand domain actions.

The FDE should narrate interpretations back to the user: “I saw you compare the serial plate with two records before opening the manual. What decision were you making?” The user supplies domain meaning; the engineer does not invent it.

Existing records and artifacts

Policies, runbooks, tickets, logs, schemas, dashboards, incident reports, training material, and support cases reveal stated expectations and historical evidence.

Limitations: artifacts can be stale, selectively retained, generated under changing definitions, or disconnected from actual behavior. A dashboard label is not a metric contract.

Reproduction and technical inspection

Sandbox calls, synthetic fixtures, schema inspection, controlled failure, and code/configuration review reveal executable behavior.

Limitations: non-production environments differ; access can expose sensitive implementation; reproduction demonstrates possibility under conditions, not production frequency.

Quantitative measurement

Defined queries and event analysis can estimate magnitude, variation, cohorts, and trends.

Limitations: collection semantics, missingness, selection, changed definitions, and incentives can invalidate apparent precision.

The discovery plan should combine methods according to the next decision and the cost of being wrong. Interviews and observation may be sufficient to reject an obviously misframed solution. A safety-relevant exception may require domain review, technical reproduction, and stronger production evidence before scope is selected.

End-to-end and front-to-back service guidance reinforces that discovery must include channels, operational processes, technology, and security rather than only the visible interface. [CLM-013]

The Orchid discovery plan

The OA-01 discovery plan turns the charter into executable work.

Decisions discovery must enable

1. Which operational decision and bottleneck should Orchid Assist address?
2. Which equipment family, region, user cohort, and exception set can represent the first safe path?
3. Which data and interfaces are necessary, and what quality/access constraints exist?
4. Which recommendations carry safety, privacy, or material operational consequence?
5. Which owners can define outcome, guardrail, approval, rollout, support, and handoff decisions?

Stakeholder hypotheses

- technicians: affected users, workflow and exception knowledge;
- dispatch/service coordinators: queue, prioritization, escalation, and handoff knowledge;
- service operations leader: outcome and process responsibility;
- equipment/maintenance specialist: qualified domain decisions and safety escalation;
- inventory/ERP owner: stock/order semantics and interface responsibility;
- ticketing/equipment registry owners: identifiers, schemas, access, and change constraints;
- regional leader: data boundary and operating authority;
- security/privacy owners: access, handling, control, and formal exception path;
- support/on-call teams: actual failure demand and ownership feasibility;
- customer engineering/platform teams: deployment, identity, network, observability, and release paths.

This is a hypothesis, not an invitation list. Discovery should deliberately test for overlooked contractors, accessibility needs, low-connectivity sites, night shifts, new technicians, and people affected by the workflow without using the new interface.

Evidence sequence

Begin with existing process/policy/support evidence and a small set of role-diverse interviews. Observe representative cases with permission. Build a provisional workflow and contradiction log. Request minimized quantitative or technical evidence only for decisions the early findings make material. Reproduce a small number of critical interface/data behaviors in a controlled environment. Validate the current-state map and remaining contradictions with users and owners.

The sequence reduces the chance of collecting large datasets before the team knows which semantics matter.

Access-purpose examples

Question	Minimum initial evidence	Avoid initially	Owner/constraint
How are equipment records matched?	supervised walkthrough, schema/field definitions, redacted mismatch examples	full ticket archive and unrelated attachments	registry/data owner; customer environment
Where does inventory fail the decision?	interface docs, error semantics, selected synthetic calls, support categories	production write credential	ERP owner; read-only/sandbox first
Which safety decisions need approval?	policy/process, qualified expert interview, recent de-identified scenarios	model deciding the policy	safety/domain owner
Does connectivity alter use?	site/network constraints, segmented latency/support evidence, supervised observation	device-wide data capture	regional IT and user consent/process

Stop and escalation conditions

Pause or reduce discovery when requested access has no accountable owner; handling requirements are unknown; observation cannot be conducted legitimately; a high-consequence domain decision has no qualified owner; sponsor access excludes materially affected users; or evidence cannot support a representative/safe path.

Stopping here protects the outcome. It prevents urgency from converting ignorance into production exposure.

Trust is built through evidence behavior

Customer trust is sometimes described as charisma, responsiveness, or executive presence. Those qualities can help, but durable technical trust comes from observable behavior:

- say what is known, inferred, and unknown;
- do not overstate domain understanding;
- use the least access required and return or expire it;
- preserve inconvenient contradictions;
- make decisions and changes traceable;
- surface bad news early with consequence and options;
- follow the customer's formal operating and incident paths;
- show working technical evidence, not only presentations;
- credit user and specialist expertise;
- close loops when someone supplies evidence or raises risk.

Trust is not permission to become informal. A close customer relationship can increase the risk of undocumented exceptions or data sharing. Good FDE practice makes legitimate collaboration easier by clarifying purpose, owner, boundary, and expiry.

Run discovery as a sequence of decisions

Discovery should have a cadence without becoming a factory of meetings. A useful cycle is: prepare, collect, synthesize, challenge, decide, and re-plan.

Prepare

Before an interview, observation, or technical session, write the decision it may affect and the current hypothesis. Review what is already known. Identify sensitive topics, required permission, and the evidence that must not be collected. Select participants because of impact or knowledge, not convenience.

For an Orchid technician observation, the preparation note might say:

Decision affected: whether the first slice must reconcile ticket and equipment identifiers before retrieving manuals. Current hypothesis: technicians routinely resolve ambiguous identifiers using visual equipment evidence and personal knowledge. Observe two selected-family service cases with user and customer approval; record decision steps and system transitions, not customer-identifying content. Do not infer frequency from the sessions.

This note stops the session from becoming open-ended surveillance.

Collect

During collection, preserve the distinction between the participant's words, the observer's description, and the observer's interpretation.

- Participant: "I never trust this field."
- Observed: technician compared the field with the equipment plate and an older ticket in both observed cases.
- Interpretation: field may be syntactically present but operationally unreliable for identification.
- Next test: review field provenance and a customer-run sample of mismatch classes.

Do not silently edit a statement into your preferred explanation. Ask neutral follow-ups and capture why a decision mattered.

Synthesize

After each small set of sessions, update the stakeholder map, evidence log, contradiction register, workflow hypothesis, and access-purpose map. Synthesis should change an artifact or explicitly state that no decision changed.

Use a short evidence review:

1. What did we learn?
2. What kind of evidence is it?
3. Which prior assumption did it support or contradict?
4. What decision changes now?
5. What consequential unknown remains?
6. What is the least costly legitimate way to resolve it?

Challenge

Ask someone with different impact or knowledge to challenge the current model. Show the evidence and uncertainty, not merely a polished diagram. A support analyst may reveal a failure cluster. A new technician may expose a hidden expertise assumption. A regional owner may invalidate a data path. A security engineer may offer a safer evidence method than the team assumed.

Challenge is not a ceremonial review. Record whether it changes the model, leaves a dispute, or identifies a decision owner.

Decide and re-plan

Discovery earns its cost when it enables a decision: narrow the problem, request specific evidence, add a stakeholder, reject a feature, stop an unsafe path, or proceed to detailed workflow mapping. Record the decision and re-plan only the remaining uncertainty.

This cadence also gives discovery an exit. The team does not need exhaustive domain knowledge. It needs sufficient evidence for the next bounded decision, explicit gaps, and a safe path to strengthen the evidence later.

Conduct sessions without taking the domain

An FDE needs enough domain understanding to build correct technical behavior, but the method must protect against confident imitation.

Ask participants to teach decisions through cases:

- "Tell me about the last time this path failed."
- "What evidence made you choose that action?"
- "What would cause you to stop or escalate?"
- "Which part is policy, which is judgment, and which is system limitation?"
- "Who would disagree with this description?"

- “What changes for a new technician, another region, or an unusual equipment state?”
- “Can you show the artifact or state you relied on, within the approved access boundary?”

Avoid solution-confirming questions such as “Would an AI summary solve this?” The participant may answer the concept rather than reconstruct the work. Ask first what evidence is sought, how confidence is judged, what errors cost, and where the next decision occurs.

At the end of a session, play back the interpretation:

“I believe the equipment ID in the ticket is treated as a search hint, not a trusted key. When multiple records match, you compare location, serial evidence, and prior service; if ambiguity remains, you escalate before selecting a manual or part. Is that accurate? Which case would break this description?”

The final question invites disconfirmation. It signals that the engineer is building a model, not claiming expertise.

If a participant supplies a safety, legal, privacy, or policy interpretation, record both the content and the authority status. A knowledgeable user can reveal the practical rule while a designated owner remains responsible for approving the rule used by the deployment.

Handle evidence you should not retain

Sometimes sensitive evidence must be inspected without becoming a project asset. A customer expert can run a query and share an aggregate. A production trace can be viewed in the customer environment while only the failure class is recorded. A qualified owner can demonstrate a case using redacted or synthetic inputs. A schema can be reviewed without copying representative records.

The evidence log should say what was observed, by whom, under which conditions, and what conclusion it supports. It should not reproduce sensitive content merely to make the note feel complete.

If the conclusion cannot be audited without retaining prohibited data, do not create a vague assurance. Record the limitation and identify an approved evidence mechanism: customer-retained query, attestation by an accountable owner, reviewed derived statistic, reproducible synthetic case, or controlled test executed by the customer.

Data minimization can reduce future harm, but it can also reduce reproducibility. That is a real tradeoff. Resolve it explicitly with provenance, customer-controlled artifacts, reviewer identity, and a repeatable method rather than by keeping unnecessary copies.

Discovery failure modes

Sponsor-only truth

The team interviews leaders and maps the desired process. Missing users, operators, support, and system owners appear later as “resistance” or “unexpected requirements.”

Repair: map impact and knowledge separately from influence; include representative users and people who handle exceptions.

Data-first certainty

The team receives a large dataset and immediately calculates distributions. Field semantics, missing populations, changed definitions, and workarounds remain unknown.

Repair: start with decision questions and semantic discovery; record query/version/population/exclusions; triangulate with workflow evidence.

Interview theater

The team conducts many interviews but produces no tested workflow model or changed decision.

Repair: connect every discovery activity to a hypothesis, artifact, contradiction, or next decision. Stop interviewing when marginal evidence no longer changes the bounded decision.

Domain performance

The engineer learns terminology and begins making expert judgments. Users stop correcting the confident outsider.

Repair: state interpretations as hypotheses, ask qualified people to explain decisions, and record where formal expertise/authority is required.

Access accumulation

Credentials and exports remain active “in case they are needed.” Ownership and deletion are unclear.

Repair: maintain an access register with purpose, approver, scope, storage, audit, expiration, and revocation evidence. Review it at each phase gate.

Contradiction collapse

The team chooses the most senior or most convenient account and deletes alternatives from the narrative.

Repair: preserve contradiction IDs and design the next observation, measurement, or reproduction to explain the difference.

Treat access as a changing production artifact

Access decisions made during discovery often survive longer than anyone intended. A temporary credential becomes part of a script. A downloaded sample is copied into a test fixture. A broad permission remains because no one recorded who should revoke it. By the time the system approaches production, the team no longer knows which access is necessary.

Prevent this by maintaining an access register from the first day. Each record should contain:

- a stable access ID;
- the question or task it enables;
- the person, service, environment, data category, and operation in scope;
- whether access is read, write, approve, administer, or impersonate;
- the accountable system/data owner and approving authority;
- the user or workload identity receiving access;
- the approved storage, transfer, logging, and retention conditions;
- the issue/change record through which it was granted;
- the grant, review, and expiry dates;
- the evidence produced through the access;
- the condition for reduction or revocation;
- revocation evidence.

The register should distinguish access to **observe, reproduce, build, release, operate, and recover**. These purposes do not imply the same permission. A discovery analyst who needs to inspect a customer-run aggregate does not necessarily need raw-record access. A deployment service that reads inventory does not need the authority to place an order. A support engineer who can inspect health may not need to view user content. A break-glass identity should not become the normal release path.

Review the register whenever the deployment changes phase. At the end of discovery, remove access that produced its intended evidence. Before the vertical slice, replace personal credentials with bounded workload or development identities where appropriate. Before rollout, test production access, monitoring, and revocation. At handoff, remove hidden FDE dependency.

This practice does more than reduce privilege. It exposes architecture. If an essential workflow decision can be made only through a person's broad administrator account, the problem is not "remember to be careful." The dependency, identity flow, and operating owner are incomplete.

Example access records

A-004 might permit a customer data analyst to execute a versioned aggregation over selected-family tickets in the customer environment. The FDE receives the query definition, population/exclusion notes,

result table, and owner review, not the raw records. The access remains with the customer analyst and expires from the discovery plan after the metric is reproduced or replaced.

A-009 might permit the FDE's named development identity to call a sandbox inventory API using synthetic equipment and part identifiers. It allows read operations only, logs calls to the customer sandbox audit trail, and expires at the architecture gate. It cannot prove production latency, capacity, or write semantics.

A-014 might be a proposed production read permission for the Orchid Assist workload identity. It remains pending during discovery because the topology, tenant/region boundary, fields, service owner, and rollout cohort have not yet been approved. Recording it now identifies a future dependency without granting it prematurely.

Make discovery inclusive of edge conditions

"Representative users" is not a synonym for "average users." A small group can be material because its consequences, environment, or exclusion risk differ.

For Orchid, the discovery plan should ask whether the selected sample covers:

- new and highly experienced technicians;
- low-connectivity and well-connected sites;
- day and after-hours work where support differs;
- common and high-consequence equipment states;
- people who use assistive technology or encounter accessibility barriers;
- contractors or regional teams with different identity and policy paths;
- support and coordination roles that absorb failures without using the interface;
- users who abandoned a previous tool and therefore disappear from activity data.

The team cannot always observe every segment before scope. It can record which segments are represented, which are missing, why the gap matters, and how later verification or rollout will contain it. The mistake is to let convenient access define the population silently.

This also changes facilitation. A group workshop may suppress a junior technician who disagrees with a leader. A translated or asynchronous method may be needed. An observation at a showcase site may miss low-connectivity behavior. An accessibility need may change the permitted artifact or session format. The FDE should work with the customer's appropriate research, accessibility, privacy, labor, and management owners rather than inventing local policy.

Produce a discovery decision packet

At the end of this chapter, do not hand the team a folder of notes. Produce a compact decision packet that someone who did not attend every session can inspect.

The packet contains:

1. **Purpose and decisions.** Which next decisions discovery must enable.
2. **Stakeholder coverage.** Impact, knowledge, access, responsibility, influence, authority, plus missing voices.
3. **Evidence index.** IDs, levels, provenance, handling, confidence, and affected hypotheses.
4. **Contradiction register.** Competing accounts, possible explanations, consequence, and next test.
5. **Access register.** Granted, pending, rejected, expired, and revoked access with purpose and owner.
6. **Emerging workflow hypothesis.** The current path and exceptions, explicitly not yet the verified OA-02 model.
7. **Decision-right gaps.** Missing or disputed accountable owners.
8. **Discovery limitations.** Population, time, environment, data, and method boundaries.
9. **Next action.** The Chapter 3 workflow-validation work and its stop conditions.

The packet should not turn uncertainty into presentation polish. Use plain language such as “observed twice under supervised conditions; prevalence unknown” and “regional owner not yet identified; production data path blocked.” Precision about weakness is a form of quality.

The stakeholder/evidence framework in this chapter is an original synthesis. It organizes durable discovery and access principles into an FDE artifact; it is not presented as a formal external standard. [CLM-014]

Conduct the discovery exercise

Prepare a two-week discovery plan for fictional Orchid without requesting broad production access. Name the decisions discovery must enable, stakeholder hypotheses, evidence ladder, complementary methods, access purpose/scope/expiry, handling/retention, stop conditions, and daily synthesis/replan cadence.

Inject three conflicts: sponsor says identifiers are reliable while technicians report ambiguity; dashboard shows fast closure while observations show transfers/rework; a useful log contains more sensitive content than the current purpose permits. Preserve the reports separately, choose the next credible test, and route the data-access decision to the named owner.

Pass when the packet distinguishes observed/reported/inferred evidence, records confidence and gaps, includes edge users/conditions, and can proceed without the FDE pretending domain authority or accumulating unnecessary access.

The entry gate

Before Chapter 3 maps the workflow, OA-01 should contain:

- a provisional responsibility charter;
- discovery decisions and questions;

- stakeholder hypotheses across all six dimensions;
- interview, observation, artifact, measurement, and reproduction plan;
- access-purpose records with handling and expiry;
- an evidence log and contradiction convention;
- accountable owners for formal decisions;
- stop/escalation conditions;
- explicit evidence and access gaps.

The gate does not require complete knowledge. It requires a legitimate path to stronger knowledge.

The most important habit is simple: never let proximity masquerade as proof. Enter the customer system with humility, precise questions, minimal access, and records that allow the evidence to disagree with you.

Chapter 3, *Map the Workflow That Actually Exists*, uses that foundation to model actors, decisions, states, data, tools, handoffs, incentives, queues, and exceptions. It will turn the sponsor's "AI copilot" request into a verified current-state problem model without erasing what remains uncertain.

3. Map the Workflow That Actually Exists

Model actors, decisions, state, data, tools, handoffs, incentives, exceptions, and failure cost without idealizing the current workflow.

The most dangerous workflow diagram is the one everyone recognizes and nobody actually follows.

It begins with a clean request. It moves through approved systems in a straight line. Each box has one owner. Data arrives before it is needed. Exceptions appear in a footnote. The diagram is easy to present because it describes how the organization believes work should happen.

A deployment enters the workflow that does happen.

That workflow contains waits, rework, personal notes, duplicated entry, judgment under incomplete evidence, inconsistent identifiers, unofficial communication, manual reconciliation, approval delays, retries, partial completion, and exceptions handled by people who may not appear in the project plan. A system designed for the ideal path does not merely miss detail. It can move the bottleneck, remove a safety check, create invisible work, or optimize the wrong result.

The purpose of current-state mapping is therefore not documentation. It is decision control. The model must be detailed enough to change the outcome contract, scope, architecture, verification, rollout, or ownership decision, and no more detailed than that.

Official service guidance supports mapping the whole user problem and the broken handoffs across channels, rather than treating one interface as the service. Observation of real users can reveal the difference between reported procedure and actual work. These principles ground the chapter; the coordinated model used here is an original FDE synthesis. [CLM-015] [CLM-016] [CLM-020]

Start with the operational result

Chapter 2 established legitimate access and an evidence log. Now choose a workflow boundary by naming an operational result from the user's perspective.

For Orchid, "use the AI copilot" is not the result. "Close a service ticket" may be the administrative endpoint, but it can still be a poor workflow boundary. A ticket can close after a temporary workaround while the equipment fails again, or after a difficult case is routed elsewhere.

A more useful provisional result is:

A technician resolves or appropriately escalates a service request with sufficient evidence, required approval, and an operable record of what was decided and why.

This boundary reaches before the proposed interface and after its output. It includes the request, equipment identification, triage, evidence gathering, decision, parts state, safety approval, work at the site,

resolution or escalation, and record completion. It may later be narrowed to a selected equipment family, region, and cohort, but the whole path must be visible before the slice is chosen.

Define the start and end events. Then ask which earlier and later events can change the meaning of the result. A technician arriving without the right part may originate in inventory data hours earlier. A repeated failure may be caused by a closure process that hides unresolved uncertainty. A privacy or safety consequence may begin with evidence collection before the user sees a recommendation.

The boundary is not universal. It serves a decision. Record which upstream and downstream paths are excluded and why.

Use coordinated views, not one perfect diagram

No single representation handles sequence, state, decision, data, ownership, incentives, and exception equally well. Trying to place everything on one canvas creates a model that is comprehensive but unusable.

Use five linked views.

1. The service journey

The journey shows the user's progress toward the operational result across channels and time. It includes visible action, waiting, handoff, failure, and recovery. It is useful for finding gaps between teams or systems that each appear healthy in isolation.

A journey row for Orchid might show:

1. request arrives through portal, phone, or email;
2. dispatch creates or amends a ticket;
3. equipment is identified or ambiguity is flagged;
4. technician receives assignment and gathers evidence;
5. site inspection changes the evidence state;
6. technician decides, seeks approval, or escalates;
7. inventory/part action occurs;
8. work is performed and verified;
9. ticket and equipment history are updated;
10. unresolved condition is handed to another owner.

The journey prevents the interface from becoming the workflow. Journey and experience maps are specifically useful for exposing cross-channel handoffs and gaps. [CLM-017]

2. The swimlane and state flow

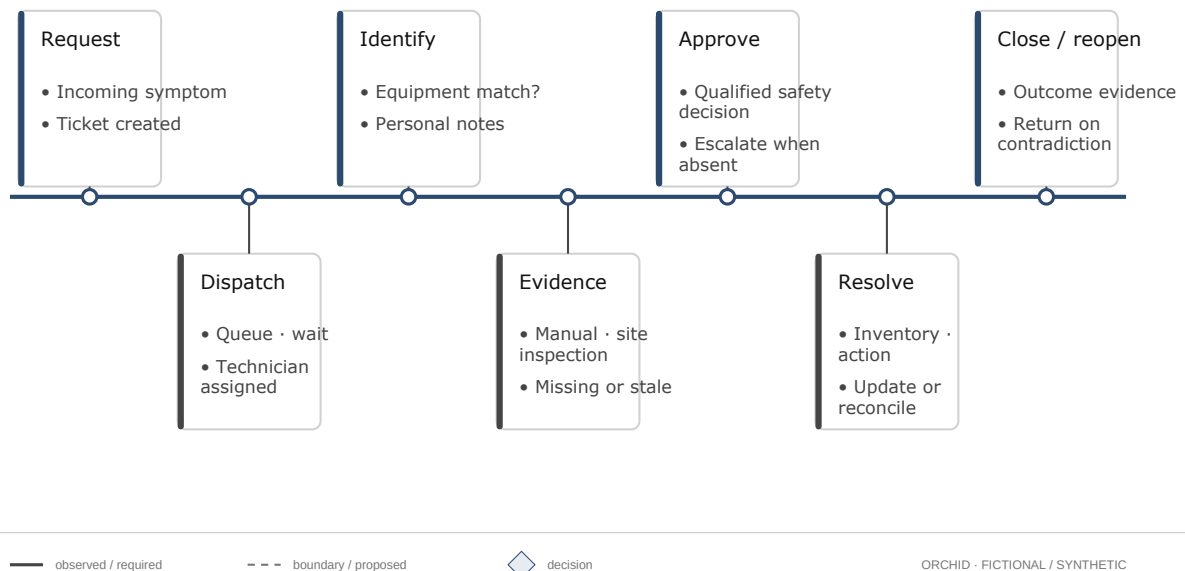
The swimlane shows actors and systems over sequence. A linked state model shows what the request, equipment match, recommendation, approval, part, and resolution can become.

For example, a request may be received, triaged, assigned, in-field, awaiting-evidence, awaiting-approval, awaiting-part, resolved, escalated, cancelled, or reopened. “Ticket open” is too coarse if different waits have different owners and consequences.

The state model should name transition evidence. What makes a request awaiting-approval rather than merely paused? Who can move it? Can a transition be reversed? Which external system is authoritative? What happens when systems disagree?

F03.1 · FORWARD DEPLOYED ENGINEERING

Orchid current workflow and state



F03.1 - Orchid current workflow and state. Observed work, waiting states, unofficial channels, decisions, and reopen loops stay visible.

Text alternative: Swimlane-style workflow from service request through dispatch, equipment identification, evidence, inspection, approval, inventory, resolution, record update, escalation, and reopen.

3. The decision and evidence table

For every material decision, record:

- decision and consequence;
- actor and required qualification;
- evidence available at that moment;

- evidence trusted, distrusted, or missing;
- rule, judgment, policy, or incentive applied;
- system action and human action;
- next state and escalation path;
- observed variation or contradiction.

Example:

Decision	Actor	Evidence available	Rule or judgment	Failure consequence
Which equipment record matches?	dispatcher or technician	ticket ID, site, serial plate, prior history	compare identifiers; resolve ambiguity manually	wrong manual, part, or history
Is suggested action eligible?	qualified technician/domain owner	equipment state, evidence, policy	deterministic/safety rule plus judgment	unsafe or prohibited action
Is the job resolved?	technician and service process	work performed, test result, remaining condition	resolution criterion or escalation	premature closure, repeat visit

The table reveals where the proposed system would act and which evidence/authority it must not replace.

4. The system and data inventory

List each system, store, document, informal tool, and identifier used by the workflow. Record its purpose, owner, authoritative claim, update path, latency/freshness, access, interface, common mismatch, and failure behavior.

Do not omit spreadsheets, messaging, paper, personal notes, phone calls, or memory because they are unofficial. They may carry the evidence that makes the official process work. Their presence is not proof that they should be automated or retained. It is evidence that the current path depends on them.

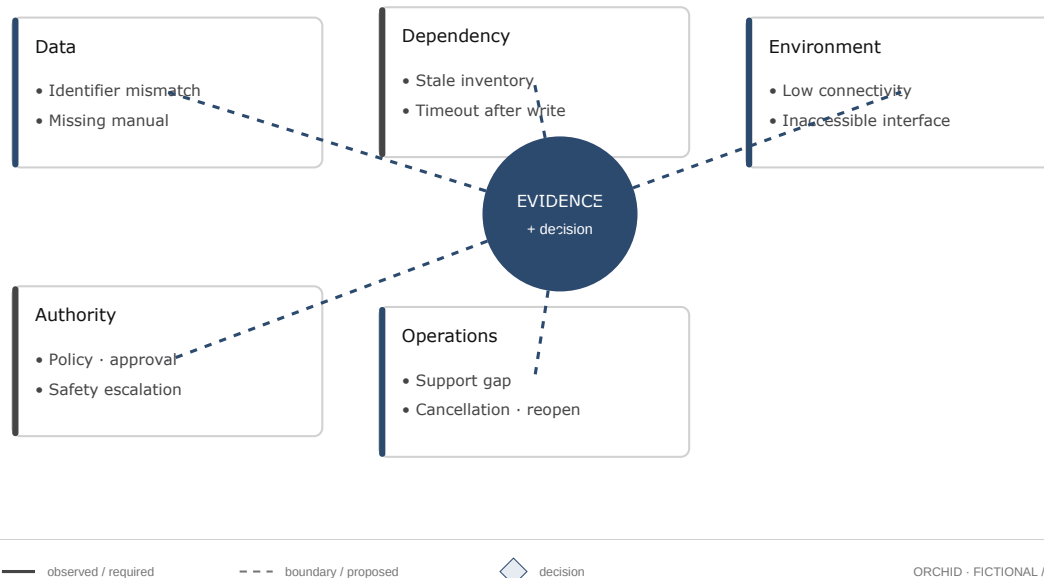
5. The exception topology

Place the normal path in the center and surround it with exception families:

- identity and matching;
- missing, late, duplicate, invalid, or contradictory data;
- connectivity and environment;
- policy, approval, and authority;
- inventory and external dependency;
- user access, capability, and accessibility;
- safety and high-consequence conditions;
- system timeout, retry, and ambiguous completion;
- handoff, ownership, and support;
- cancellation, reopen, and changed intent.

F03.2 · FORWARD DEPLOYED ENGINEERING

Exceptions define the real workflow



F03.2 - Exceptions define the real workflow. Each exception names evidence, owner, response, and a return or terminal state.

Text alternative: Exception topology around the Orchid normal path, including identifier mismatch, missing manual, stale inventory, low connectivity, approval, API timeout, accessibility, support, cancellation, and reopen.

The topology prevents exception handling from becoming “show an error.” It asks what work follows, who performs it, which state is authoritative, and how the main path resumes or ends.

Map what is observed and what is reported

Every element in the current-state model should retain evidence status.

- solid line: observed, measured, or reproduced within the recorded boundary;
- dashed line: reported or assumed, validation pending;
- conflict marker: evidence disagrees;
- gap marker: material behavior or owner unknown;
- proposed marker: future-state idea, excluded from current state.

Do not convert a dashed path to solid because several stakeholders repeat it. Repetition can reflect a shared official narrative. Do not mark a path false because an observation contradicts it once. The observation may be an exception or an effect of being observed.

Use evidence IDs on important nodes and transitions. A diagram should allow a reviewer to ask: “Why do we believe the equipment match happens here?” and reach the interview, observation, query, reproduction, limitation, and contradiction record.

The model is versioned. When new evidence changes it, record the change and affected decisions. Do not silently redraw history.

Separate process, state, and intent

Sequence alone can hide the most important technical ambiguity: whether an action completed and whether the same request still expresses the same intent.

Suppose Orchid Assist calls an ERP operation to reserve a part. The request times out. Three different states may exist:

1. the ERP never received the request;
2. the ERP committed the reservation but the response was lost;
3. the ERP accepted the request but later rejected downstream processing.

The technician sees one symptom: no confirmed result. A naive retry can create a duplicate reservation. Refusing to retry can leave the job blocked. The workflow needs a way to determine or reconcile operational state.

HTTP defines method semantics, including the idea of idempotent methods, but transport-level semantics do not by themselves settle the business meaning of a retry. First-party engineering guidance on idempotent APIs emphasizes caller intent and ambiguous completion. Technical retry behavior is therefore part of the workflow because it changes the next human and system decision. [CLM-019]

Map the intent explicitly:

- Which person/system initiated the action?
- What did they intend to happen once?
- How is that intent identified?
- Which system records acceptance and final effect?
- How can another attempt mean “same intent” versus “new intent”?
- How long is the intent identity valid?
- What does the user see while state is uncertain?
- Who reconciles disagreement?

Chapter 7 will design the contract. Current-state discovery must first establish how ambiguity is handled now, including manual calls, repeated clicks, spreadsheet checks, or silent duplicate work.

Put waiting and invisible work on the map

Value-stream diagrams often emphasize active work and duration. For FDE decisions, waiting needs ownership and reason.

Record:

- queue entered and exit condition;
- waiting owner versus action owner;
- evidence missing;
- service expectation or informal norm;
- user workaround while waiting;
- escalation threshold;
- consequence of delay;
- whether the wait is visible to the next actor.

At Orchid, a ticket may wait for equipment clarification, inventory confirmation, qualified approval, a site connection, or a regional data decision. Grouping every condition as “pending” prevents useful scope and telemetry decisions.

Invisible work includes copying identifiers, reformatting notes, reconciling systems, calling experienced colleagues, checking whether an API action took effect, and explaining system output to a user. Automation that reduces visible clicks but adds invisible verification work can worsen the workflow.

Observe who absorbs this work. Support, coordinators, junior staff, contractors, or affected users may compensate for system gaps without appearing in project measures.

Model incentives without pretending to read minds

The workflow is shaped by what people are measured, rewarded, blamed, or authorized to do. Record observable incentives and consequences, not psychological speculation.

Orchid service leaders may track ticket closure. Technicians may prioritize safe first-visit resolution. Inventory may minimize stock. Regional leaders may minimize data movement. Support may prioritize response targets. These aims can conflict without any actor behaving irrationally.

Ask:

- Which metric or policy affects this decision?
- What behavior could improve the metric while worsening the user outcome?
- Which difficult cases leave the measured population?
- Who bears rework or risk created by the local optimization?
- Which escalation is discouraged by time or performance pressure?

The answers will inform the metric tree in Chapter 4. At this stage, label them as reported, observed, or measured.

For Orchid, “time to ticket closure” can improve if difficult cases are transferred, prematurely closed, or excluded. “First-visit success” can improve if unsafe or unserviceable cases never enter the denominator. Mapping the transitions and populations is necessary before contracting either outcome.

Build an exception taxonomy that changes scope

An exception taxonomy is not a catalog of every edge case. It groups exceptions by the engineering and operating decision they require.

Use these fields:

- exception ID and family;
- trigger and evidence;
- affected actor, decision, and state;
- known frequency range or “unknown”;
- consequence and reversibility;
- current detection;
- current handling and owner;
- data/system dependencies;
- whether the first slice must handle, prevent, detect/escalate, simulate, defer, or reject it;
- validation evidence and remaining gap.

Example:

EX-ID-02: multiple equipment matches

- Trigger: ticket identifier maps to more than one active equipment record.
- Evidence: observed in supervised cases and reported by dispatch; population frequency not yet measured.
- Consequence: wrong history/manual/part; safety consequence depends on later action.
- Current handling: dispatcher/technician compares site, serial evidence, and prior history; escalates unresolved ambiguity.
- First-slice decision: detect and block recommendation until a qualified match is confirmed; include reconciliation path.

EX-ERP-04: timeout after reservation request

- Trigger: no response before client timeout.
- Evidence: API documentation reports timeout; commit semantics and occurrence unknown.

- Consequence: duplicate reservation or unresolved part state.
- Current handling: repeated request plus manual inventory check, according to one operator; reproduction pending.
- First-slice decision: interface cannot be designed until acceptance/reconciliation behavior is verified; dependency risk remains open.

The first example may belong in the slice even if rare because its consequence and central data boundary are material. The second may block automated writes while allowing a read-only recommendation path.

Apply the resolution test

How much detail is enough? Add detail when it can change at least one of these:

- the problem or outcome definition;
- the selected user, region, equipment, or exception boundary;
- a risk, guardrail, or authority decision;
- system/interface/data/identity architecture;
- an acceptance or evaluation case;
- a rollout cohort or stop trigger;
- an operating/support owner;
- a handoff or product-leverage decision.

If detail cannot change a decision, place it in a reference note or omit it. If a material decision depends on an unexplained transition, keep investigating.

The resolution test avoids two extremes. A shallow map hides constraints. An exhaustive map consumes time and becomes impossible to maintain. The goal is decision sufficiency, not a complete digital twin of the organization.

The five-view model and resolution test are an original synthesis. BPMN 2.0.2 is a standardized notation that may be useful for formal process representation, but neither the standard nor this book makes BPMN mandatory. A notation cannot compensate for weak evidence, missing users, or hidden exceptions.

[CLM-018] [CLM-020]

Derive the state model from decisions

State names are often inherited from a ticketing system. That can be useful, but the system's fields may exist for reporting or queue management rather than for the decision the deployment must support.

Derive workflow states by asking what becomes newly true, which action is now permitted, which evidence is required, and who is responsible next.

Take the generic state pending. It may conceal:

- `awaiting-equipment-match`: evidence is insufficient to bind the request to one equipment record; recommendations must remain blocked;
- `awaiting-site-evidence`: a technician must inspect or capture a condition before a decision can proceed;
- `awaiting-qualified-approval`: the proposed action exists but cannot be performed until an authorized person accepts it;
- `awaiting-inventory-confirmation`: a part candidate exists, but availability/freshness is unresolved;
- `awaiting-customer-access`: the site or system cannot be reached;
- `ambiguous-external-write`: a request timed out and the effect must be reconciled;
- `escalated-unowned`: the normal owner cannot resolve the case and a new accountable owner is not yet confirmed.

These states look similar on a dashboard if all are “pending,” but they require different data, timers, owners, alerts, user messages, and stop conditions.

For each candidate state, complete a state contract:

Field	Question
Meaning	What is true while the workflow is in this state?
Entry evidence	Which event and evidence permit entry?
Owner	Who must act or monitor now?
Allowed actions	Which operations are permitted or prohibited?
Exit transitions	Which evidence moves the workflow to each next state?
Time behavior	When does waiting become degradation or escalation?
System of record	Where is the authoritative state, and what can disagree?
User visibility	What does the affected person need to know or do?
Failure/recovery	How is an invalid, stuck, or split state detected and reconciled?

Do not assume every state belongs in the future application. Some exist only to understand current work. Some can be simplified after an architecture or policy decision. Others are vital because hiding them would create unsafe automation.

State disagreement is itself a state

When ticketing says “resolved,” inventory says “reserved,” and the equipment history has no completed work record, the deployment should not choose whichever system is easiest to query. Model the disagreement and its owner.

Ask which system is authoritative for each claim, not for the workflow as a whole. Ticketing may own assignment state. ERP may own inventory commitment. A qualified service record may own completed

inspection. An audit event may prove who approved an action. “Single source of truth” is often an unhelpful simplification when different systems legitimately own different propositions.

This distinction will later shape the interface catalog and reconciliation plan. At discovery time, it explains why users cross-check systems.

Quantify only what the decision needs

Workflow maps can become anecdotal if they contain no magnitude. They can become falsely authoritative if weak observations are converted into exact numbers. Use a measured-evidence plan tied to decisions.

Suppose scope depends on whether identifier mismatch is material. Define:

- population: selected-family service requests created in Region West during the stated 30-day window;
- unit: distinct request at initial triage;
- mismatch: no unique active equipment record can be selected using the canonical ticket identifier under rule version 2;
- exclusions: test tickets, cancelled before triage, and requests without equipment by design;
- data owner and query version;
- missingness and known correction behavior;
- segmentation: channel, site connectivity class, technician cohort, and equipment subfamily where permitted;
- decision: include reconciliation in the first slice if consequence and measured/unknown magnitude justify it; otherwise detect/escalate with a validation plan.

The threshold is not invented by the engineer. The outcome and risk owners decide how evidence affects scope. The FDE ensures the measurement definition is inspectable and the unknown is not replaced with a convenient estimate.

Some decisions do not need prevalence. A rare exception with catastrophic or irreversible consequence may require prevention, approval, containment, or explicit exclusion. A common low-cost workaround may be deferred. Frequency, consequence, detectability, reversibility, and owner capacity must be considered together.

Use ranges and categories when the evidence supports only ranges and categories. “Observed in 2 of 7 supervised cases, convenience sample, prevalence unknown” is stronger than “29 percent” because it does not pretend sampling validity.

Show how the map changes the deployment

A workflow model earns acceptance by changing downstream work. For Orchid, each finding should have a trace:

Finding: equipment identifier is not always a trusted key

- Problem effect: evidence retrieval can begin from the wrong equipment.
- Outcome effect: “faster recommendation” is unsafe without match confidence/confirmation.
- Scope effect: the first slice must detect ambiguity and include human confirmation/reconciliation.
- Architecture effect: ticket ID and equipment ID remain distinct; lineage/provenance is required.
- Verification effect: duplicate, stale, absent, and conflicting identifiers become required cases.
- Rollout effect: cohort selection should include representative mismatch conditions or explicitly exclude them with monitoring.

Finding: technicians rely on provenance to trust advice

- Problem effect: generated or ranked output without sources increases verification work.
- Outcome effect: time saved must account for evidence checking and bypass behavior.
- Scope effect: visible evidence and uncertainty are included before cosmetic breadth.
- Architecture effect: retrieval/provenance path is a first-class component.
- Verification effect: UAT must test whether users can inspect and challenge supporting evidence.
- Adoption effect: low use is diagnosed against evidence visibility, not labeled resistance.

Finding: safety-relevant decisions require qualified approval

- Role effect: the FDE cannot define or accept the safety rule.
- Scope effect: autonomous action is excluded; approval and escalation are included.
- Architecture effect: deterministic eligibility, identity/authorization, audit, and fail-closed behavior are required.
- Verification effect: prohibited/uncertain cases and authority failures are tested.
- Rollout effect: qualified approvers and escalation coverage must exist for every exposed cohort.

Finding: ERP completion can be ambiguous after timeout

- Scope effect: automatic writes may be deferred until semantics and reconciliation are verified.
- Interface effect: intent identity, status lookup, conflict behavior, and operator reconciliation become contract requirements.
- Operability effect: uncertain state requires a signal, queue, owner, and runbook.
- Recovery effect: retry/rollback cannot be designed as a transport-only choice.

If a major workflow finding has no downstream consequence, ask whether it belongs in the model. If a downstream decision has no workflow finding or explicit assumption behind it, the deployment is designing from preference.

Keep the current state ethically legible

Current-state mapping can expose workarounds, policy deviations, individual mistakes, performance concerns, or sensitive operational weaknesses. Treat this evidence according to the approved purpose and customer process.

The artifact should make systemic conditions visible without turning it into a personnel dossier. Use role labels and de-identified cases where individual identity is not required. Restrict sensitive variants. Do not publish an unofficial workaround merely because it is operationally important. Escalate immediate safety or security concerns through the customer's established path rather than waiting for a chapter artifact to be reviewed.

"Blameless" mapping does not mean pretending choices have no owners. It means asking which conditions, incentives, information, tools, authority, and safeguards made the observed choice reasonable or likely, then locating the appropriate corrective decision.

The FDE's proximity creates a special obligation here. Users who reveal the real workflow are not consenting to have every detail copied into product analytics, training data, or cross-customer stories. The evidence and access rules from Chapter 2 continue to govern the map.

Validate the model with users and owners

Validation is not asking a sponsor whether the diagram "looks right." Use role-diverse walkthroughs and concrete cases.

Normal-case walkthrough

Choose a recent representative case. Ask each actor to locate when they entered, what they knew, which state they trusted, what they produced, and what happened next. Compare the model with artifacts or records inside the approved evidence boundary.

Exception walkthrough

Choose a high-consequence or common exception. Ask what signal reveals it, who responds, what authority is required, how state is reconciled, and how the path ends. If participants disagree, retain the contradiction.

Counterfactual walkthrough

Change one condition: no connectivity, duplicate ID, unavailable expert, stale inventory, policy conflict, downstream timeout, or user unable to access the interface. Observe which assumed path collapses.

Ownership walkthrough

For each wait, control, data correction, support action, and escalation, ask the named owner to confirm responsibility and access. A label placed by the project team is not owner acceptance.

Evidence walkthrough

Ask reviewers which parts are observed, measured, reproduced, reported, or unknown. Confirm that the legend matches the evidence log.

Validation can produce three outcomes: accepted within a boundary, disputed with next evidence, or unknown with consequence/owner. “Validated” does not mean every participant agrees or every gap is solved.

Orchid Assist: current-state findings

The fictional case now has enough stated evidence to create OA-02 Verified Current-State Workflow. These are constructed pedagogical findings, not claims about a real company.

Observed or stipulated case facts

- Requests arrive through multiple channels and are normalized into tickets with inconsistent completeness.
- The ticket equipment identifier is not always a trusted key; matching can require site, serial, history, and human judgment.
- Technicians gather evidence across manuals, old tickets, inventory, equipment records, telemetry, and informal knowledge.
- Qualified approval is required for safety-relevant recommendations.
- Some sites have intermittent connectivity.
- Responsibility is split across service operations, IT, data, security, inventory, and regions.
- Technicians distrust recommendations that hide supporting evidence.

Remaining evidence gaps

- frequency and segment distribution of identifier mismatches;
- exact ERP timeout and write-completion semantics;
- inventory freshness by workflow decision;
- support demand attributable to evidence fragmentation;
- low-connectivity latency and abandonment patterns;
- authoritative safety rule and approval transition;
- baseline relationship among closure, resolution, repeat visit, and escalation.

Revised problem statement

The problem is not that technicians lack generated text. The bounded problem is that the evidence required to resolve or appropriately escalate selected service requests is fragmented, inconsistently identified, variably available, and difficult to evaluate at the point of decision. Any deployed path must improve evidence assembly and decision support while preserving qualified safety approval, visible provenance, exception handling, regional/access constraints, and operable state reconciliation.

This statement may still be rejected by later evidence. It is stronger than the initial request because it names the workflow decision, evidence conditions, and guardrails without committing to AI as the universal mechanism.

Workflow-mapping failure modes

The happy-path mural

The map describes the normal official sequence and places exceptions in a list.

Repair: build the exception topology and validate at least one material exception end to end.

The future state in current-state colors

The team draws how a new system will connect and calls it discovery.

Repair: mark proposed nodes and flows separately; require evidence IDs for current-state claims.

The unreadable master diagram

Every data field, team, system, state, and exception appears in one image.

Repair: use linked views with stable IDs and the resolution test.

The title-as-owner map

The project assigns responsibility based on job title without owner confirmation.

Repair: perform ownership walkthroughs; distinguish access, execution, responsibility, and decision authority.

False frequency

Words such as “usually,” “rarely,” or “always” become percentages, or a small observation sample becomes prevalence.

Repair: keep reported frequency as reported; measure with a defined population/method or record unknown.

Tool worship

The team debates BPMN, diagram software, or process-mining tools while evidence and semantics remain weak.

Repair: choose the smallest representation the audience can validate; keep editable sources and provenance.

Exception as error message

The technical design plans to display an error without mapping the work required afterward.

Repair: model detection, decision, owner, next state, reconciliation, return path, and terminal outcome.

Conduct the workflow-mapping exercise

Observe or use a constructed service request from arrival through closure. Produce the journey, swimlane/state flow, decision/evidence table, system/data inventory, and exception topology. Mark every statement observed, reported, inferred, or unknown.

Inject ambiguous equipment, stale manual, missing inventory response after acceptance, qualified-approval need, and offline technician work. Show waiting, rework, manual coordination, and ownership rather than hiding them in error boxes. Write the revised problem statement and identify which finding changes scope or architecture.

Validate the normal, exception, counterfactual, ownership, and evidence walkthroughs with representative fictional roles. Pass when another reader can predict state, decision, evidence, consequence, and owner for both the common and consequential exceptional paths.

The OA - 02 gate

Before Chapter 4 contracts an outcome, OA-02 should contain:

- operational result and explicit workflow boundary;
- whole journey and channel/handoff view;
- current-state swimlane and state model;
- decision/evidence/authority table;
- system/data/informal-tool inventory;
- exception taxonomy and topology;
- waits, invisible work, incentives, and metric-population risks;
- evidence IDs and observed/reported/measured/reproduced/unknown legend;
- user/owner validation records and unresolved contradictions;
- revised problem statement;
- explicit remaining gaps and their next tests.

The gate passes when the model is sufficient to change a decision and its uncertainty remains visible. It does not pass because the diagram is polished.

Chapter 4, *Contract for an Outcome*, will turn this current-state evidence into a metric tree, guardrails, adoption measures, baselines, decision rights, non-goals, and exit criteria. That contract must be able to reject a convincing prototype that improves interface activity while leaving the real workflow unchanged or worse.

4. Contract for an Outcome

Turn workflow evidence into measurable outcomes, guardrails, adoption signals, decision rights, assumptions, non-goals, and exit criteria.

A convincing prototype can be evidence of technical possibility and still be evidence against the deployment.

It may show that a model can summarize manuals, that an integration can return inventory, or that a user can complete a scripted task. It may also reveal that the team has not defined which workflow result matters, what can become worse, who may accept the consequence, how real use will be observed, or when the work should stop.

The outcome contract answers those questions before polish makes them harder to ask.

This is not a commercial contract. It does not set price, legal liability, or procurement terms. It is an engineering and decision artifact that binds a deployment hypothesis to measurable evidence, guardrails, authority, and exit conditions.

Official service guidance supports defining performance evidence early, establishing baselines, and monitoring benefit so a team can continue, change, or stop. Site reliability practice adds a compatible principle: user-relevant indicators and objectives are valuable when they lead to explicit action and tradeoffs. This chapter synthesizes those ideas into an FDE outcome contract. [CLM-021] [CLM-022] [CLM-024] [CLM-027]

Contract the change in work, not the feature

Chapter 3 reframed Orchid's request. The bounded problem is not the absence of generated text. Evidence needed for selected service decisions is fragmented, inconsistently identified, variably available, and difficult to evaluate at the point of work.

A weak success statement is:

Launch an AI copilot to all technicians by the seasonal peak.

It names a mechanism, population, and date. It does not state value, guardrails, evidence, baseline, adoption, authority, or what to do if the mechanism fails.

A stronger outcome hypothesis is:

For a selected equipment family, region, and technician cohort, an evidence-assisted workflow may reduce the effort and delay required to reach a supported resolution or appropriate escalation, while preserving qualified safety decisions, evidence visibility, data/access boundaries, service reliability, cost control, and customer ownership.

The word *may* is essential. At this point Orchid has no measured baseline or accepted target in the fictional case. The outcome contract must record the measurements required rather than invent them.

The contract begins with six questions:

1. Which operational result should change?
2. For which users, cases, environment, and time boundary?
3. Which evidence would show progress or failure?
4. Which harm, degradation, or cost must not be traded away?
5. Who has the authority to define, accept, stop, or revise each consequence?
6. What action follows each evidence state?

If the final question has no answer, the metric is observation without governance.

Build a metric tree

A metric tree connects the intended workflow result to evidence at different distances from it.

Use five branches.

Outcome measures

Outcome measures describe the operational result users or the customer ultimately need. For Orchid candidates may include time to supported resolution or appropriate escalation, first-visit success under a precise definition, repeat service within a defined window, or proportion of selected cases reaching an evidence-complete decision.

These are candidates, not accepted metrics. The customer must define the operational meaning, population, baseline, target direction or threshold, and consequence.

Leading workflow measures

Leading measures describe steps plausibly connected to the outcome: time to identify equipment, time to gather required evidence, rate of unresolved identifier ambiguity, approval wait, inventory reconciliation time, or proportion of recommendations whose evidence is inspected before decision.

They are often available sooner than outcome measures. They can be useful for diagnosis. Improving them does not guarantee the outcome.

Adoption and use measures

Adoption asks whether the intended workflow is actually incorporated into representative work. Measures may include eligible cases in which the path is used, repeated use by the target cohort, completion versus abandonment, bypass reason, evidence-view behavior, and distribution across users/sites/case types.

Use is not value. Non-use is not automatically resistance. Use, satisfaction, dropout, and segmented workflow evidence are complementary signals. [CLM-023]

System and quality measures

These describe technical behavior required for the workflow: latency by site/cohort, error/timeout rate, identifier-match state, data freshness, recommendation/evaluation category, approval/audit completeness, retry/reconciliation state, and support burden.

System measures are usually causes, constraints, or diagnostic signals. Availability does not prove usefulness; model quality does not prove adoption.

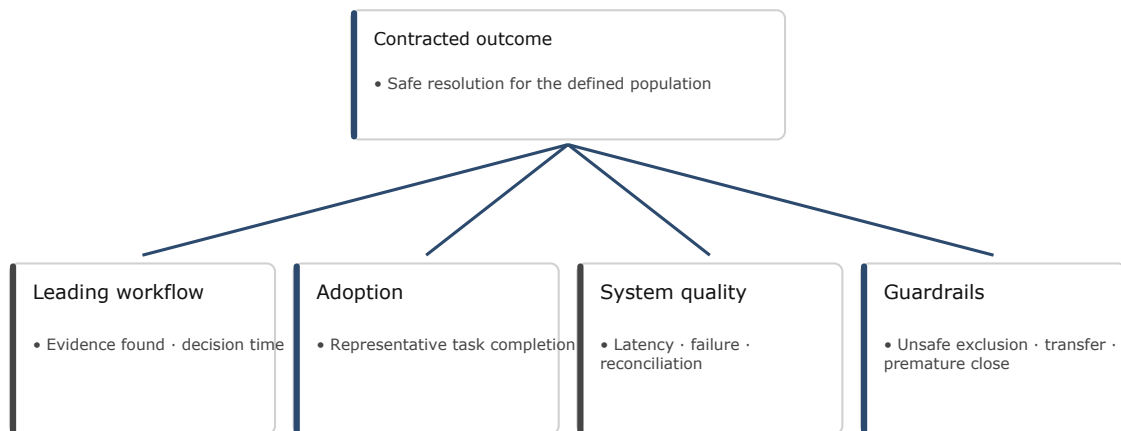
Guardrail measures

Guardrails show whether apparent benefit is achieved by an unacceptable trade. Orchid guardrail candidates include safety-rule violation or unsafe recommendation categories, missed required escalation, incorrect equipment/part binding, privacy/access exception, reliability degradation, user effort moved into verification, support load, cost per eligible case, accessibility failure, and regional-data breach.

A guardrail can stop or reduce rollout even when the primary outcome improves.

F04.1 · FORWARD DEPLOYED ENGINEERING

Outcome evidence needs guardrails



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F04.1 - Outcome evidence needs guardrails. Metric branches show hypotheses and gaming paths, not guaranteed causality.

Text alternative: Metric tree with safe resolution outcome above workflow, adoption, system quality, and guardrail branches, plus gaming paths where closure improves while safe resolution worsens.

Define every measure as a contract

A label such as “resolution time” is not a definition. A metric record should contain:

- stable metric ID and name;
- decision or promise served;
- exact numerator, denominator, unit, and event semantics;
- population, cohort, segment, exclusions, and window;
- start and end conditions;
- source systems and provenance;
- data quality/missingness limits;
- baseline method and version;
- target direction, threshold, or range, with owner;
- cadence and observation window;
- guardrail relationship;
- action when healthy, uncertain, or breached;

- accountable decision owner;
- known gaming/counterfactual risk;
- change history.

Consider “first-visit success.” It could mean no additional site visit within 30 days, issue resolved during the first arrival, or customer operation restored without escalation. It can be gamed by excluding difficult equipment, converting repeat visits into new tickets, or defining success before the equipment is tested. The useful definition must match the workflow state model and disclose exclusions.

For fictional Orchid, use placeholders such as `BASELINE REQUIRED`, `TARGET OWNER REQUIRED`, and `WINDOW TO BE APPROVED`. Do not replace missing evidence with a realistic-looking number.

Establish a baseline without freezing the past

A baseline describes the current measurement under a versioned definition and boundary. It is not “the number before launch” unless the collection semantics are comparable.

Baseline work should check:

- Does the current workflow emit the start/end events?
- Did definitions, policies, systems, or populations change during the window?
- Are difficult or abandoned cases missing?
- Can the measure be segmented by the target cohort without inappropriate data collection?
- Does measurement itself change behavior?
- Is the seasonal peak materially different?
- Which manual work is not captured?
- Can the same metric definition operate after the deployment?

If no credible baseline exists, say so. Options include a prospective baseline period, customer-reviewed sampling, a phased comparison, matched historical data with limitations, or a direction-only pilot claim. The decision owner must accept the weaker inference.

Benefit guidance emphasizes establishing a baseline and monitoring evidence that can support continuation, change, or stopping. It does not make the baseline causal proof. [CLM-022] [CLM-026]

Segment before the average reassures you

An aggregate can improve while a consequential segment worsens.

Orchid should consider segments such as equipment family/subfamily, region, site connectivity, technician experience, request channel, exception class, safety relevance, data completeness, and workflow route. Only collect/use segment attributes that are legitimate and necessary.

Suppose mean time to evidence improves. Low-connectivity sites may be slower. New technicians may abandon more often. High-risk equipment may have worse evidence retrieval. A model may perform well on common manuals and poorly on a minority equipment family.

The contract should identify **precommitted critical segments** whose results cannot be hidden in an average. It should also define minimum evidence volume or uncertainty treatment. A tiny segment should not produce a precise claim, but high consequence may still require a block or review.

Test every metric for gaming

Ask how a rational actor or system could improve the number without improving the intended result.

For each candidate metric:

1. Can cases be excluded, transferred, cancelled, or relabeled?
2. Can the start or end event move?
3. Can difficult users disappear from the denominator?
4. Can work move outside the measured system?
5. Can speed improve by reducing evidence, quality, safety, or explanation?
6. Can use be coerced or duplicated?
7. Can an error become an escalation that another team absorbs?
8. Which guardrail or audit would reveal the gaming path?

Orchid's ticket closure metric is vulnerable. A difficult case can be reassigned, prematurely closed, split, or excluded from the selected category. The metric tree therefore treats closure as a process signal at most, not the production outcome.

Adoption is a diagnostic system

Adoption combines eligibility, access, repeated use, workflow fit, trust, and retained value.

Define the adoption funnel:

1. eligible case occurred;
2. user had access and the system was available;
3. workflow path was offered or selected;
4. user began the path;
5. user reached a meaningful decision state;
6. output/evidence was inspected or used as intended;
7. user completed, escalated, or intentionally rejected the path;

8. user returns for later eligible cases;
9. support/bypass reason is recorded.

Each transition can fail for a different reason. A training campaign is not the default remedy.

A vendor customer story reports 98 percent active use by Morgan Stanley advisor teams. The figure is useful here only as an attributed example of adoption evidence; it is not independent proof of productivity, correctness, financial outcome, or customer benefit. [CLM-025]

This attribution discipline applies to Orchid too. Even if synthetic exercises later show a high use rate, the text must not convert it into a real customer outcome.

Connect service objectives to action

Technical objectives matter because a workflow depends on them. An SLO-like record should connect a user-relevant indicator to action and tradeoff, not choose a target because an industry example uses it. [CLM-024]

For example:

For eligible low-connectivity pilot requests, the evidence path must return a usable response or an explicit safe fallback within the approved latency objective for the stated proportion/window. A breach pauses cohort expansion and invokes the runbook; repeated breach triggers reduced scope or rollback under the named owner.

The objective remains incomplete until Orchid supplies the actual threshold and accepts the tradeoff. The book can teach the structure without inventing the number.

Calibrate the attribution claim

An outcome can move for many reasons: seasonality, staffing, policy, data cleanup, training, inventory availability, case mix, another system change, or simple measurement drift. The outcome contract should state what kind of inference the deployment can support.

Use an attribution ladder.

Descriptive

During the observation window, the contracted measures had these values for the stated cohort and definition.

This describes what was measured. It does not connect the change to the deployment.

Before/after association

The measure changed from the versioned baseline to the pilot window after the deployment was introduced.

This establishes timing and direction under the definitions. It remains vulnerable to case mix, seasonality, concurrent changes, and regression to the mean.

Comparative association

The exposed cohort changed differently from a defined non-exposed or earlier cohort under stated comparability assumptions.

This strengthens the evidence if assignment, populations, and measurement are credible. It does not automatically remove selection or spillover.

Plausible contribution

The deployment changed specific leading workflow states in the expected direction; outcome/guardrail movement was consistent with the mechanism; alternative explanations were investigated and remain bounded as stated.

This combines process evidence with outcome evidence. It can support a professional decision without claiming experimental causality.

Causal estimate

The study design supports a defined causal effect estimate under its assumptions.

This requires appropriate experimental or quasi-experimental design, sufficient evidence, and specialist review where consequence demands it. Most FDE pilots should not casually claim this level.

The contract chooses the intended claim level before the results are known. That reduces the temptation to promote a favorable association into causality after launch.

For Orchid, a bounded cohort before/after comparison may be feasible, but the seasonal peak, equipment mix, technician experience, and concurrent process changes could matter. The initial contract should plan to report descriptive and associated change plus mechanism/limitations unless stronger design is approved.

Design the measure from events and states

The workflow state model from Chapter 3 is the best defense against metric ambiguity.

Suppose Orchid selects “time to supported decision.” Define it from state transitions:

- Start: request first enters triaged with sufficient identity to enter the selected workflow population.
- End: request enters supported-resolution, qualified-escalation, or another approved terminal decision state.
- Paused time: decide whether customer/site unavailable time is included, excluded, or reported separately.
- Reopened work: define whether reopening continues the same episode and for how long.
- Transfers: preserve the original episode if the same operational result remains unresolved.
- Cancelled cases: classify by reason and never exclude silently.
- Missing end: retain as unresolved/censored under the defined analysis rather than dropping it.
- Version: state contract and metric definition versions remain linked.

Now the team can test event emission and query behavior before production. A dashboard label cannot replace this definition.

For “evidence-complete decision,” the contract must name required evidence classes and who defines completeness. A model citation count is not sufficient. The appropriate qualified owner may require equipment identity, current manual version, observed condition, inventory state, policy check, and approval evidence for a particular decision class.

The FDE can implement and verify the measurement. The domain owner defines the criterion where domain authority is required.

Pair rate and burden

A percentage can improve by adding hidden work. If evidence-complete decisions rise because technicians spend twice as long verifying output, the outcome may not improve.

Pair selected rates with burden evidence:

- user time and steps;
- context switches or duplicate entry;
- support contacts;
- exception queue age;
- reviewer/approver load;
- correction/reconciliation work;
- latency and abandonment;
- qualitative evidence about trust and fit.

Not every burden requires continuous tracking. The contract identifies which burden could reverse the benefit and how it will be sampled or monitored.

Measure the denominator before celebrating the numerator

If 80 of 100 surfaced recommendations are accepted, the acceptance rate is 80 percent. If only 100 of 1,000 eligible cases surfaced because difficult cases were filtered or failed earlier, the workflow story is different.

Maintain population flow:

eligible -> accessible/available -> offered -> started -> evidence produced -> decision -> completed/escalated/rejected -> retained outcome

Report loss and reason at each transition. Unknown reason is a meaningful category until resolved.

Turn evidence states into decisions

An outcome contract should include a decision table before results exist.

Evidence state	Example interpretation	Permitted action
outcome improves, guardrails healthy, adoption representative, operations ready	hypothesis supported within limits	continue/ramp under owner approval
leading measure improves, outcome window incomplete	mechanism signal only	hold scope; continue observation
outcome improves, critical segment/guardrail fails	benefit not acceptable for current path	stop/reduce/repair; no aggregate override
use low, technical health good	cause unknown	diagnose workflow/access/trust; do not force rollout
outcome unchanged, burden/support rises	hypothesis not supported	stop or redesign
evidence quality insufficient	claim cannot be made	improve measurement, reduce claim, delay, or stop
benefit observed, ownership/recovery unproven	customer outcome not yet supportable	do not exit/ramp; complete operations/handoff evidence

The table prevents result interpretation from becoming a negotiation after the deadline. It also makes a negative or inconclusive result useful. A stopped deployment can be a successful decision if it prevents unjustified exposure or investment.

Resolve competing outcomes explicitly

Stakeholders can agree that a deployment should “improve service” while optimizing different consequences.

For Orchid:

- service leadership values throughput and seasonal capacity;
- technicians value safe, evidence-supported resolution;
- customers value equipment availability and trustworthy communication;
- inventory values stock efficiency and correct reservation;
- security/privacy owners value bounded access and handling;
- support values diagnosable failure and manageable demand;
- product teams value reusable capability;
- finance may value cost predictability.

The outcome contract does not hide these interests inside one weighted score. Weighted scores can make serious guardrail failure appear acceptable because another benefit is large.

Use a hierarchy:

1. prohibited or formally constrained outcomes;
2. minimum guardrail conditions;
3. primary bounded outcome;
4. secondary benefits and burdens;
5. diagnostic/leading measures.

Where two valid outcomes conflict above a threshold, route the tradeoff to the named authority. The FDE documents alternatives, evidence, reversibility, and consequence. A metric formula should not silently accept risk.

Version the outcome contract

The contract will change as the scope and system become concrete. Version changes should name:

- evidence that triggered the change;
- affected metric/definition/population/owner;
- whether baseline comparability breaks;
- which tests, dashboards, rollout triggers, or claims change;
- who approved the change;
- effective date and prior-version disposition.

Never adjust a threshold after seeing an unfavorable result without recording the change. A legitimate definition correction is possible, but the old result and reason remain visible. Otherwise the team teaches the metric to pass.

Build a guardrail ladder

Not every guardrail uses the same response. Classify the control action:

- **prevent:** prohibited action cannot execute;
- **require approval:** qualified identity must accept before action;
- **detect and block:** evidence state triggers a stop/fallback;
- **detect and escalate:** action pauses for an owner;
- **monitor and review:** signal enters a scheduled decision;
- **limit exposure:** cohort, capability, rate, or region remains bounded;
- **stop/rollback:** threshold or event ends exposure;
- **record limitation:** residual uncertainty is visible and accepted by the authorized owner.

Safety-relevant eligibility may be prevented or require qualified approval. Low-confidence evidence matching may block recommendation. Latency degradation may limit exposure. Support demand may trigger review. The ladder connects the metric to an actual production decision.

Separate assumptions, constraints, and decisions

An outcome contract includes an assumption register.

- **Assumption:** unverified proposition with owner, consequence, and validation path. Example: selected-family manuals are sufficiently current for evidence retrieval.
- **Constraint:** real boundary shaping the solution. Example: selected regional data cannot leave an approved boundary.
- **Decision:** accepted choice with evidence, alternatives, authority, and revisit trigger. Example: pilot excludes autonomous parts ordering.

Do not promote an assumption into a constraint because it is inconvenient to test. Do not call a preference a requirement. Do not treat a constraint as immutable if the accountable owner can change it.

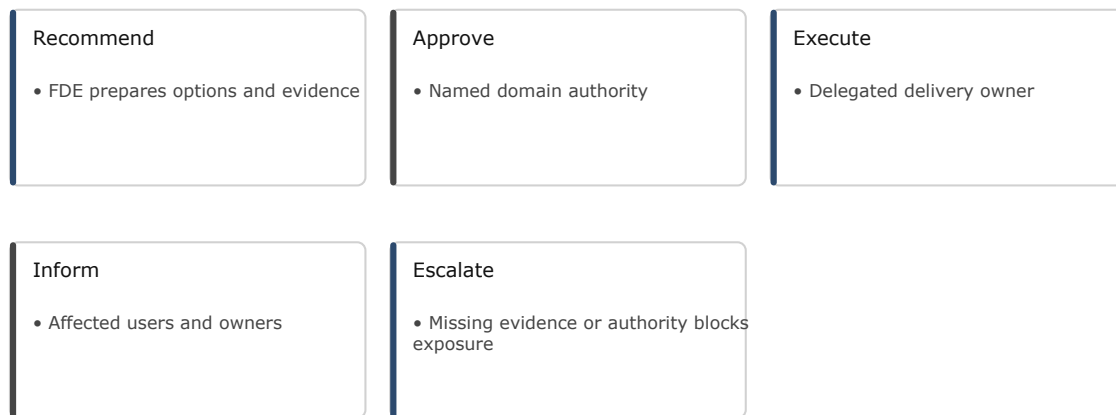
Map decision rights

For each outcome, guardrail, scope, and gate decision, name four actions:

- **recommend:** assembles evidence/options and proposes a path;
- **approve/accept:** holds authority for the consequence;
- **execute:** performs the change or operation;
- **inform:** needs timely decision/result information.

F04.2 · FORWARD DEPLOYED ENGINEERING

Decision rights stay named



F04.2 - Decision rights stay named. Recommend, approve, execute, and inform are separated for every consequential decision.

Text alternative: Decision-rights matrix for outcome, safety, privacy, security exception, pilot scope, release, go or no-go, incident command, and handoff, separating FDE recommendation from approval.

The FDE can recommend the technical path and maintain the evidence chain. Orchid's service owner defines/accepts business outcome consequences. Qualified safety owners define approval rules. Security/privacy/regional owners handle their formal decisions. The customer's rollout authority accepts go/no-go consequences. Execution can belong to different engineering or operations teams.

An owner's absence is a blocker, not permission for the FDE to inherit authority.

Write non-goals and rejection criteria

Non-goals prevent the outcome statement from becoming an unlimited promise. For the initial Orchid contract:

- not all equipment families, regions, or channels;
- not automatic safety-relevant action;
- not enterprise identifier remediation;
- not replacement of customer safety/security/privacy/support ownership;
- not proof that AI is superior to deterministic or non-AI alternatives;
- not guaranteed causal attribution from a bounded pilot;

- not platformization of customer-specific code.

Rejection criteria state when a technically impressive result is insufficient:

- output lacks evidence needed for qualified judgment;
- aggregate quality hides a critical segment failure;
- use requires unsafe access or violates a data boundary;
- the workflow adds more verification/support burden than it removes;
- no credible recovery or support owner exists;
- the outcome cannot be measured under an accepted limitation;
- the first safe path cannot be bounded before the seasonal peak.

Define entrance, exit, and stop criteria

An outcome contract governs several gates.

Discovery exit

The current workflow, material exceptions, outcome hypothesis, stakeholders, access, and evidence gaps are sufficient to define measures and safe scope.

Pilot entrance

The selected cohort/path has verified behavior, controls, operability, support/user preparation, rollback/stop triggers, and authorized risk disposition.

Pilot exit

The agreed outcome, guardrail, adoption, and operating evidence has been observed over the accepted window, with limitations and exceptions dispositioned.

Stabilization exit

Material incidents/corrective actions are addressed, signals and support ownership operate, representative users can perform the path, and residual risks are accepted.

Engagement exit

The customer demonstrates operation/change/recovery, access is transferred or revoked, outcome review is complete, and reusable work has an isolation-aware disposition.

Stop criteria

Stop or reduce the path when a prohibited event occurs, a critical guardrail breaches, required approval disappears, evidence cannot support the claim, support/recovery is not viable, or new evidence invalidates the outcome hypothesis.

An outcome contract should make continue, change, reduce, delay, or stop legitimate outcomes.
[CLM-026]

The OA-03 Orchid outcome contract

The fictional artifact now contains the structure below. Values remain intentionally unresolved until a hypothetical Orchid owner supplies or approves them.

Population. Selected equipment family, Region West, named pilot technician cohort, eligible service-request definition pending.

Outcome candidates. Time to supported resolution/appropriate escalation; evidence-complete decision rate; first-visit success under an approved definition.

Leading candidates. Equipment-match time/state; evidence-gathering time; approval wait; reconciliation wait.

Adoption candidates. Eligible cases offered/started/completed/escalated/rejected; repeated use; bypass reason; evidence-view behavior; segment coverage.

System/quality candidates. Latency by site; dependency/timeout/retry state; identifier/data quality; recommendation error class; audit/approval completeness; support demand/cost proxy.

Guardrail candidates. Safety-rule/approval violation; incorrect match/part; missed escalation; privacy/access exception; reliability; accessibility; low-connectivity degradation; verification burden; support load; cost.

Required evidence fields. BASELINE REQUIRED, DEFINITION OWNER REQUIRED, TARGET/THRESHOLD REQUIRED, OBSERVATION WINDOW REQUIRED, ACTION/OWNER REQUIRED.

Decision rights. FDE recommends/maintains evidence. Service, safety, security, privacy/regional, operations/support, and rollout owners approve within named boundaries.

Non-goals/rejection/exit/stop. As defined above and versioned as discovery/scope evidence changes.

The artifact is useful before it has numbers because it identifies exactly which numbers, definitions, owners, and actions are missing. It is not ready for go/no-go until those gaps are resolved or formally dispositioned.

Outcome-contract failure modes

Vanity success

Logins, prompts, outputs, or demos become the headline.

Repair: trace activity through the adoption funnel to workflow outcome and guardrails.

Aggregate comfort

The mean improves while a critical region, user, or equipment segment fails.

Repair: precommit material segments and consequence-based review.

Target without baseline

A realistic-looking target is chosen before event semantics and current performance are known.

Repair: define the measurement and baseline method; mark target authority/evidence as missing.

Guardrail without action

A risk metric is monitored but no threshold, owner, or stop path exists.

Repair: assign a control response and decision right.

Causal overclaim

Observed improvement during a pilot is attributed entirely to the deployment.

Repair: state design, comparison, confounders, counterfactual limits, and calibrated claim.

FDE acceptance

The FDE signs off a safety, privacy, security, business, or support consequence because the owner is absent.

Repair: escalate, reduce, delay, or stop; keep the missing decision visible.

Conduct the outcome-contract exercise

Build the fictional Orchid metric tree, then fully specify one outcome, two leading workflow measures, one adoption measure, one system-quality measure, and three guardrails. For each define population, event/state, source, formula, segment, window, baseline, threshold authority, gaming path, missing-data behavior, owner, and decision trigger.

Add a decision-rights map, assumptions with validation, non-goals, rejection criteria, and entrance/exit/stop conditions. Inject a favorable average that hides one low-connectivity cohort and a faster closure rate caused by transferred cases. Repair the contract without changing definitions after seeing results.

Pass when the contract can reject a convincing but useless or harmful deployment, preserves attribution limits, and tells the designated owner what action each evidence state supports.

The Chapter 4 gate

Before Chapter 5 scopes the path, OA-03 must contain:

- bounded population/workflow outcome hypothesis;
- metric tree with definitions and data feasibility;
- baseline plan and explicit missing evidence;
- critical segments;
- adoption funnel and bypass evidence;
- guardrails linked to controls/actions;
- assumptions, constraints, non-goals, and gaming risks;
- recommend/approve/execute/inform decision map;
- discovery, pilot, stabilization, engagement, and stop criteria;
- attribution limitations and change history.

The gate does not require invented certainty. It requires that uncertainty has a name, consequence, validation path, owner, and effect on the decision.

Chapter 5, *Scope the First Safe Production Path*, will use this contract to choose the least exposure that can produce decision-quality evidence across the real user, data, integration, control, environment, and operating boundaries.

5. Scope the First Safe Production Path

Select the least exposed end-to-end path that produces representative production evidence with explicit dependencies, uncertainty, change, and stop criteria.

The smallest feature is not always the smallest risk.

A team can remove identity, audit, recovery, exception handling, or representative integration from a pilot and call the result an MVP. The feature list becomes shorter. The most important uncertainty remains untouched. When those obligations return near launch, the project discovers that the prototype proved the easiest part.

The first safe production path has a different purpose. It is the least exposure that crosses enough of the real workflow, data, integration, control, environment, and operating boundary to produce decision-quality evidence.

Current deployment-oriented employer materials describe diagnosing and prioritizing workflows before build/deployment. Cloud architecture frameworks, though provider-specific, offer a compatible reminder: operational excellence, security, reliability, performance, cost, and related quality concerns involve contextual tradeoffs. A scoped path cannot be called safe because it ignored those concerns. [CLM-028] [CLM-029] [CLM-031]

Replace “MVP” with a decision

The term *minimum viable product* carries too many meanings. It can mean a market test, a prototype, a first sellable product, or an excuse to omit production obligations.

This book uses **first safe production path** with five conditions:

1. **End to end.** It reaches from a representative workflow trigger to an operational result or explicit safe escalation.
2. **Representative.** It crosses the boundaries where material uncertainty lives.
3. **Bounded.** User, case, capability, environment, data, time, and blast radius are limited.
4. **Reversible or containable.** Exposure can stop, isolate, or recover under known authority.
5. **Decision-producing.** Evidence can support continue, change, reduce, delay, or stop.

Benefits evidence should make all those dispositions legitimate rather than assume the project must continue. [CLM-030]

For Orchid, “chat with manuals” is small but not representative. It bypasses equipment matching, current work state, inventory, qualified approval, low connectivity, identity, audit, support, and evidence visibility.

A stronger first path is:

For a named Region West technician cohort and selected equipment family, take an eligible service request from confirmed equipment match through visible evidence retrieval and ranked next-action

suggestion, apply deterministic eligibility rules, require qualified approval for safety-relevant action, expose current inventory as read-only evidence, record the decision/audit state, and provide explicit fallback/escalation when identity, connectivity, data, dependency, or model behavior is insufficient.

This is still a hypothesis. Detailed architecture follows. The scope statement identifies the boundaries that evidence must cross.

Narrow through the scope cone

Use a sequence of filters.

1. Outcome relevance

Does the capability directly support an OA-03 outcome, guardrail, adoption, or risk-reduction hypothesis?

If no, reject or defer it. A feature can be attractive and irrelevant to the current decision.

2. Consequence and guardrail

Which harm or degradation could the path create? What must be prevented, approved, detected, contained, or excluded before exposure?

3. Uncertainty

Which unknown could invalidate the outcome, architecture, control, usability, operation, or ownership model? The first path should cross the most consequential uncertainty that can be tested legitimately.

4. Dependency reality

Which external owner, system, data, approval, environment, or operating capability is required? A dependency without owner/evidence/date is not a plan.

5. Representativeness

Would the path succeed only because it avoids the actual identifier, network, approval, data, user, or support condition? If yes, it is a demo boundary, not a production evidence boundary.

6. Reversibility and blast radius

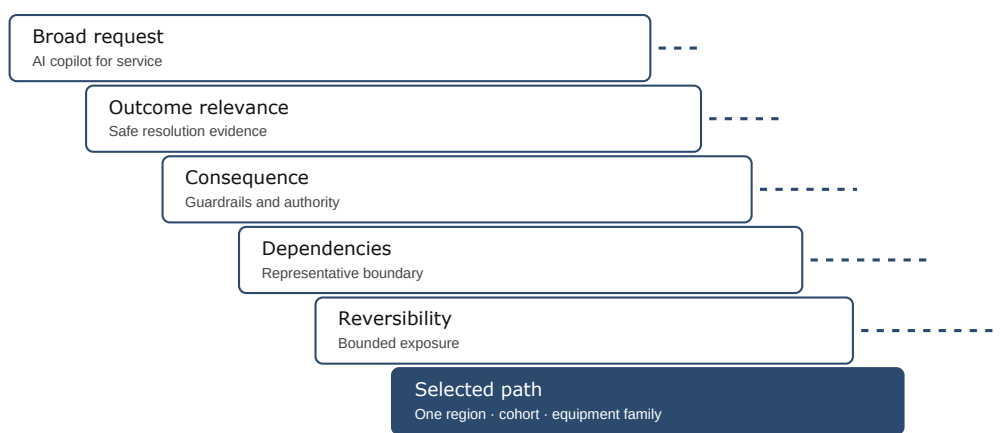
Can the team limit users/cases/actions, preserve old behavior, stop exposure, reconcile state, communicate impact, and recover?

7. Evidence yield

Which decision becomes better after the path runs? If the expected evidence cannot change a decision, the scope may be work rather than learning.

F05.1 · FORWARD DEPLOYED ENGINEERING

Narrow toward representative evidence



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F05.1 - Narrow toward representative evidence. Small means least exposed while still crossing the uncertainty-bearing boundary.

Text alternative: Scope cone narrowing a broad AI copilot request through outcome, consequence, uncertainty, dependency, representativeness, reversibility, and evidence yield to one bounded Orchid path.

The cone is an original synthesis, not a formal standard. [CLM-032]

Classify work into four buckets

Must prove

Uncertainty that must be resolved before the next commitment. Examples:

- Can equipment ambiguity be detected before evidence retrieval?

- Can a qualified approval path work under the selected identities?
- Can low-connectivity users receive evidence or a safe fallback?
- Can ERP inventory reads and timeout behavior be interpreted reliably?
- Can recommendation errors be evaluated by workflow consequence?
- Can support detect and diagnose the selected path?

“Must prove” may produce a test, simulation, customer decision, or measured evidence rather than shipped code.

Must build

Behavior required to obtain evidence safely. Examples:

- bounded request/equipment-match path;
- evidence retrieval with provenance;
- deterministic eligibility and approval gate;
- safe timeout/fallback state;
- audit event and basic signals;
- feature/cohort control;
- exception and reconciliation path.

Defer

Potentially valuable work whose absence does not invalidate the current evidence and whose risk is contained. For Orchid:

- multilingual voice;
- automatic parts ordering;
- every equipment family/region;
- broad personalization;
- general product platform;
- enterprise identifier remediation.

Each deferral needs a consequence and revisit trigger. “Later” without a trigger is hidden abandonment.

Reject

Work that does not support the bounded outcome, creates unjustified risk, duplicates another owner, or cannot be made legitimate. Rejection can change with evidence, but the current decision is explicit.

Scope by uncertainty, not by component

A common plan divides work into UI, API, model, data, and deployment components. Each team can finish while the end-to-end uncertainty remains.

Instead, define vertical milestones:

1. **Evidence skeleton.** One synthetic/approved case moves through request, equipment match, evidence retrieval, deterministic policy, approval, audit, and inspectable result using controlled adapters.
2. **Representative contract.** Validate selected schemas, semantics, identity, timeout, retry, and reconciliation behavior against customer-owned environments.
3. **Failure-bearing slice.** Inject duplicate ID, missing manual, stale inventory, low connectivity, model refusal/error, denied approval, and ambiguous dependency response.
4. **Verification slice.** Connect high-consequence workflow risks to tests/evals/UAT and limitations.
5. **Operating slice.** Signals, alerts, runbook, release/recovery, support access, and ownership rehearsal.
6. **Bounded production cohort.** Expose only when readiness evidence and authority exist.

Each milestone ends in a decision, not a percent-complete report.

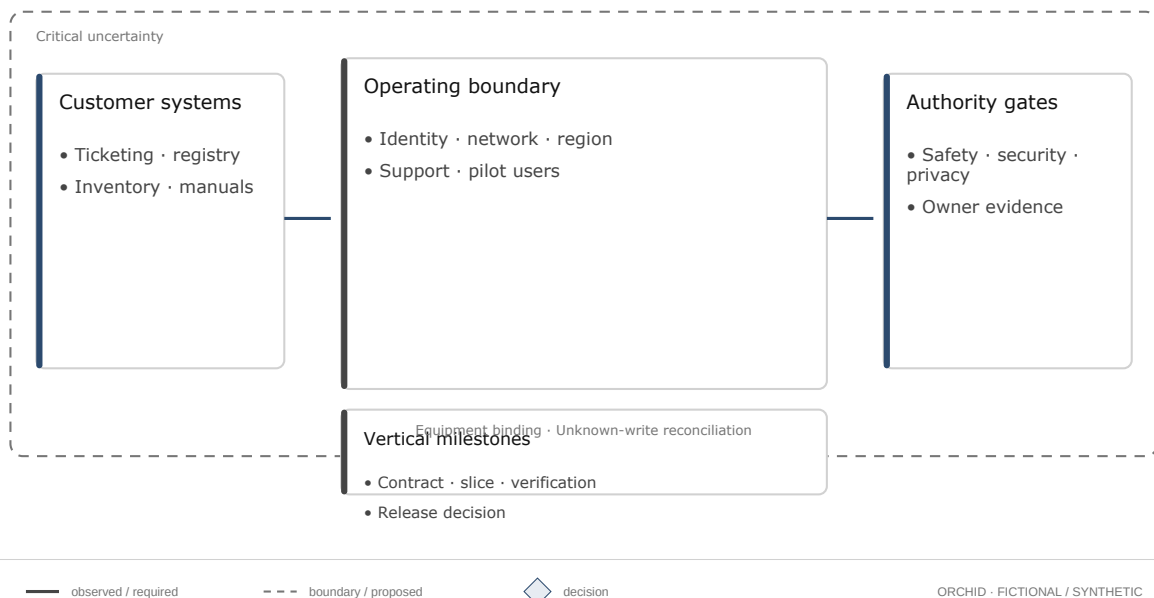
Build the dependency network

A task list hides external ownership and evidence gates. Use a dependency network with these fields:

- dependency ID and capability/evidence required;
- why the selected scope needs it;
- provider and accountable owner;
- consumer and integration owner;
- interface/decision/approval artifact;
- earliest evidence date and latest safe date;
- confidence and basis;
- failure/late consequence;
- workaround/alternative and new risk;
- escalation trigger;
- status/change history.

F05.2 · FORWARD DEPLOYED ENGINEERING

Dependencies are decision gates



F05.2 - Dependencies are decision gates. External owners and evidence gates govern sequencing more honestly than a calendar alone.

Text alternative: Dependency network linking ticketing, equipment registry, manuals, inventory, identity, network, approvals, support, and pilot users to vertical milestones and a critical uncertainty path.

Example:

DEP-ID-01 equipment match semantics

- Needed for: prevent retrieval/recommendation against wrong equipment.
- Owner: customer equipment-registry owner; domain validation by service operations.
- Evidence: field definitions, unique-match rules, mismatch sample, exception owner.
- Latest safe date: before contract/vertical-slice acceptance.
- Failure consequence: scope must block automation at ambiguity and may remain manual.

DEP-AUTH-02 qualified approval identity

- Needed for: safety-relevant recommendation path.
- Owner: safety/domain authority plus identity/platform owner.
- Evidence: approval rule, authorized role, token/identity mapping, audit requirement, fallback.
- Failure consequence: safety-relevant path excluded; non-safety evidence path may continue only if separation is credible.

The network should show where one missing decision blocks several technical tasks. Escalation happens before the deadline, not when the integration branch waits.

Convert uncertainty into an experiment or decision

Not every unknown needs production exposure.

Use an uncertainty record:

- proposition;
- consequence if false;
- current evidence/confidence;
- least risky credible test;
- required data/access/owner;
- expected decision states;
- stop condition;
- evidence produced and limitation.

Reproduce outside production

Use synthetic fixtures, sandbox, simulation, interface contract review, or customer-run query where the behavior can be represented credibly.

Observe in a controlled shadow path

Run without changing user/system decisions. Shadow evidence can test latency, data availability, matching, or recommendation behavior but does not prove usability, adoption, or action consequence.

User-test representative tasks

Use approved realistic tasks and users to test workflow fit, evidence visibility, accessibility, and decision support before live consequence.

Bound a production cohort

Use production only when real operating conditions are necessary, prior evidence supports exposure, guardrails/owners/recovery exist, and blast radius is explicit.

Production exposure should not be used merely to discover whether an unacceptable safety, privacy, or authorization control works. [CLM-033]

Make milestones reversible

A reversible milestone limits commitment while improving evidence.

Record:

- entrance evidence;
- capability/data/user boundary;
- allowed/prohibited actions;
- observation window;
- success/guardrail/stop evidence;
- authority present;
- rollback/isolation/cleanup;
- exit decision.

Reversibility is not binary. A read-only suggestion can still change human behavior. A shadow path can still process sensitive data. A small cohort can still contain high-consequence cases. Evaluate reversibility by consequence and state, not by feature label.

Estimate with ranges and evidence

FDE work contains unknown customer dependencies. A single date hides them.

Separate:

- known implementation work;
- dependency/approval wait;
- discovery/contract uncertainty;
- environment/integration uncertainty;
- verification/operating evidence window;
- contingency triggered by named risks.

Use ranges tied to assumptions and evidence dates. Example:

The local controlled slice is estimated at X-Y engineering days after contract/schema decisions. Customer-environment integration cannot be dated beyond a confidence range until DEP-ID-01 and DEP-AUTH-02 evidence exists. The production observation window is governed by eligible case volume and cannot be compressed through engineering effort.

The book does not invent the values for Orchid. The structure prevents a delivery date from silently including unresolved authority and evidence.

Compare complete scope options

Teams often compare one detailed preferred option with vague alternatives. Make each option complete enough to expose its evidence and consequence.

For Orchid, compare three paths.

Option A: manual evidence workspace

The system confirms equipment identity, gathers/manual-links evidence, and records the technician's supported decision. It uses deterministic ranking or search only; no generated suggestion.

- Evidence gained: identifier/data/integration/workflow fit, evidence visibility, latency, adoption, approval/audit, support.
- Evidence not gained: model error/control/cost behavior.
- Consequence: lower AI uncertainty; may leave more technician synthesis effort.
- Reversibility: high if read-only and cohort-bounded.

Option B: bounded suggestion path

Option A plus a replaceable model interface that summarizes/ranks evidence. Deterministic eligibility and qualified approval constrain consequence.

- Evidence gained: Option A plus task-specific model quality, trust, review burden, latency/cost, fallback.
- Consequence: adds non-deterministic error, evaluation, data/tool, monitoring, and change-control requirements.
- Reversibility: moderate; suggestions can affect decisions even when no automatic side effect occurs.

Option C: automatic parts order

Option B plus an external write that reserves/orders a part.

- Evidence gained: full action outcome and operational integration.
- Consequence: financial/stock state, intent/retry/reconciliation, approval, fraud/abuse, rollback, and customer communication.
- Reversibility: lower, because an external effect can survive application rollback.

The scope decision should not ask which option looks most innovative. It asks which option is the least risky credible way to test the contracted hypothesis.

If workflow evidence shows that evidence fragmentation is the core problem and the value of generated suggestions is uncertain, Option A may be the correct first production path while Option B remains a controlled evaluation. If model assistance is central and controls are ready, Option B may be justified. Option C requires a separate outcome and authority case.

Use an option table:

Field	A	B	C
Outcome hypothesis tested	evidence assembly	evidence plus suggestion	evidence, suggestion, action
Critical uncertainty	data/workflow fit	model behavior/review	side-effect semantics/control
New authority	limited	AI/control approval	financial/stock/action approval
Failure consequence	wrong/missing evidence	misleading suggestion	duplicate/wrong action
Required recovery	fallback/manual path	disable/fallback/review	reconcile/cancel/compensate
Exposure boundary	read-only cohort	suggestion cohort	write-enabled subset

Document why the chosen option dominates for the current decision and what evidence would move the team to another.

Build a decision-oriented risk register

A risk register should alter scope or milestones, not collect generic concerns.

Each risk record includes:

- risk ID and precise condition/consequence;
- affected outcome, guardrail, user, or owner;
- current evidence and uncertainty;
- likelihood language only at the supported precision;
- severity/consequence and reversibility;
- prevention/reduction/detection/containment/recovery actions;
- accountable risk/decision owner;
- trigger or leading evidence;
- milestone/gate affected;
- residual risk and acceptance status;
- review date.

Avoid multiplying subjective numbers into a false score. Categories can support prioritization, but high consequence, missing authority, and irreversible state should remain visible even if occurrence is uncertain.

Example:

R-IDENT-01: ambiguous equipment binding causes evidence/recommendation for the wrong asset.

- Evidence: observed ambiguity; frequency unknown.
- Consequence: wrong manual/history/part, possible safety impact.
- Scope response: confirmation/reconciliation required before suggestion; ambiguous cases block/escalate.
- Gate: must pass contract and high-consequence tests before cohort exposure.
- Owner: registry/data/domain owners for semantics; FDE for implementation/evidence; safety owner for consequence acceptance.

R-CONN-02: low-connectivity users cannot obtain evidence or complete approval within workflow need.

- Evidence: intermittent connectivity stipulated; measured performance missing.
- Consequence: abandonment, delay, unsafe workaround, hidden exclusion.
- Scope response: include representative site simulation/user evidence and safe fallback; segment rollout.
- Gate: latency/fallback evidence before low-connectivity cohort exposure.

R-OWN-03: support cannot diagnose or recover the path without FDE access.

- Evidence: operating model incomplete.
- Consequence: prolonged incident and permanent embedded dependency.
- Scope response: signals/runbook/access/recovery rehearsal are must-build/must-prove, not post-pilot work.

The register connects discovery to architecture and later readiness. It should shrink, change, or block scope when mitigation evidence is absent.

Sequence learning before irreversible commitment

Good sequencing reduces uncertainty in the order that prevents waste or harm.

For Orchid:

1. **Resolve authority blockers first.** Identify safety, regional data, identity, support, and rollout owners. No technical elegance compensates for ownerless consequence.
2. **Resolve semantic blockers before integration volume.** Confirm equipment identity, ticket state, inventory meaning, and approval rules before building broad adapters.
3. **Prove local failure behavior.** Use synthetic/controlled doubles to exercise missing/duplicate/late/timeout/deny states.
4. **Test representative customer contracts.** Validate real schemas, identities, networks, rate/timeout behavior, audit, and operating ownership within approved environments.

5. **Verify high-consequence behavior.** Connect workflow risks to tests/evals/UAT/control evidence before polishing breadth.
6. **Rehearse release/recovery/support.** A path that only the delivery team can operate is not ready for live evidence.
7. **Expose the smallest decision-valid cohort.** Choose users/cases that reveal the target uncertainty without hiding consequential segments.

This order may differ when an uncertainty is cheap and decisive. A two-day customer-run query can reject a data assumption before weeks of implementation. A short supervised user exercise can show that provenance is missing. An architecture review can reveal that a regional boundary invalidates the proposed topology.

Use **evidence gates**, not phase labels:

“Proceed to customer-environment integration when equipment-match semantics, selected fields, identity path, and interface error behavior are owner-reviewed, versioned, and testable.”

That is stronger than “integration starts in sprint three.”

Define scope debt

Deferral creates debt when the omitted behavior will become necessary or the current shortcut increases future cost/risk.

Record:

- deferred item and reason;
- affected user/case/environment;
- current containment;
- consequence if the scope expands;
- evidence or milestone that triggers repayment;
- owner;
- whether current architecture preserves a path to change.

For example, excluding low-connectivity sites is not harmless if the intended population includes them. The debt record should say the pilot claim excludes those sites, which evidence is required, how rollout prevents accidental inclusion, and who owns the expansion decision.

Deferring automatic ordering may be appropriate without architecture for it. But if the current interface contract discards caller intent or reconciliation state, later expansion becomes harder. The architecture decision should preserve relevant boundaries only when justified; it should not build the full future feature.

Scope debt prevents “out of scope” from becoming “forgotten” or from leaking into outcome claims.

Use change budgets for attention, not permission

A bounded delivery horizon benefits from a visible change budget: how much scope, dependency, or uncertainty can enter before an explicit re-plan is required.

The budget need not be a number. Triggers can include:

- new user population or region;
- new sensitive data category;
- new external write or irreversible action;
- new formal authority/control path;
- new model/tool autonomy;
- dependency owner/date change affecting critical path;
- critical segment or guardrail failure;
- outcome definition change that breaks baseline comparability.

Crossing a trigger does not forbid the change. It requires a decision showing what moves, which evidence becomes invalid, and who accepts the consequence.

Control change with the same scope cone

When an executive requests multilingual voice and automatic parts ordering for the pilot, do not answer from preference.

For each change:

1. Which contracted outcome, guardrail, or uncertainty does it serve?
2. Which new users, data, identity, model, interface, action, support, and authority enter?
3. Which milestone/evidence becomes invalid?
4. Which capability leaves the current scope or which date/evidence moves?
5. Can it be tested separately with lower consequence?
6. Who accepts the trade?

Multilingual voice

It may improve access for a target group, but it adds audio capture, transcription behavior, language/dialect evaluation, privacy/retention, latency, noise/environment, accessibility, user correction, and support constraints. If target user evidence shows it is essential for the selected cohort, the scope decision may change. Without that evidence and control design, defer it; do not call it cosmetic.

Automatic parts ordering

It adds financial/stock side effects, intent/idempotency/reconciliation, approval, rollback, fraud/abuse, inventory accuracy, and support consequences. A read-only recommended-part path can produce evidence with lower consequence. Defer automatic action until its separate outcome/control case is justified.

Change control should state the rejected alternative and evidence needed to reconsider it.

Include a stopped deployment

Consider a fictional regulated-document workflow. The team is asked to extract and route sensitive case files, but no legitimate owner can authorize the proposed data use, affected-user access is absent, and the available sample excludes the highest-consequence cases.

The correct output may be a stopped deployment with:

- verified problem and consequence;
- missing authority/access/representativeness;
- alternatives considered;
- evidence that cannot be produced legitimately;
- stop decision and owner;
- conditions required to reopen.

Stopping is not a failure of FDE ownership. Proceeding would create confidence without authority or representative evidence.

Handle evidence that cannot be learned safely in production

Some uncertainty is important precisely because failure is unacceptable. “Try it with a few users” is not a responsible default.

Separate evidence modes.

Specification and authority evidence

A qualified owner defines prohibited and required behavior. The FDE records the rule, scope, version, approval, and testable implications. Example: which equipment/action combinations require qualified approval. Production users do not discover the policy through exposure.

Static and contract evidence

Schemas, configuration, policy-as-code, permission maps, and interface contracts can prove that required structure exists. They cannot prove runtime behavior, but they can reject invalid designs early.

Simulation and fault injection

Synthetic cases exercise unsafe suggestions, denied identities, stale data, ambiguous completion, dependency outage, and low-connectivity states without creating real consequence. Simulation must state which production conditions it cannot reproduce.

Expert/user review

Qualified owners and representative users evaluate realistic cases, evidence visibility, decision fit, and fallback. Review does not substitute for live operating evidence, but it can block a path before exposure.

Shadow evidence

The system observes approved production inputs or receives mirrored events without changing the operational decision. Shadowing can reveal distribution, latency, and behavior gaps. It still has data/security/cost consequences and cannot prove adoption.

Staged production evidence

Only after earlier gates pass, expose a bounded cohort with real command, support, guardrails, recovery, and authority. Production evidence addresses conditions that cannot be credibly represented elsewhere.

Map each uncertainty to the least consequential evidence mode that can answer it. Escalate when no legitimate method can provide adequate evidence. The answer may be to exclude the capability or stop the deployment.

Defend scope at three altitudes

The same scope decision needs different explanations without changing facts.

Technical

Describe interfaces, state, failure, identity, data, controls, tests, environments, and recovery. Example: automatic ERP writes are deferred because completion semantics, intent identity, reconciliation, and compensating action are unresolved; read-only inventory evidence remains included.

Operational

Describe workflow, cohort, owner, support, exception, and effect. Example: technicians can see current part evidence but cannot create a reservation through Orchid Assist; the existing approved process remains the action path.

Executive

Describe outcome evidence, consequence, timing, and decision. Example: the pilot can test whether evidence assembly improves selected service decisions before taking financial/stock action; adding automatic orders would introduce unresolved control/recovery dependencies and move the readiness date.

The three explanations must reconcile. Executive brevity does not permit deleting a material risk. Technical depth does not permit avoiding the outcome trade.

Review scope as evidence changes

Schedule scope review at evidence gates, not arbitrary ceremonies. Ask:

- Did the outcome or critical segment change?
- Which assumption became evidence or was disproved?
- Which dependency date/owner/confidence changed?
- Did a guardrail/control become harder or easier?
- Is the selected path still the least exposure that crosses the critical uncertainty?
- Can a deferred item now be rejected, or must it enter?
- Does the estimate range or observation window change?
- Which artifacts/tests/figures/claims must be updated?

Record “no scope change” with the evidence reviewed when the decision is material. This prevents silent drift while avoiding change-control theater for trivial implementation details.

Orchid OA-04 scope and delivery plan

Selected boundary. One named equipment family; Region West; named pilot technician cohort; eligible request definition pending OA-03; evidence-assisted supported-resolution/escalation path.

Included path. Confirm equipment identity; retrieve visible evidence; show read-only inventory state; generate/rank bounded suggestion through a replaceable interface; deterministic eligibility; qualified approval where required; audit; safe fallback/escalation; basic signals and support/recovery path.

Must prove. Match ambiguity, data provenance/quality, low-connectivity behavior, dependency semantics, evaluation by consequence, identity/approval/audit, user evidence visibility, operability.

Deferred. Voice, multilingual expansion, automatic order, all families/regions, general platform, enterprise data cleanup.

Rejected for current path. Autonomous safety-relevant action; broad production export; ownerless risk exception.

Dependencies. Registry, ticketing, manuals, ERP read semantics, identity, regional topology/data decision, qualified approval, security/privacy review, support, representative users.

Milestones. Evidence skeleton; representative contracts; failure-bearing slice; verification; operations/recovery; readiness; bounded cohort.

Stop conditions. Missing formal owner, unacceptable guardrail/control gap, inability to detect equipment ambiguity, no safe fallback, no support/recovery owner, or evidence that invalidates the outcome hypothesis.

All dates, baselines, thresholds, owners, and measured frequencies remain explicit fictional-case inputs rather than invented facts.

Scoping failure modes

Feature-list slice

The path contains visible capabilities but avoids real dependencies and failures.

Repair: define the end-to-end uncertainty-bearing boundary.

Hidden dependency optimism

External approvals/systems appear as task dates with no owner/evidence.

Repair: dependency network plus latest-safe escalation.

Prototype in production

A “small pilot” processes real users/data without operating and risk obligations.

Repair: scale requirements to consequence; bound exposure but keep necessary controls/ownership.

Everything is must-have

No work can be deferred because scope is not tied to decisions.

Repair: must-prove/must-build/defer/reject with outcome/evidence trace.

Deadline as acceptance

The seasonal peak converts unresolved evidence into assumed pass.

Repair: precommit reduce/delay/stop options and authority.

False precision

One date combines engineering, approvals, dependencies, and observation volume.

Repair: ranges/assumptions/evidence dates; separate calendar wait from work.

Conduct the safe-scope exercise

Compare three complete options for Orchid: a manual evidence workspace, a bounded suggestion path, and automatic parts ordering. For each state the outcome evidence, representative constraints, dependencies, uncertainty experiments, blast radius, reversibility, milestones, ranges, debt, stop/exit, and authority.

Build the scope cone and classify work as must prove, must build, defer, or reject. Inject a late voice request, unavailable regional approval, and an ERP finality uncertainty that cannot be tested safely through uncontrolled production writes. Replan through explicit evidence/change decisions.

Pass when the selected slice crosses the consequential workflow end to end, learns the highest-value uncertainties before irreversible commitment, rejects unsafe automation, and can be defended at technical, operational, and executive altitudes.

The Chapter 5 gate

Before detailed architecture, OA-04 must show:

- selected user/case/environment/capability boundary;
- vertical-slice start/end and representative boundaries;
- must-prove/must-build/defer/reject decisions;
- dependency network with owners/evidence/dates/escalation;

- uncertainty records and least-risk tests;
- reversible milestones with entrance/exit/stop;
- scope-change rule;
- risks, assumptions, ranges, and explicit non-goals;
- trace from every included capability to outcome or risk evidence.

The path is small only if it limits exposure while retaining the boundaries needed to learn safely.

Chapter 6, *Draw the Real System Boundary*, turns this path into explicit structural, runtime, data, trust, failure, and responsibility boundaries. It will make visible what the scope expects each customer and product system to own.

Part II - Design the Deployment

Part II converts the selected workflow into explicit boundaries that reviewers can challenge. Boxes alone are not architecture. The deployment must state which system holds which responsibility, which proposition crosses each interface, how identities and regions are enforced, where uncertain model behavior belongs, and which control evidence supports formal decisions.

Chapter 6 draws structural, runtime, data, trust, failure, and responsibility boundaries. Chapter 7 makes interface semantics, time, intent, finality, error, quality, lineage, and reconciliation explicit. Chapter 8 instantiates the logical design in customer topology, identity, network, configuration, secret, tenancy, regional, capacity, connectivity, and recovery constraints. Chapter 9 permits AI only as a bounded mechanism with task-relevant evaluation, deterministic controls, qualified review, fallback, budgets, and stop behavior. Chapter 10 connects threats and risks to control objectives, implementation, evidence, authority, and expiry.

These chapters complete OA-05. Their purpose is not to select fashionable infrastructure or reproduce specialist standards. It is to make the design falsifiable before implementation hides ambiguity behind working code.

6. Draw the Real System Boundary

Express structural, runtime, data, trust, failure, and responsibility boundaries with explicit ownership and architecture decisions.

An architecture diagram can be accurate and still conceal the deployment.

It may show a user, an application, several services, and databases connected by arrows. The boxes can match the code. Yet the diagram may not say which team operates each service, which system owns a data claim, why one request is trusted, what happens when the arrow partially fails, who detects the failure, which state must be reconciled, or who may authorize recovery.

Forward deployed architecture begins where boxes and arrows stop.

The C4 model supplies useful hierarchical context and container abstractions and does not require a single notation or tool. This chapter uses those structural views and adds responsibility, trust, data, runtime, and failure annotations for deployment decisions. Those annotations are an original extension, not part of the formal C4 method. [CLM-034] [CLM-040]

Draw the boundary the outcome needs

Chapter 5 selected one Orchid path: a Region West cohort and equipment family moving from confirmed equipment match through visible evidence, bounded suggestion, deterministic eligibility, qualified approval, read-only inventory evidence, audit, fallback, signals, and support.

The architecture boundary must answer:

- Which actors and systems participate from trigger to supported resolution/escalation?
- Which runtime components belong to Orchid Assist?
- Which customer systems remain authoritative for which claims?
- Where do data and control cross organizational, regional, tenant, network, or trust boundaries?
- Which identity acts at each crossing?
- Which failure can propagate, and where can it be detected/contained?
- Which team operates, changes, supports, and approves each responsibility?
- Which assumption, dependency, and scope decision justifies the boundary?

Do not begin by selecting cloud services. Begin with responsibilities and constraints.

Build the context view

The context view should contain the smallest set of external actors and systems needed to explain the deployment.

For Orchid:

Actors

- technician using Orchid Assist;
- dispatcher/service coordinator;
- qualified approver/domain specialist;
- support/operator;
- customer engineer/release owner;
- security/privacy/regional decision owners where they interact with controls.

External customer systems

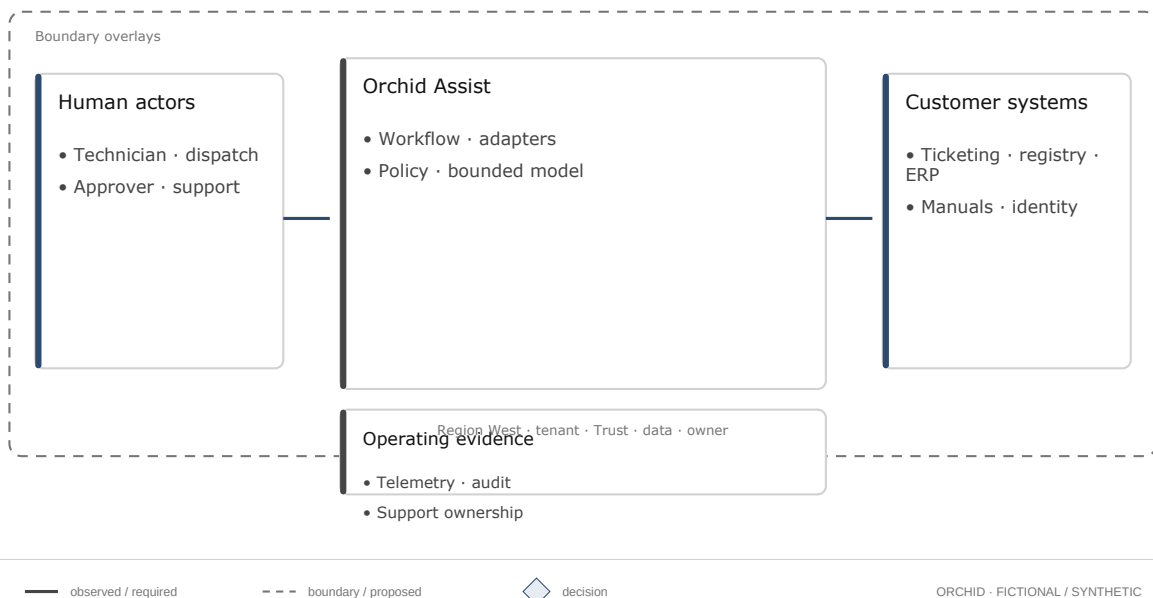
- ticketing/service-request system;
- equipment registry;
- inventory/ERP;
- curated manual/document source;
- identity provider and authorization sources;
- telemetry/operations platform;
- audit/record destination where distinct.

Orchid Assist

A bounded system that coordinates the selected workflow. It does not become the owner of every customer record or decision merely because it reads or displays them.

F06.1 · FORWARD DEPLOYED ENGINEERING

The real Orchid system boundary



F06.1 - The real Orchid system boundary. Actors, services, data, trust, region, operations, and external ownership appear in one context.

Text alternative: System context showing technicians, dispatch, approvers, support, Orchid Assist, ticketing, equipment registry, inventory, manuals, identity, telemetry, and audit within ownership and Region West boundaries.

Write a relationship sentence for every arrow:

Orchid Assist reads selected equipment identity and status fields from the customer equipment registry using the approved workload identity to bind an eligible request; the registry remains authoritative for equipment record state; ambiguous matches enter a customer-owned reconciliation path and block suggestion.

“Uses equipment registry” is not enough.

Build the container view around responsibilities

Candidate Orchid Assist containers include:

- web/user experience;
- gateway/API and workflow coordinator;
- customer integration adapters;
- evidence retrieval/index component;
- deterministic eligibility/policy component;

- optional model interface with deterministic test double;
- qualified approval workflow;
- audit/event component;
- operational signal component;
- configuration/feature/cohort control;
- local or bounded workflow state store.

Container boundaries should reflect independent runtime, data, trust, change, scaling, failure, or ownership concerns. Do not create a service for every conceptual noun. A modular component inside one runtime may be easier to understand and operate for the bounded path.

For each container, record:

- purpose and explicit non-responsibility;
- runtime/environment/region/tenant;
- owner for code, operation, data, and decisions;
- identities/permissions;
- input/output contracts;
- state held and authority of that state;
- dependencies and failure behavior;
- signals/runbook/recovery;
- change/release path;
- build/buy/reuse decision.

Distinguish six boundaries

Structural boundary

Which component/system exists and which relationship connects it.

Runtime boundary

Where code executes, scales, restarts, persists state, and receives configuration. Two modules in one repository can have different runtimes; two services can share one failure domain.

Data boundary

Which data category crosses, where it is stored/cached/derived, who is authoritative, which region/tenant applies, and how retention/deletion/reconciliation works.

Trust boundary

Where identity and authorization must be evaluated. Network location alone is not sufficient trust. Protect resources with explicit authentication/authorization appropriate to the request. [CLM-036]

Failure boundary

Which failure can propagate together, which state can diverge, where detection/containment occurs, and what recovery changes.

Responsibility boundary

Who builds, operates, supports, changes, supplies evidence, and accepts consequence. “Shared” must be decomposed into actions.

A structural diagram does not establish the other five. [CLM-039]

Annotate trust per request

Avoid labels such as “trusted network” or “internal service” as conclusions.

For each material request:

- actor/workload identity;
- credential/token mechanism and lifecycle;
- resource and operation;
- tenant/region/context;
- authorization policy and decision point;
- data/action scope;
- audit evidence;
- denial/failure behavior;
- owner and review/revocation path.

Example:

A Region West technician identity requests evidence for an assigned eligible ticket. The gateway validates the user session and assignment/region context. The workflow service uses a bounded

workload identity to read selected registry/manual/inventory fields. Safety-relevant approval requires a separately authorized qualified identity. No user identity is converted into unrestricted service access.

The exact identity mechanisms wait for Chapter 8. The architecture must reserve the decisions now.

Annotate data by proposition

Do not call an entire system “the source of truth.” Identify the proposition.

- Ticketing owns current request assignment/status under the agreed workflow definition.
- Equipment registry owns selected equipment record attributes, subject to data-quality exceptions.
- ERP owns reported inventory state and reservation/action state.
- Manual repository owns document versions/approval status.
- Orchid workflow state owns its processing, evidence references, recommendation, approval, fallback, and reconciliation status.
- Audit destination owns the retained decision/event record according to the operating policy.

The application can cache or derive data without becoming authoritative for the underlying proposition. Record freshness, provenance, invalidation, and disagreement behavior.

Use a diagram grammar reviewers can challenge

Architecture notation should reduce ambiguity rather than display tool skill.

Use a stable visual grammar:

- actor: rounded rectangle;
- system/container: squared rectangle;
- data store/artifact: cylinder or document;
- ownership boundary: labeled enclosure;
- trust/data/runtime boundary: distinct labeled line style;
- observed/approved relationship: solid;
- assumed/proposed relationship: dashed;
- external dependency: owner label plus link to dependency ID;
- failure path: warning marker and link to matrix row;
- control/approval: gate/shield marker;
- normal data/control flow: directional arrow with verb and proposition.

Color may reinforce the meaning but cannot carry it alone. Every boundary and arrow needs a text label. The web and print figure must remain legible in grayscale.

Relationship labels should answer “who does what to whom, why, with which identity/data, and under whose authority?” Examples:

- Technician **submits supported decision and evidence references** to Orchid Assist for the assigned ticket under their user identity.
- Orchid workflow **reads selected equipment propositions** from Registry under bounded workload identity; Registry owner supplies quality/availability evidence.
- Qualified approver **accepts or rejects safety-relevant action** through Approval using a role-authorized identity; Orchid records the result but does not define the rule.
- Orchid support **inspects request correlation and dependency states** through the operational plane; access excludes unrestricted content by default.

The diagram should contain stable component/boundary IDs so prose, interface catalog, risks, tests, signals, and runbooks can reference the same object.

Decompose responsibility beyond RACI shorthand

A RACI chart can be helpful, but “accountable” and “responsible” are often interpreted differently. For architecture, record concrete actions.

For each component/dependency:

- **code owner:** reviews and changes implementation;
- **runtime operator:** deploys, scales, patches, monitors, and restores runtime;
- **data/proposition owner:** defines meaning/quality/change for the relevant claim;
- **interface owner:** versions and communicates contract/error behavior;
- **control owner:** defines control objective and approves exception where authorized;
- **support owner:** receives user/system failure and coordinates response;
- **decision owner:** accepts material consequence or go/stop decision;
- **FDE role:** discover, design, implement, integrate, prepare evidence, recommend, stabilize, transfer, or escalate as named;
- **consumer:** depends on the capability and must handle its states.

One team may fill several actions. The decomposition prevents a shared service from being “everyone’s responsibility” and no one’s recovery task.

Example responsibility record:

CNT-ADAPTER-ERP

- Code owner: Orchid Assist delivery team during build; target customer/product owner at handoff.

- Runtime operator: Orchid platform operations.
- Interface owner: customer ERP team.
- Data/proposition owner: inventory operations for availability/reservation meaning.
- Support: first line Orchid support; ERP incidents escalated to ERP on-call.
- Decision owner: service/inventory owner for action semantics; security owner for access exception.
- FDE: build adapter, verify contract/failure/reconciliation evidence, prepare ownership transfer; no authority to redefine ERP or inventory policy.

If the target handoff owner is unknown, mark the architecture incomplete. Do not write “TBD” and then design a permanent component around FDE operation.

Run boundary walkthroughs

Validate architecture with scenarios, not only static inspection.

Request walkthrough

Take one eligible case. Trace user identity, request state, data propositions, dependencies, policy, suggestion, approval, audit, and user result. At each crossing ask who owns semantics and failure.

Denied request walkthrough

Use wrong region, tenant, assignment, or role. Confirm where denial happens, what the user sees, what is logged, which data did not cross, and who diagnoses a legitimate denial error.

Dependency outage walkthrough

Remove registry, manual, inventory, model, identity, audit, telemetry, or configuration plane one at a time. Ask whether the path blocks, falls back, degrades, queues, or continues; whether that behavior is allowed; and how recovery/reconciliation works.

Split-state walkthrough

Assume desired configuration is accepted on some runtimes but not others, or a migration partially completes. Identify independent health evidence, blast radius, safe operation, rollback/forward, and owner.

Ownership-loss walkthrough

The FDE leaves. Who can deploy, access logs, interpret alerts, rotate credentials, reconcile ERP state, update policy, and approve a risky change? Every “the team” answer needs a named operating role and evidence.

Regional-boundary walkthrough

A Region West request attempts to retrieve/copy evidence from another region. Show policy decision, data path, denial/fallback, audit, and formal exception owner. Do not assume network routing enforces every data rule.

Record walkthrough findings as architecture decisions, risks, or gaps. A review that produces no inspectable artifact is weak evidence.

Model correlated failure

Components drawn separately can share a failure domain:

- same runtime cluster or region;
- same identity provider or DNS/network path;
- same configuration/control plane;
- same database, queue, storage, or secret;
- same deployment artifact or change mechanism;
- same operator/on-call team;
- same upstream data-quality process;
- same capacity or rate limit.

Annotate these shared dependencies. Otherwise a diagram suggests redundancy that does not exist.

For Orchid, a manual fallback hosted behind the same unavailable identity and network path is not a fallback. A rollback tool that depends on the failed configuration store is not a recovery boundary. A second model endpoint using the same retrieval/index state may not protect evidence availability.

Ask for each critical promise:

Which independent capability must remain available to detect, decide, communicate, contain, and recover when the primary path fails?

Independence is contextual and costly. The architecture should justify where it is necessary rather than duplicate everything.

Trace architecture to outcomes and tests

Every material container or boundary should exist because of an outcome, guardrail, scope, or operating decision.

Architecture item	Upstream reason	Downstream evidence
equipment confirmation/reconciliation	wrong-asset consequence; scope must prove ambiguity	contract/exception tests, UAT, audit, match signal
visible provenance/retrieval	trust and evidence-complete outcome	retrieval/version tests, UAT, quality telemetry
deterministic eligibility	safety guardrail	rule tests, prohibited-case suite, approval evidence
qualified approval	formal authority boundary	authorization/audit/UAT/denial tests
cohort/feature control	bounded/reversible scope	rollout/config health and stop evidence
workflow state/reconciliation	partial failure/ambiguous completion	idempotency/recovery/runbook evidence
operational signals	support/ownership exit	SLO/alerts/diagnostic rehearsal

If a component has no upstream reason, challenge it. If an outcome/guardrail has no architecture item or accepted external owner, the design has a gap.

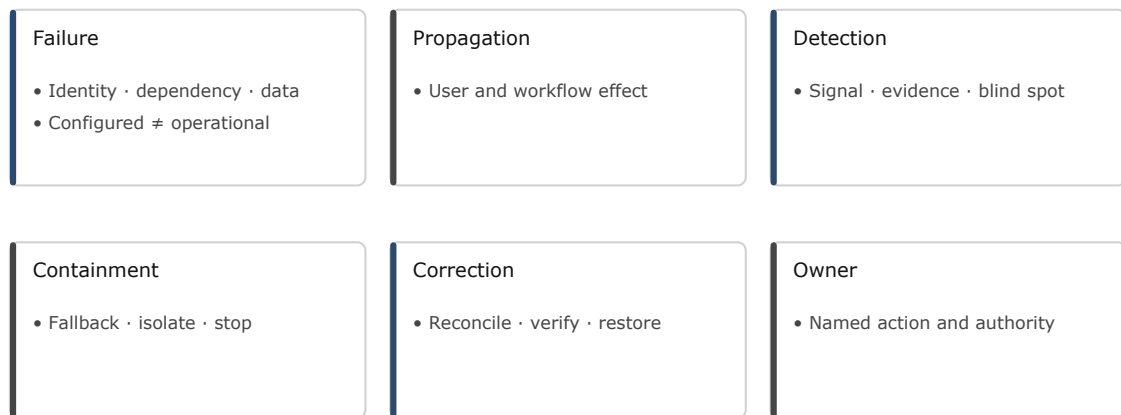
Map failure propagation and ownership

For every material dependency or container, complete a failure/ownership matrix:

Failure	User/workflow effect	Detect	Contain/fallback	Correct/reconcile	Owner
equipment registry unavailable	cannot confirm match	dependency signal + request state	block suggestion; manual approved path	restore/retry; no invented match	registry ops + Orchid support
multiple equipment matches	wrong evidence risk	validation rule	require confirmation/escalation	customer reconciliation	data/domain owner
manual retrieval stale/missing	incomplete evidence	version/provenance rule	show limitation; block affected action	source update/reindex	manual owner + retrieval owner
inventory timeout	current state uncertain	timeout + correlation	no automatic retry/action; show uncertain	status/reconcile	ERP owner + adapter owner
model unavailable/refuses	no suggestion	model-interface state	deterministic/manual evidence path	retry/change under policy	Orchid runtime owner
approval service/identity unavailable	action unauthorized	authorization failure	block/escalate	restore authorized path	identity/approval owners
telemetry unavailable	blind operation	self/independent health evidence	hold/ramp stop based on risk	restore evidence path	ops owner

F06.2 · FORWARD DEPLOYED ENGINEERING

Failure must end with ownership



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F06.2 - Failure must end with ownership. Propagation, detection, containment, correction, reconciliation, and owner remain connected.

Text alternative: Failure and ownership matrix connecting Orchid failures to propagation, detection, containment or fallback, correction or reconciliation, and accountable owner, including configured versus operational state.

The matrix exposes ownerless arrows. If the deployment has a fallback but no one can invoke it, the fallback is not operational.

Configured state is not operational state

A configuration API can accept a desired value while distributed runtime state differs. Health must not be inferred solely from successful configuration.

A first-party Cloudflare postmortem reports a 2026 outage in which configured and operational state diverged and recovery was complicated by different distributed states. The case is architecture-specific and self-reported, but the transferable lesson is useful: validate the operational effect, model divergent state, and design recovery for more than reapplying the previous configuration. [CLM-037]

For Orchid, ask:

- Did the feature/cohort policy reach every intended runtime?
- Is the identity/authorization rule active where requests execute?
- Did the manual/index version become available to all selected paths?
- Did a migration complete in every store/region?

- Can rollback operate when states differ?
- Which health evidence is independent of the control action?

The architecture should distinguish desired, accepted, applied, observed, and reconciled state.

Separate control plane and data plane carefully

Control/data separation can limit blast radius, but the control plane remains part of operation.

A first-party Cloudflare postmortem reports that control-plane and data-plane separation limited one class of customer traffic impact during a 2025 dashboard/API outage, while the unavailable API/dashboard still had operational consequence. [CLM-038]

For Orchid:

- data/workflow plane performs selected request/evidence/decision processing;
- control plane manages configuration, cohort/feature exposure, version, and perhaps deployment administration;
- operating plane supplies telemetry, alert, support, and recovery access.

Ask what happens if each plane fails alone and whether recovery of one depends on the failed plane. A feature-control service that cannot disable exposure during its own outage is a material boundary decision.

Make build, buy, and reuse a decision record

For each capability, compare:

- outcome/risk requirement;
- customer constraints;
- existing product/customer capability;
- alternatives including no new component;
- integration/identity/data fit;
- failure and recovery;
- security/privacy/governance evidence;
- operability/support/skills;
- cost and lock-in;
- migration/exit;
- reuse evidence versus customer specificity;
- owner and revisit trigger.

Example: evidence retrieval can use an existing product component if it supports approved regional storage, document provenance/version, access filters, failure signals, local testability, and an operating owner. Otherwise “reuse” may shift hidden work into customer-specific glue.

Do not build a custom component to demonstrate engineering depth. Do not buy a service because its diagram is polished. Select the boundary that makes the contracted path supportable.

Balance quality attributes explicitly

Provider architecture frameworks enumerate concerns such as operational excellence, security, reliability, performance, and cost. Use them as questions, not certification. [CLM-035]

An architecture decision record should state the trade:

Cache approved manual evidence inside the Region West boundary to meet low-connectivity latency. Accept additional freshness, storage, access, invalidation, and operating responsibilities. Do not cache unrestricted ticket notes. Validate version/freshness on request, expose provenance, and fall back safely when required evidence is unavailable. Regional data/security/operations owners approve the boundary and residual risk.

The decision does not claim maximum reliability or security. It names the consequence and owner.

Write architecture decisions so they can expire

An architecture decision record is not a defense of the selected option. It is a versioned explanation of why the option is currently justified and when it must be revisited.

Use these fields:

- decision ID, title, status, date, and owners;
- problem/outcome/guardrail/risk served;
- current evidence and constraints;
- alternatives, including no new component and reduced scope;
- selected option and rationale;
- quality/security/privacy/operating tradeoffs;
- affected boundaries, interfaces, data, identities, tests, signals, and owners;
- assumptions and evidence gaps;
- failure/recovery behavior;
- consequences and debt;
- validation evidence;
- revisit/expiry/rollback trigger;

- superseding record.

Example:

ADR-0A-006: keep inventory integration read-only in first path

Context. The outcome requires current inventory evidence. The customer requests automatic ordering, but write-completion semantics, intent identity, reconciliation, approval, compensation, fraud/abuse controls, and ownership are unresolved.

Decision. Read selected availability/status evidence. Do not create reservation/order through Orchid Assist. Preserve a separate adapter contract boundary so a later write path requires a new outcome/control decision.

Consequences. Technician continues the approved ordering process outside Orchid Assist; duplicate entry/burden must be measured. The first path cannot claim automated resolution. Data freshness/provenance and read availability remain material.

Revisit. Only after DEP-ERP-WRITE semantics/authority/recovery evidence and a separate guardrail/outcome contract pass review.

This record prevents “temporary” scope from turning into accidental architecture or an implied future commitment.

Transfer incident lessons without copying architecture

Public postmortems are useful when they change a question, not when their architecture is copied.

From the Cloudflare configured-versus-operational-state case, transfer questions such as:

- How does Orchid know a cohort/configuration change took effect on every relevant runtime?
- Can a health check validate the actual user path rather than the configuration API response?
- Can recovery handle mixed versions/state?
- Are snapshots/rollback tied to observed health and independently reachable?

Do not infer that Orchid has the same distributed design, cause, or remediation. The postmortem is first-party evidence about Cloudflare’s system. [CLM-037]

From the dashboard/API outage, transfer questions such as:

- Can a client loop/retry amplify demand?
- Are retries distinguishable from new requests?
- Does control-plane impairment block configuration, support, or recovery while workflow processing continues?
- Are capacity and rollout assumptions tested for the control/API path?

Again, do not claim the same failure. Use the case to challenge boundaries. [CLM-038]

This discipline avoids cargo-cult architecture: transplanting another organization's services without its scale, constraints, or evidence.

Review the design with accountable owners

Architecture review needs several lenses:

- workflow/domain: do decisions, evidence, approval, and exceptions match actual work?
- data/interface: are propositions, semantics, lineage, quality, and change owners explicit?
- security/privacy/governance: are access, data handling, control, evidence, and formal decisions routed correctly?
- reliability/operations: can failures be detected, contained, diagnosed, recovered, and supported?
- customer environment: are identity, networking, region, capacity, configuration, and release assumptions real?
- product/engineering: are components maintainable, testable, reviewable, and appropriately reused?
- user/accessibility: can representative users perform the path and understand evidence/fallback?

Record reviewer scope and disposition: approved within boundary, approved with owned conditions, revise, or blocked. Silence is not approval.

If reviewers disagree, locate the proposition: Is the dispute about a fact, constraint, risk appetite, authority, architecture trade, or missing evidence? Assign the next decision/test rather than averaging opinions.

The review output should update OA-05, ADRs, risks, dependencies, and downstream tests. A slide presentation with no decision record is not the gate.

Avoid architecture theater

Box ownership by assumption

The diagram labels "Customer IT" without a confirmed team/access/support path.

Repair: name responsibility actions and owner acceptance; unresolved ownership is a risk.

VPC equals trusted

Network placement substitutes for per-request identity and authorization.

Repair: annotate actor/workload/resource/operation/policy/audit/denial.

Arrow without semantics

“Calls ERP” hides timeout, intent, state, retry, and reconciliation.

Repair: link to an interface decision/gap; Chapter 7 completes the contract.

One source of truth

A system is labeled authoritative for every workflow claim.

Repair: assign authority by proposition and model disagreement.

Framework compliance

Completing provider review questions becomes proof of architecture quality.

Repair: map selected concerns to implementation/evidence/gaps; no certification claim.

Microservice reflex

Conceptual components become independently deployed services without operating justification.

Repair: choose runtime boundaries from change, scale, trust, data, failure, and ownership needs.

Product-logo architecture

Vendor names replace responsibilities and portability.

Repair: provider-neutral capability/contract first; label current implementations as replaceable.

Orchid OA-05 structural design

The first portion of OA-05 Deployment Design Dossier now contains:

Context. Technicians, dispatch, qualified approvers, support, customer engineers; ticketing, registry, ERP, manuals, identity, telemetry/audit; Orchid Assist bounded coordinator.

Containers. Web UX, gateway/workflow, adapters, retrieval, deterministic policy, optional model interface/test double, approval, audit, signals, feature/config, state.

Boundaries. Region West/tenant/data, human/workload identity, customer/product ownership, runtimes, external dependencies, failure/containment.

Proposition authority. Request, equipment, inventory, manual, workflow/approval, audit assigned separately.

Failure matrix. Registry, match, manual, inventory, model, approval/identity, telemetry paths with detect/contain/correct/owner.

Build/buy/reuse decisions. Required per capability; unresolved decisions remain explicit.

Deferred detail. Chapter 7 supplies interface/data contracts; Chapter 8 deployment/identity topology; Chapter 9 AI boundary; Chapter 10 control evidence.

No diagram implies that the FDE owns customer systems or accepts customer risk.

The Chapter 6 gate

Before designing interfaces, the architecture must show:

- context/container structure and relationship sentences;
- runtime, data, trust, failure, and responsibility annotations;
- proposition-specific authority;
- identity/resource/operation decision points;
- failure propagation, containment, correction, and owners;
- configured versus operational state;
- control/data/operating plane dependencies;
- build/buy/reuse records or explicit gaps;
- quality tradeoffs, assumptions, and revisit triggers;
- trace to OA-03 outcome/guardrails and OA-04 scope/dependencies.

The gate passes when a reviewer can identify what fails, who knows, what the user sees, what is safe, who acts, and which state must be reconciled. A polished diagram alone cannot pass.

Chapter 7, *Make Interfaces and Data Explicit*, will turn every material arrow and data claim into versioned semantics for schema, identity, ordering, timeout, retry, idempotency, quality, provenance, exceptions, and reconciliation.

7. Make Interfaces and Data Explicit

Design versioned API, event, and data contracts with explicit semantics, quality, idempotent intent, exceptions, and reconciliation.

An interface is not explicit because its JSON parses.

A schema can confirm that `equipmentId` is a string. It cannot tell you whether the value is a trusted unique key, a search hint, a historical identifier, or a value that can match multiple active records. An HTTP response can be 200 OK while a downstream operation later fails. A message can be delivered once while the business effect occurs twice. A version number can change without anyone knowing which consumer assumption broke.

Forward deployed systems fail at these semantic seams.

OpenAPI 3.2.0 and AsyncAPI 3.0.0 provide versioned machine-readable descriptions for HTTP and message-driven interfaces. JSON Schema Draft 2020-12 provides a vocabulary for structural validation. These are valuable. They do not by themselves establish business meaning, ownership, ordering, consistency, timeout, retry, observability, support, or reconciliation. [CLM-041] [CLM-042] [CLM-047]

Turn every material arrow into a contract

Chapter 6 left Orchid with named relationships. Chapter 7 creates an interface catalog, semantic data contract, and reconciliation structure. These are original synthesis tools for this book, not formal standards. [CLM-049]

Each record contains:

- interface ID, purpose, owner, consumers, and support path;
- protocol/transport and specification version;
- operation/message and business intent;
- actor/workload identity and authorization context;
- request/event schema and semantic field definitions;
- response/result and completion meaning;
- consistency, ordering, freshness, and authoritative proposition;
- timeout, retry, rate/capacity, and backpressure behavior;
- idempotency/intent/conflict/retention behavior;
- error taxonomy and user/workflow state;
- observability/correlation/audit;
- compatibility/version/deprecation/migration;
- exception and reconciliation owner;

- test/verification evidence and known gaps.

“GET inventory” is not a contract. A useful relationship might say:

Orchid reads the latest reported availability proposition for selected part/site pairs under the Region West workload identity. The response includes source version and observed-at time. It is evidence, not a reservation guarantee. A timeout or stale result moves the workflow to `inventory-uncertain`; no automatic order occurs. ERP owns meaning and availability; the adapter owns translation, validation, correlation, and safe state.

Write semantic field definitions

For each material field, record:

- name and type/format;
- business meaning and non-meaning;
- source/owner and derivation;
- units, timezone, precision, locale, encoding;
- required/optional/nullable/unknown distinctions;
- valid domain and invariants;
- freshness/observed/effective time;
- sensitivity/residency/retention;
- missing/invalid/conflicting behavior;
- version/change compatibility;
- examples using synthetic data.

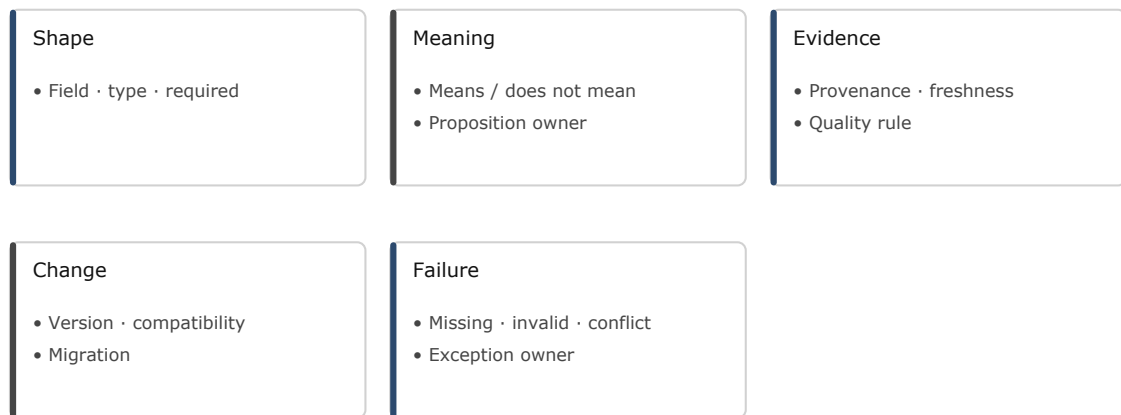
Example:

`equipmentId`

- Type: string matching the current synthetic format in the companion.
- Meaning: identifier of an equipment candidate as supplied by the equipment registry.
- Non-meaning: not proof that the candidate uniquely matches the ticket.
- Owner: registry team for identifier proposition; service/domain owner for match rule.
- Required for: confirmed equipment state, not raw request ingestion.
- Invalid/multiple: workflow blocks suggestion and enters reconciliation.
- Provenance: registry version and observation timestamp retained.

F07.2 · FORWARD DEPLOYED ENGINEERING

A contract is more than a schema



— observed / required

- - - boundary / proposed

◊ decision

ORCHID · FICTIONAL / SYNTHETIC

F07.2 - A contract is more than a schema. Meaning, provenance, quality, version, sensitivity, and exception behavior complete the field definition.

Text alternative: Data-contract anatomy for Orchid equipment evidence, contrasting a valid but ambiguous schema with semantic meaning, proposition owner, provenance, quality rule, version, sensitivity, and invalid or conflicting behavior.

The companion implements this distinction. A synthetic record can satisfy structural fields and still fail the Region West residency rule. The validator is deliberately small and provider-neutral; it demonstrates that semantic checks sit alongside structure.

Model time explicitly

Distributed customer data frequently has several times:

- event occurred;
- source observed or effective time;
- source recorded time;
- message emitted time;
- consumer received/processed time;
- cache/index updated time.

“Current inventory” is meaningless without the relevant time and decision. A part count observed hours ago may be adequate for planning and inadequate for reservation. A manual version approved last week may be current even if indexed later.

Contracts should state which time governs ordering/freshness, timezone, clock uncertainty, late-arrival behavior, and whether backdated correction is possible.

Separate missing, null, unknown, not applicable, and redacted

These states are often collapsed:

- missing: field absent;
- null: explicit empty value under schema meaning;
- unknown: value exists conceptually but is not known;
- not applicable: proposition does not apply;
- redacted/withheld: value intentionally unavailable under policy;
- invalid: supplied value violates structure/semantics;
- stale: value exceeded decision-specific freshness;
- conflicting: sources/candidates disagree.

Each can require a different workflow action. Coercing all to an empty string creates silent business behavior.

Make idempotency about caller intent

HTTP specifies method semantics, including idempotent methods. Application-level retry safety still depends on business intent and implementation. [CLM-043]

Suppose a client asks to reserve part P-7 for ticket T-9 and receives no response. A retry should represent the same intent only when the caller says it does. The service needs an intent/request identifier and a semantic fingerprint.

The contract defines:

- who creates the intent ID and its scope;
- which fields define semantic equivalence;
- behavior for same ID/same fingerprint;
- behavior for same ID/different fingerprint;
- pending/unknown/completed/rejected states;
- retention/expiry and late arrival;
- replayed result and audit;
- concurrency and race behavior;
- reconciliation after ambiguous completion;

- compensation/cancellation where relevant.

First-party AWS engineering guidance describes client request identifiers and the need to distinguish repeated requests from new intent. The pattern is useful, but no quantitative outcome is claimed here. [CLM-044]

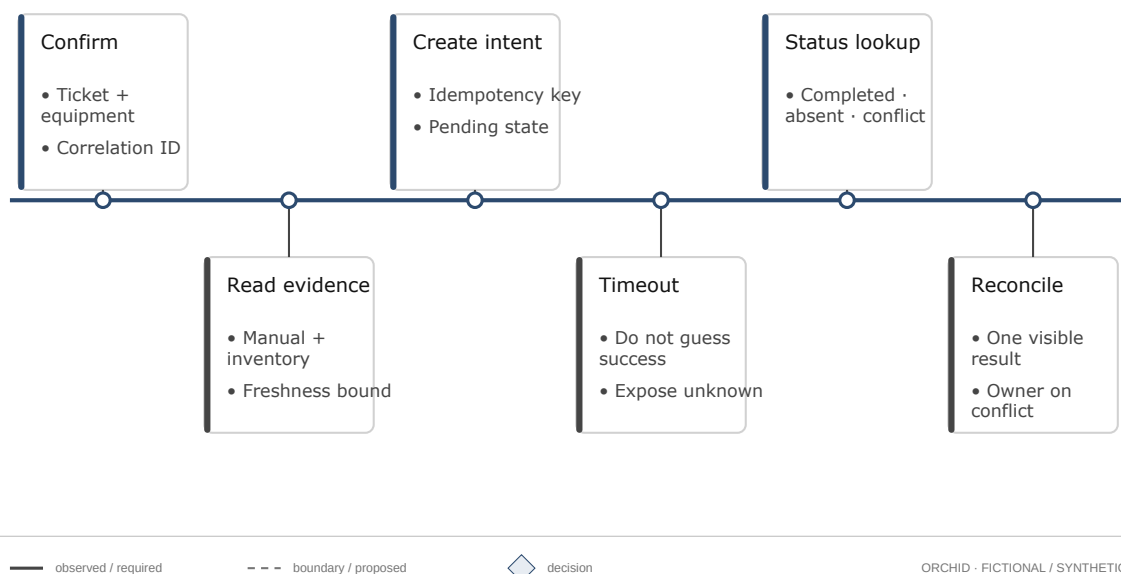
The companion IntentLedger demonstrates:

- first intent begins as pending;
- same ID/same semantics replays state/result;
- same ID/different semantics raises conflict;
- missing response moves state to unknown;
- reconciliation supplies final effect rather than blind retry.

The first Orchid scope keeps ERP integration read-only. The ledger prepares the concept without authorizing a write path.

F07.1 · FORWARD DEPLOYED ENGINEERING

Intent survives partial failure



F07.1 - Intent survives partial failure. Correlation, idempotent intent, status lookup, and reconciliation make uncertainty visible.

Text alternative: Sequence from ticket and equipment confirmation through manual and inventory reads, showing correlation, timeout, retry decision, idempotent intent, unknown state, status lookup, and reconciliation.

Do not promise exactly-once business effects

Messaging systems may provide delivery or processing guarantees under defined conditions. Business effect crosses storage, external systems, side effects, consumer state, retries, and human actions.

An “exactly once” label does not establish that:

- the same event was not published twice under different IDs;
- two consumers did not create equivalent effects;
- an external API effect and local commit remained atomic;
- replay did not repeat a non-idempotent action;
- manual reconciliation did not duplicate work;
- a late event did not override newer intent.

Exactly-once business effects are not obtained by labeling delivery exactly once. [CLM-048]

Prefer precise statements: at-least-once delivery with idempotent consumer under these keys; ordered within this partition; deduplicated for this retention window; effect reconciled against this authoritative state; duplicate/late/conflict paths tested.

Define an error taxonomy

Avoid one generic error field.

Classify:

- invalid request/schema;
- semantic validation failure;
- authentication/authorization denial;
- not found versus ambiguous match;
- conflict/version mismatch;
- rate/capacity/backpressure;
- dependency unavailable;
- timeout before/after possible acceptance;
- stale/incomplete evidence;
- policy/control block;
- internal defect;
- unknown operational state.

For each class, state:

- stable code and human-safe message;
- retryability and backoff/limit;

- workflow state and permitted actions;
- correlation/audit fields;
- data disclosure boundary;
- support owner and escalation;
- reconciliation path.

Never tell a client to retry an ambiguous operation without specifying intent and status behavior.

Specify request/response completion step by step

For each operation, draw a completion timeline:

1. caller creates intent and local state;
2. request leaves caller;
3. provider authenticates/authorizes;
4. provider validates structure/semantics;
5. provider accepts or rejects intent;
6. business effect commits, queues, or remains pending;
7. provider records result/status;
8. response returns or is lost;
9. caller records known/unknown state;
10. caller or operator reconciles if necessary.

The client timeout can occur between any two steps. A contract should say which states are possible, not pretend timeout means failure.

For a read, ambiguity may concern freshness or partial results rather than side effects. For a write, it concerns whether an effect occurred. For an asynchronous job, acceptance and completion are deliberately separate.

Define status lookup where practical:

`GET /intents/{intentId}` might return `pending`, `completed`, `rejected`, `unknown`, or `expired`, plus `result/error/provenance`. The exact endpoint is optional; the semantic capability is not when the caller can lose a completion response.

If the provider cannot expose status, document the reconciliation alternative and risk. Do not invent certainty at the adapter.

Design event contracts around facts, not commands disguised as facts

An event should state what occurred under whose authority and at which time/version.

Compare:

- `UpdateInventory`: command asking an owner to act;
- `InventoryReservationAccepted`: fact that an intent was accepted, not necessarily fulfilled;
- `InventoryReservationCommitted`: fact that the defined effect reached committed state;
- `InventoryAvailabilityObserved`: evidence of availability at an observation time, not a promise.

An event contract records:

- event ID/type/schema version;
- producer/owner and proposition;
- subject/aggregate key and tenant/region;
- occurred/effective/recorded/emitted time;
- correlation, causation, and intent IDs;
- payload semantics/provenance/sensitivity;
- ordering scope;
- delivery/replay/retention expectations;
- consumer idempotency and unknown-version behavior;
- correction/tombstone/retraction behavior;
- operational support and dead-letter/reconciliation path.

AsyncAPI can describe message-driven interfaces, but the contract still needs these domain/operating decisions. [CLM-041] [CLM-047]

Late and out-of-order evidence

Suppose availability observation A occurred at 10:00 and B at 10:05, but B is processed first. Arrival order should not silently overwrite event-time meaning. State what order matters and how late evidence is handled.

For workflow decisions, a late correction may require reopening or annotating an earlier decision rather than mutating history. The audit record should preserve what evidence was available when the person acted.

Preserve lineage through transformations

Orchid Assist derives evidence from customer records. A data contract should trace:

- source system/record/version;
- extraction/query/adaptor version;
- normalization/mapping rules;
- validation results and exceptions;
- retrieval/index version;
- derived field or ranking method;
- observation/effective/processing times;
- downstream decision/audit reference.

Lineage does not require one massive platform. For the bounded slice, stable references and transformation versions may be enough.

If a manual paragraph is presented, record document ID/version/section and retrieval/index version. If inventory is shown, record observed-at/source version and freshness state. If equipment is matched, record candidates and confirmation evidence. This enables a user, tester, incident responder, or auditor to reconstruct the basis of the decision without copying unnecessary sensitive content.

Correction behavior

When source data is corrected, decide:

- Does the derived index/cache update automatically?
- Are prior decisions re-evaluated or only annotated?
- Which active cases must be alerted?
- Can a corrected identifier merge/split histories?
- Who approves correction of an audit or business record?
- How is the original state preserved?

These are workflow decisions, not generic database updates.

Design interface security without hiding it in headers

The interface catalog should describe:

- caller identity and credential type;
- resource/operation/tenant/region authorization;
- field/data filtering by purpose;
- replay/impersonation/confused-deputy risks;

- secret/token rotation and expiry needs;
- audit/correlation;
- denial/error disclosure;
- rate/abuse limits;
- break-glass or support access where relevant.

Do not copy live credentials into examples. The companion remains local and synthetic. Chapter 8 supplies the concrete identity flow; Chapter 10 supplies the control/evidence matrix.

Make capacity and backpressure part of the contract

An interface that works for one request can fail under workflow demand or retry amplification.

Record:

- expected and bounded request/event volume;
- concurrency and burst assumptions;
- rate limits/quotas and response semantics;
- queue capacity/age and overflow behavior;
- client retry/backoff/jitter/limit;
- load shedding and priority;
- timeout budgets across dependencies;
- user-visible wait/fallback;
- capacity owner and review signal.

If an adapter retries every timeout immediately, it can turn dependency degradation into an outage. If it queues indefinitely, users may act on stale evidence. Backpressure must map to workflow state.

Test the contract at several layers

Schema tests

Accept/reject structural fixtures, including unknown fields and versions.

Semantic tests

Validate units, region/tenant, status combinations, freshness, identifier meaning, and policy invariants. The companion's Region West residency case shows a structurally plausible record failing semantics.

Consumer/provider contract tests

Verify both sides against versioned examples and errors. A mock is useful only if it represents the documented provider behavior.

Failure and timing tests

Inject timeout before acceptance, timeout after possible commit, late/out-of-order event, duplicate, conflicting intent, rate limit, partial result, unavailable status, and reconciliation.

Data-quality tests

Exercise missing, invalid, stale, conflicting, duplicated, and incorrectly mapped records with workflow actions.

Operational tests

Confirm correlation, metrics/logs, alerts/queues, runbook, support access, and recovery. Contract correctness without diagnosis is incomplete.

Compatibility tests

Run old consumer/new provider and new consumer/old provider under the declared promise. Verify rollback and migration state.

Tests are evidence for specified cases, not proof that every production interaction is safe.

Walk through the companion foundation

The companion is intentionally small.

`contracts.mjs` validates a synthetic equipment record. It checks structure-like rules and semantic rules such as Region West residency. It produces a stable fingerprint only after validation.

`reconciliation.mjs` preserves:

- no active candidate -> missing;
- one distinct active equipment ID -> confirmed;
- multiple distinct active IDs -> ambiguous;

- missing external response -> completion unknown.

intent - ledger .mjs records intent ID, semantic fingerprint, pending/completed/unknown state, replay, conflict, and reconciliation. It does not claim distributed durability or concurrency safety; it is a deterministic teaching model. Later chapters may replace in-memory storage behind the same behavior while retaining tests.

Seven initial tests prove the bounded behaviors. They do not prove customer ERP semantics, which remain a dependency gap.

Lab sequence

1. Run `npm run test:companion`.
2. Change a synthetic west record to east residency and observe semantic rejection.
3. Add two active equipment IDs and observe ambiguous rather than arbitrary selection.
4. Begin/complete an intent and replay it.
5. Reuse the ID with a different fingerprint and observe conflict.
6. Mark completion unknown after timeout and reconcile it.
7. Add a late-arrival test and define the expected state before coding.

The exercise is complete only when the learner can explain the workflow consequence of every state.

Review a sample interface record

IF-0A-INV-READ

Purpose. Supply read-only inventory evidence for a confirmed equipment/part/site context.

Owner/consumer. ERP team owns interface and inventory propositions; Orchid adapter consumes/translate; inventory operations owns business semantics; Orchid support owns first diagnosis.

Identity. Region-bound workload identity; read selected fields only; no order/reservation permission.

Request. Part ID, site ID, region/tenant, correlation; equipment/ticket context remains in Orchid audit and is minimized at ERP where not required.

Result. availability state/count where permitted, observed/effective time, source version, status; not a guarantee/reservation.

Freshness. Decision-specific threshold owner required; stale state remains visible and may block/escalate.

Timeout/error. Timeout produces inventory-uncertain; no automatic external effect. Rate/unavailable/denied/invalid are distinct.

Retry. Read may retry under bounded policy; user sees wait/fallback; retry amplification is measured.

Reconciliation. Re-read/status under owner-defined behavior; support queue for persistent conflict/staleness.

Version. Versioned schema and semantic contract; breaking meaning changes require migration/consumer tests.

Evidence. Synthetic contract/failure tests now; customer sandbox/production semantics later.

This record is longer than an endpoint declaration because it prevents each consumer from inventing the missing behavior.

Build data-quality rules from decisions

ISO/IEC 25012 defines a general structured-data quality model. Use such models as a vocabulary for contextual requirements, not universal thresholds. [CLM-045]

Orchid quality rules should trace to workflow consequence:

- equipment match uniqueness/completeness;
- manual approval/version/freshness;
- inventory observed-at/freshness for the decision;
- ticket state/assignment consistency;
- provenance/reference resolvability;
- regional/tenant correctness;
- approval identity/completeness;
- audit correlation/completeness.

Each rule contains definition, population, severity, detection, action, exception owner, measurement, and trend. A schema-valid record can fail any of these.

Design reconciliation as normal operation

Reconciliation is not only incident cleanup. Distributed boundaries can disagree during ordinary timeouts, late updates, manual corrections, and eventual consistency.

A reconciliation plan identifies:

- propositions compared;
- authoritative owner for each;
- comparison key and time/window;

- mismatch categories;
- automatic versus human decision;
- safe state during uncertainty;
- correction source and audit;
- retry/replay rules;
- queue/age/alert/support owner;
- closure evidence.

For equipment candidates, the companion preserves missing, confirmed, and ambiguous states. It does not choose the first valid record. For external completion, it preserves unknown when a response is absent.

Version the public contract

Semantic Versioning communicates compatibility expectations only after a public API is declared. It does not solve deployment or data migration compatibility by itself. [CLM-046]

Record:

- contract version and public compatibility promise;
- provider/consumer versions in use;
- backward/forward compatibility;
- additive field behavior and unknown-field rules;
- default/semantic changes;
- deprecation notice/owner/date;
- dual-read/write or translation window;
- data migration/backfill;
- rollback compatibility;
- contract tests and production evidence.

A change from “equipmentId is unique” to “equipmentId is a candidate” may be semantically breaking even if the JSON type remains string.

Use interface specifications honestly

OpenAPI, AsyncAPI, and JSON Schema can generate docs, clients, validators, mocks, and contract tests. Version them with the implementation.

Do not infer:

- runtime availability from generated docs;
- authorization correctness from a security-scheme declaration;
- semantic compatibility from schema compatibility;
- delivery/ordering guarantees from a message channel name;
- safe retry from a method label;
- support ownership from a contact field.

The interface catalog links formal specs to the missing operational/semantic records. [CLM-047]

Handle contract change without semantic drift

Consider three changes.

Add a field

The registry adds `equipmentClass`. Structurally, an optional field may be backward compatible. Semantically, consumers may start using it before its coverage, authority, and change process are understood.

Record population/unknown semantics, provenance, owner, quality, version adoption, and what behavior is prohibited when absent. Unknown fields should not crash a tolerant consumer, but tolerated fields do not automatically enter decision logic.

Change meaning without changing type

`status: active` once meant installed and serviceable. It now means not retired, including equipment awaiting commissioning. The JSON remains valid and the enum remains unchanged. The change is semantically breaking for service eligibility.

Use semantic contract versions, owner notice, consumer impact review, historical/backfill behavior, and dual interpretation/migration where necessary. A schema diff will not find it.

Split one identifier into candidates

The `ticket` field previously assumed one equipment ID. Discovery proves it can contain an alias that resolves to multiple records.

Do not preserve the illusion through arbitrary selection. Change the workflow contract to candidate set plus confirmed match evidence. This breaks consumers that require one ID, but it repairs a false invariant.

Compatibility should protect valid consumer expectations, not preserve a dangerous meaning.

Minimize data at the interface boundary

An interface can be semantically rich and still transfer too much data.

For each field ask:

- Which operation/decision needs it?
- Can a reference, category, derived signal, customer-run query, or redacted value suffice?
- Does the provider need the caller's full workflow context?
- Does the consumer need the source payload or only selected propositions/provenance?
- How long must it persist and where?
- Can logs/errors expose it?
- Which region/tenant/purpose applies?

For Orchid, inventory may need part/site identifiers but not full technician notes. Retrieval may need an approved equipment/manual scope but not unrestricted service history. Audit may need decision references and identity/control evidence without copying every retrieved paragraph.

Minimization can complicate diagnosis. Solve that with correlation, customer-controlled evidence, derived state, and approved support access rather than default replication.

Treat reconciliation workload as a capacity decision

An exception queue can grow faster than humans resolve it. A design that safely detects ambiguity but creates an unstaffed queue is not operable.

Estimate or bound:

- arrival by exception family/segment;
- age and consequence;
- handling time and required qualification;
- owner capacity and hours/regions;
- duplicate/related-case grouping;
- escalation and expiry;
- automation permitted for classification versus decision;
- signal/alert thresholds and backlog stop rule.

The fictional case lacks measured arrival/handling values, so OA-05 records them as required before rollout. The first scope can limit cohort volume and block expansion if the queue exceeds the accepted state.

Create a contract review packet

Before accepting an interface, provide:

- specification/schema/version;
- semantic dictionary and examples;
- identity/authorization/data handling;
- time/consistency/ordering;
- errors/timeouts/retry/intent;
- capacity/backpressure;
- quality/lineage/reconciliation;
- compatibility/migration/deprecation;
- ownership/support/operability;
- test evidence and gaps;
- affected workflow/outcome/guardrails;
- reviewer/decision dispositions.

Reviewers should be able to reject “technically valid” behavior that violates workflow meaning.

Orchid OA-05 interface and data section

The dossier now includes:

Interface catalog. Ticket, equipment, manual, inventory, identity/approval, audit, signals/config relationships with owners and support.

Data contract. Equipment/request/evidence/inventory/approval fields with proposition meaning, provenance/time, residency/sensitivity, quality, invalid/unknown/conflict behavior.

Intent and completion. Correlation and idempotent intent model; ERP writes remain deferred; ambiguous completion has explicit unknown/reconciliation state.

Errors. Stable taxonomy mapped to workflow states, retry rules, user messages, audit, support.

Quality/reconciliation. Decision-linked rules and owned queues/closure evidence.

Versioning. Public contract, consumer compatibility, migration/deprecation/rollback evidence.

Companion. Synthetic validation, candidate reconciliation, completion classification, and intent ledger tests run locally.

Interface failure modes

Schema equals contract

Repair: add proposition meaning, time, owner, failure, operation, and reconciliation.

Retry equals resilience

Repair: define intent, ambiguity, limits, backoff, status, and effect.

Exactly-once label

Repair: state delivery/processing/effect boundary and tests precisely.

Canonical-model overreach

One model tries to represent the enterprise before the bounded workflow is understood.

Repair: define the minimum semantic contract and translations for the selected path.

Null coercion

Repair: preserve missing/unknown/not-applicable/redacted/invalid/stale/conflicting states.

Version is a number only

Repair: declare compatibility promise, consumers, migration, rollback, deprecation, tests.

Reconciliation has no owner

Repair: make uncertain state visible with queue, age, decision, owner, and closure evidence.

The Chapter 7 gate

Before fitting the customer environment, OA-05 must contain:

- complete interface catalog for selected path;
- schema plus semantic field contracts;
- proposition ownership, time, provenance, and quality;
- identity/authorization context requirements;
- timeout/retry/idempotent-intent/completion states;
- error taxonomy and workflow actions;
- ordering/consistency/backpressure where relevant;
- version/compatibility/deprecation/migration;
- reconciliation and support ownership;
- executable companion tests for critical missing/ambiguous/conflict behavior.

The gate passes when a consumer can implement safe behavior without guessing what a string, timeout, duplicate, missing value, or version means.

Chapter 8, *Fit the Customer Environment*, places these contracts inside real topology, regional/tenant, network, identity, secret, configuration, capacity, and environment constraints.

8. Fit the Customer Environment

Translate the logical design into explicit deployment topology, identity, secrets, configuration, tenancy, regional, network, capacity, and environment responsibilities.

A logical design does not run in a customer environment.

Code runs in a region, cluster, machine, process, network path, tenant, and release configuration. It acts through identities whose permissions expire or drift. It depends on DNS, certificates, secrets, rate limits, capacity, clocks, storage, egress, and customer-operated systems. A staging environment may share the API shape and differ in the behavior that matters.

The FDE's job is to translate the logical boundary into deployable responsibility without hiding customer work behind a box labeled "infrastructure."

Provider architecture guidance offers question sets for reliability, security, performance, operations, cost, and capacity. NIST zero trust guidance rejects implicit trust based only on network location. Use these principles in context; do not call a design safe because it is inside a VPC or follows a provider diagram. [CLM-050] [CLM-051] [CLM-055]

The environment-difference matrix and topology, identity, and configuration records in this chapter are original synthesis tools rather than formal external standards. [CLM-056]

Draw the deployment topology

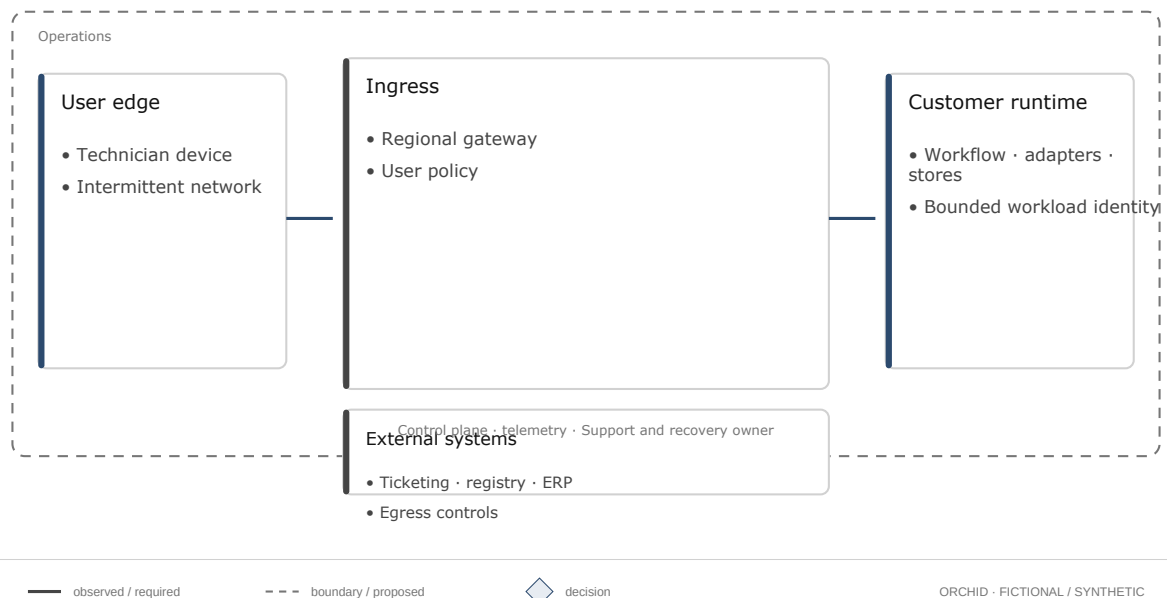
Start from Chapter 6 containers and place each into its actual or proposed runtime.

For Orchid, the topology must show:

- technician device/browser and low-connectivity path;
- customer ingress/gateway or approved access path;
- Orchid Assist web/API/workflow runtime;
- adapters and outbound paths to ticketing, registry, ERP, manuals, identity, audit, telemetry;
- state/cache/index stores;
- Region West and tenant/data-residency boundaries;
- control/configuration/release plane;
- operational signals/support/recovery access;
- external network/DNS/certificate dependencies;
- approved egress and denied paths.

F08.1 · FORWARD DEPLOYED ENGINEERING

Region West deployment topology



F08.1 - Region West deployment topology. Connectivity, tenancy, regional, capacity, and operating assumptions are visible before launch.

Text alternative: Topology from technician device through network and ingress to customer runtime, workflow services, adapters, stores, customer systems, control plane, telemetry, and support with region and tenant boundaries.

Do not imply a runtime choice before it is decided. Use capability labels first; record selected provider/product as a versioned alternative.

Create an environment matrix

Perfect parity is often impossible. The professional task is to expose differences that can change behavior.

For local, test, staging, and production, record:

- runtime/topology/scale;
- identity/token/authorization source;
- tenant/region/data category;
- dependencies: stub, sandbox, or live;
- data distribution/volume/quality;
- network, latency, rate, timeout, and failure behavior;
- secrets/certificates/DNS;
- configuration/feature/cohort controls;

- telemetry/audit/support access;
- release/migration/recovery mechanism;
- policy/control/approval evidence;
- known difference, consequence, test/monitor/control, and owner.

Environment parity is incomplete when consequential differences remain. Those differences need explicit tests, controls, monitoring, or accepted gaps. [CLM-053]

Example:

Difference	Consequence	Compensating evidence
local adapter uses deterministic stub	real timeout/rate/auth semantics unproven	contract tests plus customer sandbox/live bounded verification
staging uses test identity	production role/tenant/region mapping unproven	production authorization dry run and denied-case evidence
staging dataset is small/clean	capacity and data-quality tails hidden	synthetic scale/failure fixtures plus production cohort signals
recovery uses local script	customer permissions/control-plane path unproven	customer-environment recovery rehearsal

The companion `compareEnvironments` function makes differences explicit. It does not judge them safe; the record adds consequence/owner/evidence.

Design human and workload identity separately

Human identity answers who the user is and which assigned context/role applies. Workload identity answers which software process may call which resource operation. Do not pass unrestricted human authority through the application.

For a technician request:

1. authenticate the human under customer-approved identity;
2. resolve tenant, region, assignment, and role context;
3. authorize the user action on the Orchid resource;
4. let the bounded workload identity call selected dependencies;
5. enforce downstream resource/operation/field constraints;
6. record correlation, actor, workload, policy decision, and result;
7. deny safely when context is missing or mismatched.

For safety approval, require a separately authorized qualified identity. The FDE does not define qualification.

F08.2 · FORWARD DEPLOYED ENGINEERING

Identity is bounded at every hop



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F08.2 - Identity is bounded at every hop. Human context informs policy but never becomes unrestricted service authority.

Text alternative: Identity flow from technician and approver sessions through gateway policy, bounded workload identity, downstream authorization, secret retrieval, audit, expiry and revocation, with denied region, tenant, and role paths.

The companion authorization model checks principal, resource, operation, tenant, region, permission, and expiry. It is a teaching model, not a production identity provider.

Write an identity and access record

For each principal-to-resource relationship, record:

- principal type/identity source;
- authentication method and assurance required by the owner;
- tenant, region, assignment, device/session, or workload context;
- resource and operations;
- permission source/policy version;
- grant/approval/provisioning owner;
- token/credential audience, scope, lifetime, rotation/revocation;
- service-to-service delegation or impersonation behavior;
- sensitive fields/data purpose;
- audit/correlation;

- denied/expired/mismatch behavior;
- support and break-glass path;
- test evidence and review date.

Example:

IAM-OA-APPROVER

- Principal: customer human identity mapped to qualified service-approver role for Region West.
- Resource: Orchid safety-approval record for assigned/eligible ticket.
- Operations: view supporting evidence; approve/reject/escalate; not edit safety policy.
- Context: tenant Orchid, Region West, active assignment/qualification.
- Lifetime: bounded customer session; role/qualification changes revoke future authority.
- Audit: human identity, role/policy version, ticket/evidence version, decision, time, correlation.
- Denial: workflow remains awaiting-qualified-approval; no fallback that bypasses approval.

The record separates identity proof, qualification, assignment, and permission. A valid login is not enough.

Confused deputy risk

Orchid Assist has workload access to customer systems. It must not use that access for a user request the user is not permitted to initiate.

Bind user intent/context to the service operation. Authorize at the application resource and again at downstream resource boundaries where required. Minimize fields and operations. Audit both human and workload identities. Do not accept arbitrary resource identifiers simply because the workload credential can access them.

Map network flows as policies

For every ingress/egress relationship, record:

- source/destination runtime and owner;
- protocol/port/name resolution;
- identity/authentication and encryption;
- operation/data category/region/tenant;
- direction and initiation;
- proxy/gateway/firewall/egress control;
- timeout/rate/connection assumptions;
- audit/telemetry;

- failure/fallback;
- change/review owner.

Avoid broad “allow customer network” rules. The architecture should be able to answer why Orchid workflow needs to reach manual/registry/inventory endpoints and why it cannot reach unrelated systems.

DNS and certificates are dependencies. Expired certificates, failed rotation, wrong name, or split DNS can mimic application failure. Put them in the signal/runbook/ownership plan.

Choose placement from constraints

Common placement patterns include:

- customer-managed runtime;
- vendor-managed regional runtime;
- split plane with customer-side connector;
- on-premise or edge component;
- approved managed service inside customer account/subscription;
- local/offline component for selected behavior.

Compare each on:

- data/residency and customer control;
- identity/network integration;
- update/release velocity;
- observability/support access;
- capacity/reliability/recovery;
- secrets/key ownership;
- customer skills/operations;
- cost and exit/migration;
- product reuse versus customer-specific coupling.

There is no universal best placement. A customer-managed runtime may satisfy control and create upgrade/support burden. A vendor-managed runtime may improve consistency and violate a data/network constraint. A connector can reduce inbound access and become a difficult stateful dependency.

Record the decision and revisit trigger rather than presenting the provider name as architecture.

Promote configuration with evidence

Configuration should move through an evidence path:

1. schema/semantic validation;
2. code review and owner approval for material changes;
3. versioned immutable artifact or record;
4. environment-specific binding without secrets;
5. dry run or policy evaluation where possible;
6. bounded rollout;
7. observed runtime/health verification;
8. rollback/forward and reconciliation;
9. audit and change history.

Configuration examples include region/tenant maps, feature cohorts, timeout/retry, manual/index version, policy/model selection, data freshness thresholds, and endpoints. A configuration change can have code-level consequence and must receive proportionate tests/review.

The environment record contains reference IDs to secrets, not secret values. The companion's rejection is intentionally obvious; real systems need secret scanning, access control, and runtime mechanisms beyond it.

Treat clocks and time zones as environment dependencies

Chapter 7 distinguished event times. The environment must provide usable clock behavior.

Record:

- authoritative time source/synchronization expectation;
- UTC storage and local display rules;
- tolerated skew for tokens, ordering, and correlation;
- daylight-saving/local schedule effects;
- behavior when device clocks are wrong/offline;
- timestamp provenance (client versus server/source);
- monitoring and incident diagnosis.

An expired-token error caused by clock skew, or a late event sorted by device time, can create authorization and workflow failures. Do not assume timestamps are comparable because they share a format.

Make capacity assumptions falsifiable

Create a capacity envelope:

- eligible cases/time and peak pattern;
- concurrent users/workflows;
- dependency calls per case and fan-out;
- payload/index/model context size;
- connection/thread/process limits;
- rate quotas and shared tenancy;
- queue/backlog growth and drain;
- storage/log/audit volume and retention;
- timeout/retry amplification;
- cost per case and budget owner;
- load/failure test and production signal.

Use ranges tied to evidence. A pilot cohort still needs protection against loops and bursts. A small user count can generate large model/retrieval/dependency demand.

Define overload behavior: reject, shed optional work, queue, degrade to evidence-only/manual, limit cohort, or stop. Preserve high-consequence approval/audit behavior; do not degrade by skipping a control.

Review a denied case end to end

Use this synthetic scenario:

1. An East-region technician has a valid customer session.
2. They present a West-region ticket identifier, accidentally or maliciously.
3. Gateway authenticates the identity but detects tenant/region/assignment mismatch.
4. The request is denied before West data retrieval.
5. User receives a safe message without disclosing West resource details.
6. Audit records principal/context/resource type/policy/result/correlation, within privacy constraints.
7. Support can distinguish legitimate assignment error from attempted misuse through authorized evidence.
8. No workload call to registry/manual/inventory occurs.

The companion test proves a region mismatch is denied in the teaching policy. It does not prove production identity mapping, gateway enforcement, or downstream isolation. Those require customer-environment evidence.

Create the environment readiness packet

Before Chapter 9, package:

- topology/flows and placement decision;
- environment-difference matrix;
- identity/access records and denied cases;
- network/egress/DNS/certificate flows;
- tenant/region/data/storage/telemetry boundaries;
- secret/configuration lifecycle and promotion;
- capacity/latency/connectivity envelope;
- clock/time assumptions;
- support/operating/recovery/break-glass access;
- tests, simulations, owner reviews, and remaining production gaps.

The environment is “fit” only within the selected scope and evidence, never as a blanket property.

Apply OAuth guidance only where OAuth is used

RFC 9700 is the current OAuth 2.0 security best current practice at the research cutoff. A deployment using OAuth 2.0 should account for it within the actual flow/threat context. [CLM-052]

Do not add OAuth terminology to an architecture that uses another mechanism. If OAuth is selected, document:

- client type and authorization flow;
- redirect/issuer/audience validation;
- token sender constraints where applicable;
- scope/resource/tenant mapping;
- refresh/rotation/revocation/expiry;
- storage and browser/mobile considerations;
- consent/delegation versus application permission;
- replay/substitution/mix-up and logging risks;
- failure/user/support behavior.

A standards reference does not replace a threat model or correct implementation.

Treat secrets as lifecycle responsibilities

Never place secrets in source, manuscript, fixtures, logs, screenshots, or static configuration records.

For each secret/credential/certificate:

- purpose, owner, consumer, resource/operation scope;
- creation/provisioning channel;
- storage/injection and exposure boundary;
- rotation/expiry/revocation;
- audit/detection;
- recovery/break-glass;
- environment separation;
- cleanup at handoff.

The companion environment validator rejects configuration records containing fields named as secret material. That is a narrow guard, not secret scanning or production security.

Separate configuration from secret and code

Version non-secret configuration, including:

- environment/tenant/region;
- endpoint/reference IDs;
- timeout/retry limits;
- feature/cohort policy;
- schema/model/prompt/policy versions;
- freshness/quality thresholds when approved;
- signal/alert settings;
- dependency modes and fallback;
- release version.

Define validation, default, owner, change/review, rollout, and observed-state evidence. A default that broadens access or exposure is not safe merely because it is convenient.

Design tenant and regional isolation

Tenant and region must appear in identity, data keys, storage, cache/index, logs, queues, configuration, and operational access where relevant.

Ask:

- Can one tenant/region reference another's identifier?
- Does cache/index key include the boundary?
- Can support query across boundaries, under what authority/audit?

- Where do backups/telemetry/error payloads reside?
- What happens when dependency data belongs elsewhere?
- Can configuration accidentally enable a cohort in another region?
- How is deletion/retention/revocation scoped?

Private networking or IP allowlisting does not establish isolation or data-residency compliance. [CLM-055]

The companion tests reject a west resource requested by an east principal and reject west configuration with east residency.

Budget capacity and latency across the path

Start from workflow need and allocate a budget across:

- device/network/ingress;
- authentication/authorization;
- workflow coordinator;
- equipment/manual/inventory dependencies;
- retrieval/model/policy/approval;
- state/audit/telemetry;
- retries/queues/fallback.

Record peak/cohort assumptions, concurrency, request size, rate limits, connection pools, queue age, cold start, cache behavior, and low-connectivity segments. The fictional case keeps numeric budgets unresolved.

If the total budget cannot be met, choices include narrower evidence, precomputation/cache with freshness/control cost, asynchronous flow, explicit wait, offline/fallback, reduced cohort, or rejection. Do not hide the miss behind an average.

Design for intermittent connectivity

For Orchid field sites:

- distinguish no network, slow network, request timeout, and dependency failure;
- preserve user intent and local workflow state where appropriate;
- avoid duplicate actions after reconnect;
- show evidence freshness and incomplete state;
- queue only operations safe to delay/replay;
- protect local data/secrets;

- define conflict/reconciliation;
- provide approved manual/escalation path;
- segment signals by connectivity/site.

Offline support is not automatically required. The scope may exclude offline action while offering clear fallback. The decision traces to outcome/guardrail and cohort.

Plan break-glass without making it normal

Recovery access must be usable when the primary path is impaired, tightly controlled, time-bounded, logged, reviewed, and rehearsed.

A first-party Cloudflare postmortem reports that an access dependency complicated rollback and discusses break-glass access/known-good recovery identifiers. The transferable question is whether recovery shares the failed dependency, not that routine controls should be weakened. [CLM-054]

Record:

- trigger and authorized requester/approver;
- independent identity/path/dependencies;
- least emergency permission;
- duration/expiry;
- immutable audit and communication;
- credential custody/rotation;
- rehearsal and after-action review;
- return to normal access.

Define environment failure behavior

Walk through:

- identity provider unavailable;
- token/certificate expired;
- DNS/certificate/network route failure;
- region/tenant mismatch;
- secret/config unavailable or stale;
- configuration partially applied;
- rate/capacity exceeded;
- telemetry/control plane unavailable;

- dependency accessible from staging but not production;
- recovery operator lacks permission.

For each, identify user state, safe fallback, detection, containment, owner, and recovery.

Adapt the same design to three customer patterns

The durable contract should survive placement changes even when implementation differs.

Customer cloud account/subscription

Orchid components run under customer-managed cloud ownership. Customer networking/identity/operations teams control foundational access; product/FDE supplies artifacts and perhaps deployment automation.

Questions:

- Who applies infrastructure/runtime changes?
- How do product updates arrive and receive review?
- Can the product team diagnose without broad customer access?
- Who owns backups, patching, capacity, certificates, and incident command?
- How are customer-specific changes prevented from forking permanently?

Vendor-managed regional service

The vendor operates the runtime and customer connects approved systems/data.

Questions:

- Which data may cross and remain in the vendor region?
- How are tenant isolation, customer keys, support access, export/deletion, and audit evidenced?
- How do customer identities/roles map without over-delegation?
- Which outage/recovery/communication obligations exist?
- Can the customer exit and retrieve required records?

On-premise or disconnected edge

Selected components run near field sites with constrained connectivity.

Questions:

- Which capability must work disconnected, and which must fail closed or defer?
- How are software/configuration/policy/manual versions signed, distributed, observed, and rolled back?
- How are identity, revocation, time, audit, and queued synchronization handled?
- What happens to conflicting updates after reconnection?
- Who maintains hardware/runtime and supplies support evidence?

Do not teach three separate products. Keep the logical interfaces, workflow states, controls, evidence, and ownership contract stable where possible; instantiate topology and operating decisions per environment.

Test environment assumptions before the launch gate

Evidence should include:

- clean deployment from approved artifact/config;
- positive and denied identity/authorization paths;
- region/tenant isolation attempts;
- dependency connectivity, TLS/certificates, DNS, rate/timeout;
- representative data volume/quality and low-connectivity behavior;
- secret injection/rotation/revocation without source exposure;
- configuration promotion and observed application;
- telemetry/audit/support access;
- failure/fallback and recovery/break-glass rehearsal;
- owner review and unresolved-gap disposition.

Production may still differ. The evidence packet states which claims are from local, test, staging, customer sandbox, or bounded production. An environment name is not evidence provenance.

Avoid test identities that are more powerful than production

A shared staging administrator can make every integration appear functional while hiding role/region/tenant denial behavior. Use representative least-privilege test roles, including denied and expired states. Where customer identity cannot exist in staging, simulate the claims locally and schedule production dry-run evidence before cohort exposure.

Avoid clean data as the only environment difference

Use synthetic missing, conflicting, stale, and cross-region fixtures. Customer-controlled test data can add realistic semantics within approved boundaries. Never copy production data casually to create “parity.”

Keep the FDE from becoming the environment

If only the FDE knows which configuration, credential reference, network exception, manual command, or dashboard makes the deployment work, the architecture is not operational.

For every environment-specific action, require:

- versioned procedure or automation;
- owner and backup owner;
- least access and expiry;
- observable result;
- failure/recovery path;
- handoff demonstration.

The FDE may execute early work, but the target operating model must be visible from design. Convenience access should shrink as automation/ownership evidence grows.

Orchid OA-05 environment section

Topology. Region West customer runtime, user/network path, Orchid components/stores, external systems, control/operations planes, owners/boundaries.

Identity. Human technician/approver plus bounded workload identities; resource/operation/tenant/region authorization; expiry/revocation/audit; denied paths.

Secrets/config. No source secrets; lifecycle records; versioned non-secret config with validation/observed state.

Isolation. Region/tenant in identity/data/cache/log/queue/config/support/recovery.

Environment matrix. Local/test/staging/production differences, consequences, tests/monitoring/owners.

Capacity/connectivity. Segmented budget assumptions, rate/backpressure, low-connectivity fallback/reconciliation.

Recovery access. Independent, least, time-bound, audited, rehearsed break-glass plan.

Companion. Authorization and environment/config validation with 4 additional passing tests.

Conduct the environment review exercise

Take the logical Orchid design and instantiate it for three fictional environments: a customer cloud account, a vendor-managed regional service, and a disconnected edge site.

For each, produce topology, identity flow, network policy, secret/config lifecycle, tenant/region isolation, capacity/connectivity budget, telemetry/support access, break-glass path, and owner matrix. Then inject:

- expired technician permission;
- workload identity scoped to the wrong tenant/region;
- blocked egress or DNS failure;
- stale configuration with successful deployment status;
- secret rotation while a worker remains active;
- clock skew affecting expiry/audit order;
- capacity saturation plus retry demand;
- unavailable normal identity during recovery.

Record observed state, user/fallback behavior, data/external effects, evidence, owner, and recovery. Pass when the same logical workflow contracts remain clear while environment-specific responsibilities and failure behavior are explicit. Fail if an administrator identity, copied production data, hidden FDE command, or environment name substitutes for representative evidence.

Environment failure modes

VPC as identity

Repair: per-request principal/resource/operation/context authorization.

Staging equals production

Repair: environment-difference matrix and compensating evidence.

Shared long-lived credential

Repair: workload identity, least operation, rotation/expiry/audit/revocation; route exceptions to owner.

Secret in configuration

Repair: separate secret lifecycle/injection; keep records secret-free.

Region as a label only

Repair: enforce in identity, storage, cache, telemetry, support, config, recovery.

Break-glass depends on failure

Repair: independent path/known-good access and rehearsal.

Capacity after launch

Repair: budget/assumptions/failure/load evidence and cohort control before exposure.

The Chapter 8 gate

Before bounding AI behavior, OA-05 must show:

- deployment topology and operating ownership;
- environment-difference matrix;
- human/workload identity flows and authorization context;
- secret/configuration lifecycle and observed state;
- tenant/region/data isolation;
- network/egress/DNS/certificate dependencies;
- capacity/latency/rate/backpressure and connectivity assumptions;
- break-glass/recovery access;
- failure walkthroughs and companion tests;
- explicit gaps/owners/approvals.

The gate passes when “works in the customer environment” is decomposed into testable runtime and ownership claims.

Chapter 9, *Put AI Inside a Bounded Workflow*, decides whether uncertainty-producing model behavior belongs in this environment and how task, data, tools, evaluation, controls, latency, cost, fallback, and human authority remain visible.

9. Put AI Inside a Bounded Workflow

Decide when AI belongs, decompose tasks, bound model and tool behavior, evaluate by workflow consequence, and preserve deterministic controls and accountable human decisions.

AI should enter the workflow only after the workflow can say no to it.

If a deployment cannot define the task, error consequence, permitted data/tools, deterministic rules, human authority, evaluation evidence, fallback, latency/cost budget, version, and stop condition, it is not ready to add uncertain model behavior.

The question is not “Where can we use AI?” It is:

Which bounded decision or evidence task benefits from uncertainty/flexibility enough to justify its measurement and control burden?

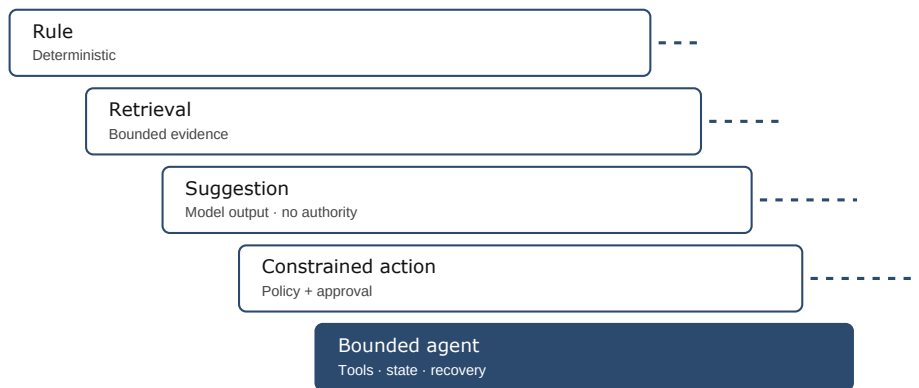
NIST AI RMF 1.0 and its Generative AI Profile support lifecycle/use-case risk framing. Version matters: the framework was under active revision at the research cutoff. Current security/risk taxonomies can prompt discovery but do not provide complete coverage or certification. [CLM-057] [CLM-064] [CLM-065]

Climb the mechanism ladder only as needed

Consider five mechanisms:

1. deterministic rule/calculation;
2. structured search/retrieval;
3. model suggestion/classification/summarization;
4. constrained model-selected tool/action under policy;
5. bounded agentic planning/execution.

Use the least uncertain mechanism



F09.1 - Use the least uncertain mechanism. Every step upward adds evaluation, permission, monitoring, recovery, latency, cost, and authority needs.

Text alternative: Mechanism ladder from deterministic rule through retrieval, model suggestion, constrained action, and bounded agent, with Orchid stopping at evidence retrieval plus suggestion and qualified approval.

Choose the lowest mechanism that satisfies the outcome. First-party agent engineering guidance similarly recommends simple composable workflows when sufficient and emphasizes ground truth, checkpoints, and stopping conditions for agents. It is vendor practice, not a universal standard. [CLM-058] [CLM-060]

For Orchid:

- equipment eligibility and safety rules remain deterministic;
- manual/evidence retrieval is structured and provenance-visible;
- a model may summarize/rank evidence or propose next inspection steps;
- safety-relevant action requires qualified approval;
- automatic parts ordering and open-ended agents remain out of scope.

Decompose the task

“Assist technicians” is not evaluable. Decompose:

- identify/confirm equipment: deterministic/data reconciliation plus human confirmation;
- retrieve approved evidence: search/retrieval with access/provenance;

- summarize evidence: bounded model task;
- rank possible next inspection steps: bounded model task plus deterministic eligibility;
- decide safety-relevant action: qualified human/domain decision;
- check inventory: deterministic interface read;
- order part: external action, deferred;
- record decision: deterministic workflow/audit.

For each subtask record input, output, success criteria, errors, consequence, fallback, owner, data/tools, latency/cost, and evaluation.

Write an AI decision record

Include:

- workflow task and outcome/guardrail served;
- non-AI alternatives and why insufficient/sufficient;
- input data/provenance/sensitivity;
- model purpose and prohibited use;
- output meaning/non-meaning;
- tools/actions and permissions;
- deterministic pre/post controls;
- human role, qualification, evidence, authority, workload;
- error taxonomy and consequence;
- evaluation data/graders/segments/threshold owners;
- latency/cost/capacity budget;
- fallback/abstain/escalate/stop;
- model/prompt/retrieval/tool/policy versions;
- monitoring/change/release/rollback;
- residual limits and formal owners.

The model is one component in a broader production system of data, interfaces, controls, evaluation, monitoring, operations, and users. [CLM-061]

Build an error taxonomy from workflow consequence

For Orchid:

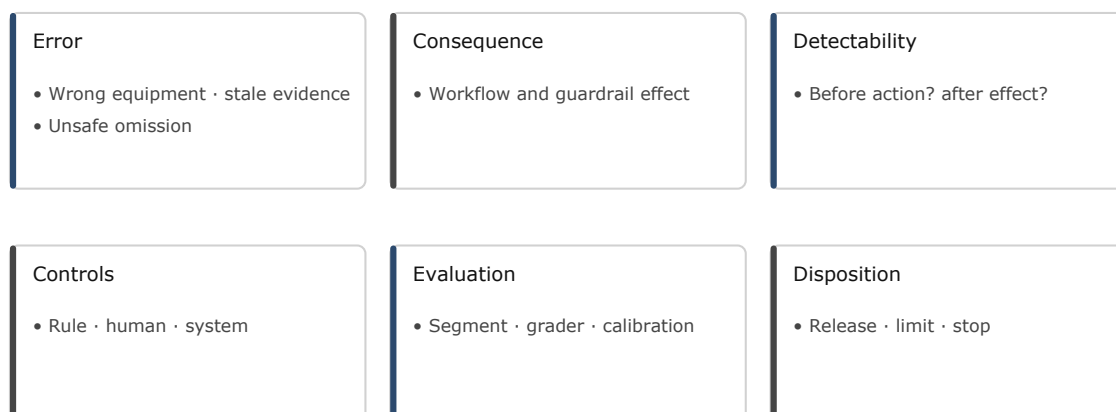
- wrong evidence cited;

- relevant evidence omitted;
- unsupported statement;
- wrong equipment context;
- outdated manual/source;
- unsafe/prohibited action suggested;
- required escalation omitted;
- overconfident language;
- refusal/abstention when help was possible;
- excessive latency/cost;
- tool request outside permission;
- privacy/sensitive-data disclosure;
- inconsistent output across equivalent cases.

Record consequence, detectability, control, evaluation, threshold owner, and release action.

F09.2 · FORWARD DEPLOYED ENGINEERING

Evaluate by workflow consequence



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F09.2 - Evaluate by workflow consequence. Aggregate quality cannot hide a critical segment or an error without a safe control.

Text alternative: AI risk and evaluation matrix mapping error type to workflow consequence, detectability, deterministic, human and system controls, evaluation segment and grader, and release or stop threshold.

Design evaluation as a system

Useful distinctions include task, trial, dataset/case, grader, transcript/trace, outcome, harness, suite, capability evaluation, and regression evaluation. [CLM-059]

Task and case

Define the exact workflow behavior and case context. Include common, boundary, exception, high-consequence, adversarial/misuse, denied, and fallback cases.

Data provenance

Record source, permission, construction, version, split, segment, leakage risk, correction, and representativeness. The companion uses synthetic cases only.

Grader

Use deterministic checks where possible: required evidence ID, prohibited action, policy state, schema, latency/cost. Human/domain graders handle contextual quality under rubric/qualification. Model graders may scale judgment but require calibration and can share failure modes.

Harness and version

Freeze model interface, model/version, parameters, prompt/system instructions, retrieval/index, tools, policy, environment, and grader versions. A score without system version is weak evidence.

Segments

Report by workflow consequence, equipment family, evidence quality, language/environment, user cohort, or other legitimate critical segment. Do not rely on one average.

Decision

Precommit pass/guardrail/stop/waiver authority. Evaluation is useful only when it changes release or scope.

The companion evaluation preserves common and safety-critical segments, demonstrating how an aggregate could conceal a critical failure.

Separate capability from regression

Capability evaluation asks whether the approach can meet the task under defined conditions. Regression asks whether a changed system preserves accepted behavior.

Early exploratory cases should not become the only release suite. Curate validated cases, maintain held-out/rotating/adversarial sets, and guard against tuning directly to every known example.

Production traces can reveal new cases, but they require permission, minimization, de-identification/handling, provenance, review, and separation from evaluation leakage.

Build the evaluation dataset from workflow states

Do not begin by collecting “interesting prompts.” Derive cases from OA-02 decisions, states, exceptions, and OA-03 guardrails.

For Orchid include:

- confirmed equipment with complete current evidence;
- multiple/absent equipment candidate;
- stale/missing/conflicting manual or inventory evidence;
- normal versus safety-relevant decision;
- required escalation;
- low-connectivity/timeout/fallback;
- prohibited action;
- adversarial/untrusted manual content;
- irrelevant or sensitive data request;
- ambiguous user instruction;
- no answer supported by evidence;
- equivalent cases with formatting/order changes;
- high-consequence equipment subfamily;
- reviewer disagreement.

Each case record contains:

- case ID/version/segment;
- fictional/synthetic or approved-source provenance;
- workflow state/task and user goal;
- input evidence/permissions;
- expected acceptable behaviors, prohibited behaviors, and abstention conditions;
- grader/rubric and qualified owner;
- consequence/severity/detectability;

- control expected to catch the error;
- capability/regression/monitoring use;
- known limitation and change history.

Avoid one gold string where several answers are acceptable. Grade properties: cites required evidence, does not invent missing facts, stays within task, triggers required escalation, avoids prohibited action, uses calibrated uncertainty, and meets format/budget.

Prevent leakage

Separate development examples from held-out acceptance. Track whether production traces were seen during tuning. Rotate or refresh cases when the system memorizes a small suite. Keep high-consequence regression cases stable while adding new variants.

Leakage does not make a case useless; it changes the claim. A memorized case can be a regression check and weak capability evidence.

Calibrate graders

Deterministic graders are ideal for exact invariants:

- required evidence ID present;
- prohibited action absent;
- JSON/schema valid;
- policy state/approval correct;
- latency/cost within test budget;
- tool call/argument allowed;
- no cross-region reference.

Human/domain graders evaluate contextual usefulness, completeness, uncertainty, and professional judgment. Provide rubric, examples, qualification, blinding/order where feasible, disagreement handling, and calibration cases.

Model graders can assist with scale. Compare their decisions against qualified human labels across critical segments. Investigate shared bias, prompt sensitivity, position/order effects, verbosity preference, and inability to verify external evidence.

Report grader uncertainty and disagreement. Do not hide it inside the system score.

Evaluate the whole decision path

A good model output can fail inside a bad system. Evaluate:

1. correct equipment/context binding;
2. permitted evidence retrieval;
3. evidence quality/provenance/freshness;
4. model input construction;
5. output behavior;
6. deterministic policy;
7. human review/decision;
8. tool/action execution where allowed;
9. audit/state/user message;
10. outcome/fallback/support behavior.

This prevents model-only evaluation from masking retrieval, identity, UI, policy, or approval failures.

For example, a summary may accurately reflect the wrong equipment manual. Model scoring alone can pass it. End-to-end evaluation must fail at equipment binding/provenance.

Threat-model model input and tools

Treat customer documents, retrieved content, user text, and tool results as untrusted inputs according to context.

Ask:

- Can content instruct the model to ignore policy or expose data?
- Can one tenant/region influence another through shared index/cache/context?
- Can output cause unauthorized tool selection or arguments?
- Can sensitive data appear in prompt, output, logs, traces, graders, or vendor retention?
- Can repeated requests exhaust cost/capacity or probe policy?
- Can a user rely on fabricated citations or hide evidence?
- Can tool output be forged/stale/misbound?
- Can an attacker manipulate evaluation data or production feedback?

Mitigations can include data/tenant isolation, input labeling, retrieval allowlists, output schema, deterministic authorization/policy, tool allowlists, argument validation, least workload identity, evidence verification, rate/budget limits, redaction/minimization, audit, monitoring, and human approval.

OWASP's current community guide can prompt categories, but it is not a complete system threat model or certification. [CLM-064]

Design abstention and fallback

Abstention is a product state, not an error string.

Define when the system must abstain:

- equipment/context unconfirmed;
- evidence absent/stale/conflicting;
- task outside approved scope;
- policy requires qualified decision;
- output cannot meet confidence/evidence rule;
- model/tool unavailable or budget exceeded;
- authorization/region/tenant denied;
- detected attack/unsafe input;
- evaluation/monitoring stop state.

For each abstention, tell the user what is known, what is missing, what action is permitted, and how to escalate. Do not expose sensitive policy internals unnecessarily.

Fallback can be evidence-only search, deterministic checklist, manual workflow, qualified escalation, deferred queue, or stop. The fallback itself needs access, usability, support, and recovery evidence.

Bound agent loops

If a later deployment introduces an agent, define:

- goal/task boundary and termination state;
- maximum steps/time/cost/tool calls;
- tool allowlist and per-call authorization;
- state/memory/data retention;
- checkpoints and human approval;
- ground truth/status verification;
- duplicate/retry/idempotency/reconciliation;
- loop/stall detection;
- interruption/resume/cancel;
- audit/trace and privacy;
- compensation/recovery;
- eval suite for multi-step behavior.

An agent should not continue because a model says it is not done. Termination and budget enforcement live outside model output.

Orchid's first scope does not require an agent. Keeping this explicit prevents "agentic" capability from entering through a library default.

Govern model and prompt change

Record a system version bundle:

- model/provider/version;
- prompt/system instructions;
- retrieval/index/corpus versions;
- tool definitions/permissions;
- deterministic policy/control version;
- dataset/grader/harness versions;
- application/config/release version.

A model alias can change behavior without code change. A retrieval corpus update can change answers without model change. A prompt fix can interact with tool schema. Evaluate and release the bundle.

Classify changes by risk and required evidence. Run targeted plus regression suites. Canary with quality and workflow signals. Preserve rollback to an actually available/compatible bundle. If provider version retirement prevents rollback, plan a validated alternative before migration.

Decide when not to use AI

Reject AI when:

- deterministic rules solve the task with less uncertainty;
- required data cannot be used legitimately;
- errors cannot be detected/contained relative to consequence;
- no qualified owner can define acceptable behavior;
- no representative evaluation can be built;
- latency/cost/availability break the workflow;
- tool permission would be too broad;
- fallback/support/recovery is unavailable;
- expected value does not justify control/operation burden.

The decision can be “retrieval only,” “human workflow with better evidence,” or “stop.” That is an engineering outcome, not a failure to innovate.

Build a control ladder

Controls should match consequence:

- remove AI from deterministic task;
- constrain input/data/retrieval scope;
- constrain output format;
- block prohibited tool/action;
- require evidence/provenance;
- validate against deterministic policy;
- abstain/fallback on insufficient evidence;
- require qualified human approval;
- limit cohort/rate/action;
- monitor quality/incident/support;
- disable/rollback/stop.

The companion app `lyControlLadder` blocks ambiguous equipment, prohibited suggestions, and missing qualified approval before presenting the suggestion.

Human review is a designed control

Human review does not automatically create safety, attentiveness, scale, qualification, or accountability.
[CLM-063]

Define:

- which cases require review;
- reviewer qualification/authority;
- evidence and uncertainty shown;
- time and workload;
- independent decision versus rubber stamp;
- approve/reject/edit/escalate actions;
- disagreement/audit;
- reviewer error evaluation;
- coverage when unavailable;

- consequence of automation bias.

For Orchid, the model never defines the safety rule. The qualified approver receives evidence, rule state, suggestion, limitations, and identity/audit context.

Bound tools and actions

If a model can call tools, define:

- allowlisted tool/operation;
- read/write/irreversibility;
- argument schema and semantic validation;
- identity/tenant/region/data scope;
- deterministic policy before execution;
- user/human approval;
- intent/idempotency/reconciliation;
- result/error disclosure;
- rate/cost/time limits;
- audit/monitoring/recovery;
- stop/kill mechanism.

Do not give a model a broad credential and rely on instructions for access control. Permissions and policy live outside the model.

Budget latency, cost, and capacity

Define budgets per eligible case and segment. Include retrieval, model calls, tool calls, retries, review, and fallback.

The companion `runWithBudget` returns visible fallback on cost or timeout. It uses synthetic “cost units,” not vendor prices. Real pricing and performance are volatile and require current evidence.

If the model times out, the user should see an approved evidence-only/manual path, not a silent spinner or repeated expensive calls.

Use deployment-like simulation carefully

Static evaluation can miss multi-turn, tool, timing, environment, or behavioral context. A first-party research article reports that deployment-like simulation can expose some missed behavior while retaining production-tail limitations. [CLM-062]

Simulate:

- changing/contradictory evidence;
- tool unavailable/denied/slow;
- low connectivity;
- prompt injection/untrusted content;
- user correction/pressure;
- long context/state;
- reviewer unavailable;
- model/version change;
- cost/latency limits;
- repeated/adversarial attempts.

Simulation does not authorize live exposure. It is one evidence layer.

Read an evaluation report without being fooled

Suppose a synthetic suite contains 900 common cases and 100 safety-critical cases. The system passes 880 common and 40 safety-critical cases. Aggregate pass rate is 92 percent. The safety-critical pass rate is 40 percent.

The aggregate is mathematically correct and operationally misleading if the critical segment governs exposure. The report should show:

- case counts/provenance by segment;
- pass/fail/error/abstain categories;
- consequence-weighted review without hiding raw counts;
- confidence/uncertainty and grader disagreement;
- failure taxonomy distribution;
- controls that caught or missed each failure;
- comparison to previous system bundle;
- known coverage gaps;
- release disposition and owner.

The book does not claim these synthetic numbers for Orchid. They illustrate why the companion test preserves segment failure.

Abstention can be correct

If evidence is missing, abstention may pass while an unsupported confident answer fails. But excessive abstention can make the workflow useless. Measure coverage and appropriate abstention separately from correctness on answered cases.

Control performance matters

Record whether the raw suggestion was unsafe and whether deterministic policy/approval blocked it. A safe final system can contain a weak model under strong controls for a limited task, but control burden, user experience, and residual failure remain visible. Do not publish only the post-control score if the model change affects workload/cost.

Connect AI errors to production decisions

Use a matrix:

Error class	Consequence	Required control/evidence	Release action
wrong equipment context	wrong evidence/action	match confirmation; provenance; end-to-end tests	block exposure
unsupported statement	mistrust/wrong decision	evidence-required output; grader; UAT	repair or restrict
unsafe/prohibited suggestion	harm	deterministic policy; critical suite; qualified approval	zero accepted violations under owner rule; block
appropriate abstention	delay/manual work	fallback and support capacity	monitor burden
excessive abstention	low value/adoption	segmented coverage eval	reduce scope/change
latency/cost breach	abandonment/capacity	budgets/fallback/cohort	hold ramp
tool permission violation	unauthorized effect	external authorization/allowlist/audit	block/incident

Threshold language belongs to the owner and evaluation plan. “Zero accepted violations” can apply to observed test/release criteria but is not proof that no production violation can occur.

Design the review interface as part of the control

The reviewer should see:

- task/request and confirmed equipment context;
- source/evidence references, versions, freshness, and gaps;
- model suggestion separated from verified facts;
- deterministic policy result and prohibited state;
- uncertainty/abstention reason;

- approve/reject/edit/escalate choices appropriate to authority;
- consequence and next workflow state;
- audit/privacy notice where required.

Avoid interfaces that place the suggestion prominently and evidence behind extra navigation, or default to approve under time pressure. Measure review time, evidence inspection, disagreement, override, and support burden.

If the qualified reviewer cannot understand why evidence was selected, the system may shift verification work rather than reduce it.

Run a pre-mortem

Before rollout, ask: “The bounded AI path caused a serious incident or failed adoption. What happened?”

Candidate conditions:

- wrong equipment binding bypassed evaluation;
- retrieved manual was obsolete;
- untrusted content manipulated output;
- reviewer approved by habit under workload;
- low-connectivity users saw stale partial evidence;
- model alias changed behavior;
- fallback was unavailable;
- traces logged sensitive data;
- tool permission was broader than intended;
- aggregate monitoring hid the critical segment;
- support could not identify the system bundle.

For each, identify prevention/detection/containment/recovery/owner and add missing cases or controls. A pre-mortem is not evidence of control effectiveness; it improves the test plan.

Keep current tools volatile

Vendor evaluation APIs and agent frameworks evolve. The OpenAI Evals platform documented at the research cutoff was scheduled for late-2026 deprecation. Teach portable evaluation artifacts and interfaces rather than durable UI/API promises. [CLM-066]

The companion’s deterministic model interface can be replaced by a provider adapter later; tests/control behavior remain stable.

Orchid OA-05 AI section

Decision. Use AI conditionally for bounded summary/ranking; deterministic retrieval/policy/approval remain separate; non-AI fallback exists.

Inputs. Confirmed equipment, approved evidence with provenance, selected workflow context; no unrestricted ticket archive.

Outputs. Suggestion with evidence references and uncertainty; not authority or fact by itself.

Errors/evals. Workflow consequence taxonomy, critical segments, synthetic suite, deterministic/domain graders, capability/regression separation.

Controls. Match confirmation, data/tool scope, evidence requirement, deterministic eligibility, abstention/fallback, qualified approval, cohort/budget, monitoring/disable.

Versions. Model interface, model/test double, prompt, retrieval/index, tool, policy, grader, dataset.

Companion. Deterministic model, budget fallback, control ladder, segmented evaluator; 4 additional tests.

Conduct the bounded-AI design exercise

Start with the stakeholder request use AI to recommend the fix. Decompose the workflow into equipment confirmation, approved evidence retrieval, inventory state, candidate generation, eligibility policy, qualified review, and recorded action.

For each step decide deterministic logic, search/retrieval, model behavior, human judgment, or rejection. Justify why AI is needed, what non-AI baseline exists, which errors matter, and which action remains prohibited.

Build a small versioned evaluation set covering clear, ambiguous, missing, conflicting, stale, adversarial, low-connectivity, and critical equipment states. Define graders and segment dispositions before results. Inject an aggregate pass with one critical prohibited suggestion and require a blocking decision. Calibrate a model grader against qualified human labels and preserve disagreement.

Then run the companion under missing evidence, exhausted budget, policy denial, expired qualification, and feature-disabled conditions. Confirm abstention/fallback is visible, dependencies/tools remain scoped, and the model never becomes the authorization or approval authority.

Pass when a reviewer can see the task, evidence, system bundle, errors, controls, owners, residual limits, and stop/disable path. Fail if the proposal relies on prompt wording, an aggregate benchmark, or human in the loop without qualification, evidence, workload, authority, and interface design.

AI failure modes

AI-first decomposition

Repair: define workflow task and compare deterministic/retrieval alternatives.

Aggregate benchmark

Repair: task cases, consequence segments, graders, system version, release decision.

Prompt-only control

Repair: enforce permissions, policy, approval, budgets, and audit outside model.

Human-loop theater

Repair: qualification, evidence, authority, workload, evaluation, fallback.

Vendor interface as architecture

Repair: stable provider-neutral model/tool/eval interfaces and versioned adapter.

Production traces without governance

Repair: purpose, permission, minimization, provenance, handling, leakage control.

Silent fallback

Repair: explicit state, user path, signal, support, decision.

The Chapter 9 gate

Before governance/control evidence is finalized, OA-05 must contain:

- AI decision record and non-AI alternative;
- task decomposition and prohibited uses;

- data/tool/identity/action boundary;
- error/consequence/detectability taxonomy;
- evaluation cases/provenance/graders/segments/versions;
- deterministic/human/system control ladder;
- latency/cost/capacity/fallback;
- monitoring/change/release/disable;
- authority and residual limitations;
- passing deterministic companion tests.

The gate passes when the system can reject AI, abstain safely, explain its evidence boundary, and stop under precommitted conditions.

Chapter 10, *Build Security and Governance Into Delivery*, connects all system/AI risks to control objectives, implementation, verification evidence, formal approval, and escalation without turning framework names into compliance claims.

10. Build Security and Governance Into Delivery

Turn threats and risks into proportionate controls, implementation evidence, minimized access and data, and decisions routed to designated authorities.

Security does not arrive when a reviewer opens a checklist. Governance does not arrive when a committee name appears on a diagram.

They exist when a deployment can answer, with evidence:

- what must be protected and why;
- what can go wrong, through which path, and with what consequence;
- which control objective addresses that risk;
- where the control is implemented;
- how the implementation is verified;
- what evidence the verification produced;
- who owns operation and remediation;
- which designated authority approves the residual risk or exception.

An FDE helps build this chain while the system is still changeable. The FDE does not become legal counsel, privacy officer, security approver, safety authority, independent auditor, accessibility authority, or business risk owner. Outcome ownership means making a decision ready for the correct authority, not absorbing authority that was never delegated. [CLM-078]

Start with consequence, not framework names

The fictional Orchid Assist deployment has a bounded purpose: help qualified technicians identify equipment, retrieve approved evidence, and consider next inspection steps. It also has concrete exposure:

- a person could act on evidence for the wrong equipment;
- one customer or region could see another's records;
- an unqualified actor could approve a safety-relevant step;
- a secret could be placed in configuration or a log;
- an interface could retry an action whose completion is unknown;
- an obsolete manual could be presented as current;
- a model could invent support, follow untrusted instructions, or request an unauthorized tool;
- the system could become unrecoverable because emergency access depends on the failed control plane;
- audit collection could preserve more sensitive content than the business purpose requires;

- an inaccessible review interface could prevent a user from examining evidence or exercising a required decision.

These statements are more useful than “follow NIST,” “be zero trust,” or “meet OWASP.” A framework can organize questions and desired outcomes. It cannot know Orchid's actual assets, actors, deployment boundary, contractual obligations, regional rules, safety consequence, control implementation, or decision authority.

Use frameworks as versioned lenses. Record the selected scope, mapping, evidence, gap, and accountable owner. Never turn alignment into certification language. NIST CSF 2.0 is an outcome-oriented framework, not a prescribed implementation or certificate. [CLM-069]

Build the evidence chain

Use this sequence:

1. **Asset or obligation.** What has value or must be preserved: equipment identity, technician authorization, customer records, service availability, safe workflow state, evidence provenance, secrets, audit integrity.
2. **Threat or failure condition.** What actor, event, defect, dependency, misuse, or operational condition could cause loss?
3. **Risk statement.** Under a defined context, what could happen, to whom, and with what consequence?
4. **Control objective.** What must be prevented, detected, contained, recovered, or decided?
5. **Implementation.** Which code, configuration, workflow, process, training, physical measure, or contractual boundary realizes that objective?
6. **Verification.** Which inspection, analysis, test, exercise, review, or monitoring query challenges the implementation?
7. **Evidence.** Which versioned result shows what happened, in which environment, at what time, with which limits?
8. **Owner disposition.** Who operates/remediates the control, and which designated authority approves, rejects, restricts, or accepts an exception?

F10.1 · FORWARD DEPLOYED ENGINEERING

Control claims require executed evidence



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F10.1 - Control claims require executed evidence. Contextual risk flows through objective, implementation, verification, ownership, and authority.

Text alternative: Chain from asset and threat through contextual risk, control objective, implementation, verification, versioned evidence, operating owner, and designated approval or exception, with failed verification returning to discovery.

This is an original synthesis structure for FDE delivery, not a formal external standard. [CLM-075]

Write risks as decisions can use them

Avoid vague entries such as “data breach,” “AI risk,” or “API security.” A useful statement identifies condition, consequence, and affected obligation:

If a support identity can read equipment records across tenants, a compromised or misused credential could disclose another customer's operational data, violating the isolation expectation and expanding incident scope.

Do not pretend an ordinal score is objective. Likelihood, consequence, detectability, exposure, and risk tolerance depend on context and owner judgment. If the customer uses a scoring method, record the method/version, inputs, uncertainty, owner, and decision rule. Preserve narrative consequence even when a number exists.

Separate inherent, controlled, and residual views

- **Inherent view:** the condition before the proposed control set.

- **Controlled view:** expected condition if the controls operate as designed.
- **Residual view:** what remains after evidence and known limitations.

The residual view is not “low” because a control appears in a spreadsheet. It depends on implementation and evidence. A missing negative test, untested recovery path, unowned alert, or unsupported environment can keep material uncertainty open.

Turn a vague control into a verifiable record

“Use least privilege” is a control intention. “Encrypt data” is a mechanism. “Require approval” is a workflow statement. None is complete evidence.

A control record needs:

Field	Question
control ID/version	Can evidence and change history identify the same control?
risk/control objective	What consequence must be prevented, detected, contained, recovered, or decided?
scope	Which actors, resources, operations, tenants, regions, environments, and states?
implementation	Which actual enforcement point, configuration, process, and responsible component?
verification	Which positive, negative, boundary, failure, and recovery tests?
evidence	Which immutable-enough result, version, environment, time, and limitations?
operating owner	Who keeps it working and responds to failure?
decision authority	Who can approve the residual condition or exception?
monitoring/review	Which signal, cadence, trigger, retention, and escalation?
known gap/expiry	What remains uncertain, and when must the decision be revisited?

A framework label or policy sentence becomes useful only when it connects to implementation, verification, evidence, ownership, and disposition. [CLM-076]

Example: qualified approval

Risk. A technician without the required qualification approves a safety-relevant next step for the wrong region/equipment family.

Objective. Only a currently qualified, region-authorized actor who is not the requester can approve that step for the bound equipment.

Implementation. The authorization service evaluates principal, tenant, region, equipment family, operation, qualification, expiry, and requester/approver separation. The workflow rejects missing/expired/mismatched grants. The UI displays evidence and does not default to approve.

Verification. Test a valid approval plus wrong tenant, wrong region, wrong equipment family, expired qualification, requester-equals-approver, ambiguous equipment, and authorization dependency unavailable. Inspect server-side enforcement rather than UI visibility alone.

Evidence. Record test bundle version, policy version, environment, results, failed cases, reviewer, and open limitations. Do not copy sensitive payloads into the report.

Owners. The identity/platform owner operates enforcement. A designated safety/business authority defines the qualification and accepts residual workflow risk. The FDE prepares and routes evidence.

Threat-model the workflow and system together

A component-only threat model misses handoffs. A workflow-only model misses trust boundaries. Join OA-02 workflow states and OA-05 architecture.

For each state transition ask:

- Who initiates it, under which identity?
- Which resource, tenant, region, and operation are involved?
- What evidence binds the request to the correct equipment and customer?
- What untrusted input can influence the decision?
- What happens if an interface times out, duplicates, reorders, or returns stale data?
- What privilege changes the workflow state?
- Which information is read, derived, stored, transmitted, logged, cached, or exported?
- Which detection would reveal misuse or control failure?
- Can the system contain impact and recover without widening routine access?
- Who has authority to stop, override, or accept the condition?

Use misuse, failure, and recovery cases. A malicious actor is not required for harm: an expired role, misbound tenant, copied secret, stale index, broken clock, partial deployment, inaccessible UI, or exhausted reviewer can create the same consequence.

Model trust per request

Network placement is not identity. A private subnet, VPN, VPC, or IP allowlist can reduce exposure, but it does not prove the user, workload, tenant, resource, action, or current authority. Zero trust architecture focuses protection on resources and rejects implicit trust based only on network location. [CLM-067]

For Orchid, authorization evaluates:

principal + resource + operation + tenant + region + qualification + context + expiry

The decision is enforced near the resource/action, not only in the browser. Service identities receive separate grants from human users. Support access is time-bounded, purpose-bound, recorded, and revocable. Cache and search layers preserve tenant/region boundaries rather than assuming the calling UI filtered correctly.

Where OAuth 2.0 is actually used, record the flow and apply RFC 9700 security best current practice to that context. The protocol name itself is not a security outcome. [CLM-068]

Minimize data across the entire path

“We do not store it” is incomplete. Data may still appear in requests, queues, memory, logs, traces, analytics, caches, backups, search indexes, model inputs, grader datasets, support exports, screenshots, or incident artifacts.

Create a data-handling record for each category:

- purpose and prohibited secondary use;
- data subject/owner and source/provenance;
- classification/sensitivity;
- fields required, derived, or explicitly excluded;
- collection/input path;
- storage/cache/index/backup locations;
- tenant and regional boundary;
- access roles and approval;
- transmission and processor/subprocessor path;
- retention, expiry, deletion, and legal hold owner;
- logging, trace, evaluation, support, and export treatment;
- encryption/key responsibility where applicable;
- verification and incident contact;
- unresolved legal/privacy decision and designated authority.

Minimization is a design activity. Prefer an equipment identifier plus approved evidence references over an unrestricted ticket transcript. Prefer a derived eligibility flag over a full qualification dossier when the consumer only needs the decision. Prefer allowlisted audit fields over raw request bodies.

The NIST Privacy Framework can structure privacy risk conversations, but it is not legal advice or a jurisdiction-specific compliance decision. [CLM-071]

Design auditability without creating a second exposure

An audit event should let an authorized investigator reconstruct the consequential decision path without turning the audit system into a copy of all operational data.

Useful event fields may include:

- stable event ID and timestamp/time source;
- actor/workload ID and authentication context reference;
- action and resource ID;
- tenant/region and environment;
- request/intent/correlation ID;
- policy/control/version decision;
- result/reason code;
- approval or exception reference;
- evidence/model/retrieval bundle reference where relevant;
- integrity, retention, and access metadata.

Avoid raw secrets, credentials, full documents, unrestricted prompts, sensitive free text, and entire payloads by default. Use references, classifications, hashes where appropriate, and narrowly approved enrichment. Restrict who can query/export audit data. Test tenant isolation, retention, clock behavior, event loss/duplication, redaction, and investigation usability.

More logging is not automatically more accountable. Excess content increases privacy and security exposure and can make useful signals harder to find. Audit collection must remain purpose-bound and minimized. [CLM-077]

Treat secrets and keys as operational systems

Do not place secrets in source, example configuration, test fixtures, tickets, prompts, logs, screenshots, or runbooks. A secret reference is not enough unless runtime identity, retrieval, rotation, revocation, audit, availability, and break-glass behavior are designed.

Record:

- secret/key owner and allowed workloads;
- environment and tenant/region scope;
- creation, distribution, storage, use, rotation, expiry, and revocation;
- deployment and rollback behavior during rotation;
- detection for unauthorized use or disclosure;
- recovery when the secret manager or identity dependency is unavailable;
- evidence that old material no longer authorizes access.

Never copy a real secret into a test to prove redaction. Use synthetic canaries and scanning. If a scanner finds a credential, treat rotation/revocation and exposure assessment as separate from removing the string.

Separate duties where consequence requires it

Separation of duties prevents one identity from initiating, approving, executing, and concealing a consequential action. Apply it proportionately:

- code author versus protected-branch/release approval;
- access requester versus approver;
- safety-step requester versus qualified approver;
- exception proposer versus accepting authority;
- audit producer versus evidence reviewer where independence is required.

Avoid cargo cult separation. If a team is small, the accountable owner may select compensating controls such as time delay, two-person review, immutable audit, limited scope, heightened monitoring, or post-action review. The FDE documents options and evidence; the owner decides.

Design break-glass as a different controlled path

Emergency access is not “admin without rules.” Define triggering conditions, eligible actors, approval where feasible, scope, duration, authentication independent enough to survive the incident, actions allowed, notification, recording, post-use review, credential rotation, and recurring exercise.

Test break-glass under the failure it is meant to survive. If the access system, control plane, DNS, network, or secret manager is down, a recovery path that depends on it is fictional. At the same time, do not weaken routine controls merely to simplify a rare emergency.

Integrate security into the development lifecycle

Security work begins before code and continues through support:

1. **Discovery:** obligations, assets, users, abuse/failure, authority, data/access minimization.
2. **Architecture:** trust/data/failure boundaries, identity, tenancy, region, secrets, recovery.
3. **Contracts:** validation, semantics, authorization, idempotent intent, error/reconciliation.
4. **Implementation:** secure defaults, dependency/source integrity, code/config review, tests.
5. **Verification:** positive/negative/boundary/failure/recovery checks in representative environments.
6. **Release:** evidence bundle, known gaps, decision owner, exposure/ramp/rollback.
7. **Operation:** monitoring, incident/support paths, access review, patch/change, evidence refresh.

8. **Retirement:** revoke access, dispose data, remove integrations, preserve required records.

NIST SSDF 1.1 supports integrating secure practices across the lifecycle instead of postponing security to a final gate. Cite the final version in use; do not silently substitute draft or later guidance. [CLM-070]

Use verification standards carefully

OWASP ASVS 5.0.0 provides versioned application-security verification criteria. Select relevant scope and level with security ownership; preserve IDs/version in mappings; connect each selection to implementation and evidence. “ASVS aligned” without scope, version, results, gaps, and authority is not a useful release claim. [CLM-072]

WCAG 2.2 provides testable web accessibility success criteria. Automated checks can find some failures, while keyboard use, focus order, semantics, screen-reader behavior, zoom/reflow, error recovery, cognitive load, and real task completion require additional inspection and user evidence. Conformance alone does not prove complete usability or accessibility for every person and context. [CLM-073]

ISO/IEC 25010:2023 records a product quality model useful for remembering that security, reliability, performance efficiency, interaction capability, maintainability, flexibility, and safety-related qualities can interact. This book does not reproduce the copyrighted requirements. The practical point is that one quality attribute cannot erase the others. [CLM-079]

Keep AI inside the same governance chain

AI does not receive a separate, weaker definition of evidence. Begin with the Chapter 9 decision record and add system threat paths:

- untrusted retrieved content influences output or tool choice;
- one tenant's content enters another tenant's context;
- sensitive data enters prompts, traces, graders, or provider retention;
- a model invents evidence, hides uncertainty, or suggests a prohibited action;
- a tool request exceeds identity/tenant/region/operation scope;
- evaluation data is poisoned, leaked, or unrepresentative;
- reviewer workload makes approval nominal;
- model/prompt/index/policy changes invalidate prior evidence;
- latency/cost exhaustion removes the safe fallback.

Controls live across data selection, identity, retrieval, model interface, output schema, deterministic policy, authorization, qualified review, tool execution, audit, evaluation, monitoring, and stop/rollback. Prompt instructions alone are not an enforcement boundary.

NIST AI RMF 1.0, its Generative AI Profile, and current community guides can prompt discovery. They are versioned and incomplete for a specific deployment. They do not replace the system threat model or create certification. [CLM-074]

For Orchid, AI is still limited to evidence summary/ranking. No model output accepts safety risk, approves a safety step, widens access, bypasses tenant/region boundaries, or orders a part.

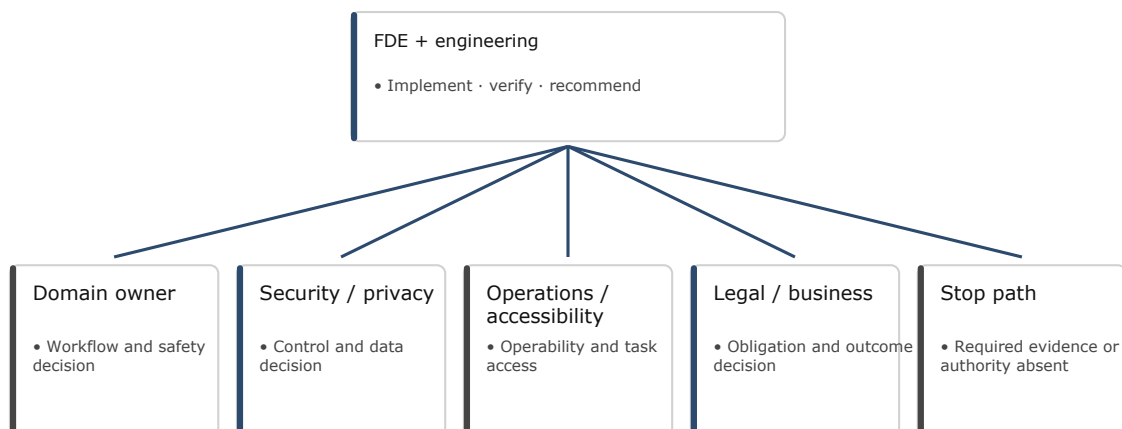
Map authority before an exception appears

Create an authority map for each consequential decision:

Decision	Evidence preparer	Consulted owner	Designated authority	FDE boundary
access scope	engineering/FDE	identity/security/data owners	named access authority	propose/test; cannot self-grant
regional data handling	engineering/FDE	privacy/legal/security	named organizational authority	map flow/gaps; no legal conclusion
safety workflow	engineering/FDE	domain/safety owner	designated safety/business authority	implement evidence path; no safety acceptance
security exception	engineering/FDE	control/system owner	security/risk authority	options/evidence; no self-approval
accessibility release	product/engineering	accessibility/user owner	designated release authority	test/remediate; no universal conformance claim
production exposure	delivery/product/operations	all relevant owners	named business/release authority	recommend cohort/stop; no unilateral risk acceptance

F10.2 · FORWARD DEPLOYED ENGINEERING

Evidence flows to authority



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F10.2 - Evidence flows to authority. The FDE prepares and recommends; designated authorities approve, reject, or request evidence.

Text alternative: Authority map routing FDE and engineering evidence to domain, security, privacy, operations, accessibility, legal, and business owners, with missing evidence returning and absent authority blocking exposure.

The map must name actual roles or groups in the customer context, delegation scope, quorum if relevant, response time, escalation, expiry, and substitute. A generic “security approved” field is weak if no accountable identity, scope, date, evidence bundle, or conditions are recorded.

Make exceptions expire

An exception record contains:

- exact control/risk/scope;
- reason and alternatives considered;
- evidence and uncertainty;
- affected users/data/environments;
- compensating controls;
- monitoring and incident trigger;
- accepting authority and delegation basis;
- decision date, expiry, review trigger;
- remediation owner and target;
- automatic restriction when expired where feasible.

“Temporary” without expiry and owner means permanent by accident. An ownerless exception is an unresolved blocker, not approval.

Complete Orchid's OA-05 governance section

The design dossier now records:

Threat and risk model

Assets: equipment binding, approved evidence, customer isolation, qualified authority, workflow state, secrets, audit integrity, availability/recovery.

Principal conditions: identity misuse, cross-tenant/region access, stale/malicious evidence, interface ambiguity, excessive privilege, secret exposure, model/tool misuse, log overcollection, reviewer failure, recovery dependency.

Risk entries preserve consequence, affected obligation, exposure, existing/proposed controls, verification, uncertainty, operating owner, and decision authority.

Control matrix

Every selected control has objective, scope, implementation, positive/negative/failure/recovery verification, evidence reference, owner, monitoring, gap, and formal disposition. Framework mappings are secondary metadata, not the control itself.

Data-handling record

Orchid accepts confirmed equipment/context and approved evidence references. It excludes unrestricted customer archives by default. Tenant/region boundaries apply to operational storage, retrieval, cache, audit, evaluation, support export, and backup. Retention/deletion and legal questions remain explicitly owner-routed.

Access and secrets

Human/workload identities are separate. Authorization binds resource, operation, tenant, region, qualification, and expiry. Support elevation is purpose/time bounded. Configuration contains secret references, never secret values. Rotation/revocation and dependency failure are tested.

AI controls

AI remains suggestion-only. Confirmed equipment, allowlisted evidence, deterministic policy, qualified approval, segmented evaluation, budget fallback, version bundle, and kill/rollback bound behavior.

Evidence plan

The plan includes contract tests, authorization denials, tenant/region isolation, environment/config/secret checks, dependency/static analysis, AI critical-segment cases, audit redaction, break-glass exercise, accessibility evidence, failure/recovery tests, and owner review.

The executable companion adds four tests: documentation-only control failure, evidence-pending-approval, designated/scope-bounded authority, and allowlisted/redacted audit events. These tests demonstrate the boundary; they do not certify a production system.

Run a control-evidence review

For each control, ask:

1. Is the risk/consequence specific enough to choose a control?
2. Is the objective stated independently of one tool?
3. Is enforcement actually in the path that matters?
4. Does verification include denied, boundary, dependency-failure, and recovery cases?

5. Does evidence identify version, environment, time, and limitations?
6. Can the operating owner detect and remediate failure?
7. Is the formal authority named and within delegated scope?
8. Does an exception expire and restrict exposure?
9. Could the control itself create availability, privacy, accessibility, or support harm?
10. What change would invalidate the evidence?

If the answer to 3 or 4 is no, the control is probably documentary. If 6 or 7 is no, the decision is ownerless. If 10 is unknown, evidence can silently become stale.

Common governance failures

Checklist theater

Symptom: every row is green because a document exists.

Repair: objective, implementation, verification, evidence, owner, disposition, and limitations.

Framework-name compliance

Symptom: “NIST/OWASP compliant” without version, scope, mapping, tests, gaps, or authority.

Repair: use versioned selections as lenses; remove certification language unless a qualified formal process supports it.

Security at the final gate

Symptom: architecture/data/access choices are already expensive to change.

Repair: lifecycle integration, incremental evidence, and explicit stop decisions from discovery onward.

Excessive access for delivery speed

Symptom: shared admin credentials or broad production copies become the default discovery path.

Repair: question-bound access, synthetic/minimized evidence, workload identities, time-bound elevation, revocation, audit.

Logging everything

Symptom: sensitive payloads/prompts/documents accumulate in a broadly searchable system.

Repair: purpose-based event schema, allowlist/redaction, access/retention/export controls, investigation usability tests.

Human approval theater

Symptom: reviewer lacks qualification, evidence, time, interface support, or authority.

Repair: qualification/scope checks, evidence-first UI, workload measurement, separation where needed, fallback/escalation.

Ownerless exception

Symptom: “business accepted” without a named delegated authority, expiry, or compensating control.

Repair: block exposure or route a complete decision record.

Break-glass that shares the broken dependency

Symptom: emergency recovery cannot authenticate or reach the control plane during the target failure.

Repair: scenario-based exercise, limited independent path, notification, post-use review, and rotation.

Exercise: repair five controls and route two decisions

Start with these vague statements:

1. “Use encryption.”
2. “Only authorized users can approve.”
3. “Keep logs for audit.”
4. “Follow OWASP for the AI.”
5. “Have a break-glass account.”

For each, produce:

- contextual risk and consequence;
- control objective and scope;
- implementation/enforcement point;

- positive, negative, failure, and recovery verification;
- evidence/version/environment/limitations;
- operating and remediation owner;
- formal decision authority;
- monitoring, expiry, and invalidation trigger.

Then route:

- a temporary regional logging exception;
- a production exposure decision with an unresolved break-glass exercise.

Pass only if the FDE prepares evidence and alternatives without fabricating approval. If the required authority is absent, record the decision as blocked or restrict the exposure. Do not convert silence into acceptance.

The Chapter 10 gate

Before implementation begins, OA-05 must include:

- workflow/system/AI threat and risk model;
- risk-to-control-to-implementation-to-verification-to-evidence chain;
- data-handling and minimization record across logs, traces, support, evaluation, and backups;
- human/workload identity, least privilege, tenant/region, secrets, and separation design;
- audit event purpose, schema, access, retention, redaction, and investigation path;
- secure development and representative verification plan;
- AI-specific threats connected to deterministic/system/human controls;
- break-glass and recovery boundaries;
- versioned framework mappings with gaps and no certification language;
- named operating owners and formal decision authorities;
- exception/expiry/escalation path;
- passing companion governance tests.

The gate passes when every material control can be traced to an implementation and verification plan, every open risk has an owner and disposition path, and the FDE's evidence role remains distinct from formal approval.

Chapter 11, *Build a Production Vertical Slice*, will build one vertical slice under these constraints. It will not postpone identity, failure behavior, observability, accessibility, rollback, or control evidence until after the happy path works.

Part III - Build Evidence Into the System

Part III changes the central question from `does it work?` to `which production claims does the current evidence support?`

Chapter 11 builds one small but consequentially complete vertical slice. It preserves truth across configuration, contracts, equipment identity, evidence, external finality, bounded suggestion, deterministic policy, qualified approval, audit, telemetry, failure, and support. Chapter 12 traces workflow risk and acceptance criteria into typed verification, evaluation, security/accessibility review, UAT, simulation, limitations, and release disposition. Chapter 13 carries those claims into promise-linked signals, cohorts, alerts, diagnosis, privacy-safe telemetry, support, capacity, cost, and runbooks. Chapter 14 binds the release identity and evidence to progressive exposure, compatible migration, actual-state recovery choice, tested restore, break-glass, and rehearsal.

The dossier progresses through OA-06, OA-07, and OA-08. The provider-neutral companion is intentionally synthetic: it makes behavior reviewable and reproducible, while preserving the real integration, identity, performance, accessibility, UAT, and production limitations it cannot prove.

11. Build a Production Vertical Slice

Implement one small, reviewable path across contracts, data, integrations, bounded AI, authorization, audit, telemetry, configuration, and controlled failure.

A vertical slice is small in breadth and complete in consequence.

It follows one user intent across the boundaries that will determine whether the deployment can become real: interface, data, identity, external dependency, policy or AI, human authority, audit, signals, configuration, failure, and support. It is not a thin screen over a thick pile of mocks. It is not every feature built shallowly. It is not a happy path renamed “production-grade.”

The purpose is to turn design assumptions into reviewable implementation evidence while the path is still small enough for one team to understand end to end.

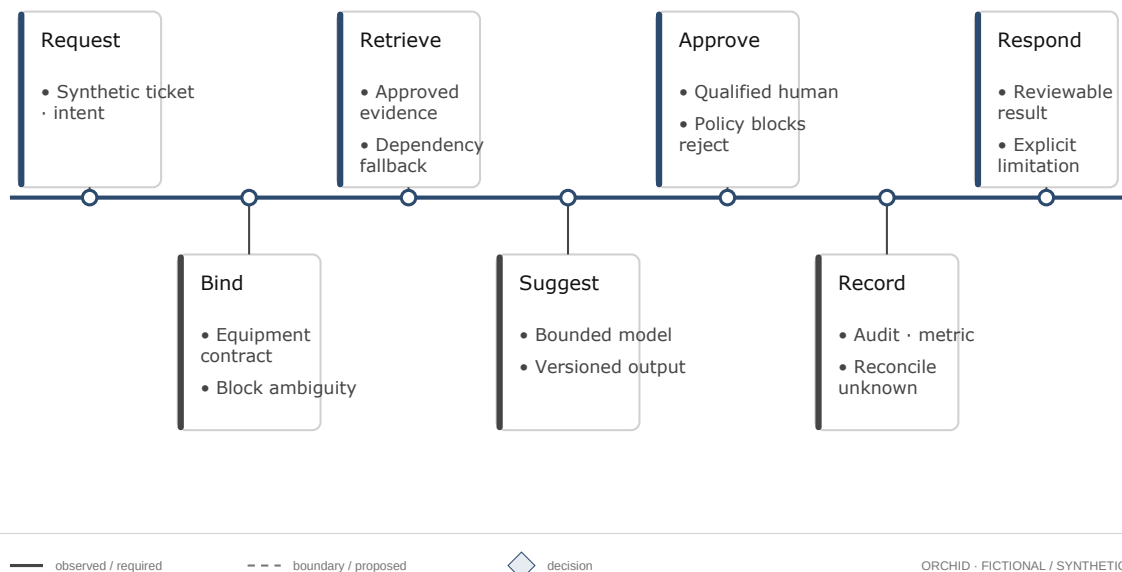
Define the slice by a user intent

For fictional Orchid Assist, the slice begins with a technician opening a synthetic ticket and ends with a bounded ranked inspection suggestion ready for qualified review. The system must:

1. accept a ticket with correlation and intent identifiers;
2. reconcile equipment candidates and stop on missing/ambiguous identity;
3. validate the confirmed equipment's structure and regional semantics;
4. retrieve approved evidence for the bound equipment/region;
5. query a synthetic inventory adapter with explicit completion state;
6. run a deterministic model double within budget;
7. apply deterministic policy outside the model;
8. require a qualified, tenant/region-scoped approval for the safety-relevant step;
9. emit a minimized audit event and inspectable metric;
10. return a visible state: ready for review, blocked, fallback, or reconciliation.

F11.1 · FORWARD DEPLOYED ENGINEERING

One consequential path, end to end



F11.1 - One consequential path, end to end. The slice crosses contracts, evidence, policy, approval, audit, metrics, and controlled failure.

Text alternative: End-to-end trace from synthetic ticket and intent through environment control, equipment reconciliation, evidence store, inventory adapter, bounded model, qualified approval, deterministic policy, audit, metric, and response.

That sentence is the implementation contract. It includes the failure states, not just the successful result.

Apply the vertical-slice coverage test

A candidate slice is representative only if it crosses the constraints that could invalidate the outcome.

Coverage dimension	Orchid evidence
real user intent/state	technician ticket to reviewable next step
representative interface	versioned input/output/error meanings
representative data semantics	equipment ID, tenant, region, provenance, freshness
external boundary	inventory adapter with completed/rejected/unknown behavior
identity/authorization	resource/operation/tenant/region/qualification/expiry
AI/policy	deterministic double plus policy and approval outside model
audit/telemetry	allowlisted event and correlation-safe metric
environment/config	validated configuration, no embedded secret, feature default closed
controlled failure	dependency failure, rejection, ambiguity, timeout/budget, unknown completion

Coverage dimension	Orchid evidence
support/recovery	reason code, trace, reconciliation state, owner path
review/test	clean command, automated positive/negative tests, known non-proof

If a row is not relevant, record why. Do not delete difficult rows until the slice appears complete.

This coverage test is an original FDE synthesis, not a formal standard. [CLM-081]

Keep the slice small enough to review

The vertical slice should minimize parallel novelty. Orchid uses:

- Node.js built-ins and ECMAScript modules;
- in-memory synthetic fixtures;
- a deterministic model double;
- a small injected evidence store and inventory adapter;
- pure functions for contracts, reconciliation, policy, authorization, and evidence;
- Node's built-in test runner;
- structured JSON output for inspection.

No paid service, secret, customer data, container, or provider account is required. This makes the path reproducible. It does not make it representative of production dependency latency, permissions, quotas, regional controls, failure distributions, or operational load.

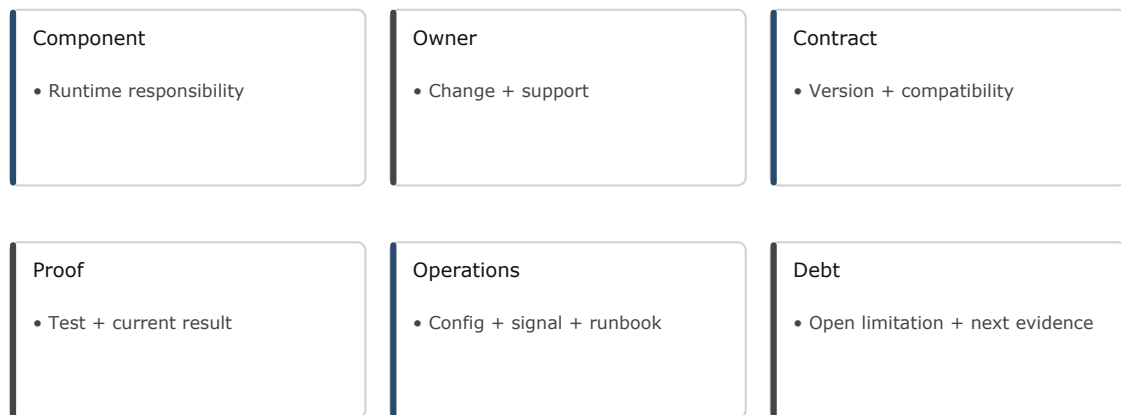
Choose realism deliberately. A local double is appropriate for deterministic contract/failure tests. A representative customer integration environment is required before claiming the real adapter works.

Make the repository explain ownership

For every component, record purpose, owner, contract, configuration, signal, failure behavior, test, runbook, and debt.

F11.2 · FORWARD DEPLOYED ENGINEERING

Code is not supportability



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F11.2 - Code is not supportability. Every runtime component connects to owner, contract, test, configuration, signal, failure, runbook, and debt.

Text alternative: Supportability matrix for orchestrator, equipment contract, evidence store, inventory adapter, model, policy, authorization, audit, metrics, and feature configuration across owner, test, signal, runbook, and debt.

The companion's map is intentionally compact:

Component	Purpose	Failure state	Evidence
vertical-slice.mjs	orchestrate one intent	blocked/fallback/reconcile	end-to-end tests and trace
contracts.mjs	structural/semantic equipment validation	blocked	positive and regional negative tests
reconciliation.mjs	equipment and external completion truth	missing/ambiguous/unknown	state tests
environment.mjs	validate environment/secret-free config	blocked	configuration tests
synthetic-fixture.mjs	reproducible local dependencies/data	non-production proof only	fixture review
model-interface.mjs	bounded replaceable suggestion interface	abstain/fallback	budget/abstention tests
policy.mjs	deterministic control ladder	block/require approval	policy tests
authorization.mjs	scoped permission/qualification	denied	mismatch/expiry tests
control-evidence.mjs	evidence, authority, minimized audit	incomplete/pending	governance tests

A filename is not ownership. The production dossier names the customer/product/platform/support owners and change path. The map exists so a reviewer can find where behavior lives and who must respond when it fails.

Implement contracts before orchestration

The slice reuses Chapter 7's contract and intent foundations. The equipment record requires:

- an OES- identifier pattern;
- supported region and lifecycle status;
- observation timestamp and source version;
- west-region data in west residency;
- semantic validation beyond JSON shape.

OpenAPI 3.2.0, AsyncAPI 3.0.0, and JSON Schema Draft 2020-12 can describe structures and make selected properties machine-testable. They do not establish workflow semantics, correct authorization, safe retry, reconciliation, support, or operations. [CLM-082]

In production, keep interface descriptions versioned near implementation and test providers/consumers against agreed examples and errors. In the companion, the small pure functions expose the same separation without generating a large specification artifact for a single local path.

Return states that preserve truth

The orchestrator returns four top-level states:

- ready-for-review: the bounded path completed and policy permits presentation;
- blocked: a precondition/control/dependency response makes continuation inappropriate;
- fallback: an expected dependency/model limitation requires the non-AI/manual path;
- reconcile: an external boundary may have accepted work but final completion is unknown.

Do not collapse them into success/error. Support, retry, UI, monitoring, and user guidance differ.

An HTTP timeout after an accepted request does not prove failure. Method-level idempotency does not preserve application intent by itself. The intent identifier, semantic fingerprint, completion record, reconciliation query, and late response behavior must agree. [CLM-083]

Bind configuration before dependency access

The slice validates environment, tenant, region, data residency, and release version before using stores or adapters. It rejects configuration that embeds secret values. The bounded-suggestion feature must be explicitly enabled; the default is closed.

This order prevents a disabled or invalid path from touching dependencies “just to see what happens.” It also makes the trace show which gate made the decision.

Configuration remains distinct from code and secret material, but code validates the configuration's meaning. A syntactically correct region string can still violate residency or ownership requirements.

Treat feature controls as lifecycle artifacts

A feature flag is not a safety control by name. Record:

- owner and purpose;
- default and fail state;
- environment/tenant/region/cohort scope;
- prerequisites and incompatible flags;
- signal and support visibility;
- approval required to widen exposure;
- rollback/disable behavior;
- expiry/removal condition;
- tests for both states and material combinations.

Without these fields, a flag can create divergent paths, stale code, support confusion, and accidental exposure. [CLM-088]

Orchid's local feature has one purpose: include or exclude the bounded suggestion path. When false, orchestration stops before equipment/evidence/inventory access. A production flag would additionally require external configuration ownership, audited change, cohort visibility, and removal.

Reconcile identity before retrieval

The ticket can contain zero, one, or multiple active equipment candidates. The slice:

- blocks missing and routes to evidence/customer correction;
- blocks ambiguous and asks for confirmation;
- continues only on confirmed;
- then validates the selected record's semantics.

This is deliberately before evidence retrieval. Otherwise a system can retrieve a real manual for the wrong real equipment and produce a convincing wrong answer.

The trace includes the state but not the full ticket payload. The output exposes only identifiers/evidence needed by the review path.

Inject dependencies at explicit boundaries

The evidence store and inventory adapter are injected objects with narrow methods:

- `findApproved({ equipmentId, region });`
- `check({ intentId, equipmentId, tenant, region }).`

This shape supports deterministic testing and future provider adapters. It also makes contract review possible: inputs, outputs, errors, timeouts, identity, authorization, and completion meaning are visible.

State what the doubles do not prove

The synthetic evidence store proves only that the orchestrator:

- binds lookup to equipment and region;
- stops when approved evidence is absent;
- falls back on dependency failure;
- passes evidence identifiers rather than unrestricted content to the double.

It does not prove the real store's authorization, tenant isolation, freshness, indexing, availability, latency, retention, audit, or source approval.

The inventory double proves completed/rejected/unknown branching. It does not prove real inventory semantics, reservation behavior, capacity, concurrency, rate limits, credentials, or reconciliation APIs.

A test double's passing result is evidence about the code and encoded double, not the real dependency.
[CLM-087]

Demonstrate upstream success followed by downstream rejection

In one test, approved evidence is found and then inventory rejects the request or reports unavailable. The orchestrator returns `bLocked`, does not call the model, and does not produce suggestions.

This matters because “upstream succeeded” can tempt a workflow to continue with partial confidence. The correct state is determined by the full intent and control chain.

In another test, the adapter says it accepted the request but no response/final state arrived. The orchestrator returns `reconcile` with the original intent ID. It does not guess or blindly retry.

Keep model, policy, and authority separate

The deterministic model double receives a case ID and approved evidence IDs. It can suggest, abstain, or fail to a visible budget fallback. It never receives authority.

The deterministic policy then evaluates:

- confirmed equipment;
- suggestion state;
- prohibited-action marker;
- whether a safety-relevant step has qualified approval.

Authorization evaluates the principal against the safety-approval resource, operation, tenant, region, qualification/permission, and expiry. OAuth-specific designs, if later introduced, must follow the recorded flow and applicable RFC 9700 practices; the local double does not pretend to implement a full identity protocol.

The model is one replaceable component. Contracts, access, evidence, policy, audit, operation, and failure remain system responsibilities. [CLM-085]

Review the human path too

The companion stops at ready-for-review; it does not implement a production UI. OA-06 records UI requirements:

- confirmed equipment and ticket context;
- evidence identifiers/version/freshness;
- suggestion visibly distinct from fact;
- policy and approval state;
- uncertainty/fallback/reconciliation state;
- allowed review actions;
- no default approval or hidden evidence;
- keyboard/focus/error/accessibility behavior;
- correlation/support reference.

A backend test cannot prove that the user can perform the task. Chapter 12 will add task-based UAT and selected accessibility evidence.

Emit evidence, not sensitive exhaust

The slice emits:

- a trace of named steps and outcome codes;
- one completion metric with correlation ID, state, and reason;

- one allowlisted audit event for the consequential policy result.

It does not log the raw ticket, manual content, model prompt, secret, or free-text note. This makes the trace inspectable while preserving Chapter 10's minimization boundary.

The trace is local evidence, not full production observability. Chapter 13 will define latency, dependency, workflow, segment, quality, cost, security, and support signals plus retention/cardinality/privacy constraints.

Use a review lens that crosses concerns

Review each change through seven lenses:

Contract

- Are structural and semantic rules explicit?
- Are error and completion states stable?
- Are compatibility/version/migration needs recorded?

Failure

- What happens on missing, ambiguous, denied, timeout, exception, partial success, duplicate, late response, and recovery?
- Is the user/system state truthful and actionable?

Security and governance

- Are identity, resource, tenant, region, operation, qualification, secrets, data, audit, and authority enforced at the right boundary?
- Which control evidence and owner disposition result?

Observability

- Can an operator find the request and decision without sensitive payloads?
- Does the signal distinguish blocked, fallback, reconcile, and ready?

Operation

- Who deploys, configures, supports, disables, reconciles, recovers, and removes the path?
- Which dependency/runbook/alert must exist?

Test evidence

- Which risk/requirement is each test challenging?
- What does the fixture/environment fail to prove?

Ownership and debt

- Is every compromise named, bounded, owned, time-limited, and connected to an exit trigger?

Secure development and application verification belong inside implementation/review, not after functional completion. [CLM-084]

Debug across boundaries, not by component reputation

When the slice fails, begin with the externally visible state and correlation identifier. Follow evidence in sequence rather than assuming the newest or most complex component is responsible.

Symptom: no suggestion is shown

Ask:

1. Was the feature/environment gate accepted?
2. Was equipment missing, ambiguous, or semantically invalid?
3. Did approved evidence exist for the bound equipment and region?
4. Did inventory complete, reject, or become unknown?
5. Did the model suggest, abstain, time out, or exceed budget?
6. Did deterministic policy block a prohibited/missing state?
7. Did qualified approval succeed for the correct tenant/region and current permission?
8. Did the UI render the returned state and support reference correctly?

“The model failed” is not a diagnosis when the system correctly blocked ambiguous equipment before calling it. “The API failed” is not a diagnosis when an accepted request has unknown completion and requires reconciliation.

Symptom: a suggestion appears for the wrong context

This is more consequential. Preserve the trace and version bundle, restrict the feature/cohort if needed, and test:

- ticket-to-equipment binding;
- cache/search tenant and region keys;
- evidence source/provenance/freshness;
- model input construction;
- policy/equipment confirmation;
- UI display of selected context;
- audit/correlation consistency.

Do not repair only the final prompt. The fault may be retrieval, identity, stale data, UI state, or cache isolation.

Symptom: retries increase load

Separate new user demand from repeated attempts. Inspect timeout location, client retry policy, intent reuse, server completion state, downstream capacity, and reconciliation availability. A frontend polling loop can become a backend incident even when each request is individually valid. The UI belongs inside system performance and failure reasoning.

Record the diagnostic result

A useful debug record contains:

- symptom and user/cohort consequence;
- correlation/intent/release/config identifiers;
- first known good and first known bad boundary;
- evidence inspected and access constraints;
- competing hypotheses and falsification results;
- containment and recovery state;
- defect/control/debt classification;
- owner, follow-up test, and runbook update.

This turns one incident into reusable product/operating knowledge without copying sensitive payloads into a ticket.

Review change size and seams

A vertical slice can become unreviewable if it arrives as one enormous change. Sequence implementation so each checkpoint preserves a testable contract:

1. pure contracts and reconciliation states;

2. environment/configuration gates;
3. injected adapter interfaces and deterministic fixtures;
4. model/policy/authorization composition;
5. audit/metric/result state;
6. end-to-end positive and negative paths;
7. representative adapters/UI/exporters behind reviewed seams.

Do not merge knowingly insecure intermediate behavior into an exposed environment merely because a later change will add the control. Keep incomplete paths unreachable by default.

At each seam review:

- whether the abstraction represents a stable responsibility or only hides uncertainty;
- whether errors preserve dependency truth;
- whether timeouts/cancellation/retries/late results are defined;
- whether the adapter receives the minimum identity/data;
- whether test fixtures model the intended semantics, not just convenient output;
- how real integration evidence will replace the double's non-proof;
- how a future implementation can be removed without rewriting domain logic.

A provider-neutral interface is useful only if it preserves the consequential differences that callers need. An over-generic `execute(input)` method can erase intent, authorization, completion, and failure semantics.

Define safe defaults at construction time

Safe defaults reduce the chance that a new environment, tenant, or operator accidentally widens exposure. For Orchid:

- bounded suggestion defaults disabled;
- missing evidence causes fallback, never invention;
- ambiguous equipment blocks retrieval/action;
- unknown completion requires reconciliation;
- safety-relevant work requires current qualified approval;
- secret values are rejected from configuration;
- audit uses an allowlist;
- no provider adapter is selected implicitly;
- synthetic fixtures are visibly named and cannot be confused with customer data.

Defaults are not enough. Test configuration drift, explicit override, environment promotion, failure of the configuration source, and visibility to support. A silent default that changes between environments creates a different system.

Keep migrations and compatibility visible even in the first slice

The current companion has in-memory state, so it does not execute a schema migration. OA-06 still identifies migration seams:

- equipment/source version changes;
- ticket/status vocabulary changes;
- intent-ledger retention/record format;
- policy/permission identifier changes;
- audit schema evolution;
- model/evidence bundle version changes.

For each future persistent change, define old/new readers and writers, compatibility window, backfill or dual-write decision, validation, rollback/roll-forward limits, and recovery evidence. “We can change the database later” is not a vertical-slice plan if the first customer data makes the shape irreversible.

Chapter 14 will execute migration and recovery fixtures. Chapter 11 creates the seams and version identifiers so that work is possible.

Make debt a decision

The slice intentionally carries debt:

Debt	Why accepted now	Consequence/non-proof	Exit trigger	Owner
in-memory stores	deterministic local run	no persistence/concurrency/integration proof	representative adapter test	integration owner
deterministic model	no paid dependency/uncertainty	no provider/model quality proof	bounded provider evaluation decision	AI owner
JSON trace only	inspectable local evidence	no production telemetry pipeline	Chapter 13 signal adapter	operations owner
no production UI	focus on orchestration	no task/accessibility/UAT evidence	review interface test	product owner
local timing	reproducible behavior	no capacity/latency evidence	representative load/soak plan	platform owner

An unowned TODO is not a debt decision. Preserve reason, consequence, owner, trigger, and exit condition. [CLM-090]

Generated code, if used later, receives the same review, tests, source/license/security checks, ownership, and maintenance expectations as human-written code. “The tool generated it” is not provenance sufficient for release.

Reproduce from a clean clone

The companion supports:

```
npm run test:companion
npm run demo:companion
```

The demo emits deterministic synthetic JSON. The tests cover contracts, authorization, environment, intent/reconciliation, bounded AI/evaluation, governance, and the end-to-end slice.

Automated setup and consistent release processes reduce manual variance. They do not replace representative verification, review, approval, or recovery. [CLM-086]

A clean local path is valuable reproducibility evidence and onboarding/support infrastructure. It is still separate from evidence in a customer-like environment. [CLM-089]

Trace the slice as a reviewer

Use one synthetic intent and read the trace in operational order:

1. Configuration is parsed and rejected if a secret field or wrong residency appears.
2. The feature control defaults closed; no dependency call occurs until explicitly enabled for the synthetic scope.
3. The ticket's equipment candidates remain missing, confirmed, or ambiguous rather than being coerced into a match.
4. Contract validation checks structural and semantic equipment/region rules.
5. Evidence and inventory adapters receive only the confirmed, authorized context.
6. Unknown inventory completion preserves the intent and enters reconciliation rather than blind retry.
7. The deterministic model double suggests only when approved evidence exists and budget permits; otherwise it abstains/falls back.
8. Deterministic policy and qualified approval remain separate decisions.
9. The final state, minimized metric, and audit event share safe correlation identifiers.

At each step ask: what input was trusted, what state can emerge, what external effect can occur, what evidence is emitted, and who owns the next action? A function-level review can miss a semantic break between individually correct components.

Run the consequential failure drill

Execute the happy path once to establish the trace shape. Then inject:

- ambiguous equipment before evidence access;
- successful evidence retrieval followed by inventory rejection;
- missing response after external inventory acceptance;

- expired or wrong-region authorization;
- model budget exhaustion;
- prohibited policy result;
- feature disabled at construction.

For each injection, record expected and observed state, dependencies contacted, external effects, audit/metric evidence, user/fallback behavior, owner, and non-proof. The test passes only when failure remains truthful and bounded. Error returned is insufficient if an external effect may already have happened or sensitive work continued after a failed boundary.

Review supportability from a clean checkout

Give the repository to a reviewer who has not run it. They should locate setup, commands, module boundaries, configuration schema, synthetic fixtures, tests, failure injections, evidence output, and debt without private instructions.

Record every hidden step: assumed runtime, global package, missing environment value, undocumented port, stale command, secret request, or unclear owner. Repair the repository or add an explicit prerequisite. The original author succeeding from a warmed environment does not prove reproducibility.

This exercise also tests the repository map. If the reviewer cannot identify where a contract, policy, adapter, test, signal, or debt is owned, the structure does not yet support handoff.

Failure modes and repairs

Happy path becomes production proof

Repair: inject consequential failures, preserve explicit states, and record the real integration/UI/environment/performance evidence still missing.

Mock becomes the dependency contract

Repair: state what the double encodes and does not prove; validate representative adapter semantics and failures later. [CLM-087]

Orchestration hides contract meaning

Repair: implement/validate semantic contracts first and keep intent, finality, error, authorization, and reconciliation visible.

Feature flag becomes permanent architecture

Repair: owner, default, scope, interaction tests, evidence, expiry, and removal. [CLM-088]

Local setup depends on the author

Repair: clean-checkout reproduction with documented runtime, deterministic fixtures, no secrets, and inspectable failures. [CLM-089]

TODO hides accepted risk

Repair: debt reason, consequence, owner, trigger, exit, and release effect. [CLM-090]

Production quality is a portfolio of evidence

Functional correctness competes and cooperates with reliability, security, performance, cost, usability/accessibility, maintainability, operational support, and recovery. Provider architecture guidance and ISO's quality-model record help enumerate concerns, but no provider checklist decides Orchid's tradeoffs. [CLM-080]

For example:

- stricter authorization can increase dependency/recovery complexity;
- richer audit can increase privacy/cost exposure;
- aggressive retries can increase downstream overload and duplicate work;
- a fast model can reduce review time while increasing unsupported suggestions;
- a feature flag can reduce exposure while multiplying configuration paths;
- a sophisticated integration can increase realism while reducing local reproducibility.

Record the tradeoff, evidence, owner, and stop/exit condition. Do not hide it in “non-functional requirements.” These qualities determine whether the function produces a supportable outcome.

Complete OA-06

The vertical-slice artifact now contains:

- synthetic ticket/equipment/evidence/inventory fixtures;
- environment and feature-control record;
- equipment contract and reconciliation;

- evidence and inventory adapters;
- deterministic model interface and budget fallback;
- policy and qualified approval;
- minimized audit and metric;
- end-to-end trace and explicit states;
- repository/supportability map;
- review lens and debt register;
- clean run/test commands;
- known non-proof for every double.

It also records future production seams: real stores/adapters, workload identity, external config/secret manager, review UI, telemetry exporter, support/reconciliation API, representative environment tests, and release pipeline.

The Chapter 11 gate

Before expanding the feature surface, the slice must:

- cross one real user intent end to end;
- validate environment/config before access;
- preserve tenant/region/equipment binding;
- use explicit contracts and completion meanings;
- stop early on missing/ambiguous/denied states;
- demonstrate downstream rejection after upstream success;
- route unknown completion to reconciliation;
- keep model, policy, authorization, and authority separate;
- emit minimized audit/trace/metric evidence;
- default the feature closed;
- run from a clean clone without secret/paid service/customer data;
- pass positive and consequential negative tests;
- state what every double does not prove;
- link owners, runbooks, and debt exit conditions.

The gate passes when a reviewer can trace one intent across every consequential boundary, reproduce its safe failure behavior, and identify the evidence still required before production.

Chapter 12, *Prove Behavior Before Production*, will connect every material risk and acceptance claim to the cheapest credible verification method, representative data, observed result, limitation, and release disposition.

12. Prove Behavior Before Production

Trace workflow risk and acceptance claims to typed verification evidence, representative segments, calibrated graders, task-based UAT, limitations, and release disposition.

A test is useful only in relation to a claim.

“The suite passed” does not say which risk was challenged, which environment and system version ran, which data or users were represented, what was observed, what remains unknown, or who can release the system despite a gap.

Verification turns a deployment promise into decision-quality evidence:

requirement or risk -> criterion -> method and data -> expected result ->
observed result -> limitation -> owner disposition

The FDE builds this trace across the whole workflow. The FDE does not convert finite test passage into proof of safety, security, accessibility, quality, compliance, or absence of failure. [CLM-093]

Use the cheapest credible evidence

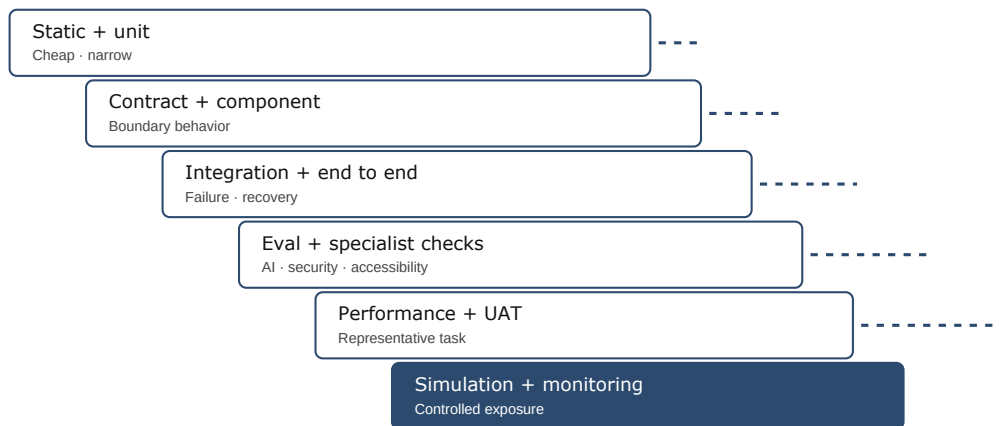
Choose the least expensive method that can credibly support the specific claim. “Cheapest” does not mean weakest. “Credible” depends on consequence and uncertainty.

Examples:

- A pure function mapping unknown completion to reconciliation can be verified with a deterministic unit test.
- Consumer/provider compatibility may require contract tests plus representative integration behavior.
- Tenant isolation requires negative authorization tests at the enforcement boundary, not a UI screenshot.
- Recovery requires an executed rehearsal from a representative state, not code review alone.
- AI quality needs datasets, graders, segments, versions, and uncertainty, not one example.
- Review-interface usability needs representative task observation, not only DOM assertions.
- A real dependency's quotas, latency, identity, and failure behavior need evidence in a representative environment; a local double cannot provide it.

F12.1 · FORWARD DEPLOYED ENGINEERING

Use the cheapest credible evidence



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F12.1 - Use the cheapest credible evidence. Higher, more representative layers complement rather than replace lower, faster checks.

Text alternative: Verification stack from static review and unit tests through contracts, integration, end-to-end failure, AI evaluation, security and accessibility, performance, UAT, simulation, and controlled production monitoring.

This selection rule and traceability matrix are original FDE synthesis tools, not a formal standard.
[CLM-091]

Build OA-07 as a verification matrix

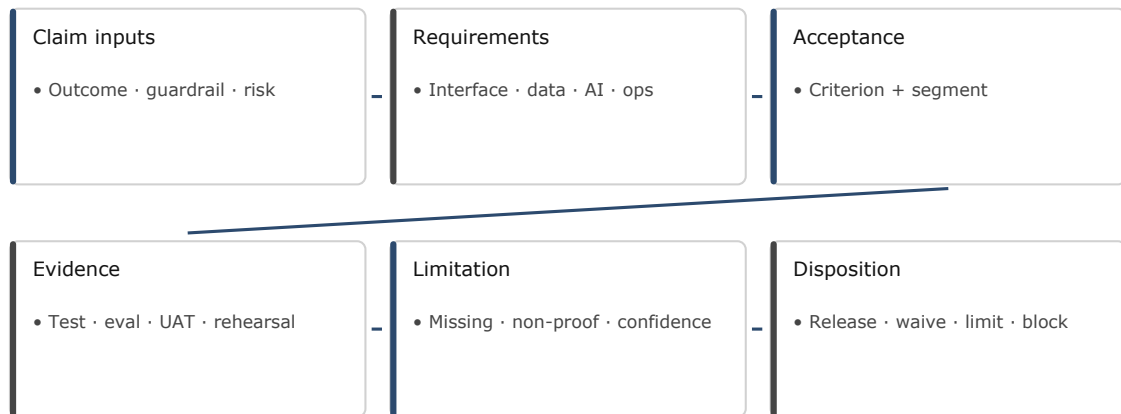
Each row contains:

- requirement, risk, control, or acceptance-criterion ID;
- exact criterion and consequence of failure;
- evidence type and method;
- data/fixture/user/environment provenance;
- system/config/policy/model/evidence versions;
- expected result and threshold owner;
- observed result, date, and evidence location;
- segment/cohort and uncertainty;
- known limitation/non-proof;
- pass/fail/inconclusive state;

- release disposition, owner, authority, waiver/expiry where relevant.

F12.2 · FORWARD DEPLOYED ENGINEERING

Every release claim needs a trace



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F12.2 - Every release claim needs a trace. Requirements flow to evidence, limitations, and a named release or waiver disposition.

Text alternative: Traceability map from outcomes, guardrails, risks, interface, data, AI and operating requirements through test, evaluation, UAT, simulation and rehearsal evidence to limitations and release disposition.

Do not write one row called “security testing” or “AI accuracy.” Decompose until a failed observation produces a meaningful action.

Example matrix rows

Risk/claim	Criterion	Method/type	Current observation	Limitation/disposition
wrong equipment drives evidence	ambiguous equipment never reaches retrieval	end-to-end deterministic assertion	blocked before store call	synthetic fixture; candidate
external completion is misread	accepted/no response becomes reconciliation	contract/end-to-end deterministic assertion	reconcile with intent ID	no real adapter; candidate only locally
prohibited suggestion reaches user	prohibited output is blocked for every critical case	deterministic policy plus segmented eval	local cases blocked	provider/adversarial coverage incomplete; no production release
unqualified safety approval	wrong tenant/region/expiry/per mission denied	authorization negative tests	local denials pass	production identity/qualification mapping absent
audit leaks payload	event contains allowlisted fields only	deterministic assertion/static review	secret/raw payload absent	exporter/storage/access/retention untested
user cannot inspect evidence	representative users complete review task	task-based UAT/user acceptance	not yet observed	release of safety-relevant path blocked

The last row is not a failed automated test. It is missing user evidence with release consequence. “Not tested” must remain distinct from passed.

Define criterion before running

Precommit the expected behavior, threshold, stop/waiver rule, and authority before seeing the result. Otherwise teams can move the target after an inconvenient failure.

For a statistical criterion, record numerator, denominator, exclusions, confidence/uncertainty method where meaningful, critical segment floors, and minimum sample adequacy. For a deterministic invariant, define exactly which versions and environments it covers. For expert judgment, define qualification/rubric/disagreement handling. For UAT, define task completion and unacceptable assistance or error.

Threshold selection is a risk/business/domain decision supported by evidence. The FDE can show tradeoffs and sensitivity but should not invent tolerance.

Keep evidence types distinct

The companion recognizes four types:

- **deterministic assertion:** for a fixed input/version/environment, a defined property held or failed;
- **statistical estimate:** observed rate/distribution with sample, segments, uncertainty, and assumptions;
- **expert judgment:** qualified reviewer decision under rubric and stated evidence;
- **user acceptance:** representative user completed or failed a defined task under recorded conditions.

These support different claims. Do not average them into one “confidence” score. [CLM-103]

Derive tests from risk and workflow state

Start from the workflow map, outcome contract, architecture, contracts, AI error taxonomy, and control matrix. For Orchid, the critical families are:

Binding and data

- missing/ambiguous/retired equipment;
- correct structure but wrong regional residency;
- stale/conflicting/missing manual;
- wrong equipment evidence returned by cache/index;
- source version changes;

- incomplete/duplicate/out-of-order data;
- cross-tenant/region query attempts.

Intent and dependencies

- same intent/same semantics replay;
- same intent/different semantics conflict;
- upstream success/downstream rejection;
- accepted request/no final response;
- late success after timeout;
- dependency timeout/unavailable/malformed response;
- rate/capacity constraint and retry amplification.

Identity and controls

- missing/expired identity;
- wrong tenant, region, resource, operation, or qualification;
- requester equals approver where separation is required;
- secret in configuration/log;
- feature disabled or wrong cohort;
- audit event missing or over-collecting;
- approval evidence complete but formal decision pending.

AI and review

- supported suggestion with correct evidence;
- unsupported statement or fabricated citation;
- prohibited/safety action;
- missing evidence requiring abstention;
- untrusted retrieved instruction;
- tool request outside permission;
- timeout/cost fallback;
- aggregate quality hiding a critical family failure;
- reviewer unable to inspect evidence or detect wrong context.

Operation and recovery

- signal/alert/runbook path for each material state;
- migration compatibility and partial deployment;
- rollback impossible after data change;
- backup present but restore invalid;
- break-glass dependency unavailable;
- support cannot identify release/config/policy bundle.

Chapter 12 records the method and current result; Chapters 13-14 build remaining signal/recovery evidence.

Layer the verification stack

No single layer dominates every claim.

Static analysis and review

Find suspicious source/config/dependency patterns, secret material, unreachable branches, unsafe defaults, and unclear ownership early. Static tools can produce false positives/negatives and cannot establish runtime semantics.

Unit and property tests

Challenge pure contracts, state transitions, authorization predicates, redaction, budgets, and invariants quickly. Add generated boundary inputs where useful. They do not prove adapter/environment behavior.

Contract tests

Verify provider/consumer expectations, examples, errors, compatibility, and semantic additions beyond schema. Test both sides when possible. A shared mistaken contract can still pass.

Integration/component tests

Exercise real protocol/identity/configuration and representative dependency behavior. Include denied, timeout, quota, partial, malformed, duplicate, and late-response paths. A test environment may still differ materially from production.

End-to-end tests

Trace representative user intent across components and controls. Keep a small consequential set: broad E2E suites can become slow, flaky, opaque, and expensive. Use lower layers for precise coverage.

Failure, load, and recovery evidence

Inject dependency impairment, concurrency, backpressure, retry amplification, capacity pressure, migration state, and recovery actions. Measure user-visible state and containment, not merely process survival.

Evaluation, security, accessibility, and UAT

Apply their specialized data, rubric, version, qualification, and user-context requirements. They complement the engineering stack rather than becoming an end-stage stamp.

When a high-level test fails, add a lower-level regression near the defect plus preserve the high-level case when it represents a consequential promise.

Control flakiness without erasing failure

Repeatedly rerunning until green destroys evidence. For each nondeterministic failure:

- preserve seed/input/system/environment and raw result;
- classify product defect, test defect, environment defect, or unknown;
- measure recurrence and segment;
- quarantine only with owner, consequence, expiry, and release rule;
- create a deterministic reproduction or improved diagnostic where possible;
- do not exclude the case from the aggregate without disclosure.

AI outputs may be probabilistic, but policy and authorization invariants can still be deterministic. For statistical behavior, run recorded trials, report distributions and failure classes, and retain critical failures instead of averaging them away.

Verify data and answer correctness separately

A query can execute successfully and still answer the wrong question. A retrieval can return a valid document and still bind the wrong equipment/version. Verify:

- source authority and provenance;

- selection/filter/join/binding semantics;
- freshness/completeness/consistency rules;
- missing/duplicate/conflicting states;
- transformation/aggregation meaning;
- tenant/region access;
- answer/evidence relationship;
- user interpretation and action.

ISO/IEC 25012's official record supports use of a structured data-quality model, but Orchid's concrete criteria and thresholds remain local. The book does not reproduce copyrighted requirements.

Plan representative performance evidence

Local execution time is not a capacity claim. Define workload from workflow evidence:

- arrival rate, concurrency, bursts, request sizes, cohort/site patterns;
- dependency quotas and connection pools;
- model/retrieval latency/cost distribution;
- timeouts, retries, cancellation, backpressure;
- warm/cold/cache conditions;
- data volume and migration/maintenance overlap;
- acceptable user-visible latency and fallback;
- capacity owner and scaling/stop action.

Measure tails and segments, not only averages. Verify that overload fails safely: reject/defer/fallback with truthful state rather than queueing invisibly, duplicating effects, or exhausting a shared dependency. Do not extrapolate production capacity from a laptop benchmark.

Test the contracts and their semantics

OpenAPI, AsyncAPI, and JSON Schema artifacts can anchor structural compatibility. Add semantic assertions for:

- resource/tenant/region binding;
- authorization and qualification;
- accepted/rejected/unknown completion;
- request intent equivalence/conflict;
- ordering, duplication, freshness, and reconciliation;
- error visibility and retry guidance;

- data-quality and provenance rules.

Schemas cannot prove that a valid equipment ID belongs to the ticket or that a timestamp is fresh enough for the decision. HTTP method semantics cannot make an external business effect safe to retry without application intent and completion rules. [CLM-092]

Use producer/consumer examples, negative cases, compatibility checks, and representative adapter tests. Freeze the contract version with the evidence result.

Design the AI evaluation as a system test

An evaluation needs explicit anatomy:

- **task**: bounded workflow behavior;
- **case/trial**: one context/input/evidence/permission state;
- **dataset**: versioned set with provenance and segments;
- **grader/rubric**: how acceptable/prohibited behavior is judged;
- **trace/outcome**: intermediate actions and final workflow result;
- **harness**: system bundle and execution/recording logic;
- **suite**: decision-relevant collection;
- **purpose**: capability, regression, release, diagnosis, or monitoring.

Portable concepts matter more than a vendor evaluation UI. [CLM-094]

Separate capability from regression

Capability asks whether an approach can satisfy the task under defined conditions. Regression asks whether a change preserves accepted behavior. [CLM-095]

An exploratory case can find a possible design. It should not automatically become the only release case. Curate validated cases, held-out and rotating variants, adversarial cases, stable high-consequence regressions, and new production-derived cases under permission/minimization.

Freeze the system bundle

Record:

- code/release and environment/config;
- model/provider/version/parameters;
- prompt/system instructions;
- retrieval corpus/index/chunking/ranking;

- tools/adapters and permissions;
- policy/authorization/approval;
- dataset and graders;
- latency/cost budgets;
- time and dependency versions.

A changed index or policy can invalidate a “same model” score.

Govern evaluation data

For every dataset/case record:

- source and permission;
- synthetic, constructed, public, customer-approved, or production-derived status;
- collection/construction procedure;
- version/change/correction history;
- development/validation/test/held-out/monitoring use;
- segment and consequence;
- leakage/tuning exposure;
- sensitive-data handling/retention;
- representativeness and known omissions.

The companion uses synthetic cases only. Production traces are not a free dataset. They may include customer content, identities, sensitive decisions, and model/provider retention issues.

Leakage changes the claim. A seen/memorized case can still be useful for regression but is weak independent capability evidence. [CLM-097]

Preserve segments and uncertainty

Suppose 99 common cases pass and one safety-critical equipment-family case fails. The aggregate is 99 percent. The release disposition is still blocked if the owner precommitted that critical failure as unacceptable.

Segment by legitimate decision factors such as:

- workflow consequence;
- equipment family;
- evidence quality/freshness;
- normal/exception/adversarial/fallback state;

- user role/qualification;
- tenant/region/site/connectivity;
- language or accessibility context;
- model/policy/version;
- latency/cost band.

Do not slice so finely that every cell hides uncertainty. Report sample counts, failure counts, disagreement, missing coverage, and uncertainty. Avoid false precision when examples are constructed or few.

An aggregate can conceal unacceptable segment failure. [CLM-096]

The companion verification report keeps common and safety-critical segments separate and blocks a critical failure even when most rows pass.

Calibrate graders

Deterministic graders

Use for exact invariants: schema, required evidence ID, prohibited action absent, policy state, permission, latency/cost budget, audit redaction, tenant/region binding.

Human/domain graders

Use for contextual usefulness, completeness, calibrated uncertainty, evidence interpretation, and workflow appropriateness. Record qualification, rubric, examples, order/blinding where feasible, disagreement, and escalation.

Model graders

Use only after comparing against qualified human labels across critical segments. Test prompt/order/verbosity sensitivity and shared failure modes. A model grader can prefer fluent unsupported answers or mirror the evaluated model's blind spot.

The companion calibration output reports agreement and case-level disagreement instead of promoting a grader to authority. Graders need rubrics/calibration and are not objective by assumption. [CLM-098]

Add adversarial and misuse cases

Use the threat model to test:

- retrieved content instructing the model to ignore policy;
- cross-tenant evidence/reference injection;
- fabricated/stale tool results;
- encoded or indirect prohibited requests;
- secret/sensitive-data extraction attempts;
- permission and argument escalation;
- repeated calls exhausting budget/capacity;
- manipulation of evaluator or feedback data;
- reviewer automation bias and concealed evidence;
- malformed/oversized/unexpected interface input.

Security guides can prompt categories, but the cases must match Orchid's architecture and workflow. Layered safeguards reduce risk; passing them does not establish a complete guarantee. [CLM-101]

Select security and accessibility evidence by version

With accountable owners, map relevant NIST SSDF/OWASP ASVS practices to implementation and verification. Record ASVS 5.0.0 selections, scope, result, gaps, and evidence rather than claiming broad alignment.

For WCAG 2.2, combine automated checks with keyboard, focus, semantics, zoom/reflow, error, and assistive-technology inspection plus real task evidence where appropriate. Passing selected criteria does not establish complete usability/accessibility for every context.

ISO product/data quality records can help organize attributes, but operational criteria/thresholds remain contextual. Versioned criteria support evidence, not automatic certification. [CLM-101]

Make UAT a protocol, not a demonstration

Define:

- representative roles, qualifications, contexts, devices/connectivity, and access needs;
- exact tasks and starting state;
- acceptance/guardrail criteria;
- data and environment;
- observation and evidence method;
- facilitation limits and assistance;
- issue severity, stop, retest, and sign-off path;
- participant consent/privacy and data handling;

- known non-represented users/conditions.

Observe whether the user can confirm equipment, distinguish evidence from suggestion, notice uncertainty, recover from fallback/reconciliation, exercise approval correctly, and find support.

UAT is not “we showed it and they liked it.” It does not transfer formal risk acceptance to participants. [CLM-102]

Work one Orchid UAT scenario

Role: a qualified technician reviewer using the intended device under degraded but supported connectivity.

Starting state: ticket has two plausible equipment candidates, one stale manual, current approved evidence for one candidate, and inventory whose first response is lost after acceptance.

The participant must:

1. recognize ambiguity and avoid premature evidence/model access;
2. resolve equipment using the approved workflow;
3. inspect source/version and distinguish evidence from suggestion;
4. handle the stale source visibly;
5. route qualified approval under the correct regional permission;
6. recognize unknown inventory completion and reconcile rather than resubmit;
7. use fallback/support when the connectivity window is exceeded;
8. complete the non-digital handoff and locate the audit/support record.

Acceptance evidence includes correctness, prohibited action absence, state interpretation, time within the owned window, assistance, fallback, accessibility observations, and participant reasoning at decision points. Stop if the protocol could create unsafe/external effects. Use synthetic or appropriately governed data.

A passing participant does not approve production risk. The result is one evidence row with stated representation and limitations.

Use simulation and monitoring for different evidence

Static evaluation may miss multi-step behavior, timing, tool interaction, reviewer workload, and operational adaptation. Deployment-like simulation can expose more of that path but still cannot reproduce every real user, dependency, adversary, workload, or production tail. [CLM-099]

Production monitoring remains necessary for data/workflow/model quality, drift, cohorts, latency/cost, fallbacks, overrides, support, security, and outcomes. Service availability alone cannot establish usefulness or trustworthiness. [CLM-100]

Do not use “we will monitor” to waive a foreseeable high-consequence control that should be verified before exposure. Monitoring detects after or during behavior; it is not always prevention.

Handle failures, limitations, and waivers honestly

Every failed/inconclusive row receives:

- consequence and affected segment;
- containment or scope restriction;
- root-cause/hypothesis status;
- remediation/verification owner;
- release effect;
- formal waiver authority if permitted;
- compensating controls and monitoring;
- expiry/retest trigger;
- evidence link and decision date.

Known limitations belong in the release packet, support material, and user/operator communication where relevant. A waiver cannot change an observed failure into a pass. [CLM-104]

The FDE prepares evidence/options. Designated authorities decide risk within delegated scope.

Conduct the chapter verification exercise

Start from five risks:

- wrong equipment/evidence binding;
- duplicate external effect after unknown completion;
- wrong tenant/region/qualification;
- prohibited or unsupported suggestion;
- low-connectivity timeout/retry amplification and unusable fallback.

For each, write the requirement/control, observable criterion, cheapest credible method, data/provenance/version, expected result, observed result, evidence type, segment, limitation, owner, and release disposition.

Then build evidence at several layers. A unit assertion can test a state transition. A contract test can test schema plus semantic invariants. An integration test can exercise adapter finality. End-to-end evidence can prove the bounded synthetic path. Evaluation can measure critical AI behavior. UAT can observe a representative workflow. Security/accessibility review can test versioned scoped criteria. Simulation can combine timing and failure. None replaces the others automatically.

Calibrate two graders against human labels. Preserve disagreements and inspect shared failure modes. If both model graders were trained on similar assumptions, agreement is not independence. Use the rubric and decision purpose to decide whether the grader is advisory, blocking, or unsuitable. [CLM-098]

Create an aggregate that appears to pass while one high-consequence equipment family fails. The correct disposition is not aggregate pass. Restrict release, repair or formally disposition the segment through designated authority, and keep the limitation visible. [CLM-096]

Finally, conduct the UAT scenario above and write one failed/inconclusive row. The exercise passes when the packet tells the release owner exactly which claims are supported, which are not, and what exposure those limitations permit.

Failure modes and repairs

Test count becomes confidence

Repair: trace consequence and requirement to credible observed evidence; keep suite conditions and non-proof explicit. [CLM-093]

Schema pass becomes semantic correctness

Repair: assert workflow meaning, authorization, quality, intent, finality, and reconciliation as well as structure. [CLM-092]

Aggregate conceals a critical segment

Repair: preserve segments, counts, uncertainty, and blocking disposition. [CLM-096]

Evaluation data leaks across decisions

Repair: provenance, permissions, versions, splits, leakage checks, correction history, and representativeness limits. [CLM-097]

Model grader becomes objective authority

Repair: explicit rubric, calibration, disagreement review, shared-failure analysis, and appropriate human/domain ownership. [CLM-098]

Demonstration becomes UAT

Repair: representative roles/tasks/context, predefined criteria, observed assistance, stop/retest, privacy, and limitations. [CLM-102]

Waiver erases failure

Repair: preserve failed observation, consequence, scope, conditions, owner, authority, expiry, and release effect. [CLM-104]

Orchid OA-07 current packet

The current companion provides 28 cumulative tests across:

- structure/semantics and region residency;
- missing/confirmed/ambiguous equipment;
- intent replay/conflict and unknown completion;
- authorization/qualified approval;
- environment/secret configuration;
- model abstention/budgets/policy;
- critical-segment evaluation;
- control evidence/authority/audit minimization;
- end-to-end success/rejection/reconciliation/default-closed feature;
- verification record completeness, evidence type, critical disposition, grader disagreement.

This is strong deterministic evidence for the synthetic local bundle. It is not evidence for real customer integrations, performance, production identity, UI/UAT/accessibility, provider-model quality, real monitoring, migration, release, or recovery. Those limitations remain explicit.

The Chapter 12 gate

Before release planning, OA-07 must contain:

- risk/requirement/control-to-evidence matrix;
- unit, property, contract, integration, end-to-end, failure, and recovery plans;
- AI task/case/data/grader/trace/harness/suite/purpose records;
- capability/regression and held-out/leakage boundaries;
- provenance/version/system bundle;

- consequential segments and uncertainty;
- grader calibration/disagreement;
- adversarial/security/accessibility selection and scope;
- task-based UAT protocol;
- observed results, limitations, owners, and disposition;
- waiver authority/expiry where allowed;
- passing local companion plus explicit non-proof.

The gate passes when every material release claim has the cheapest credible evidence available now or an explicit owned limitation that restricts release.

Chapter 13, *Make the System Observable and Operable*, carries these claims and limitations into signals, objectives, alerts, diagnostic paths, runbooks, capacity/cost, and explicit blind spots.

13. Make the System Observable and Operable

Design promise-linked signals, actionable objectives and alerts, privacy-safe cohort views, support feedback, and evidence-first diagnostic paths.

An observable system helps an operator decide which evidence to inspect next. An operable system gives that operator the authority, runbook, configuration, recovery path, and ownership needed to act.

A dashboard can exist without either. [CLM-116]

For Orchid Assist, “healthy” must be decomposed:

- Is the service reachable and within latency/capacity limits?
- Can users complete the ticket-to-review workflow?
- Is equipment/evidence data correct, current, and properly bound?
- Are model suggestions supported and policy controls working?
- Are qualified approvals and audit events occurring correctly?
- Are users adopting the path or escaping to manual work?
- Which sites/cohorts are degraded?
- Is support demand rising?
- Is the deployment affordable at the observed workload?
- Can owners diagnose and recover when a promise fails?

Availability alone cannot answer these questions. [CLM-108]

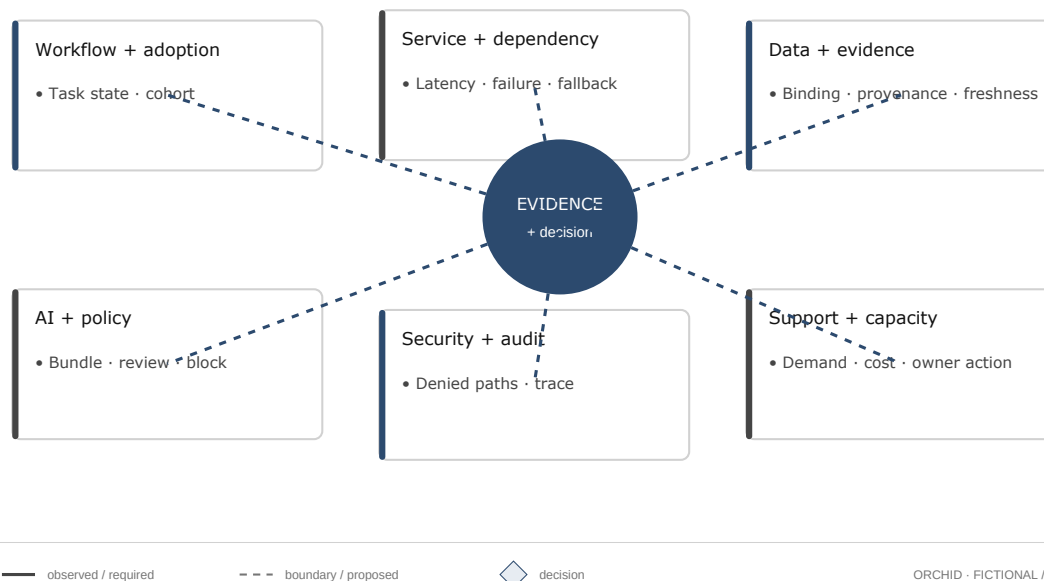
Start from promises and failure modes

For every production promise, record:

```
promise -> failure condition -> signal -> segment -> objective/threshold ->  
owner -> alert/first action -> runbook -> blind spot
```

F13.1 · FORWARD DEPLOYED ENGINEERING

Observe the workflow, not only infrastructure



F13.1 - Observe the workflow, not only infrastructure. Correlation and version identity connect signals while privacy, cardinality, and retention bound collection.

Text alternative: Signal map connecting workflow promises to service, dependency, adoption, data, AI, policy, security, audit, support, capacity, cost, and outcome signals.

Examples:

Promise	Failure	Signal/segment	First action
reviewer receives truthful state	requests fail, stall, or misstate unknown completion	state/rate/latency by site and completion state	inspect trace and dependency completion
evidence belongs to equipment	wrong/stale/missing source	binding/freshness/missing/conflict by equipment family	inspect provenance/index/version
safety step needs qualified approval	bypass/denial/expired qualification	policy and authorization outcomes by tenant/region	inspect policy/identity bundle
bounded AI remains useful	abstention, unsupported output, prohibited action, latency/cost	eval/feedback/fallback/override by risk segment	compare system bundle and evidence quality
audit is useful and minimized	missing event or sensitive overcollection	event completeness/redaction/export/access	restrict exporter and inspect schema
users can adopt the service	abandonment/manual escape/support burden	completion/dropout/rework/support by role/site	observe workflow and support cases

This map and the diagnostic tree are original synthesis tools. [CLM-105]

Turn one promise into an operating record

Take a qualified reviewer can reach a truthful evidence-backed state within the workflow window.

- **Failure states:** request unavailable, equipment ambiguous but shown as confirmed, evidence stale/missing, dependency completion unknown, policy/authorization wrong, review latency too high, or fallback absent.
- **Signals:** completed/blocked/fallback/reconcile states; equipment confidence; evidence version/freshness; dependency finality; policy/authorization; review latency; support reason.
- **Segments:** site/connectivity, equipment family, qualified role, release/config/model/policy bundle, and clear/ambiguous/missing state.
- **Decision:** hold expansion if a critical segment has wrong binding or no truthful fallback; investigate latency only after state correctness is preserved.
- **Owner:** workflow owner for the promise and boundary owners for contributing service/data/model/identity state.
- **Blind spot:** offline completion may arrive late; freshness and coverage remain visible.

The record refuses a common shortcut: treating a successful HTTP request as a successful review. It also shows why one promise needs several signals but one accountable operational decision.

Maintain linked operating views

Service and dependency

Request volume, success/error, latency distribution, timeout, saturation, queue, connection pool, retry versus new demand, dependency state, deployment/config/version.

Workflow and adoption

Tickets entering/completing/blocked/fallback/reconcile, time in state, rework, abandonment, manual escape, qualified-review load, evidence inspection, cohort ramp.

Data and evidence

Missing/ambiguous equipment, freshness, source version, conflicts, reconciliation age, tenant/region denials, index lag, approved-evidence coverage. Data/product quality models can structure questions but do not choose Orchid's thresholds. [CLM-114]

AI, policy, and review

Suggestion/abstention/fallback, unsupported/prohibited cases from sampling/evals, retrieval/evidence quality, policy blocks, approval/override/disagreement, model/prompt/index/policy bundle, latency/cost.

Production AI operation monitors the whole decision path, not model uptime alone. [CLM-110]

Security and audit

Authentication/authorization denials, elevation, cross-boundary attempt, secret/config scan, audit completeness/redaction/access/export, exception expiry, control failure.

Support, capacity, and cost

Demand by reason/cohort, resolution/escalation, recurring confusion, workload/capacity headroom, unit-cost proxies, budget fallback. Support is a product/operating signal, not cleanup. [CLM-111]

Use metrics, traces, and logs deliberately

- **Metric:** aggregated numerical behavior suited to trends/objectives/alerts.
- **Trace:** causal path across a request/workflow using correlation/span context.
- **Log/event:** discrete structured fact useful for state, audit, and diagnosis.

OpenTelemetry documents these as distinct signals; APIs and vendor support remain version-sensitive. [CLM-107]

Do not force one signal to do every job. A metric can reveal a cohort change, a trace can locate a boundary, and a structured event can explain a policy reason. Correlate by safe identifiers and release/config/policy versions.

The companion's local JSON metric/trace is inspectable and provider-neutral. It is not a production collector, exporter, storage, query, or alerting proof.

Define SLI/SLO/action records

An SLI/SLO record contains:

- user promise and affected cohort;
- indicator definition, numerator/denominator, exclusions;
- source, window, freshness, missing-data behavior;
- objective and owner/authority;
- consequence and tradeoff;

- alert condition/urgency/first action;
- response, exposure, release, or investment decision;
- known blind spot and review trigger.

Objectives should represent user-relevant behavior and change a decision. [CLM-106]

Do not copy universal targets. A 99.9 percent availability target can coexist with unusable evidence, slow low-connectivity sites, or unsafe critical-segment behavior. Thresholds belong to accountable owners using outcome/risk/cost evidence.

Specify an indicator so two operators calculate the same thing

Example bounded workflow indicator:

Numerator: eligible Orchid review intents in the internal pump-family cohort that reach ready-for-review or an approved truthful fallback within 30 seconds. Denominator: all eligible intents during the window, including blocked, failed, abandoned, and unresolved completion states. Exclude only preidentified synthetic rehearsal intents. Segment by connectivity and equipment-state class. Source is versioned workflow events; coverage heartbeat must be current. If coverage is stale, status is unknown, not green.

The objective and 30-second value are fictional and require an owner. The mechanism is definitional: eligible population, good state, time boundary, exclusions, segmentation, source, missing behavior, and decision.

Pair the indicator with action. A critical wrong-binding event stops the affected path. Sustained latency may hold widening and activate fallback. A low-volume ambiguous segment may require targeted evidence rather than an availability page.

Preserve cohorts and tails

An aggregate can be green while one site is unusable. In the companion, nine of ten connected samples succeed while the low-connectivity cohort has zero success and 5-second average latency. The cohort view exposes what a combined rate would soften.

Segment by legitimate diagnostic factors:

- site/connectivity/region/tenant;
- role/qualification/access needs;
- workflow state and exception/fallback;
- equipment/evidence family/quality;
- release/config/policy/model bundle;
- latency/cost tail;

- new versus retry demand.

Avoid unbounded high-cardinality labels such as raw customer text or every identifier. Use correlation for targeted tracing, bounded dimensions for aggregation, and controlled investigation access.

Global averages can conceal consequential degradation. [CLM-109]

Read aggregate and cohort together

Suppose 90 of 100 intents complete. If all 10 failures belong to the only low-connectivity site, the conclusion is not mostly healthy; one intended cohort cannot use the workflow.

If that site has only two events and both fail, the evidence is serious but statistically thin. Hold unsupported expansion, inspect the mechanism, and gather representative evidence. Do not dilute it into the aggregate or pretend two events estimate a universal rate.

Always report count, denominator, window, coverage, and tail/segment with a rate. A percentage without exposure can hide rare critical failure and meaningless small samples.

Make alerts actionable

Every alert needs:

- promise/failure and exact condition;
- affected segment and evidence link;
- owner/on-call route and urgency;
- first discriminating action;
- runbook and access prerequisites;
- suppression/deduplication/escalation;
- recovery/clear condition;
- review of noise, misses, and cost.

An ownerless alert with no first action is noise, not control. [CLM-115]

Alert on user consequence or a strong leading indicator, not every metric movement. Page for urgent actionable conditions; use tickets/review queues for slower degradation. Test routing and runbook access.

Design one actionable alert

Orchid critical evidence-binding failure might contain:

- condition: any review-ready state where confirmed equipment and evidence equipment ID disagree;

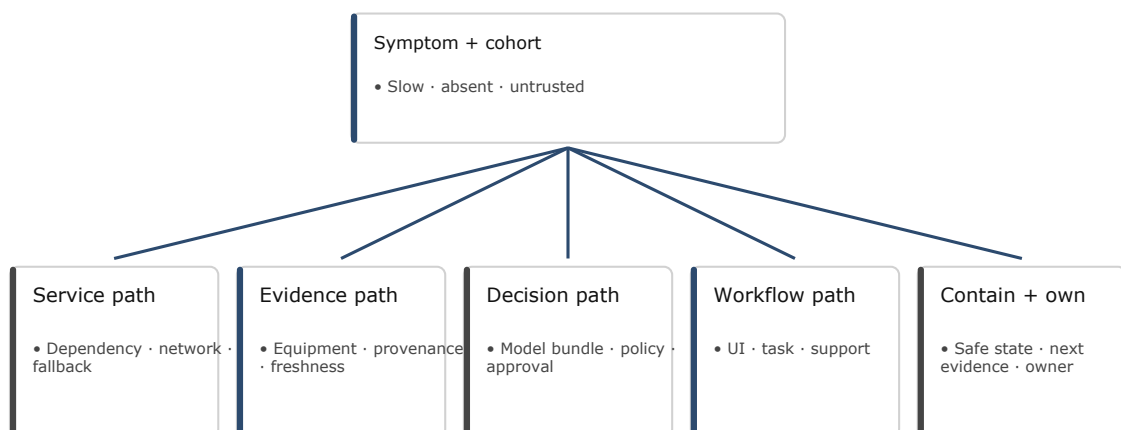
- segment: tenant, region, equipment family, release/config/policy bundle; no raw ticket text;
- urgency: page workflow on-call and release owner immediately;
- first action: stop affected suggestion path, preserve identifiers/versions, verify whether external approval occurred;
- runbook: binding/reconciliation response with access prechecked;
- escalation: designated security/safety/customer authority by consequence;
- clear condition: affected state reconciled, control restored, and owner approves bounded re-entry;
- review: every event becomes verification/regression evidence.

Inject a synthetic mismatch. Confirm the route reaches a qualified owner, the link/query works, the first action is possible, and no sensitive content leaks into the notification.

Follow an evidence-first diagnostic tree

F13.2 · FORWARD DEPLOYED ENGINEERING

Diagnose from symptom to boundary



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F13.2 - Diagnose from symptom to boundary. Evidence-first branching prevents an immediate and often-wrong code change.

Text alternative: Diagnostic tree for a slow or untrusted recommendation, branching from user symptom and cohort through dependency, connectivity, equipment binding, evidence freshness, model bundle, policy, authorization, UI, support, containment and owner action.

For “slow or untrusted recommendation”:

1. Confirm scope, cohort, time, release/config/policy/model version.
2. If service unhealthy, inspect correlated service/dependency trace and saturation.

3. If one cohort degraded, compare connectivity, latency tail, payload size, retry/fallback, site dependency.
4. If evidence untrusted, inspect equipment binding, provenance, freshness, index/cache, tenant/region.
5. If policy blocked, inspect policy version, authorization, qualification, approval and exception.
6. If the system path is healthy, inspect adoption, review UI, task evidence, support and outcome.
7. Contain exposure, preserve evidence, assign owner, update tests/signals/runbook.

The companion returns the next discriminating evidence rather than a confident root cause.

Walk a diagnosis without guessing

Symptom: technicians at one site report slow and untrusted suggestions.

1. Compare affected/unaffected sites and current release/config/model/policy versions.
2. Aggregate service latency is normal; the site cohort shows a long tail and frequent fallback.
3. Trace a representative approved/synthetic intent. Evidence waits after repeated inventory completion checks.
4. Inspect intent state: requests are unknown completion and clients retry as new work.
5. Contain by holding the site cohort and using truthful manual fallback; preserve and reconcile intent records.
6. Verify evidence provenance. If it remains correct, do not mislabel the event an AI-quality failure.
7. Assign retry/dependency correction, capacity test, cohort signal, and runbook update.

Each step narrows hypotheses. The tree avoids jumping from untrusted AI to prompt changes when the observed boundary is retry/finality under weak connectivity.

Bound telemetry by privacy, security, cost, and usefulness

Do not log raw ticket payloads, prompts, manuals, secrets, documents, or free text by default. Define:

- purpose and fields;
- classification and allowed users;
- tenant/region and export boundary;
- retention/deletion/legal hold owner;
- cardinality/volume/cost budget;
- redaction/sampling and missing-data consequence;
- integrity/time source;
- investigation and audit separation.

More data can create more exposure and noise. Telemetry must be minimized and cost/cardinality constrained. [CLM-112]

The companion rejects forbidden telemetry fields even when the sample is otherwise valid.

Treat missing telemetry as a state

No events can mean no traffic, broken instrumentation, blocked exporter, wrong query, clock skew, dropped sampling, or access failure. Define heartbeat/coverage/freshness signals and fail the dashboard visibly when evidence is stale or incomplete.

Instrumentation changes require review and tests like product code. A renamed label can silently empty an alert. Version schemas and queries; compare pre/post deployment.

Connect incidents to improvement

For a material event:

- detect and validate scope/consequence;
- contain/disable/isolate/fallback as authorized;
- preserve minimized evidence;
- communicate and escalate;
- recover and verify user-visible state;
- analyze technical/workflow/organizational contributors;
- update control, test, signal, runbook, support, ownership, and product pattern.

NIST SP 800-61r3 integrates incident response with cybersecurity risk management; organizations still define their own command/authority. [CLM-113]

The FDE can diagnose and coordinate within delegated scope but does not impersonate incident commander, legal/privacy/security authority, or customer spokesperson.

Turn quality evidence into operating signals

Do not run every expensive evaluation on every request. Layer the plan:

- online deterministic invariants for equipment/evidence binding, tenant/region, policy, permission, required approval, budgets, and audit shape;
- sampled qualified review for contextual usefulness and unsupported behavior;
- scheduled regression suites on frozen and rotating cases;

- targeted evaluation after model, prompt, index, policy, or tool change;
- governed support/feedback cases curated into datasets;
- workflow/adoption/outcome signals that reveal whether correct-looking suggestions help.

Record sampling and blind spots. Reviewing only completed requests hides abandonment, fallback, and blocked states. Voluntary feedback can exclude silent cohorts.

Connect data quality to consequence: equipment correction rate, evidence freshness, conflicting-source age, index lag, reconciliation age, semantic rejection by source version, cross-boundary denial, and manual escape. A freshness distribution without an acceptable age and owner is not actionable.

Operate support as part of the service

Classify support demand by workflow state and reason:

- access or qualification;
- equipment/evidence mismatch;
- latency/connectivity;
- fallback/reconciliation;
- confusing suggestion/evidence;
- inaccessible interface;
- training/process gap;
- dependency incident or defect;
- feature/config/cohort mismatch.

Track first useful response, resolution/escalation, repeat contact, unresolved age, and recurrence. Protect support content; extract minimized categories and approved cases. Feed recurring demand into product, documentation, tests, signals, rollout, and ownership.

Operate capacity and cost together

Build the capacity model from intent rate, concurrency, evidence size, quotas, connections, retrieval/model use, retries, fallback, reviewer capacity, telemetry, and support.

Watch headroom, saturation, queue age, rejection/defer behavior, expensive failure paths, cost per eligible/completed intent, model budget exhaustion, telemetry cardinality/storage/query cost, and reviewer/support load.

Do not optimize cost by removing required control/evidence. Do not add infrastructure when the constraint is stale data or qualification. When capacity is insufficient, rate-limit, visibly queue, restrict cohort, or fall back under owner rules. Silent latency creates abandonment and retries.

Write and test runbooks

A runbook contains:

- symptom/alert and affected promise;
- access, authority, prerequisites, and cautions;
- first query/evidence and interpretation;
- diagnostic branches;
- containment choices/consequences;
- escalation/communication owner;
- recovery verification/clear condition;
- evidence/privacy handling;
- last rehearsal and known gaps.

Test it with someone other than its author. Verify links, access, queries, feature controls, reconciliation, and recovery under the target failure. A document that assumes unavailable identity or control plane is not an operating path.

Rehearse under the failed dependency

Give an operator a synthetic alert while the normal identity path is unavailable. Observe whether the operator can authenticate through approved emergency access, find the cohort and versions, choose containment, preserve minimized evidence, communicate through the named owner, and verify recovery.

Record assistance, timing, stale links, missing access, ambiguous steps, unsafe commands, and gaps. Repair and rerun. The goal is not a fast theatrical drill; it is a path another qualified operator can execute under the failure conditions it claims to address.

Failure modes and repairs

Dashboard equals observability

Repair: connect promises to failures, segments, owners, next evidence, runbooks, authority, and recovery. [CLM-116]

Availability equals health

Repair: include workflow, data/evidence, AI/policy/review, security/audit, support, adoption, capacity, and cost. [CLM-108]

Global green hides a cohort

Repair: preserve legitimate segments, counts, tails, coverage, and consequence. [CLM-109]

Every metric pages

Repair: page urgent actionable conditions; route slow degradation to review. Bind owner, evidence, first action, and clear condition. [CLM-115]

Telemetry becomes sensitive exhaust

Repair: purpose-bound schemas, allowlists, safe identifiers, retention/access/export rules, and cardinality/cost budgets. [CLM-112]

Missing data looks healthy

Repair: heartbeat, freshness, coverage, versioned queries, and explicit unknown state.

Runbook exists but cannot run

Repair: exercise access, queries, controls, communications, and recovery under the representative failed dependency.

Keep a blind-spot register

For every missing, delayed, or sampled signal record the promise/failure, affected cohort, alternative detection, consequence, release/exposure restriction, owner, target, and trigger for urgency.

“We cannot measure it yet” is acceptable only when it changes the decision. A hidden blind spot is not.

Write a blind-spot record

Example:

Offline technicians can complete manual fallback without emitting an event until reconnect. Promise affected: truthful completion and support visibility for low-connectivity sites. Consequence: the

dashboard can undercount fallback burden and overstate timely completion. Alternative evidence: delayed sync plus support sample and scheduled task observation. Release restriction: do not widen to additional offline sites. Owner: workflow operations. Trigger: implement signed queued state with freshness/coverage signal before the next cohort review.

The record changes exposure and evidence collection. It is not a backlog note hidden beneath a green dashboard.

Work capacity, cost, and support as one operating decision

Suppose the Orchid evidence path has 40 concurrent review intents, inventory allows 20 concurrent calls, the model budget allows 15 suggestions per minute, and three low-connectivity sites retry after eight seconds. Qualified reviewers can process 12 critical cases per hour.

An incomplete capacity plan scales only the application service. A complete plan asks:

- can intent identity/reconciliation prevent retry demand from becoming new external work?
- where does bounded queuing preserve truthful user state?
- when should the system abstain or use manual fallback as model budget is exhausted?
- can reviewer capacity preserve the qualified-control latency?
- which evidence, telemetry, and support work grows with each cohort?
- what cost is per eligible, completed, fallback, and failed intent?
- which cohort/feature stops first under saturation?

Precommit behavior. New suggestion work may queue visibly or fall back, while safety-relevant review cannot be bypassed for throughput. Capacity limits should stop before external mutation where possible. Cost optimization cannot remove required evidence or authorization.

Connect support cases to the same state model

Do not classify all tickets as user issue or AI quality. Use equipment state, evidence state, dependency finality, policy/authorization, connectivity, review, fallback, and release bundle.

For each support reason, link count/cohort, first useful response, resolution, recurrence, defect or knowledge classification, owner, and product/test/signal/runbook update. A repeated question about evidence provenance may indicate interface design debt. A spike in unknown-completion tickets may reveal reconciliation or connectivity failure. The same state language lets support evidence join engineering diagnosis without copying sensitive content.

Conduct the operability exercise

Give a second operator three symptoms: a global-green but failed low-connectivity cohort, a critical evidence-binding alert, and a dashboard with stale coverage. The operator must select next evidence, use approved queries/access, contain the affected path, communicate through authority, and choose a runbook/recovery action.

Then inject retry demand plus model/reviewer budget pressure. Require an explicit queue/restrict/fallback decision and record user state, guardrails, support, and cost. Pass when the operator acts without raw sensitive payloads or hidden FDE knowledge and every missing signal changes the release or investigation decision.

Complete OA - 08 operating layer

The artifact now includes:

- signal catalog across eight linked views;
- SLI/SLO/action records with contextual thresholds pending owner approval;
- cohort/tail query designs and explicit blind spots;
- alert owner/urgency/first action/runbook;
- diagnostic tree and containment/escalation;
- telemetry privacy/retention/cardinality/cost rules;
- support and feedback loop;
- local companion evidence with four new tests.

The 32 cumulative tests prove signal-record completeness, cohort visibility, sensitive-field rejection, and diagnostic routing in the synthetic bundle. They do not prove a production telemetry pipeline or threshold.

The Chapter 13 gate

Before release/recovery design completes, every critical promise must have:

- a signal or explicit blind spot/consequence;
- user/workflow/data/AI/security/support/capacity/cost coverage;
- segment/tail visibility;
- release/config/policy/model correlation;
- owner, threshold authority, urgency, and first action;
- runbook/access/escalation;
- privacy/retention/cardinality/cost boundary;
- missing/stale telemetry behavior;
- incident-to-improvement loop;

- passing companion signal tests.

The gate passes when an operator can move from symptom to the next discriminating evidence and authorized action without exposing sensitive content or trusting a global green average.

Chapter 14, *Engineer Release and Recovery*, will use these signals to gate change and demonstrate rollback, roll-forward, isolation, restoration, and stop decisions across representative state.

14. Engineer Release and Recovery

Bind versioned artifacts and evidence to progressive release gates, compatible migration, explicit recovery choice, verified restoration, and executed rehearsal.

A release is a decision to expose a particular system state to a particular cohort under particular evidence and recovery conditions.

A pipeline can repeat that decision. It cannot decide that the evidence is sufficient, that a migration is reversible, that an exception is authorized, or that a backup will restore.

Release engineering makes change reproducible. Recovery engineering makes failure state explicit and rehearsed.

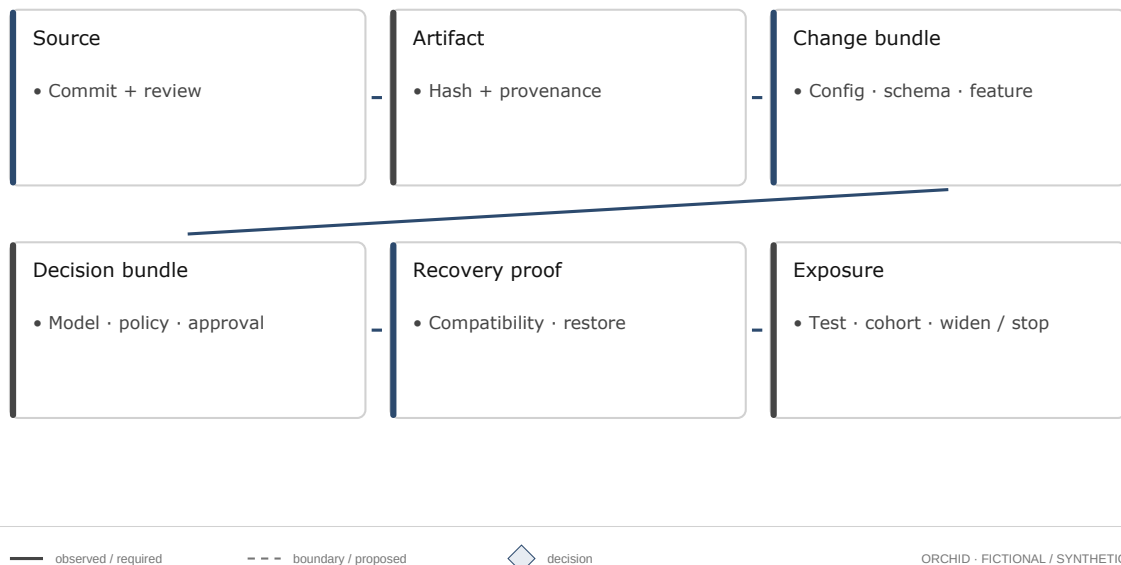
Bind the release as one versioned record

Record together:

- source commit and immutable artifact hash/provenance;
- build/toolchain/dependency evidence;
- code, API, configuration, schema, migration, data, model, prompt, index, policy, and grader versions;
- feature-control states and cohort;
- verification/control/security evidence bundle;
- known limitations, exceptions, authority, and expiry;
- compatibility window and migration state;
- health/quality/adoption/cost gates;
- owner, approver, schedule, support/on-call;
- stop/abort condition and recovery path.

F14.1 · FORWARD DEPLOYED ENGINEERING

Release identity binds the whole change



F14.1 - Release identity binds the whole change. Artifact, configuration, migration, policy, approvals, compatibility, and recovery move together.

Text alternative: Release path from source and hashed artifact through configuration, schema and migration, feature controls, model and policy bundle, approval, compatibility and recovery record to test, canary, cohort, wider exposure or stop.

The companion validates this binding and computes a deterministic SHA-256 hash over the synthetic artifact record. A hash identifies bytes/record content; it does not prove provenance, security, or correctness by itself. [CLM-117]

Semantic Versioning can signal compatibility expectations for a declared public API. It does not solve configuration, schema, data, deployment, or behavior compatibility. [CLM-120]

Review the complete release identity

Imagine commit abc123 produced artifact hash H1, but the canary uses policy bundle P4, prompt M7, schema S3, configuration C9, and feature state on for a named cohort. A rollback that changes only code to the prior artifact does not recreate the prior system. Schema/data, policy, index, configuration, and external effects may remain new.

The release record should let an operator reconstruct the exact bundle and see which parts can change independently. Store links and hashes where appropriate, not secrets or sensitive content. If a material artifact cannot be identified, the release evidence is incomplete.

Review provenance separately from identity. A hash can confirm equality with a recorded byte sequence; it does not prove who built it, which source/dependencies produced it, whether review occurred, or whether it is safe. Bind build and approval evidence explicitly.

Design the release path as evidence gates

A provider-neutral path can be:

1. source review and required checks;
2. reproducible artifact/build provenance;
3. unit/contract/security/static companion evidence;
4. migration compatibility and backup/restore prerequisites;
5. representative environment and failure/recovery tests;
6. owner-approved release record and change window;
7. internal/synthetic or least-exposed cohort;
8. observation window using absolute and comparative gates;
9. widen, hold, restrict, or recover;
10. close evidence, remove/expire controls, and update operations.

Automation and consistency reduce manual variance, but can consistently deploy a bad decision. Evidence/authority remain necessary. [CLM-118]

Walk one change through the gates

For Orchid, suppose the change improves evidence retrieval and requires a new index plus backward-compatible evidence metadata.

1. Contract tests prove old/new readers handle the declared metadata window.
2. The index is built from approved synthetic/controlled evidence and receives a content/version record.
3. Critical-segment evaluation binds model, prompt, index, policy, and grader versions.
4. Migration capacity is checked before any persistent record changes.
5. A representative environment injects stale index, missing evidence, partial migration, and unavailable identity recovery.
6. The release owner reviews limitations and selects an internal cohort.
7. Absolute wrong-binding/prohibited-action gates and comparative latency/fallback gates are watched.
8. The team widens, holds, or recovers according to the precommitted record.

The sequence is valuable because each gate can stop the change and names the evidence needed to continue. CI passed is only one input.

Make each gate actionable

For every gate define:

- required evidence and version;
- pass/fail/inconclusive interpretation;
- owner and decision authority;
- affected cohort and exposure duration;
- signal freshness/coverage requirements;
- stop condition;
- recovery action and access;
- exception rule and expiry.

Do not let a missing signal appear green. Do not silently ignore failed critical-segment verification because infrastructure is healthy.

Use progressive exposure with both relative and absolute checks

Small, time-limited changes can reduce blast radius and reveal environmental behavior. Compare control and changed cohorts where meaningful, but also apply absolute guardrails.

A canary can look “better than control” while both violate an acceptable latency or error boundary. Cohorts can differ in site, workload, evidence quality, user role, or connectivity. Low volume can hide rare/high-consequence failure. Preproduction parity is incomplete.

Record:

- why the cohort is representative for the intended claim;
- baseline/control and possible confounders;
- minimum observation/volume/critical-case coverage;
- service, workflow, data, AI/policy, security, support, capacity, and cost gates;
- authority to widen/hold/stop;
- rollback/roll-forward/isolation feasibility.

Progressive exposure is useful only with meaningful cohorts, absolute criteria, and precommitted recovery. [CLM-119]

Version configuration and feature controls

Configuration changes system behavior and needs review, provenance, validation, promotion, rollback/roll-forward analysis, access control, audit, and environment-difference evidence.

A feature-control record includes owner, purpose, default, environment/tenant/region/cohort scope, prerequisites/interactions, signal, decision authority, expiry/removal, and recovery behavior. [CLM-127]

Avoid long-lived flags that create multiple untested systems. Test on/off and material combinations. Remove the flag and dead path after the decision, unless the control is an intentional operational switch with continuing ownership.

Plan migrations around compatibility and externally visible state

For every persistent/API/event change define:

- old/new readers and writers;
- expand/contract or other compatibility sequence;
- backfill, dual-read/write, verification, and reconciliation;
- load, connection, locking, duration, pause/resume;
- partial completion and idempotent migration intent;
- feature/config coordination;
- destructive step and point of no return;
- backup/restore scope;
- rollback, roll-forward, isolation, or stop;
- data owner/authority and evidence.

Create a compatibility table across old/new readers, writers, schema, and externally visible effects. State which combinations are allowed and for how long. An expand/contract path might add nullable/new fields, deploy tolerant readers, deploy new writers, verify/backfill, then remove old behavior only after consumers and recovery paths are ready.

Do not call a destructive step reversible because the migration script has a down function. Test whether information, external effects, and consumer interpretation can actually return. When old and new states must coexist, expose that window in monitoring and support.

Capacity-gate before mutation

The companion migration checks requested connections against a capacity limit before changing state. When requested capacity exceeds the limit, it stops with zero applied records.

This demonstrates an important order: validate prerequisites before irreversible work. A production migration also needs realistic workload, connection-pool, lock, replica, storage, backup, and maintenance-window evidence.

Expose partial/divergent state

The synthetic failure after one of two records produces schema versions [2, 1] and the state divergent. It does not relabel the migration failed and assume old state.

Readers/writers must tolerate the compatibility window or exposure must remain isolated. Recovery then chooses based on actual state, not desired version.

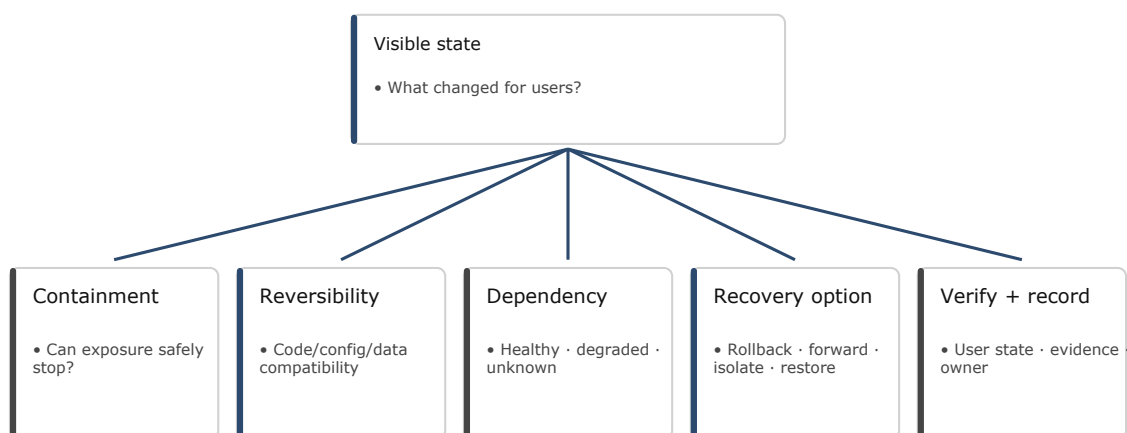
Preserve intent during recovery

Migration/recovery retries need stable intent and semantic equivalence. An accepted write with missing final response is unknown until reconciled. Reissuing can duplicate or conflict. HTTP method-level idempotency is insufficient without application state. [CLM-121]

Choose recovery from state and consequence

F14.2 · FORWARD DEPLOYED ENGINEERING

Recovery follows external state



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F14.2 - Recovery follows external state. Rollback, roll-forward, isolate, restore, and stop are selected by state and consequence.

Text alternative: Recovery decision tree beginning with user-visible state and containment, then branching on dependency health, reversibility, external data change, compatibility, restore evidence, and consequence before verification.

Ask in order:

1. What user/external state exists now?

2. Is immediate isolation/disable/fallback required to contain consequence?
3. Are dependencies and recovery access healthy?
4. Is the change actually reversible without losing or misinterpreting state?
5. Did destructive/external writes occur?
6. Can old/new code coexist with current schema/data?
7. Is a small roll-forward repair safer/faster?
8. Is a tested, scoped, integrity-checked restore available?
9. What authority and communication are required?
10. How will the recovered user-visible state be verified?

The answer can be:

- **rollback:** compatible reversible code/config change with preserved state;
- **roll-forward:** current state cannot safely return, and a bounded repair exists;
- **isolate:** unhealthy dependency/cohort or containment first;
- **restore:** verified backup/restoration path fits scope/consequence;
- **stop/escalate:** no verified safe path.

Rollback is not universally safest after destructive or external change. [CLM-122]

This tree is original synthesis, not a formal standard. [CLM-124]

Compare recovery options for one partial migration

State: one of two records is schema v2, one is v1; old/new readers are compatible, no irreversible external action occurred, and a bounded v2 repair is tested.

- **rollback code only:** may leave v2 data under old assumptions; reject unless compatibility and state restoration are proven.
- **reverse migration:** risks losing new fields and requires an equally tested path; reject for this rehearsal.
- **roll forward:** complete the second record, reconcile both, verify readers and user state; selected.
- **isolate:** useful if consumers cannot tolerate divergence while repair runs.
- **restore:** unnecessary here and could lose newer valid state; retain only if integrity/compatibility evidence supports it.
- **stop/escalate:** required if authority, access, or state evidence is insufficient.

The selection is not a preference for roll-forward. It follows the actual state and verified path.

Treat backup and restore as different evidence

A backup record needs creation time/version/scope/retention/encryption/access/region/dependencies. Recovery evidence additionally needs:

- restore environment/procedure/access;
- integrity and semantic validation;
- compatibility with code/config/schema;
- missing interval and external-effect reconciliation;
- observed time and bottlenecks;
- owner and unresolved gaps.

The companion deliberately rejects a backup that lacks `restoreTestedAt` and `integrityVerified`. A backup is not recovery evidence by existence. [CLM-123]

Test stale/corrupted/missing backup, permission failure, wrong region/key, capacity, partial restore, and service verification. Protect restored sensitive data.

Exercise break-glass under the failed dependency

Emergency recovery access must define eligible actors, trigger, scope, duration, authentication, notification, audit, actions, review, and rotation.

Exercise it when the target identity/control plane/network/secret dependency is unavailable. A runbook that depends on the failed service is fictional. Do not weaken routine access merely to simplify the exercise. [CLM-128]

The FDE can execute/record within delegated scope; customer security/operations authorities own emergency access and risk decisions.

Record a recovery rehearsal

For each scenario capture:

- scenario/initial externally visible state;
- artifact/config/schema/migration/dependency versions;
- preconditions/access/authority;
- action selected and why alternatives were rejected;
- expected state and target;
- actual steps/state/timing;
- signals/evidence/integrity/user verification;
- communication/escalation;

- gap, owner, correction, and retest;
- environment and production limitation.

The local companion:

1. creates two schema-v1 synthetic records;
2. fails after migrating one, producing divergent state;
3. roll-forwards both records to v2;
4. verifies completed state;
5. records 42 milliseconds against a 100-millisecond local target;
6. states explicitly that this is not a production RTO claim.

A local observation cannot become a production commitment. [CLM-126]

Conduct the release and recovery exercise

Ask a second operator to reproduce the bundle from the release record, then run the synthetic migration. Inject capacity pressure before mutation and verify zero records change. Rerun within the limit, fail after one record, and confirm the state is labeled divergent. The operator must choose among rollback, roll-forward, isolation, restore, and stop using current evidence, then execute and verify the selected path.

The rehearsal record must include rejected alternatives, access/authority, observed state, integrity/user verification, timing, assistance, and production limitations. Break one dependency used by recovery and prove the emergency path does not secretly depend on it.

Pass when another operator can repeat the result and explain why the action is safe for the tested state. Fail if success depends on the original author narrating hidden steps, if only the infrastructure process is verified, or if local timing becomes a production RTO promise.

Manage feature controls as temporary release state

A feature flag can reduce exposure, but it also creates multiple system configurations. Record the default, scope, owner, creation decision, prerequisites, interacting controls, on/off behavior, signals, stop/recovery use, expiry, removal criteria, and data/schema compatibility.

Test at least the material combinations. Turning off the interface may not stop background work. Turning off the model may still leave stale suggestions in a cache. A cohort flag may not reverse external inventory effects. A recovery switch that was never exercised can fail when needed.

At each release review classify a control as:

- active experiment/exposure control with current decision window;
- intentional long-lived operational control with permanent owner/tests;
- expired/dead state awaiting removal;

- unsafe/unowned divergence that blocks widening.

Remove code/config/data paths when the decision closes. If removal itself is risky, plan and verify it as a release.

Design a restore test that proves usable state

Choose a representative backup version and record what it contains and omits: data, schema, configuration, encryption/key dependency, index/evidence versions, audit, and external effects. Restore into an approved isolated environment using the access path expected during incident conditions.

Verify:

- cryptographic or storage integrity where appropriate;
- schema/application/config compatibility;
- record counts plus semantic invariants;
- tenant/region and access boundaries;
- missing interval and external-effect reconciliation;
- user-visible workflow state;
- observed time, bottlenecks, and assistance;
- cleanup/protection of restored sensitive data.

The test can reveal that a backup is valid bytes but unusable with the current application, or that inventory reservations after the backup cannot be reconstructed. Those are recovery gaps, not minor documentation issues.

Exercise break-glass without normalizing it

Disable the normal identity dependency in the rehearsal. The authorized operator must invoke the approved emergency path, authenticate independently enough to survive the failure, receive only the required scope/duration, perform named recovery actions, notify designated owners, generate audit evidence, and exit/rotate access afterward.

Record every dependency. If emergency access relies on DNS, secrets, approval service, device posture, or network control that shares the incident failure, repair the design or state the limitation. Do not respond by leaving permanent broad credentials available.

The FDE can help implement and exercise the mechanism. Customer security/operations authorities decide eligibility, conditions, and residual risk. [CLM-128]

Transfer the recovery tree across changes

For a stateless UI defect with no external effects, rollback may be simple. For an append-only event schema, roll-forward with compatible readers may be safer. For a corrupted tenant-specific index, isolate and rebuild may preserve other cohorts. For a regional data loss with verified backup, restore plus reconciliation may be necessary. For unknown authority or state, stop and escalate.

Using several change types prevents the team from memorizing one preferred action. The tree is a decision method: actual external state, reversibility, compatibility, access, evidence, and consequence determine the path.

Failure modes and repairs

“Pipeline means safe”

Repair: evidence/authority gates, artifact binding, cohort/stop/recovery.

“One-click rollback”

Repair: inspect external/data state, compatibility, point of no return, and rehearse the actual path.

“Backup exists”

Repair: restore/integrity/compatibility/timing/access evidence.

Hidden destructive migration

Repair: explicit destructive step, approval, backup, compatibility, roll-forward/restore plan, capacity gate.

Flag as permanent architecture

Repair: owner/default/scope/tests/signals/expiry/removal.

Recovery credential untested

Repair: exercise under dependency failure, minimize scope, review/rotate.

Health gate without user meaning

Repair: Chapter 13 promises/cohorts/quality/support, absolute guardrails, signal freshness.

Complete 0A - 08

The operating artifact now includes:

- version-bound release record and artifact hash;
- evidence-gated progressive path;
- configuration/feature lifecycle;
- migration compatibility/capacity/partial-state plan;
- recovery decision tree;
- backup/restore and break-glass evidence;
- executed synthetic roll-forward rehearsal;
- owner/authority/support/runbook/improvement path.

Operational readiness joins change, release, recovery, support, ownership, and improvement with executed evidence rather than documents alone. [CLM-125]

The Chapter 14 gate

Before Chapter 15 evaluates production crossing, require:

- bound code/artifact/config/schema/migration/model/policy/evidence/cohort versions;
- quality/authority/exception gates;
- compatibility window and destructive point;
- migration load/partial/idempotent/reconciliation evidence;
- progressive cohorts plus absolute criteria;
- tested feature-control stop/removal;
- rollback/roll-forward/isolate/restore/stop choice by actual state;
- verified restoration and break-glass access;
- executed recovery rehearsal and user-visible verification;
- explicit local-versus-production limitations;
- passing companion tests and reproducible command.

The gate passes when a reviewer can reproduce the intended release, force a representative failure, choose recovery from state/consequence, and verify the recovered result without inventing rollback or recovery claims.

Chapter 15, *Cross the Production Threshold*, will decide whether the combined outcome, verification, operating, release, and recovery evidence justifies exposing a real cohort.

Part IV - Launch Into Reality

Production exposure does not resolve uncertainty. It changes who and what can be affected. Part IV teaches how to cross that boundary with named authority, respond when evidence changes, and leave a system that users and customer operators can actually own.

Chapter 15 assembles the readiness packet and supports go, conditional go, reduced scope, delay, or stop. Cohort, observation, critical-case coverage, triggers, command, support, communication, and recovery are part of the decision. Chapter 16 operates within customer incident command to establish impact, preserve calibrated truth, contain and restore, deliver durable correction, and prove stabilization exit. Chapter 17 diagnoses adoption without blaming users and requires demonstrated customer capability across knowledge, access, telemetry, runbook, release, recovery, support, decisions, open risk, and change.

The dossier advances through OA-09 and OA-10. Handoff is not a folder or meeting. It is a bounded state in which representative users can complete the workflow, customer operators can act without hidden FDE dependence, and remaining exceptions have owners and exits.

15. Cross the Production Threshold

Use readiness evidence, unresolved risk, authority, cohort design, signals, communications, and recovery triggers to choose go, conditional go, reduced scope, delay, or stop.

Production is not a place. It is exposure to real users, data, dependencies, authority, support, and consequence.

The readiness decision is therefore not “Is the build green?” It is:

Does the current evidence justify this exact cohort and duration under these conditions, signals, owners, communications, and recovery triggers?

The answer can be go, conditional go, reduced scope, delay, or stop. A review that permits only “go” is ceremony. [CLM-134]

Assemble the readiness evidence packet

Bring forward the authoritative artifacts:

- outcome/guardrail/decision-rights contract (OA-03);
- scope, dependencies, uncertainty, blast radius (OA-04);
- architecture/interfaces/environment/AI/governance (OA-05);
- runnable slice and debt (OA-06);
- verification/evaluation/UAT/limitations (OA-07);
- signals/objectives/alerts/runbooks/capacity/cost/release/recovery (OA-08);
- open risks, exceptions, owners, authority, expiry;
- user/operator/support/accessibility preparation;
- cohort/ramp/observation/communication/command schedule.

F15.1 · FORWARD DEPLOYED ENGINEERING

Readiness is an evidence decision



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F15.1 - Readiness is an evidence decision. Named authority chooses go, conditional go, reduced scope, delay, or stop from explicit dispositions.

Text alternative: Readiness gate where outcome, scope, design, implementation, verification, operations, recovery, ownership, and risk evidence receive met, gap, exception, reduced-scope, or block dispositions.

For each criterion record status:

- **met:** current executed evidence supports the criterion;
- **gap:** evidence missing, stale, inconclusive, or failed;
- **exception:** designated owner accepts stated residual condition under scope/conditions/expiry;
- **reduced scope:** evidence supports only a narrower cohort/capability;
- **block:** consequence or authority rule prohibits exposure.

Missing evidence is never silently “not applicable” or passed. [CLM-130]

The five-way disposition and packet are original synthesis tools. [CLM-129]

Build a criterion table that can reject the launch

Every row needs criterion, consequence, required evidence/version, observed evidence, freshness/coverage, disposition, owner/authority, condition, and next trigger.

For fictional Orchid:

Criterion	Evidence state	Disposition
equipment/evidence binding across clear, ambiguous, and missing states	current contract and slice tests pass	met for bounded pump family
prohibited suggestion blocked before approval	critical-segment evaluation passes; limited production-like volume	met for internal cohort, uncertainty recorded
qualified review	representative internal task evidence passes	met for internal cohort only
low-connectivity fallback	synthetic test passes; target field-site evidence missing	reduced scope; exclude low-connectivity sites
break-glass recovery under failed identity dependency	not executed	gap; blocks safety-relevant/wider cohort
support/command schedule	named and rehearsed for 48-hour window	met under stated window

The table prevents a green aggregate from hiding an unexecuted consequential control. It also lets supported value continue in a narrower cohort without pretending the gap disappeared.

Separate facts, unknowns, judgments, and preferences

Facts

Versioned observations: tests passed/failed, restore executed/not executed, cohort signal coverage, known latency/capacity, approvals, artifact hash, open defect.

Unknowns

Unrepresented users/sites, production tails, dependency behavior, weak sample, missing UAT, unexecuted recovery, unresolved legal/security/privacy/safety decision.

Judgments

Risk tolerance, threshold, representativeness, acceptable residual condition, business value, authority disposition.

Preferences/pressure

Deadline, announcement, commercial expectation, stakeholder enthusiasm, sunk cost.

Pressure matters to the decision but cannot become technical evidence. [CLM-137]

Build a defensible recommendation

The FDE should state:

- recommended disposition;
- exact cohort/scope/duration;
- evidence that supports it;
- gaps/uncertainty and consequence;
- alternatives and tradeoffs;
- conditions/compensating controls;
- success/guardrail/stop/recovery triggers;
- decision authority and required signatures;
- communications/support/command plan.

The FDE can recommend; designated customer authorities accept risk. [CLM-138]

Example:

Recommend delay for the safety-relevant cohort because qualified-review UAT and break-glass restoration are unexecuted. A reduced-scope, non-safety internal cohort is supportable for 48 hours if evidence binding, critical policy, latency/fallback, audit, support, and stop gates remain green. Release authority must sign; any cross-boundary denial anomaly, critical-quality failure, unreconciled intent beyond the owner limit, or unavailable recovery access stops exposure.

This is not indecision. It separates supported value from unsupported consequence.

Present real options in the meeting

Do not frame the decision as `launch` or `fail`. Present go for the exact supported cohort; conditional go under a legitimate exception; reduced scope that removes the unsupported state; delay until named evidence/authority arrives; and stop when a bounded safe path does not exist.

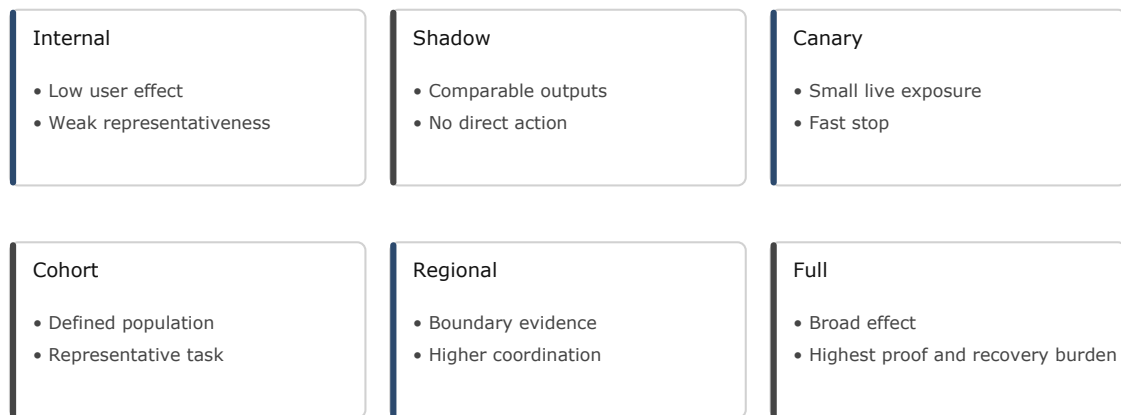
For each option, state value gained or delayed, users affected, residual uncertainty, operating cost, reversibility, evidence learned, and recovery. A reduced scope that excludes the hardest representative condition may produce little learning. A delay can preserve a safer migration window. A stop can be correct.

The designated authority chooses. Preserve the FDE recommendation, dissent, conditions, and trigger so future readers do not confuse the final decision with unanimous technical belief.

Choose rollout pattern by evidence need

F15.2 · FORWARD DEPLOYED ENGINEERING

Rollout patterns buy different evidence



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F15.2 - Rollout patterns buy different evidence. Pattern selection balances representativeness, blast radius, reversibility, support, and stop needs.

Text alternative: Comparison of internal, shadow, canary, cohort, regional, and full rollout across user effect, evidence, comparability, representativeness, blast radius, reversibility, duration, support and recovery.

Internal

Real system under limited authorized internal users. Useful for operations/workflow rehearsal; may not represent customer users/data/dependencies.

Shadow

Observe/calculate without affecting user decision. Useful for comparison/capacity/data quality; carries data/privacy/cost risk and does not prove user behavior.

Canary

Partial/time-limited changed cohort versus control plus absolute criteria. Useful for release regression; weak if volume/cohort/quality signals cannot reveal target failure. [CLM-131]

Cohort or region

Selected users/sites/capability/region. Useful for representative learning and blast-radius control; requires legitimate selection, support, isolation, rollback/recovery.

Full

All intended exposure. Appropriate only when evidence/ownership/capacity/recovery justify it; still monitored and reversible where possible.

A small cohort is not automatically low consequence. One safety-critical site or irreversible external action can be high risk. [CLM-132]

Match pattern to the uncertainty

Choose shadow when the main unknown is data/quality/capacity and user effect must remain zero, while still governing data and cost. Choose canary when changed behavior can be compared meaningfully and stopped. Choose cohort/region when workflow, support, or environment representation matters. Use internal exposure for rehearsal, not as a substitute for customer evidence. Full exposure is not a learning shortcut.

Record why rejected patterns do not answer the decision. If Orchid's uncertainty is whether technicians can evaluate evidence under intermittent connectivity, an office internal cohort is not representative. If safety-relevant recommendations cannot be safely shadowed because reviewers may treat outputs as advice, the shadow design needs stronger isolation or rejection.

Define the observation window

Record:

- start/end and minimum volume/time;
- expected workload/events/critical cases;
- baseline/control and confounders;
- signal freshness/coverage;
- service, workflow, data, AI/policy, security, support, capacity, and cost criteria;
- owner on watch and decision cadence;
- criteria to widen, hold, reduce, stop, or recover.

Do not choose the window after seeing a convenient green period. A short quiet canary may prove little. Simulation strengthens evidence but cannot remove production-tail unknowns. [CLM-135]

Define minimum critical-case coverage as well as time and volume. A 48-hour window with no ambiguous equipment or degraded dependency cannot support those claims. If a needed case does not occur naturally and safe simulation is credible, inject it under controlled conditions and label the evidence type.

Precommit triggers and actions

Examples:

- wrong equipment/evidence binding: stop affected path, preserve/reconcile, notify owner;
- critical prohibited suggestion reaches review: disable bounded suggestion and investigate bundle/control;
- tenant/region boundary anomaly: isolate and follow security/incident authority;
- unknown intent beyond limit: stop new effects, reconcile;
- low-connectivity cohort latency/failure: hold/reduce cohort, use fallback;
- recovery access unavailable: stop widening, repair/rehearse;
- support load exceeds capacity: pause ramp and fix workflow/training/support;
- cost/capacity guardrail: restrict/defer/fallback.

Decide actions before impact. [CLM-140]

For every trigger bind detection, threshold/evidence state, affected scope, immediate action, owner/authority, communication, recovery path, and re-entry criterion. A dashboard threshold without an action is not a gate. A trigger decided after impact is a reaction, not a precommitment.

Make exceptions formal and expiring

An exception requires:

- exact criterion/risk/consequence;
- scope/cohort/duration;
- alternatives/reduced scope considered;
- compensating control and evidence;
- monitoring/stop/recovery;
- designated authority and delegation basis;
- remediation owner/date;
- expiry/review trigger;
- communication to operators/support/users where needed.

Silence, attendance, or deadline is not acceptance. [CLM-136]

The companion makes an exception invalid without designated authority and expiry. It yields conditional go, never full go.

An exception cannot convert failed evidence into success. It says a named authority accepts a stated residual condition for a bounded scope under explicit controls. If authority, consequence, or expiry is missing, the row remains a gap.

Example:

Conditional go for internal non-safety users for 48 hours despite incomplete field-site connectivity evidence. Product release owner accepts reduced representativeness; low-connectivity sites are excluded, offline fallback remains available, support is staffed, and latency/fallback use is reviewed every four hours. Expiry is the end of the window. Any included-site loss of fallback or evidence-binding anomaly stops exposure. This exception does not permit the safety-relevant or low-connectivity cohort.

Prepare people and command

Readiness includes:

- release/incident decision roles and contact schedule;
- on-call/support staffing and access;
- user/operator notice, eligibility, fallback, support path;
- customer/security/privacy/safety/legal communications owners;
- runbooks, dashboards/queries, reconciliation, feature controls;
- change freeze/dependency/vendor coordination;
- status cadence and evidence location.

Technical mechanics without operational ownership are not ready. [CLM-133] [CLM-139]

Resolve the Orchid deadline injection

At deadline, Orchid has passing synthetic evidence, a local recovery rehearsal, and an unexecuted customer recovery-access/control test.

It must not label the control met because documentation exists. The appropriate options are:

1. delay all exposure until executed evidence;
2. reduced-scope internal/non-safety cohort that does not rely on the missing control;
3. formal conditional exception only if designated authority can legitimately accept the consequence and recovery conditions.

The companion test chooses delay for an unresolved gap even when the cohort is one site. Deadline pressure is recorded, not promoted.

Conduct the readiness review

Give participants the packet plus conflicting positions: a product stakeholder wants the announced date, operations wants a smaller cohort, security notes unexecuted recovery access, and technicians want the evidence display fixed.

The review must:

1. confirm decision authority and the exact decision;
2. separate facts, unknowns, judgments, and pressure;
3. disposition every criterion;
4. state the consequence of the material gap;
5. compare go, conditional, reduced, delay, and stop;
6. select cohort, pattern, window, and critical-case coverage;
7. precommit widen, hold, stop, and recovery actions;
8. staff command, support, and communications;
9. record decision, conditions, dissent, and next trigger.

Pass when a reviewer can see why the exposure is supported and which evidence would change it. The exercise fails if the date determines evidence state or the FDE self-approves an exception.

Failure modes and repairs

Missing becomes not applicable

Repair: missing, stale, inconclusive, failed, and unexecuted evidence stays a gap until justified scope exclusion or formal decision. [CLM-130]

The small pilot is called safe

Repair: examine consequence, representativeness, external state, critical-case coverage, support, and recovery rather than user count. [CLM-132]

Canary ends when the chart is green

Repair: predefine time, volume, critical segments, absolute/comparative criteria, confounders, and decision actions. [CLM-131]

The meeting allows only go

Repair: present all five dispositions with consequence and opportunity cost. Readiness is a decision system, not launch theater. [CLM-134]

The FDE accepts adjacent risk

Repair: recommend and escalate; named customer authorities own formal exceptions and risk decisions. [CLM-138]

Triggers are invented during impact

Repair: bind detection, action, authority, communication, recovery, and re-entry before exposure. [CLM-140]

Operate the decision after exposure begins

A signed go record is not the end of readiness. During the observation window, preserve the selected cohort, bundle, evidence coverage, staffing, and conditions. Record every hold, exception, configuration change, support spike, missing signal, and departure from the plan.

At each cadence, choose one of:

- **widen:** criteria and critical-case coverage support the next named cohort;
- **continue:** the current window needs more evidence and conditions remain bounded;
- **hold:** stop widening while a gap is investigated;
- **reduce:** remove an unsupported cohort/capability without pretending full success;
- **recover:** execute rollback, roll-forward, isolate, restore, or fallback by actual state;
- **stop:** end exposure and preserve evidence when the path is not supportable.

The post-window record should compare expected and observed behavior, not merely say pilot complete. Include outcome/guardrail/adoption signals, critical segments, incidents/support, capacity/cost, recovery, limitations, confounders, and the next authority decision.

Keep the decision bundle stable enough to interpret

If code, configuration, model, prompt, index, policy, cohort, training, and support all change during one window, a favorable or unfavorable result becomes hard to interpret. Freeze material parts where possible and record necessary changes as new evidence versions.

An urgent correction may be necessary. Do not delay it for experimental purity. Contain harm, version the change, reset the claim/window if needed, and state what evidence is no longer comparable.

Distinguish rollout from migration cutover

A cohort feature can often be disabled. A data migration or external workflow cutover may mutate state that cannot be assigned back to a control group. For these changes, readiness emphasizes compatibility, capacity, reconciliation, recovery access, cutover command, and the last safe decision point.

Suppose a fictional analytics deployment replaces a nightly file with streaming events. A 5 percent consumer cohort does not limit upstream schema or backlog consequence if all events enter the new pipeline. The rollout unit must reflect the actual state boundary, not the UI audience. This non-AI transfer case changes the decision and prevents small cohort language from hiding shared infrastructure risk.

Write the final go/no-go record

Use this decision summary:

- exact decision, scope, cohort, bundle, and window;
- evidence packet version and critical claims;
- facts, unknowns, judgments, and pressure;
- met/gap/exception/reduced/block table;
- options and consequences;
- FDE recommendation and dissent;
- designated authority and decision;
- conditions, owners, expiry, and communications;
- widen/continue/hold/reduce/recover/stop triggers;
- post-window evidence and next decision.

Another reviewer should be able to understand why the authority chose the path even if they disagree. That auditability matters when later production evidence contradicts the original assumptions.

Complete OA-09

The readiness artifact includes:

- criterion/evidence/disposition table;
- facts/unknowns/assumptions/preferences;
- options: go/conditional/reduced/delay/stop;

- calibrated recommendation and consequence;
- cohort/pattern/ramp/observation;
- success/guardrail/stop/recovery triggers;
- command/support/user/stakeholder communication;
- designated authority/decision/signatures/conditions/expiry;
- post-window decision and evidence update.

The companion adds five tests: missing evidence invalid, gap delays a small cohort, formal exception yields conditional go, reduced scope cannot become full go, and go/no-go records preserve unknowns/triggers/communications/recovery.

The Chapter 15 gate

Before exposure:

- all required evidence is current or visibly gapped;
- owner-accepted exceptions are formal/expiring;
- readiness reviews operability/ownership proportional to risk;
- cohort/pattern can reveal the target uncertainty;
- observation has relative and absolute criteria;
- command/support/user preparation and access are verified;
- stop/recovery/communication triggers are precommitted;
- named authority selects and signs the disposition;
- FDE recommendation preserves alternatives and consequence;
- companion readiness tests pass.

The gate passes when the decision record remains defensible even if the answer is delay or reduced scope.

Chapter 16, *Stabilize Under Real Conditions*, begins when production reality contradicts assumptions and the team must contain impact, communicate calibrated facts, restore service, correct the system, and prove stabilization.

16. Stabilize Under Real Conditions

Operate inside customer incident command, contain impact, communicate calibrated facts, restore service, correct systemic conditions, and prove stabilization exit.

When reality contradicts readiness, first protect people and the workflow. Then establish truth.

The FDE may be the deepest technical lead, but operates inside the customer's incident structure. Incident command, customer/public communications, legal/privacy/security/safety decisions, and business authority remain explicitly designated. [CLM-142] [CLM-151]

Establish command and impact before deep debugging

Confirm:

- incident commander and deputies;
- technical, customer-impact, and communications leads;
- affected users/cohorts/workflow/data and consequence;
- current exposure and containment authority;
- evidence/timeline/decision log location;
- update cadence, escalation, and next checkpoint;
- access, privacy, safety, legal, and support constraints.

Do not begin with "root cause." Initial priorities are impact, command, containment options, and evidence-safe coordination.

The companion rejects a record where an FDE silently names themselves commander without explicit designation.

Write the first ten-minute record

Before a deep technical theory, record who declared the incident and who commands, the affected workflow/cohorts, observed and worst credible near-term consequence, current exposure, containment options, first evidence requests, next update, and authorized communicator.

For a fictional Orchid low-connectivity cohort:

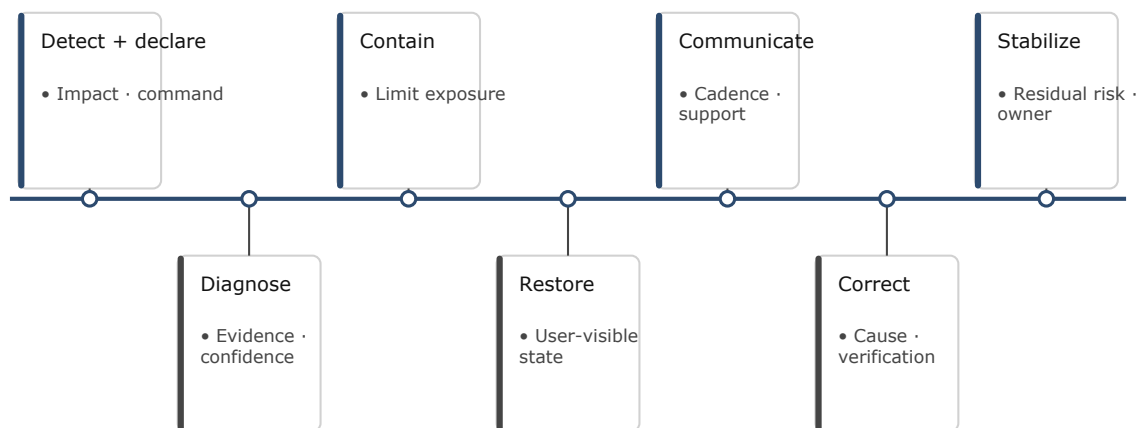
At 10:07 support confirmed technicians at two sites cannot complete evidence review within the operating window. No prohibited approval is confirmed. Expansion is held. Customer operations commands; the FDE leads technical diagnosis. Current unknowns are site connectivity, retry amplification, stale configuration, and evidence-service latency. Offline fallback remains available but adds manual time. Next update is 10:25.

This is safer than system slow, engineering investigating. It states impact, boundary, containment, uncertainty, and cadence without inventing cause.

Run parallel incident tracks

F16.1 · FORWARD DEPLOYED ENGINEERING

Incident work runs in parallel



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F16.1 - Incident work runs in parallel. Impact, diagnosis, recovery, communications, and correction share one confidence-tagged timeline.

Text alternative: Parallel incident timeline with impact and command, diagnosis and evidence, containment and recovery, communication and support, and correction and verification tracks, showing restoration begins stabilization.

Impact and command

Who is affected, what can/cannot be done, how consequence changes, which authority decides.

Diagnosis and evidence

Correlated metrics, traces, events, workflow/data/model/policy/support evidence; hypotheses and falsification.

Containment and recovery

Disable/hold/isolate/fallback/reconcile/rollback/roll-forward/restore/stop, then verify user-visible state.

Communications and support

Confirmed impact/actions/unknowns/next cadence through authorized owners; support scripts and escalation.

Correction and learning

Systemic condition, durable change, regression evidence, monitoring window, owners/dates.

These parallel tracks are original synthesis, not a universal incident standard. [CLM-141]

Coordinate tracks through decisions

Parallel does not mean independent. Each track reports decisions and evidence through command.

If diagnosis finds retry amplification, containment must choose whether to isolate the cohort, disable suggestion, cap retry work, or activate fallback. Communications explains user effect without exposing speculation. Evidence preservation captures version/correlation state before configuration changes. Correction records the lasting system change after recovery.

Use a decision log entry with timestamp, decision, facts/unknowns, options, consequence, authority, action owner, review trigger, and linked evidence. The log prevents a fast response from becoming an unauditable sequence of chat messages.

Keep a confidence-tagged timeline

Each entry includes time/timezone, track, statement, confidence, evidence, actor/decision, and correction history.

- **confirmed:** supported by named evidence;
- **probable:** current hypothesis with basis and test;
- **unknown:** important unresolved question.

Do not rewrite old entries to look certain. Append corrections and decision changes. The companion requires evidence for a confirmed entry and preserves chronology.

An example sequence:

- 10:07 confirmed: two sites exceed the evidence-review latency window; linked support cases.
- 10:10 confirmed: expansion held by release owner.

- 10:13 probable: retry demand may amplify evidence-service saturation; trace sample supports correlation, capacity evidence incomplete.
- 10:17 decision: isolate two sites and cap new suggestion work; operations authority; review at 10:30.
- 10:23 disproved: stale configuration not present; release/config hashes match.
- 10:28 confirmed: retry queue drains after isolation; user fallback remains available.

The timeline preserves the disproved hypothesis because it explains why the team stopped pursuing it.

Metrics, traces, logs, workflow state, data/model/policy, and support answer different questions. [CLM-145]

Communicate calibrated facts

An update states:

- observed impact/scope;
- confirmed facts and evidence time;
- unknowns/hypotheses labeled;
- containment/restoration actions and consequence;
- next update time;
- approved workaround/support path;
- ETA only when authorized and genuinely supported.

The companion rejects a “probable” ETA presented as ETA. [CLM-143]

Use a fixed update structure

Write impact, confirmed facts, labeled unknowns/hypotheses, actions/owners, user fallback, next cadence, and authority in that order. If no credible restoration estimate exists, say so and give the next evidence checkpoint. A confident cadence is not a speculative resolution time. Customer/public communications remain with the authorized owner. [CLM-151]

Avoid speculative cause, blame, invented reassurance, and sensitive technical detail in broad channels. Communications use the customer-approved owner/process.

Diagnose by consequence and boundary

For Orchid low-connectivity review failure:

1. confirm affected cohort and whether safety workflow is blocked/misleading;
2. hold expansion and enable truthful fallback;

3. compare release/config/site/dependency and retry/new-demand signals;
4. trace equipment/evidence/inventory/model/policy/approval path;
5. inspect capacity, stale configuration, timeout and retry amplification;
6. preserve minimized evidence and reconcile unknown intents;
7. choose containment/recovery using Chapter 14's state tree.

For an AI-quality incident, service health can remain green. Investigate equipment binding, evidence freshness, retrieval, model/prompt/index, policy, reviewer workload, critical segments, and support reports. [CLM-150]

Follow a boundary-first diagnostic sequence

Ask which boundary can produce the observed state: user/device/connectivity; identity/tenant/region; interface semantics; dependency capacity/finality; retrieval/provenance; model/prompt/policy/reviewer; release/config/migration; or telemetry/support measurement.

For each candidate, name the next discriminating evidence before broad searching. Compare affected and unaffected cohorts, versions, and workflow states. Preserve tails and critical segments. Metrics show magnitude/trend; traces show path/timing; logs show selected events; workflow state shows user consequence; support reports show lived symptoms. None is a universal truth source. [CLM-145]

Balance evidence preservation and containment

Preserve release/config/policy/model identifiers, correlation/intent IDs, timeline, decisions, minimized telemetry, relevant artifacts, and access/audit state.

Do not delay necessary harm reduction merely to collect perfect evidence. Do not casually copy sensitive payloads or alter/delete evidence. Follow customer incident, privacy, security, safety, and legal procedures. [CLM-146]

Use an evidence-preservation decision: what is needed to understand or prove the state, what is safe/legal to retain, who can access it, and whether collection delays containment. Preserve identifiers, versions, decisions, and minimized diagnostic state before copying raw content. If an authorized forensic process exists, follow it; the FDE does not improvise specialist collection.

When immediate shutdown would erase volatile evidence but continued operation can harm users, consequence comes first. Record the lost evidence and why containment was chosen. Honest uncertainty is preferable to additional harm for a perfect timeline.

Separate restore from correct

Restore: return an acceptable user-visible state: disable feature, isolate cohort, fallback, rollback/roll-forward, restore, reconcile.

Correct: remove/mitigate contributing system conditions, verify regressions, update controls/signals/runbooks/ownership, and monitor.

A workaround can restore quickly and remain dangerous as permanent architecture. Restoration does not end stabilization. [CLM-144]

For the Orchid example, isolating two sites and activating manual evidence review may restore an acceptable user path. It does not correct retry amplification, weak capacity gating, or the signal that failed to distinguish new demand from retries. The restoration record states current limitations and support burden. The corrective record changes budgets, queue behavior, tests, alerts, and the runbook, then verifies the affected cohort.

Define restoration acceptance separately from stabilization exit. Restoration may require an acceptable bounded workflow now; stabilization requires durable correction and evidence over time.

Find contributing conditions, not a person to blame

Ask how design, review, data, defaults, access, alerts, rollout, workload, incentives, documentation, ownership, or recovery made the event possible or harder.

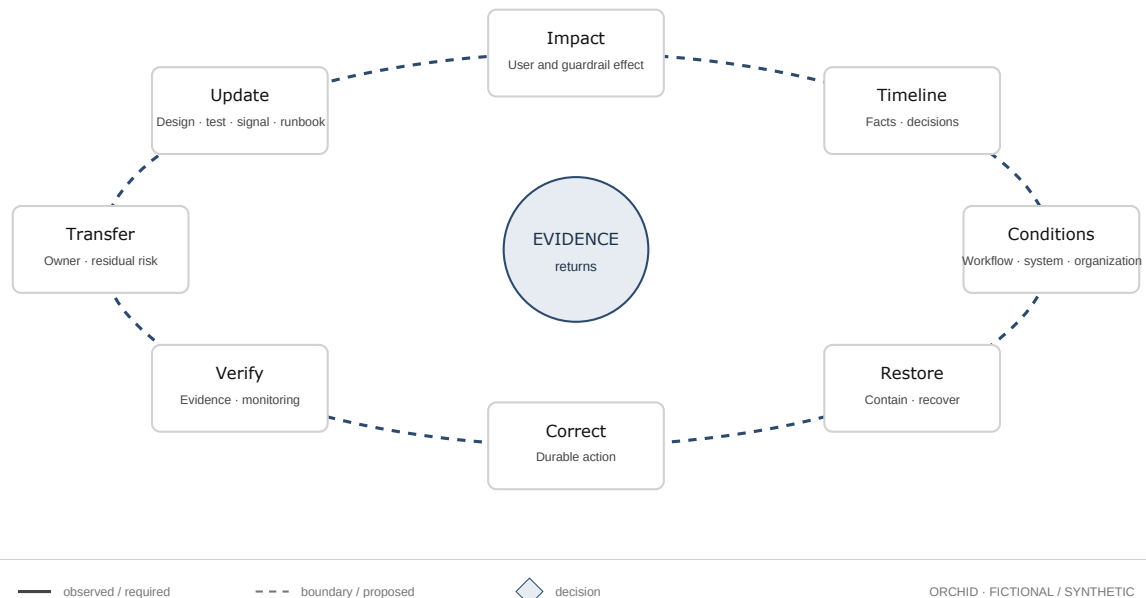
Blameless learning is not accountability-free. It preserves evidence, actions, owners, local legal/safety/employment obligations, and real decision responsibility. [CLM-147]

Avoid stopping at a label such as operator error, bad deploy, model hallucination, or customer network. Ask what system conditions made the action possible, hard to detect, or hard to recover: confusing defaults, missing review, ambiguous ownership, inadequate capacity test, hidden configuration, poor evidence display, unavailable access, incentive conflict, or weak rollout.

A contributing-condition analysis can include workflow, software, data, environment, controls, tools, documentation, staffing, communication, and decision rights. It should not speculate about individuals. Where misconduct, safety, legal, or employment processes apply, route them to the designated authority rather than using a technical post-incident review as a substitute.

F16.2 · FORWARD DEPLOYED ENGINEERING

Learning changes the system



F16.2 - Learning changes the system. A durable incident response updates design, tests, signals, runbooks, ownership, and product patterns.

Text alternative: Learning loop from user impact and detection through timeline, contributing workflow, system and organizational conditions, containment, restoration, corrective action, verification, owner transfer and updated system artifacts.

Write corrective actions as system changes

Each action contains:

- contributing condition/consequence;
- system/control/test/signal/runbook/process change;
- owner/due date/priority;
- verification method/data/environment;
- observed evidence and residual gap;
- escalation if overdue.

“Be more careful” is not a corrective system action. [CLM-149]

For retry amplification: separate new/retry budgets, preserve intent/reconciliation, cap/queue/fallback, add cohort capacity tests/signals, and verify failure recovery.

For stale configuration: version/promote/validate config, fail closed, correlate release state, add drift signal and rollback/roll-forward exercise.

For quality regression: restrict model/cohort, restore known bundle/fallback, add critical cases/segments/graders, verify policy/review, monitor the correction.

Verify corrective action, not ticket closure

For retry amplification, a completed action is not added `limit`. It includes a stable intent/reconciliation contract, separate new/retry capacity, bounded queue/fallback, a representative failure test, observable cohort signals, an owner, and evidence that recovery no longer amplifies load under the tested condition.

For stale configuration, verify version binding, promotion checks, drift detection, fail-closed behavior, and recovery with the affected environment. For a quality regression, bind model/prompt/index/policy/grader versions and confirm critical segments plus human-review behavior. Close only when the observed evidence meets the stated pass condition.

Define stabilization exit evidence

Require:

- impact contained and affected state reconciled;
- service/workflow restored under current scope;
- durable correction implemented and reviewed;
- regression/failure/recovery evidence passes;
- monitoring window complete across affected cohorts/tails;
- support/adoption state understood;
- corrective actions owned/tracked;
- customer operators own current runbooks/access;
- residual risk/exception formally disposed;
- communications closeout authorized.

The companion refuses exit when only `impactContained` and `serviceRestored` are true. [CLM-148]

Conduct a timed stabilization exercise

Inject a low-connectivity latency increase followed by retry amplification. Give participants changing evidence: first a support report, then healthy aggregate availability, then cohort latency, then a trace sample, then evidence that configuration is current.

Evaluate whether the team:

1. establishes customer command and technical roles;
2. states impact and holds expansion;

3. keeps confirmed/probable/unknown distinct;
4. chooses next discriminating evidence;
5. contains consequence without destroying evidence casually;
6. communicates fallback and cadence without speculative ETA;
7. restores an acceptable workflow;
8. finds systemic contributing conditions;
9. writes verifiable corrective actions;
10. refuses stabilization exit until correction, regression evidence, observation, ownership, and residual-risk disposition exist.

The output is the timeline, decision log, updates, containment/recovery record, contributing conditions, corrective actions, verification window, and stabilization decision. Speed matters, but coherence and safe judgment matter more.

Failure modes and repairs

Debugging starts before command

Repair: name impact, commander, leads, exposure, containment authority, log, and cadence first.

A hypothesis becomes external fact

Repair: preserve confidence labels, evidence, corrections, and authorized communications. Do not promise an unsupported ETA. [CLM-143]

Healthy infrastructure hides workflow harm

Repair: inspect workflow, data, model/policy, approval, cohort, and support evidence as well as service metrics. [CLM-145] [CLM-150]

Restoration closes the incident

Repair: keep stabilization open through durable correction, regression evidence, observation, ownership, and residual-risk disposition. [CLM-148]

Blame replaces explanation

Repair: identify systemic contributing conditions and evidence-bearing corrective actions while preserving real accountability and formal processes. [CLM-147]

A permanent workaround becomes the architecture

Repair: record its limitations/expiry and deliver the durable system change or an explicit accepted exception.

Assemble the stabilization report

The report is not a polished story that removes uncertainty. It is the versioned bridge from incident state to owned operation.

Include:

- incident ID, scope, cohort, duration, and user/workflow consequence;
- designated command and communications authority;
- confidence-tagged timeline and decision log;
- releases/config/model/policy/dependency states involved;
- containment and restored user path;
- reconciled external/data state and remaining unknowns;
- contributing conditions across workflow, system, controls, and ownership;
- corrective actions with verification evidence;
- observation window across affected segments and tails;
- support/adoption impact and communication closeout;
- residual risk/exception and formal authority;
- operator/runbook/access transfer;
- stabilization decision and review trigger.

Continue the Orchid scenario through exit

At 10:17 the two affected sites are isolated and new suggestion work is capped. At 10:28 the retry queue drains; manual fallback works but adds delay. By 10:50 the team confirms that clients treated unknown inventory completion as new requests after a connectivity timeout. No wrong evidence binding is observed. The restored path is acceptable for the bounded cohort, but expansion remains held.

Correction separates new and retry capacity, preserves intent IDs, forces reconciliation before another external effect, adds queue/fallback limits, and instruments low-connectivity plus retry demand. A regression suite injects lost responses and verifies one external effect. A failure rehearsal verifies recovery and a customer operator executes the updated runbook.

The observation window includes both affected sites, sufficient unknown-completion injections, support load, latency tails, and reconciliation age. One known limitation remains: the real inventory provider's rare finality behavior is not represented by the local double. The integration owner schedules representative evidence before wider exposure.

Stabilization can close for the current bounded path if the designated authority accepts that limitation and the existing scope does not depend on the unsupported case. It cannot close for a wider claim. This distinction preserves value without laundering a gap.

Transfer the method to a non-AI incident

Consider a fictional public-sector document migration where a partial schema rollout makes some records unreadable. The same method applies: establish command/impact, expose divergent state, stop mutation, select recovery by compatibility and consequence, communicate verified user paths, correct the migration/control, and prove restored records plus operator ownership.

The diagnosis does not mention models, prompts, or graders because they are irrelevant. What transfers is confidence-tagged evidence, actual-state recovery, designated authority, systemic correction, and stabilization exit. This case checks that the chapter teaches deployment operations rather than an AI-specific incident playbook.

Review the timed exercise

After the simulation, ask participants to identify:

- which early fact changed containment;
- which hypothesis was disproved and how;
- whether any update overstated certainty or authority;
- what evidence was lost or protected during containment;
- where restoration was mistaken for correction;
- whether corrective actions have verifiable system effects;
- which criterion actually permitted stabilization exit;
- what remains unknown and who owns it.

A fast containment with an incoherent log needs repair. A precise log that delays harm reduction also fails. Professional performance balances consequence, evidence, authority, and time.

Complete 0A-10 stabilization layer

The artifact contains command/roles, impact, chronological confidence-tagged timeline, decision log, customer updates, evidence, containment/restoration, contributing conditions, corrective actions, verification window, residual risk, and stabilization disposition.

Incident response connects detection/response/recovery/improvement to organizational risk management. [CLM-152]

The companion adds five tests for evidence-backed chronology, command boundary, speculative ETA rejection, systemic corrective action, and restoration-versus-stabilization.

The Chapter 16 gate

Stabilization exits only when:

- customer command/communications authority is explicit;
- impact/facts/unknowns/decisions/timeline remain auditable;
- containment and recovered user state are verified;
- evidence is preserved without delaying harm reduction;
- durable correction and regression evidence exist;
- observation covers affected cohorts/quality;
- corrective actions and residual risks are owned;
- customer operators/support accept the current state;
- companion incident tests pass.

Chapter 17, *Make the Solution Usable and Ownable*, tests whether users can succeed and customer operators can release, recover, support, and decide without hidden FDE dependency.

17. Make the Solution Usable and Ownable

Diagnose adoption friction without user blame and prove customer capability across use, operation, support, release, recovery, access, decisions, and change ownership.

A deployed system is not successful if users must work around it or if only the FDE can operate it.

Usability asks whether representative people can complete the real workflow. Ownership asks whether the customer can observe, support, release, recover, decide, and change the system without hidden FDE dependency.

Documents support both. They prove neither. [CLM-158]

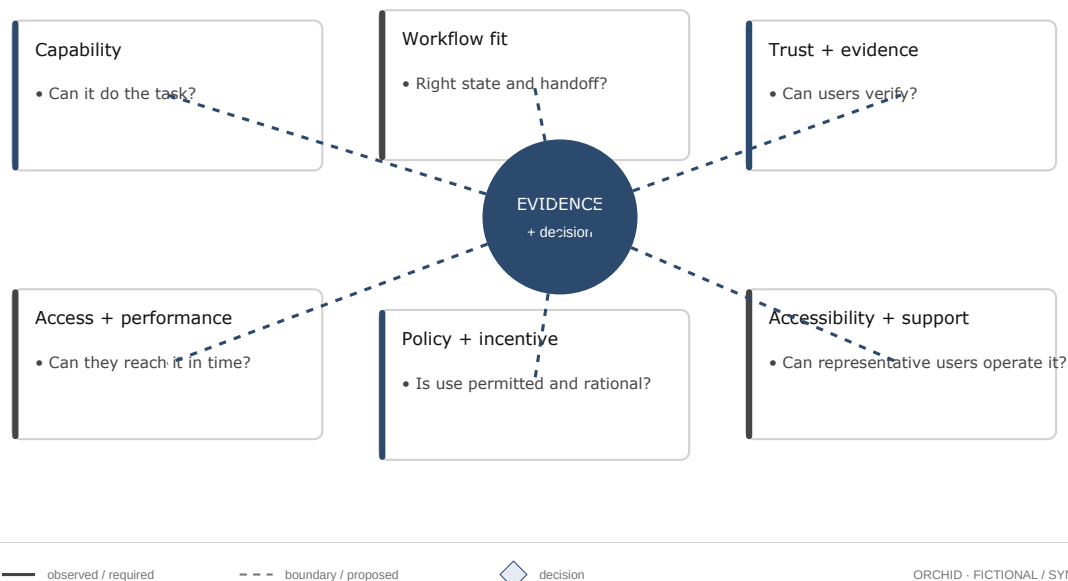
Diagnose adoption friction without blame

Use ten categories:

- **capability:** the function cannot perform the needed task;
- **workflow fit:** sequence/handoff/exception conflicts with work;
- **trust/evidence:** users cannot verify provenance, uncertainty, or consequence;
- **access:** identity, qualification, device, network, or permission blocks;
- **performance:** latency/reliability/connectivity makes use impractical;
- **incentive:** local costs/metrics/authority discourage the intended path;
- **policy:** required procedure/approval conflicts or is unclear;
- **accessibility:** interaction excludes users/assistive contexts;
- **support:** help/escalation/recovery is unavailable;
- **knowledge:** users/operators lack correct mental model or practice.

F17.1 · FORWARD DEPLOYED ENGINEERING

Non-use is diagnostic evidence



F17.1 - Non-use is diagnostic evidence. Repair follows a tested friction hypothesis, not a judgment that users resist change.

Text alternative: Adoption diagnostic map branching non-use, bypass, dropout and support signals into capability, workflow fit, trust, access, performance, incentive, policy, accessibility, support and knowledge.

The companion requires evidence, a next test, and owner for each diagnosis and marks userBlame : false. This taxonomy is original synthesis. [CLM-153] [CLM-154]

Turn a non-use signal into a testable diagnosis

Start with observed behavior, not an attitude label.

Weak entry: Technicians resist the new workflow.

Evidence-bearing entry:

In 8 of 12 observed pump-family tasks, technicians opened the suggestion but returned to manual search before approval. The interface does not show source version or the evidence passage beside the proposed action. Hypothesis: trust/evidence friction. Next test: show evidence-first review to the same role across representative clear, ambiguous, and missing-evidence tasks. Owner: product workflow owner. Alternative hypotheses: latency, equipment-match confidence, policy, or incentive.

The second entry can be disproved. If bypass remains after evidence visibility improves, inspect the alternatives. Perhaps the suggestion arrives after the technician has completed manual search, or

supervisors measure another completion path. The taxonomy routes evidence; it does not diagnose personality.

Keep a friction record with signal, cohort/task, candidate categories, evidence, confidence, next discriminating test, owner, repair, observation window, and outcome/guardrail effect. Close rejected hypotheses rather than leaving every possible cause open.

Five non-use patterns imply different tests:

1. **Never starts:** verify eligibility, access, device/network, awareness, and whether the workflow trigger exists.
2. **Starts then abandons:** inspect latency, state clarity, evidence, accessibility, error recovery, and time pressure.
3. **Completes but bypasses later:** inspect trust, result quality, downstream handoff, incentives, and support burden.
4. **High use with weak outcome:** inspect coerced use, wrong task, gaming, rework, guardrail harm, and attribution.
5. **Works for one segment only:** inspect role, site, language, assistive technology, connectivity, data quality, equipment family, and consequence.

Triangulate adoption evidence

Do not use one metric:

- eligible use and repeat use;
- task completion/time/error/rework/fallback;
- outcome/guardrail movement with attribution limits;
- support demand/reason/resolution;
- satisfaction/trust/evidence inspection;
- dropout/abandonment/manual bypass;
- role/site/region/accessibility/connectivity/risk segments;
- qualitative observation/interview.

Login or usage can be coerced by policy. Satisfaction excludes people who left. Low support can mean ease or inaccessible support. Outcome can change for unrelated reasons. Signals are complementary. [CLM-155]

Build an adoption evidence table

For each target cohort, put signals beside the claims they can and cannot support:

Signal	Can support	Cannot establish alone
eligible/repeat use	exposure and repeated interaction	correct use, outcome, voluntariness
task completion/error/time	workflow performance under observed conditions	sustained outcome or every segment
fallback/bypass/dropout	friction location and recovery behavior	cause without another test
support reason/resolution	operating burden and recurring defects	absence of unreported friction
evidence inspection/approval	review behavior and control use	correctness of the underlying suggestion
satisfaction/trust report	perceived usefulness/confidence	behavior, accessibility, or outcome
outcome/guardrail movement	bounded operational change	attribution to one feature or transferability

Segment the table and record missingness. If users who abandon the workflow also stop answering surveys, a favorable satisfaction average is selected evidence. If telemetry excludes offline completion, low use may be a measurement boundary rather than rejection.

Support demand deserves special care. Rising tickets can mean worsening quality, wider legitimate adoption, improved reporting, or a newly visible limitation. Classify reasons and connect them to releases, cohorts, and resolution evidence. [CLM-161]

Observe the whole task

For Orchid, observe a representative technician:

1. receive/open ticket;
2. confirm equipment or resolve ambiguity;
3. inspect evidence/source/version;
4. distinguish verified fact from suggestion;
5. handle missing evidence/fallback/reconciliation;
6. route qualified approval;
7. complete non-digital inspection/handoff;
8. record decision and find support.

Whole-journey evidence includes non-digital steps, workarounds, exceptions, and support. [CLM-162]

The injected bypass has a specific cause: recommendations omit visible supporting evidence. Classify trust - evidence, test an evidence-first review design, and observe task behavior. Do not send generic retraining first.

Run a whole-task observation

Prepare representative scenarios, roles, devices, connectivity, permissions, and exceptions. Ask the participant to work as normal. Avoid coaching unless safety or the protocol requires it; record every intervention.

Capture the task trigger, prior context, equipment/data state, evidence inspected, decisions, hesitation, backtracking, workarounds, non-digital actions, handoffs, error/fallback/recovery, completion, time, correctness, assistance, accessibility context, and what the user expected next.

Afterward, ask why at specific moments without treating the explanation as complete truth. Combine observation, artifact state, telemetry, and interview evidence.

For Orchid, a successful click path still fails if the technician carries the wrong part because inventory was stale, or if a qualified approver cannot reconstruct the evidence. The unit is the operational task, not the screen. [CLM-162]

Use training for knowledge, not design debt

Training is appropriate when the workflow/design/control is sound and the remaining gap is knowledge/practice. It should contain role-specific tasks, examples/exceptions, safety/authority boundaries, fallback/support, and verification.

Do not train around missing access, misleading state, slow performance, inaccessible UI, invisible evidence, bad policy, or absent support. [CLM-156]

Verify learning through consequential work. Technicians should identify evidence, uncertainty, fallback, and escalation. Qualified reviewers should distinguish a suggestion from an approved decision. Support should diagnose a cohort without exposing payloads. Operators should release, hold, recover, and reconcile. Decision owners should locate current limitations and exceptions.

Evidence is a completed task, accurate explanation, correct escalation, or successful rehearsal, not attendance. If a person succeeds only while the FDE prompts each step, record assisted performance and retest after repair.

Combine accessibility criteria and task evidence

Apply relevant WCAG 2.2 criteria with keyboard/focus/semantics/zoom/reflow/error/assistive-technology checks. Then observe representative users performing consequential tasks.

Conformance evidence does not alone prove usability for every context. [CLM-157]

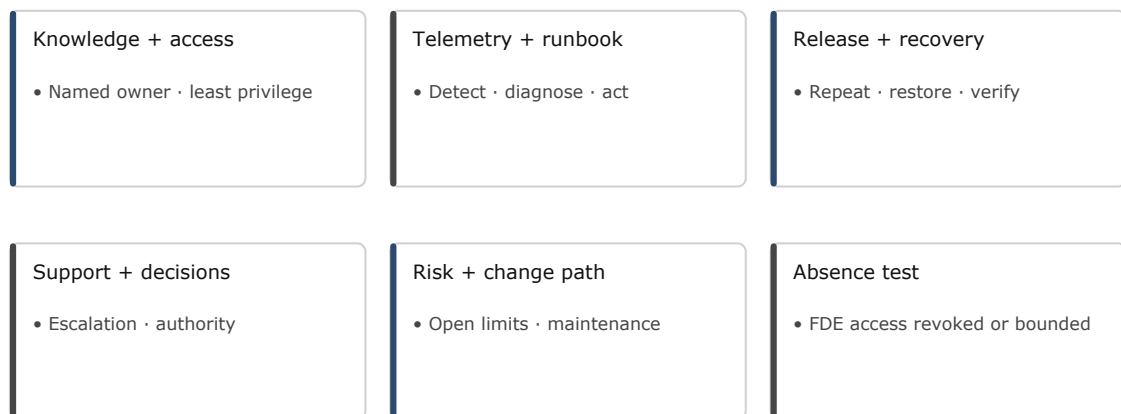
Accessibility work has two linked layers. Criteria-based checks find barriers in semantics, keyboard path, focus, contrast, reflow, error identification, labels, status messages, and assistive-technology compatibility. Representative task observation tests whether the complete workflow works in its intended context, including time pressure, evidence comparison, offline behavior, and support.

Do not claim accessible from an automated scan. Record criteria/version/scope, methods, tools, manual results, assistive contexts, defects, owners, retests, and limitations. Designated accessibility expertise owns formal interpretations where required; the FDE integrates evidence and repairs into delivery.

Build the ownership acceptance model

F17.2 · FORWARD DEPLOYED ENGINEERING

Ownership must be demonstrated



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F17.2 - Ownership must be demonstrated. Customer authority accepts ten operating areas and hidden FDE dependency is removed.

Text alternative: Ownership-transfer model across knowledge, access, telemetry, runbook, release, recovery, support, decisions, open risk, and change path, each with customer owner and demonstrated evidence.

For each area require owner plus demonstrated evidence:

1. knowledge: explain system/workflow/limitations;
2. access: reach required systems under least privilege;
3. telemetry: diagnose injected symptom/cohort;
4. runbook: execute/escalate and improve steps;
5. release: prepare/evidence-gate/hold a release;
6. recovery: choose/execute/verify a recovery;
7. support: triage/user communication/escalation;
8. decisions: identify authority and record disposition;
9. open risk: understand exceptions/expiry/residual support;

10. change path: safely modify/test/release and feed product learning.

Ownership is demonstrated capability, not delivery of a folder or RACI. [CLM-159] [CLM-164]

Define acceptance evidence precisely

For each area, write a scenario and observable pass condition. Has runbook is weak. Customer operator detects an injected low-connectivity degradation, locates the current runbook, chooses the correct cohort hold, and records escalation without FDE intervention is testable.

Evidence can be pass, assisted, failed, not run, or exception. Assisted records what the FDE did because it identifies the remaining dependency. A signed document cannot overwrite a failed demonstration.

Use more than one operator for high-consequence areas. One expert's success can hide a single-person dependency. Test backup access, off-hours escalation, and the failed dependency where relevant. Keep evidence synthetic or appropriately governed; ownership testing does not justify unsafe production experiments.

Run supervised acceptance

Customer operators, not the FDE, perform:

- identify a degraded cohort from signals;
- inspect a correlated trace without sensitive payload;
- hold/disable a feature cohort;
- reconcile an unknown intent;
- execute a synthetic release and partial-migration recovery;
- restore/verify from approved evidence;
- handle support/escalation;
- record a decision/open risk.

The FDE observes and records assistance. If intervention is necessary, ownership is not yet demonstrated for that area.

Run the Orchid acceptance session

Begin with the current bounded scope and limitations. Name customer decision authority and the FDE's observer/support role. Then execute a sequence:

1. A technician encounters an ambiguous equipment match and uses the stop/fallback path.
2. A qualified reviewer inspects source/version and rejects a suggestion lacking evidence.

3. Support identifies a degraded low-connectivity cohort without reading sensitive payloads.
4. Operations holds that cohort, uses the runbook, and communicates through the named channel.
5. Release owner reviews the packet and declines a release with a critical gap.
6. Operators run the partial-migration rehearsal, choose roll-forward from actual state, and verify recovery.
7. A customer owner records an expiring open-risk exception within delegated authority.
8. The team locates the change path, tests a bounded correction, and names product-learning ownership.

Record who acted, access used, evidence, assistance, result, gap, owner, and retest. Do not convert a synthetic rehearsal into a production recovery claim.

Transfer and revoke access

Inventory human/workload/support/emergency permissions. Ensure customer owners have required release/recover/support access, test it, and remove FDE privileges.

The companion blocks transfer when customer permission is missing or the FDE retains a required privileged permission. [CLM-160]

Access transfer is a state transition, not an email request. Inventory accounts, roles, groups, tokens, workload identities, emergency credentials, support paths, local devices, and third-party consoles. For each, record purpose, owner, scope, environment, expiry, last use, approval, test evidence, and revocation state.

Test customer access before removing the last working FDE path, but do not leave both paths indefinitely. Sequence transfer, demonstration, rollback for the access change itself, revocation, verification, and audit review. Rotate or invalidate shared material rather than assuming removal from a group eliminates copied credentials.

If support access continues, define ticket/incident trigger, approved actor, time bound, permitted actions, customer visibility, audit, and removal. The customer must retain an operating path that does not depend on the FDE being available.

Continuing support access must be explicit, purpose/time/scope bounded, approved, audited, revocable, and not the only operating path.

Make handoff exceptions explicit

If the customer cannot yet own an area, record:

- missing capability/evidence and consequence;
- interim owner/support service;

- exact FDE/vendor dependency;
- access/control/monitoring;
- authority and accepted residual risk;
- delivery/retest date;
- expiry/escalation/exit.

Do not disappear or stay forever. [CLM-163]

An exception is acceptable only when the residual dependency is visible and governed. For example:

Recovery key rotation remains assisted for 14 days because the backup operator has not completed the failed-identity exercise. Customer security owns the exception; the FDE may observe but not approve emergency use. Scope is the internal cohort. Monitoring is the access audit plus recovery readiness check. Retest is 30 August. If it fails or the cohort changes, exposure remains held. FDE privileged access expires after the successful retest or on 31 August, whichever comes first.

This is different from FDE will help as needed. It states consequence, scope, authority, evidence, and exit.

Failure modes and repairs

Logins become adoption

Repair: combine whole-task completion, outcome/guardrail, support, dropout, satisfaction, and segments; preserve attribution limits.

Users are blamed for bypass

Repair: describe observed behavior, use the friction taxonomy, run the next discriminating test, and give the repair an owner. [CLM-154]

Training conceals product debt

Repair: use training only for verified knowledge gaps; repair workflow, access, evidence, performance, policy, accessibility, or support defects in the system. [CLM-156]

Accessibility is an automated score

Repair: combine versioned criteria/manual/assistive checks with representative end-to-end task evidence. [CLM-157]

Handoff is a document transfer

Repair: require customer demonstrations for release, recovery, support, decisions, access, and change. Record assistance. [CLM-158]

The FDE remains the hidden operator

Repair: test customer paths, revoke privilege, and formalize residual support with expiry and exit. [CLM-160] [CLM-163]

Exit happens while customer readiness is unresolved

Repair: choose explicit delay, reduced scope, supported exception, or disengagement. Preserve the owner and consequence; do not silently disappear.

Conduct the chapter exercise

Given five fictional non-use signals, classify likely friction categories but select only the next discriminating test, not a final diagnosis. Then run a ten-area ownership acceptance tabletop with at least one failed and one assisted result.

Produce a segmented adoption evidence table, whole-task observation record, friction hypotheses/tests/repairs, role-specific enablement evidence, accessibility record, ten-area results, access transfer/revocation inventory, and explicit exception or final acceptance.

Pass when representative users can complete the task, customer operators demonstrate consequential operating work, all assistance is visible, and the designated authority accepts the bounded state.

Turn diagnosis into a repair experiment

A friction hypothesis is useful only if it changes the system or the next evidence.

For each hypothesis record:

- observed behavior and affected segment;
- candidate category and confidence;
- alternative explanations;

- smallest ethical/representative test;
- repair owner and authority;
- expected task/outcome/guardrail effect;
- observation window and success/failure evidence;
- rollback or stop condition;
- result and next decision.

For Orchid's evidence-visibility hypothesis, compare the current suggestion-first review with an evidence-first version in representative synthetic tasks. Preserve equipment state, evidence quality, connectivity, and reviewer role. Observe evidence inspection, wrong approval, time, fallback, confidence, and support questions. If inspection increases but task time becomes unacceptable, the next repair may be layout/progressive disclosure, not removal of evidence.

Do not optimize adoption by making consequential controls easier to skip. Higher use is not automatically better if evidence inspection or qualified approval falls.

Separate product repair, enablement, and operating repair

Use the diagnosis to route work:

- capability/workflow/evidence/accessibility/performance defect: product or integration repair;
- knowledge gap after sound design: role-specific enablement and practice;
- access/policy/authority gap: named customer owner and formal decision;
- support/runbook/telemetry gap: operating-model repair;
- incentive or organizational conflict: stakeholder/decision owner, not covert interface manipulation.

Some problems need several repairs. A slow review flow can combine dependency latency, insufficient reviewer capacity, poor evidence layout, and a supervisor metric that rewards rapid closure.

Negotiate the engagement exit explicitly

An FDE should neither disappear at document delivery nor remain the permanent operator. Before exit, compare four states:

1. **accepted ownership:** all required areas demonstrated and FDE privilege removed;
2. **bounded exception:** one area remains under explicit residual support with authority, scope, monitoring, expiry, and retest;
3. **reduced ownership/scope:** customer owns a narrower cohort/capability while unsupported work remains closed;
4. **not ready to hand off:** consequence requires delay, different operating owner, or stopped engagement.

Record support channels, response expectations, escalation, change ownership, product-learning path, unresolved decisions, access state, and a future reopen trigger. Commercial support terms belong to the responsible business/legal owners; the FDE states the technical and operating dependency without inventing a contract.

Test the absence of the FDE

Run a final scenario while the FDE observes silently. A user encounters ambiguity, support diagnoses the cohort, operations holds exposure, release authority reviews evidence, and a customer operator executes recovery. The team must locate current limitations and route an exception.

If someone asks the FDE for a hidden command, credential, dashboard interpretation, or approval, record the dependency. Repair and retest, or make it an explicit exception. The absence test is often more revealing than another walkthrough because it exposes where ownership still lives socially rather than in the system.

Complete OA-10

The artifact now includes adoption review, friction hypotheses/tests, enablement, user/operator/support material, accessibility/task evidence plan, RACI plus demonstrated acceptance, release/recovery exercise, open risks/exceptions, access transfer/revocation, and signed ownership decision.

Support demand remains an operating/product signal. [CLM-161]

The companion adds five tests: trust-friction diagnosis, document-only rejection, customer/FDE access transfer, retained-FDE-access block, and complete ten-area acceptance.

The Chapter 17 gate

Handoff passes only when:

- adoption is triangulated/segmented without user blame;
- representative users complete whole-journey tasks;
- accessibility and usability evidence are distinct;
- customer owners demonstrate all ten areas;
- support/decision/open-risk/change paths are explicit;
- customer access is tested;
- privileged FDE access is revoked or formally bounded;
- exceptions are owned/expiring;
- signed authority accepts ownership;

- companion ownership tests pass.

Chapter 18 will convert bounded delivery evidence into product leverage without generalizing one customer workaround into a platform feature.

Part V - Turn Delivery Into Leverage

The final part asks what can legitimately travel beyond one engagement. Field experience is valuable, but it becomes product leverage only after customer-specific data, authority, policy, workflow, and implementation are separated from a repeated invariant.

Chapter 18 closes the bounded outcome review, maintains a pattern ledger, classifies artifacts, prices compatibility and maintenance, and writes propose/defer/reject product packets with validation and disconfirmation. Chapter 19 allocates attention across deployments using consequence, evidence gap, reversibility, and decision latency; it delegates with authority and evidence; communicates shared facts at multiple altitudes; converts recurring escalation into system capacity; and plans growth through capability evidence rather than title or burnout.

OA-11 and the dossier index close the lifecycle. Closure does not mean every uncertainty vanished. It means the canonical source/build, limitations, risk/reuse state, decisions, and next owners are visible and resumable.

18. Turn Field Evidence Into Product Leverage

Classify customer-specific work and propose reusable code, configuration, service, architecture, playbook, documentation, testing, or product feedback only when field evidence earns the abstraction.

The first successful customer implementation is evidence of one context, not a platform mandate.

Product leverage begins by separating the repeated invariant from customer-specific workflow, data, authority, integration, and workaround. “DRY” is a code smell heuristic, not product strategy.

Employer evidence shows FDEs may codify field learning into building blocks, playbooks, and product feedback. It does not provide a universal abstraction threshold. [CLM-166]

Close the bounded outcome review

For Orchid record:

- outcome/guardrail/adoption observations and attribution limits;
- current scope, cohorts, failures, support, open risks;
- stable customer ownership and revoked/bounded FDE access;
- one-off code/config/data/operations;
- repeated seams observed elsewhere, if any;
- proposals, rejected/deferred ideas, owners, next evidence.

Do not turn observed use or satisfaction into proof of transferable product value. [CLM-175]

Run the review as a bounded argument

An outcome review is not a celebration document and not a product request queue. It is a versioned argument about what changed, what probably contributed, what remains unknown, and what can legitimately travel beyond the engagement.

Start with the exact OA-03 contract. For each outcome and guardrail, place the baseline, current observation, cohort, evidence window, known confounders, and data-quality limitation beside one another. Then add adoption and support evidence from Chapter 17. A measure belongs in the review only if a reader can trace its definition and provenance. A favorable number with a changed definition or unrepresented cohort is not comparable.

For fictional Orchid, assume the bounded cohort shows shorter evidence-search time and fewer inventory calls for the covered pump family. That observation is useful, but several explanations remain possible: the curated manual set improved, the selected technicians were more experienced, the ticket mix was easier,

the inventory API was faster during the window, or the suggestion path actually helped. The review can say that the combined deployment coincided with the observed improvement. It cannot assign the improvement to a reusable model or interface seam without a design that supports that attribution.

Review guardrails with the same seriousness. Suppose no prohibited suggestion was approved, but qualified reviewers inspected only a small number of rare critical cases. The correct entry is not safe. It is no observed prohibited approval in the stated sample; critical-case coverage remains limited. That limitation affects both rollout and reuse. A product proposal built on thin critical-segment evidence inherits the gap.

The outcome review should finish with four sets:

- **supported:** observations whose definitions, cohorts, provenance, and limitations are clear;
- **plausible:** explanations worth another test but not yet distinguished from alternatives;
- **unsupported:** attractive stories that the evidence cannot carry;
- **open decision:** a named owner, consequence, next evidence, and review trigger.

This structure prevents two common distortions. The delivery team cannot turn every favorable signal into credit, and the product team cannot dismiss bounded field evidence merely because it is not a controlled experiment. The review preserves useful evidence at its actual strength.

Maintain a pattern ledger

Each entry contains:

- observation/problem and affected workflow;
- contexts and frequency;
- invariant and variance;
- consequence and current workaround;
- evidence quality/limits;
- customer/data/contract isolation;
- candidate leverage type;
- maintenance/compatibility/security/support cost;
- disconfirming evidence;
- next validation and owner.

The ledger and abstraction test are original synthesis. [CLM-165]

Reuse must be earned across contexts



F18.1 - Reuse must be earned across contexts. Isolation, variance, maintenance, compatibility, ownership, validation, and disconfirmation gate every step.

Text alternative: Reuse ladder from isolated customer-specific material through configurable pattern, stable shared contract, operational playbook, and product proposal, with evidence gates at every transition.

The companion rejects an entry that omits disconfirming evidence and defers one-context patterns.
[CLM-167]

Distinguish a pattern from a pile of anecdotes

Count alone is weak. Ten customer requests can share a phrase while describing different workflow states, authorities, data semantics, and consequences. Two contexts can reveal a strong invariant if the same contract failure appears under meaningfully different implementations.

For every candidate, ask what is invariant at the decision boundary. Consider three fictional observations:

1. Orchid needs a stable intent identifier because an inventory reservation may complete after the client loses the response.
2. A constructed document workflow needs a stable submission identifier because a filing may be accepted after a gateway timeout.
3. A read-only analytics deployment retries a metadata lookup whose result has no external effect.

The first two support a possible invariant: a caller must reconcile an externally consequential intent whose completion can become unknown. The third is superficially similar at the transport layer but does not carry the same state or consequence. Treating all three as `retry handling` would create an abstraction too broad to be useful and too weak to be safe.

A pattern ledger entry should therefore contain negative cases. Record where the proposed invariant did not apply, where variance required a different authority or recovery rule, and what evidence would make the common seam uneconomic. Disconfirmation is not pessimism. It is how a reusable design avoids becoming a collection of hidden customer switches.

Work a ledger entry end to end

For the intent and reconciliation candidate, the entry might read:

- **observation:** Orchid inventory reservation can be accepted while the final response is lost;
- **affected workflow:** technician review must not reserve twice or report false failure;
- **contexts:** Orchid plus one constructed filing workflow; a third independent write context is required;
- **invariant:** one semantic intent has one identity, equivalent retries replay a recorded result, different semantics conflict, and unknown completion reconciles before another effect;
- **variance:** resource identifiers, finality evidence, cancellation, retention, authority, and compensation differ by provider and workflow;
- **consequence:** duplicate external effect, incorrect user state, or blocked work;
- **current workaround:** provider-specific lookup and manual reconciliation;
- **evidence quality:** executable synthetic tests and two bounded context models, no representative third integration;
- **isolation:** customer payloads, identifiers, permissions, and provider rules remain behind adapters;
- **candidate leverage:** contract plus reference state machine and test harness, not a universal service yet;
- **maintenance cost:** versioned public semantics, storage/retention, adapter conformance, migration, observability, and support;
- **disconfirmation:** contexts have incompatible intent identity/finality, or support cost exceeds duplicated local implementation;
- **next validation:** implement the contract against a third independent write boundary and run failure/reconciliation tests;
- **owner:** integration product lead, with security/data authorities for retained state.

Notice the proposed reusable object is smaller than the complete Orchid integration. That is usually healthy. Field leverage often comes from a seam whose semantics can be stated and tested, not from lifting the surrounding application into a platform.

Use the reuse ladder

1. customer-specific implementation/data mapping;
2. configurable adapter or local template;
3. reusable library/contract/test harness;

4. shared service or reference architecture;
5. playbook/documentation/evaluation pattern;
6. product capability or roadmap proposal.

Move only as far as evidence supports. Reuse can be non-code. [CLM-172]

Classify before you generalize

Use a three-pass classification for every artifact.

First, identify embedded customer specificity:

- customer or user identifiers;
- workflow states and names;
- regional or contractual rules;
- qualifications and decision rights;
- data-retention and privacy assumptions;
- environment, topology, and dependency behavior;
- thresholds learned from one cohort;
- support and operating ownership.

Second, identify the smallest portable behavior. It might be an interface contract, validation rule, test fixture pattern, decision record, diagnostic tree, runbook structure, or reference boundary. Remove the customer values, but do not erase the semantic constraint that made the artifact useful.

Third, decide the leverage form. A playbook is often safer than shared runtime code when variance is high. A test harness can spread a failure lesson without imposing one implementation. A reference architecture can communicate boundaries without creating a supported service. Product feedback can be correct even when no reusable implementation exists.

Twelve-artifact Orchid exercise

Classify these artifacts and write the reason, isolation rule, owner, and next evidence:

Artifact	Initial disposition	Reason
Orchid equipment-family mappings	reject from reusable package	customer semantics and data
regional qualification matrix	reject from generic policy	formal authority and local rules vary
ticket adapter implementation	local adapter	provider schema and operational behavior
canonical ticket contract	candidate interface template	portable fields only if semantics remain explicit
inventory intent/reconciliation harness	defer as reusable module	promising invariant; third context missing
approved-manual fixture generator	reusable test utility	synthetic and customer-free with clear limits

Artifact	Initial disposition	Reason
deterministic eligibility engine shape	reference architecture	actual rules remain authority-owned configuration
model/prompt bundle	local versioned artifact	evidence and behavior are task/context specific
critical-segment evaluation structure	reusable evaluation pattern	criteria/data stay context specific
recovery rehearsal record	reusable playbook/template	process structure transfers; targets do not
ownership acceptance checklist	reusable template	customer roles/access/evidence must be filled locally
observed time reduction	product feedback evidence	bounded outcome observation, not reusable code or prevalence

The table is a starting hypothesis, not the final decision. For example, a fixture generator may still reveal confidential schema shapes. A reusable template remains unsafe until its sample values, field descriptions, license, provenance, and distribution path have been reviewed.

Orchid classification

- equipment identifiers, regional mappings, qualification rules: customer-specific configuration/authority;
- ticket/manual/inventory adapters: reusable interface shape, provider-specific implementations;
- intent/conflict/reconciliation ledger: candidate reusable module after independent contexts validate the invariant;
- deterministic model/policy/evaluation seams: reusable test/architecture pattern, not universal policy;
- recovery/readiness/ownership records: portable playbook/templates;
- Orchid data, manuals, outcomes: never reusable product data.

Customer-specific data, IDs, permissions, and assumptions must remain isolated. [CLM-168]

Test abstraction timing

Ask:

- Are contexts genuinely independent?
- Is the invariant stable and consequential?
- Is variance understood/configurable rather than hidden?
- Can customer specifics be isolated and removed?
- Does a reusable contract reduce total risk/cost?
- Who maintains, versions, secures, documents, supports, migrates, and deprecates it?
- What evidence would disconfirm/reverse the proposal?
- Is a playbook/test/document enough instead of shared runtime code?

A public contract creates compatibility/maintenance obligations. [CLM-169] Semantic Versioning can signal declared API compatibility but cannot choose the abstraction. [CLM-170]

Premature abstraction hides meaningful differences and multiplies support burden. [CLM-174]

Price the contract you are creating

Shared code is not free code. The decision creates future promises even when nobody writes them down.

For a proposed library, service, or public contract, make the promise explicit:

- which inputs and semantics are supported;
- which compatibility changes require migration or deprecation;
- how tenant, identity, region, and customer data remain isolated;
- which performance and availability behavior is actually owned;
- how security fixes and dependency changes are delivered;
- who answers integration failures and ambiguous behavior;
- how usage, failure, and cost are observed without leaking data;
- how consumers test upgrades;
- how the capability is retired.

If there is no owner or budget for those obligations, the proposal is not ready. That does not mean the learning is discarded. Keep it as a playbook, test suite, or reference implementation until the operating model exists.

Compare two options for the reconciliation seam.

Option A: shared service. Central state can simplify consistent finality and observability, but introduces a new availability, latency, tenancy, retention, and incident boundary. Every consumer depends on its semantics and operational health.

Option B: contract and test harness. Each deployment keeps local state behind a common behavior suite. This reduces central runtime dependency but permits implementation drift and distributes maintenance.

The right answer depends on consequences, context count, operational capacity, isolation, and change rate. Shared is not inherently more mature than local.

Use a reversible abstraction decision

Record the decision as a hypothesis with stages:

1. **observe:** preserve context, variance, and negative cases;
2. **describe:** state the smallest candidate invariant;

3. **simulate**: build customer-free fixtures and failure tests;
4. **validate**: apply against an independent context;
5. **pilot**: expose a bounded consumer with compatibility/support ownership;
6. **promote**: declare and version the supported contract;
7. **reassess**: measure maintenance, exceptions, disconfirming contexts, and total cost;
8. **retire or narrow**: remove a harmful abstraction rather than protecting sunk cost.

Version the evidence and decision at every stage. A candidate can move down the ladder when new variance appears. Reversibility is a sign of disciplined product learning, not failure. [CLM-176]

Distinguish repeated pattern from severe gap

A single deployment can expose a severe safety, security, accessibility, reliability, or workflow gap worth immediate product attention. That is a priority argument, not a prevalence claim. [CLM-171]

Label the packet accurately: “high-consequence gap observed in one bounded deployment; validate breadth” rather than “customers need.”

Two different product conversations

Suppose Orchid reveals that a prohibited suggestion can appear valid when evidence provenance is visually hidden. Even one verified high-consequence occurrence can justify urgent review of the product's evidence-display boundary. The packet should say why the consequence is severe, where the defect can exist, what immediate containment is available, and what product owner must decide. It should not claim the problem is common.

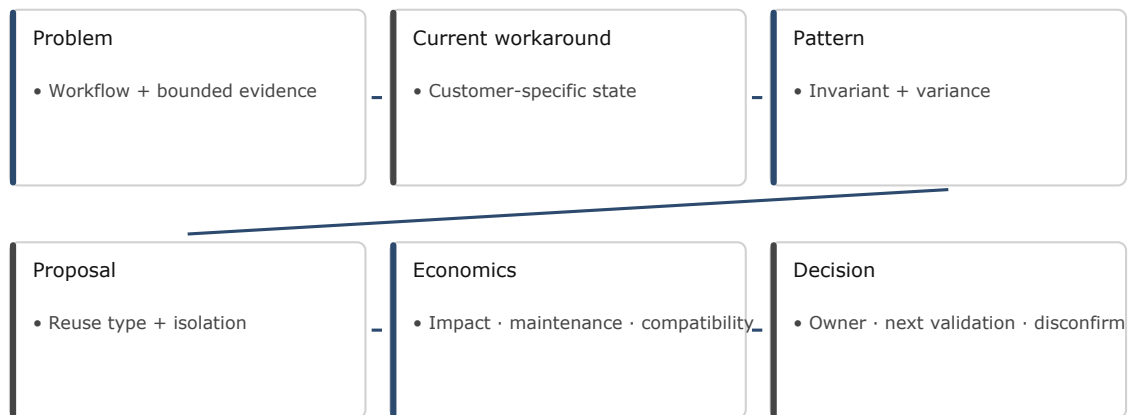
Now suppose three integrations each implement semantically identical unknown-completion reconciliation behind different providers. That supports a repeated-pattern conversation. The question is whether a stable contract and test harness reduce risk and cost across contexts.

The first argument is consequence-led. The second is recurrence-and-invariant-led. Mixing them produces bad prioritization: serious single-context defects can be ignored for lack of frequency, while frequent low-value similarities can become unnecessary platforms.

Write the field-to-product packet

F18.2 · FORWARD DEPLOYED ENGINEERING

A field observation is not yet a product claim



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F18.2 - A field observation is not yet a product claim. The packet preserves evidence, variance, cost, uncertainty, ownership, and a next validation decision.

Text alternative: Field-to-product packet from problem and bounded evidence through workaround, invariant and variance, proposal and reuse type, isolation, impact, uncertainty, maintenance cost, owner, validation and decision.

The companion requires every field, including maintenance and compatibility. [CLM-173]

Fill the packet with decision-grade evidence

A useful field-to-product packet can be reviewed without replaying the entire engagement. It contains:

1. **Decision requested.** For example: fund a third-context validation of the intent/reconciliation contract; do not approve a shared service yet.
2. **Workflow problem.** State the user decision and external effect, not only an API timeout.
3. **Bounded evidence.** Link the exact contexts, traces/tests, outcome or incident observations, and evidence limitations.
4. **Current workaround.** Include operating burden, failure modes, and who carries it.
5. **Invariant and variance.** Separate stable semantic behavior from provider, customer, authority, and data differences.
6. **Proposed leverage.** Name library, service, adapter, configuration, reference architecture, playbook, documentation, evaluation, test, or feedback.
7. **Isolation.** Explain how customer data, identifiers, policy, permissions, and contract constraints stay out.

8. **Expected impact.** State the decision or failure expected to improve and how it will be observed.
9. **Cost and obligations.** Compatibility, maintenance, security, operations, support, migration, documentation, and deprecation.
10. **Uncertainty and disconfirmation.** Say what would cause defer, narrow, reject, or retire.
11. **Owner and authority.** Product owns roadmap priority; relevant technical and specialist owners review their boundaries.
12. **Validation plan.** Name next context, artifact, pass criteria, review date, and decision trigger.

Review the proposal from four positions

The deployment engineer asks whether the packet represents field evidence accurately. The product owner asks whether the capability belongs in product strategy and opportunity cost. The platform or service owner asks whether the contract can be operated and evolved. Security, privacy, safety, legal, accessibility, or other designated specialists decide formal questions in their scope.

The FDE should not obtain roadmap authority by writing a persuasive memo. The role is to make the problem, evidence, boundary, options, and consequences unusually clear.

Example disposition:

Defer platformization of Orchid's complete adapter stack. Propose validation of the intent/reconciliation contract across a third independent write integration. Preserve provider/resource semantics as adapters. Reject reuse of Orchid equipment mappings/data. Owner: integration product lead. Disconfirm if read-only/transaction-specific semantics dominate or support cost exceeds duplicate local implementations.

The statement contains three decisions rather than one vague request:

- **reject** copying the complete customer integration;
- **defer** a shared runtime platform;
- **propose** a bounded validation investment for a smaller semantic seam.

That precision lets a product leader accept one action without accidentally approving the others.

Failure modes and repairs

DRY becomes the product strategy

The team sees duplicated adapter code and centralizes it before comparing semantics.

Repair: compare workflow decisions, authority, failure state, and maintenance burden. Share a test or interface only at the earned invariant.

Configuration hides customer forks

A generic engine accumulates dozens of flags whose combinations are not tested or owned.

Repair: expose the actual variance, define supported combinations, isolate true customer extensions, and reject a shared runtime when configurability merely conceals incompatibility.

The happy customer becomes market evidence

One favorable deployment is used to claim broad demand or value.

Repair: preserve the bounded outcome and attribution limit; ask for independent contexts and disconfirming evidence. [CLM-167] [CLM-175]

Customer material leaks into the reusable package

Fixtures, logs, prompt examples, mappings, or documentation retain customer identifiers or assumptions.

Repair: apply data/provenance review, synthetic reconstruction, semantic isolation, access control, and customer/contract approval where required. If the insight cannot be separated safely, do not distribute it. [CLM-168]

Maintenance is assigned to the platform team

The proposal names no actual owner, compatibility policy, support path, or budget.

Repair: keep the artifact local or documentary until ownership is explicit. Include retirement and escalation, not only creation.

A severe gap waits for pattern count

A high-consequence defect is dismissed because only one customer exposed it.

Repair: route a consequence-led product/risk decision while keeping prevalence unknown. [CLM-171]

The abstraction cannot be reversed

Consumers depend on undocumented behavior and customer-specific extensions before validation finishes.

Repair: stage the contract, bind versions, constrain consumers, test compatibility, and maintain a narrow/retire path.

Conduct the chapter exercise

Take twelve artifacts from a completed deployment. For each, produce:

- current context and evidence;
- customer-specific elements;
- candidate invariant and known variance;
- leverage form and ladder position;
- isolation and provenance rule;
- compatibility/maintenance/security/support consequence;
- owner and authority;
- next validation and disconfirmation;
- disposition: reuse, propose, defer, reject, or retain local.

Then write one field-to-product packet and one explicit reject/defer record. Ask a reviewer who did not work on the deployment to identify the decision being requested and the evidence that would reverse it. If that reader cannot, the packet is not decision grade.

Completion evidence is not the number of artifacts labeled reusable. It is the quality of the distinctions, including work correctly kept local or stopped.

Adapt leverage to organizational maturity

In a fictional early-stage founding-FDE team, the same engineer may own customer integration, core product seams, and the initial pattern ledger. Speed is real, but the decision record still separates customer-specific material, declared product contracts, and temporary local code. A small team cannot avoid maintenance obligations by omitting the owner; it should choose smaller leverage forms it can actually support.

In a mature multi-role organization, product, platform, security, legal/privacy, support, and customer teams may review the packet separately. More roles do not guarantee a better abstraction. The FDE must preserve one evidence record so each review does not reinterpret the field observation. Decision authority and maintenance ownership can be distributed while the invariant/variance and customer-isolation facts remain shared.

The transferable method is the same. Organizational maturity changes who reviews, implements, operates, and funds the proposal; it does not convert one context into prevalence.

Run the product-learning review

Use a 45-minute decision agenda:

1. restate the workflow problem and bounded evidence;
2. show the candidate invariant, variance, and negative cases;
3. confirm customer/data/contract isolation;
4. compare leverage forms, including no shared runtime;
5. price compatibility, maintenance, security, operations, support, and retirement;
6. review disconfirmation and next validation;
7. choose propose, defer, reject, retain local, or escalate a severe gap;
8. name owner, authority, date, and evidence trigger.

Do not spend the meeting redesigning the complete product. The output is the next bounded decision. If the group cannot state what evidence would reverse the proposal, return the packet for repair.

Preserve rejected learning

A rejected platform proposal can still yield a regression test, contract checklist, migration lesson, support taxonomy, or design warning. Record why the runtime abstraction was rejected and which smaller artifacts remain valuable. This prevents a future team from repeating the same failed generalization while avoiding maintenance of an unsupported component.

Complete OA - 11 product layer

The dossier adds outcome review, pattern ledger, twelve-artifact classification, one bounded proposal, one defer/reject record, and field-to-product memo.

Reuse decisions remain versioned/reversible as evidence changes. [CLM-176]

The Chapter 18 gate

- evidence/attribution limits explicit;
- invariant/variance/contexts/frequency recorded;
- customer data/assumptions isolated;
- reuse type chosen rather than default “platform”;
- compatibility/maintenance/security/support cost owned;
- disconfirmation/validation defined;
- roadmap authority remains product-owned;
- one-customer prevalence claims absent;

- companion leverage tests pass.

Chapter 19 will allocate attention across deployments and growth using bounded evidence rather than loudness, title, or false numeric certainty.

19. Lead Beyond One Deployment

Allocate attention across deployments, delegate with authority and evidence, communicate shared facts at multiple altitudes, and grow capability without abandoning engineering depth or inflating authority.

Leading beyond one deployment means increasing the quality and capacity of decisions without becoming distant from evidence. The portfolio record, attention model, delegation contract, multi-altitude memo, and capability-evidence path are original synthesis tools rather than formal standards. [CLM-177]

It does not mean serving the loudest request, creating a colorful numeric heatmap, retaining every escalation, or replacing specialists and customer authorities.

Keep a portfolio record

For each engagement record:

- outcome/guardrail/adoption trajectory;
- current consequence and affected cohort;
- reversibility and blast radius;
- evidence gap/quality/confidence;
- decision latency and deadline context;
- dependency/blocker age and owner;
- readiness/stabilization/ownership state;
- leverage/reuse status;
- next decision, authority, opportunity cost, review trigger.

This keeps portfolio discussion connected to real deployment artifacts. [CLM-184]

Treat the record as a decision interface

A status report describes activity. A portfolio record supports allocation. It must let a reviewer answer: what decision matters next, what is the consequence of delay, what evidence is missing, who has authority, and what receives less attention if this work moves first?

Use a compact record with links to authoritative artifacts rather than copying every engagement document. A useful review row includes:

Field	Question it answers
outcome/guardrail/adoption trajectory	Is the deployment producing bounded value, and for whom?
consequence	What becomes harmful, unavailable, irreversible, or expensive if wrong?

Field	Question it answers
reversibility	Can exposure or state be safely undone?
evidence gap/confidence	Which assumption or decision lacks credible evidence?
decision latency	When does waiting remove a safer option or delay valuable learning?
dependency age/owner	Is a named external decision blocking the path?
readiness/stabilization/ownership	Which lifecycle gate is actually open?
leverage state	Is there a bounded product-learning decision, or only local work?
next decision/authority	Who must choose what, using which evidence?
opportunity cost	Which other review, implementation, or recovery loses capacity?
trigger	What event or evidence changes the allocation?

Keep the row versioned. When consequence or confidence changes, preserve why. Otherwise a portfolio heatmap becomes a memory of colors rather than an auditable decision history.

Avoid averaged engagement health

An engagement can be green on schedule and red on ownership. It can show strong adoption while accessibility evidence is incomplete. It can have stable infrastructure while model quality fails a critical segment. Do not compress these into one health value.

For Orchid, the outcome and adoption trajectory may be bounded-positive, readiness closed, and customer ownership accepted. Product leverage remains deferred pending another context. Those states can coexist. The portfolio record should not turn the deferred reuse proposal into a deployment failure or the successful handoff into proof of platform readiness.

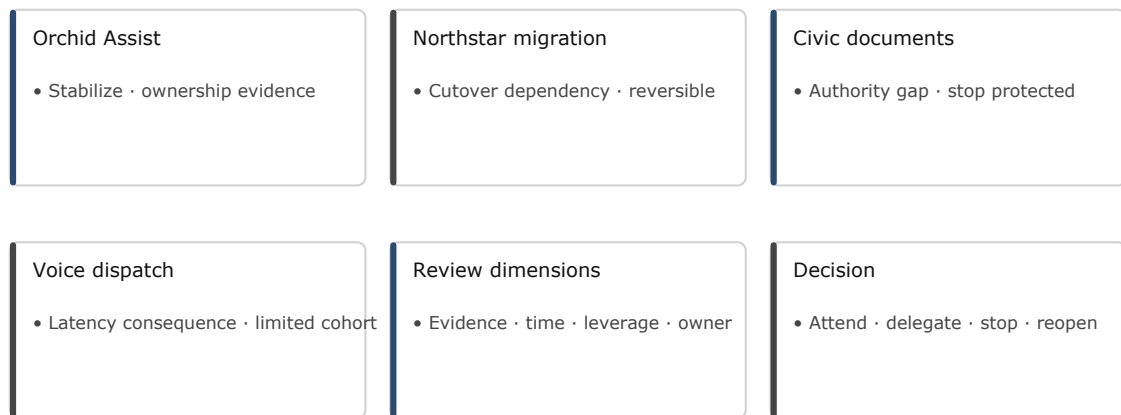
Allocate attention with ordinal judgment

Consider, in order:

1. consequence if wrong or delayed;
2. evidence gap/uncertainty;
3. low reversibility/external state;
4. decision latency/time to safe action;
5. leverage and opportunity cost.

F19.1 · FORWARD DEPLOYED ENGINEERING

Portfolio attention follows decisions



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F19.1 - Portfolio attention follows decisions. No summed risk score replaces consequence, evidence quality, timing, reversibility, and ownership.

Text alternative: Portfolio heatmap with fictional deployments across outcome, guardrail, adoption, consequence, reversibility, evidence gap, dependency age, decision latency, readiness, leverage, owner, confidence and review trigger.

Use low/medium/high/critical as discussion labels with narratives, not measured quantities. The companion sorts ordinally and states that no cardinal risk score exists. [CLM-178]

Make the comparison in a fixed order

When capacity is contested, compare engagements through a repeatable conversation:

1. **Consequence:** Which decision can create or prevent the most serious user, operational, security, safety, data, or business effect within the role's scope?
2. **Evidence gap:** Where is the current decision most likely to be wrong because a material fact, test, owner, or authority is missing?
3. **Reversibility:** Which path loses a safe option, mutates external state, or expands blast radius soonest?
4. **Decision latency:** Which decision has a closing window, and can a smaller action preserve it?
5. **Leverage and opportunity cost:** Where can bounded attention unlock other owners or reusable capacity, and what is displaced?

The order is not a universal formula. It is a guard against common distortions: revenue loudness replacing consequence, activity replacing decisions, and reuse excitement replacing unresolved production risk.

Write a short rationale after ordering. For example:

Harbor is first because a low-reversibility migration decision occurs before peak load and current restore-access evidence is stale. Civic is second because the active cohort has strong use but unresolved privacy/accessibility authority blocks expansion. Orchid is third because ownership is stable and the next reuse decision can wait for a third context. Pulse remains stopped; attention is limited to preserving lessons and contractual closure.

This explanation can be challenged. A score such as 83 cannot be challenged meaningfully unless its units and model are defensible.

The loudest customer is not necessarily the highest-consequence decision. [CLM-179]

Fictional portfolio exercise

- **Orchid:** bounded adoption/ownership, reuse proposal deferred; next decision is evidence for a third integration context.
- **Harbor migration:** critical availability/data state, low reversibility, near decision; highest direct attention.
- **Civic documents:** strong reported use but incomplete privacy/accessibility guardrails; designated authorities and evidence gap drive hold/escalation.
- **Pulse voice:** stopped engagement because latency/consent/support constraints cannot meet guardrails; preserve learning, do not rescue sunk cost.

All are constructed examples, not employer/customer outcomes.

Work the four-engagement exercise

Harbor migration. A fictional availability migration has a fixed compatibility window, data state that cannot be trivially rolled back, and stale recovery access. The next decision is whether to proceed, reduce scope, or delay before peak load. The designated release/data authorities own the choice. Direct senior technical attention is warranted because the decision is near, low-reversibility, and evidence-poor.

Civic documents. A constructed public-sector document workflow shows frequent use and improved processing time, but screen-reader task evidence and privacy review for a wider cohort are incomplete. The record separates bounded adoption from guardrail readiness. The correct move is to hold expansion, bring the designated accessibility/privacy owners into the decision, and fund the missing representative evidence. The FDE cannot accept the gaps because the schedule is attractive.

Orchid. The fictional deployment has passed ownership acceptance for its bounded cohort. A proposed intent/reconciliation seam is deferred. The next action belongs mainly to a product owner who must choose a third-context validation. The portfolio leader schedules a review trigger instead of retaining weekly escalation.

Pulse voice. A constructed real-time voice engagement cannot meet latency, consent, and support guardrails within current architecture and staffing. It is stopped. Portfolio discipline means protecting the stop decision from sunk-cost rescue while preserving the evidence, obligations, and reopen conditions.

After ordering these engagements, state which work does not happen. If Harbor receives the portfolio leader's direct review and a senior engineer for two days, Orchid's reuse workshop moves to its evidence trigger rather than occupying standing capacity. Opportunity cost is part of an honest allocation. [CLM-187]

Establish an operating cadence

Use different cadences for different decision horizons:

- **event-driven:** incident, material guardrail failure, irreversible-state decision, authority gap, or recovery trigger;
- **daily during exposure/stabilization:** consequence, cohorts, evidence freshness, blockers, next decision;
- **weekly portfolio review:** ordinal attention, dependency age, ownership, staffing, opportunity cost, review triggers;
- **monthly learning review:** repeated escalations, product packets, reusable playbooks, maintenance burden, stopped work;
- **quarterly capability review:** system capacity, role coverage, specialist access, mentoring, and growth evidence.

Do not turn every engagement into a meeting at every cadence. The portfolio record should let stable, owned work stay asynchronous until a trigger. Meetings exist for decisions, contested interpretations, and coordination that the record cannot resolve.

For each recurring review define input cutoff, decision owner, required attendees, output record, escalation path, and cancellation rule. A review without a decision or evidence update should be shortened, changed, or removed.

Delegate through a contract

Define:

- objective/outcome and non-goals;
- delegated authority and decisions retained;
- required evidence/artifacts;
- constraints/guardrails/access/data;
- review points and quality bar;
- escalation triggers/path;
- owner and handoff.

Task transfer without authority or escalation produces waiting or unsafe improvisation. [CLM-180]

Delegation is successful when the assignee can make bounded decisions and return evidence, not when the senior person disappears.

Delegate a readiness review without fabricating authority

Suppose a developing FDE will lead the technical readiness review for a fictional internal cohort.

The contract might state:

- **objective:** assemble the readiness packet, run the review, and recommend go, reduced scope, delay, or stop for the internal cohort;
- **authority delegated:** request evidence, disposition technical completeness as met/gap, schedule the review, and recommend a cohort;
- **authority retained:** release owner signs exposure; security/privacy/safety authorities decide formal exceptions; portfolio lead changes staffing;
- **required evidence:** current artifact/evidence hashes, critical-segment results, recovery rehearsal, support/command plan, limitations, and decision record;
- **constraints:** synthetic or approved data only, no production expansion, no unrecorded exception, no bypass of critical failure;
- **review points:** packet outline, pre-read, decision review, and post-decision handoff;
- **escalation:** unresolved authority, material gap, evidence conflict, or deadline pressure goes to the named owner immediately;
- **quality bar:** another reviewer can trace every recommendation to current evidence and see the consequence of each gap.

If the assignee must ask the portfolio leader before every ordinary evidence request, authority is too narrow. If the assignee can approve a customer risk exception, authority is too broad. Good delegation defines the useful middle.

Review without taking the work back

At a review point, ask the assignee to show the decision, evidence, uncertainty, alternatives, and next trigger. Correct a dangerous boundary or unsupported conclusion directly. For less consequential differences, prefer questions and explicit acceptance criteria over rewriting the packet yourself.

When the same quality gap appears repeatedly, improve the system: a template, example, test, reviewer checklist, mentoring exercise, or clearer authority map. Permanent senior rework is a capacity failure disguised as quality control.

Communicate at multiple altitudes

Preserve shared facts, uncertainty, decision, authority, consequence, and review trigger.

Executive

Outcome/guardrail, material consequence, options/tradeoff, decision/owner, opportunity cost, next trigger.

Operational

Cohort, support/command, signals/gates, staffing/access, recovery, schedule/cadence.

Technical

Architecture/state, evidence, failure hypotheses, compatibility, controls/tests, correction/recovery.

Detail changes; truth does not. [CLM-181]

Write the same escalation three ways

Shared facts:

- customer recovery access has not been executed under the identity dependency failure;
- the planned wider cohort begins in 48 hours;
- current non-safety internal exposure is stable;
- release authority owns widen/hold; security owns emergency-access conditions;
- review trigger is an executed recovery test with current access and evidence.

Executive version

Recommend holding wider exposure. The current internal cohort is stable, but the recovery path for the wider cohort depends on access that has not been executed under the failure it must survive. Widening now exchanges schedule for an unbounded recovery gap. Release authority decides after the security-owned access conditions and recovery test are complete.

Operational version

Maintain the internal cohort and current support schedule. Do not widen. Operations will run the recovery scenario at 10:00 with security and the release owner present, record access/action/timing/integrity evidence, and post a disposition by 13:00. If access remains unavailable, use the Chapter 15 delay path and notify affected stakeholders.

Technical version

Artifact/config/schema are unchanged and current cohort signals are within the stated window. The recovery rehearsal is incomplete because the emergency principal has not been authenticated while the normal identity dependency is unavailable. Execute the scoped access path, force the

representative partial-state condition, roll forward or isolate according to the state tree, verify user-visible and integrity state, and bind evidence to the release record.

The versions differ in action detail, not facts or certainty. None invents an ETA for technical success.

Example shared fact: customer recovery access is unexecuted. Executive: hold expansion. Operational: maintain reduced cohort/support. Technical: execute identity-dependency recovery test. Same fact, different action detail.

Convert permanent escalation into capacity

If the same escalation recurs, create:

- clearer decision right/control;
- reusable product capability;
- tested playbook/runbook;
- staffed specialist/owner;
- better interface/evidence;
- formally accepted residual gap.

Heroic coordination is not a durable operating model. [CLM-186]

Convert an escalation through a capacity ladder

Classify why the escalation repeats:

- unclear decision right;
- missing evidence or signal;
- fragile interface or product gap;
- absent specialist capacity;
- under-specified runbook/playbook;
- ownership or access dependency;
- capability gap in the delivery team;
- genuinely exceptional high-consequence judgment.

Then choose a response:

1. clarify the decision record and authority;
2. automate or instrument evidence collection;
3. build a bounded product/control/interface improvement;
4. create and rehearse a playbook;

5. staff or schedule the required specialist;
6. train through supervised evidence-bearing work;
7. accept an explicit residual dependency with owner and expiry;
8. retain senior review only for the truly exceptional decision.

Measure whether the change reduces waiting, unsafe improvisation, repeated rework, or hidden dependency. Merely routing the same escalation through a new channel is not system capacity.

Staff around decisions and boundaries

Do not staff only by component backlog. Map the upcoming decisions and the expertise/authority they require.

A bounded deployment may need a strong integrator as its day-to-day owner, a part-time security specialist at threat/control and readiness gates, an accessibility reviewer before representative UAT, and a senior production engineer during migration/recovery. Keeping all specialists embedded full time can be wasteful; bringing them after an irreversible decision can be dangerous.

Record the engagement owner, specialist triggers, backup owner, review capacity, and escalation availability. If no qualified owner can be present before a critical decision, reduce scope or delay. Schedule pressure does not create expertise.

Preserve engineering depth

Portfolio leaders still need enough code, architecture, data, security, reliability, and workflow depth to review evidence, find hidden failure, challenge abstraction, and delegate responsibly. [CLM-185]

Direct work shifts toward the highest-leverage/highest-uncertainty seams; it does not vanish.

Maintain a technical evidence habit

Portfolio-level depth can be preserved through deliberate contact with consequential artifacts:

- review one failure trace and recovery record, not only a dashboard summary;
- inspect a contract or state transition at the integration seam;
- run a representative companion or rehearsal path;
- challenge a critical-segment evaluation result;
- read a migration or corrective-action diff;
- join a bounded user or operator observation;
- rotate deep reviews across engagements rather than hovering over all code.

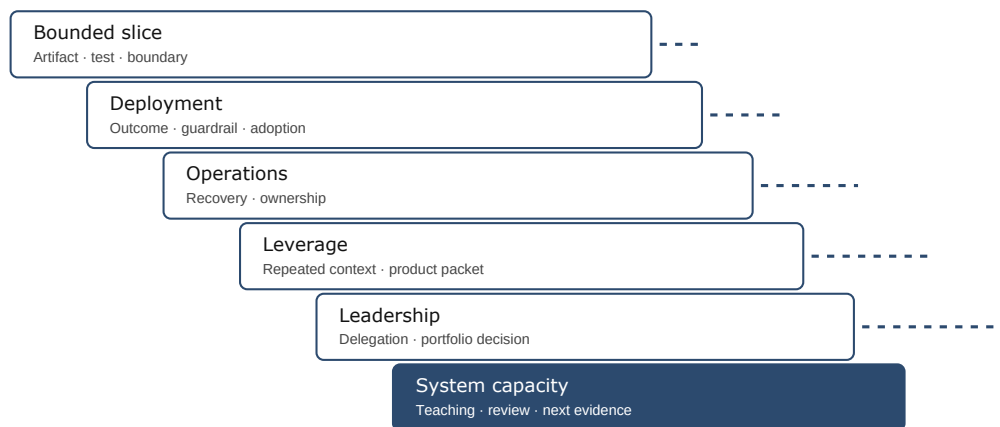
The goal is not to out-code every owner. It is to remain capable of detecting when an abstraction hides state, evidence, authority, or failure consequence. [CLM-185]

Set a rule for direct intervention. Enter the code or incident path when consequence, evidence gap, uncertainty, or missing capability justifies it. State the objective and exit. If the leader becomes the permanent implementer, the portfolio has not gained capacity.

Grow through capability evidence

F19.2 · FORWARD DEPLOYED ENGINEERING

Capability is evidence, not tenure



— observed / required

- - - boundary / proposed

◇ decision

ORCHID · FICTIONAL / SYNTHETIC

F19.2 - Capability is evidence, not tenure. Growth follows demonstrated outcome ownership, boundary judgment, recovery, leverage, delegation, and system capacity.

Text alternative: Capability path from bounded slice through end-to-end deployment, recovery and ownership, repeated product leverage, delegation and portfolio decisions, and system capacity, with required evidence at each step.

Build a six-month growth plan across:

- deeper production implementation/diagnosis;
- stronger workflow/outcome discovery;
- security/privacy/safety boundary judgment;
- verification/AI evaluation/recovery;
- stakeholder decisions and writing;
- adoption/ownership;
- reuse/product learning;

- delegation/portfolio review.

For each: target capability, current evidence, stretch task, mentor/reviewer, artifact, pass criteria, boundary/escalation, review date.

Growth is evidence, not tenure/title/travel/burnout. [CLM-183]

Seniority increases decision-quality responsibility, not adjacent authority. [CLM-182]

Build a six-month evidence plan

An example plan for an FDE moving from bounded-slice ownership toward end-to-end deployment leadership:

Months 1-2: diagnosis and contracts. Lead discovery for one unfamiliar workflow, produce an evidence log/state model, and review an interface contract with a senior engineer. Pass when users validate exceptions and contract tests cover unknown completion. Escalate domain and data authority rather than self-approving semantics.

Months 2-3: verification and recovery. Own the verification matrix and run a partial-state recovery rehearsal. Produce evidence/limitations and a decision record. Pass when a reviewer can reproduce the failure and recovery without the FDE narrating hidden steps.

Months 3-4: readiness and communication. Lead a bounded readiness review under a delegation contract. Write executive and technical versions of one material gap. Pass when the designated authority can choose among real alternatives and the evidence survives challenge.

Months 4-5: ownership transfer. Supervise customer operators performing release, recovery, support, and decision tasks. Pass when assistance is recorded, access is transferred, and hidden FDE dependency is removed or explicitly excepted.

Months 5-6: leverage and teaching. Produce a pattern ledger entry with negative cases and review another engineer's deployment packet. Pass when the reuse disposition states invariant, variance, isolation, cost, owner, validation, and disconfirmation.

The dates do not guarantee promotion. They create inspectable evidence for the next capability decision. Travel volume, long hours, customer visibility, and title are not substitutes. [CLM-183]

Close the dossier

The companion dossier index requires:

- source hash and build hash;
- GitHub issue/phase state;
- open risk/exception state;
- reuse proposal/defer/reject state;

- current limitations;
- customer/product/operations next owners.

This makes closure resumable/auditable. [CLM-188]

Review closure as if the original FDE disappears

Ask a reviewer who did not lead the engagement to locate:

- the canonical source and build identified by hashes;
- the final scope, cohort, outcomes, guardrails, adoption, and limitations;
- current issue/phase state;
- open risks/exceptions and designated owners;
- access and ownership acceptance;
- reuse proposal/defer/reject records;
- product/customer/operations next decisions;
- the trigger for any future review.

If the reviewer must rely on a private message, an individual's memory, or unexplained local files, the dossier is not closed. Closure does not require that every risk disappear. It requires that remaining state be visible, owned, bounded, and resumable.

Failure modes and repairs

Loudness becomes priority

Repair: compare consequence, evidence gap, reversibility, and decision latency; record opportunity cost and rationale. [CLM-179]

The heatmap becomes arithmetic truth

Repair: use ordinal labels plus narrative evidence and confidence. Never sum incomparable categories merely to rank them.

Delegation is task forwarding

Repair: define bounded authority, evidence, constraints, review points, escalation, and owner. Test whether the assignee can act without unsafe improvisation. [CLM-180]

The leader keeps every escalation

Repair: classify the recurring cause and create a control, capability, playbook, owner, or explicit residual dependency.

Executive communication removes uncertainty

Repair: reduce detail but preserve the uncertainty, authority, consequence, and review trigger.

Strategy loses technical contact

Repair: maintain rotating deep evidence reviews and enter direct work at the highest-consequence uncertain seams with an exit condition.

Growth rewards burnout

Repair: evaluate artifacts, decisions, outcomes, boundary judgment, teaching, and progressively harder scope. Do not treat travel, availability, or exhaustion as competence.

Seniority expands authority by implication

Repair: preserve the same legal, security, privacy, safety, audit, accessibility, and risk-decision boundaries. Seniority improves the quality and timing of escalation; it does not erase it.

Conduct the chapter exercise

Build four complete portfolio records from the fictional cases. Order them, then write the rationale and displaced work. For the highest-priority engagement:

1. define the next decision and designated authority;
2. write a delegation contract for a bounded workstream;
3. write executive and technical versions of one escalation;
4. choose an event/daily/weekly/monthly review cadence;
5. convert one recurring escalation into system capacity;
6. identify what direct technical evidence the portfolio leader will inspect;
7. define the review trigger that changes the allocation.

Finally, create a six-month capability plan with artifacts and pass criteria. A peer should be able to challenge every allocation using the recorded evidence, not inferred status or title.

Run the weekly portfolio decision review

Send current records before the meeting. Review only changes, contested evidence, aging dependencies, and decisions. A practical agenda is:

1. incidents or irreversible-state decisions that changed consequence;
2. material evidence gaps and authority deadlines;
3. readiness/stabilization/ownership gates needing disposition;
4. dependency age and escalation path;
5. staffing/delegation changes and specialist triggers;
6. product-leverage proposals at their review trigger;
7. stopped/deferred work and opportunity cost;
8. next decisions, owners, and event-driven review triggers.

Record the before/after allocation and why. If every engagement stays high priority, the review has avoided its job.

Protect stopped work from silent restart

A stopped engagement needs a closure state: reason/evidence, consequence, obligations, preserved artifacts, access/data cleanup, owner, reopen conditions, and displaced capacity. New enthusiasm or a senior request cannot silently restart it. Reopen through a new decision using the changed evidence.

For fictional Pulse voice, the reopen trigger might require a consent design owned by the correct authority, an architecture that meets the stated latency under representative conditions, and a staffed support model. Until then, experiments remain bounded or closed.

Adapt leadership to team maturity

In a founding-FDE setting, one leader may hold several engagement records and write code directly. The operating model should still expose which decision is being neglected and where missing specialist authority requires reduced scope. Small team is not permission to absorb every role.

In a mature multi-role team, delegation and specialist coverage can increase, but status layers can distance leaders from evidence. Rotate deep technical reviews, require shared-fact memos, and keep the next decision/authority visible. More process should reduce ambiguity and dependency, not merely multiply reporting.

Leadership maturity is measured by better decisions and greater system capacity: fewer hidden escalations, clearer ownership, faster safe evidence, and more people able to act within boundaries. It is not the number of meetings or people reporting to the portfolio lead.

Complete OA - 11

Add portfolio review, two-altitude escalation, operating cadence, staffing/delegation contract, opportunity cost, review triggers, and capability growth plan.

Opportunity cost and consequence of delayed attention remain explicit. [CLM-187]

The Chapter 19 gate

- portfolio records preserve outcomes/guardrails/adoption/evidence/owners;
- attention is ordinal/explainable, not false arithmetic;
- loudness/seniority do not replace consequence;
- delegation has authority/evidence/review/escalation;
- multi-altitude messages share facts/uncertainty;
- recurring escalations become system capacity;
- growth is artifact/evidence based;
- adjacent authority remains bounded;
- dossier hashes/state/next owners exist;
- companion portfolio tests pass.

The FDE lifecycle closes where it began: accountable for a deployed outcome, rigorous about evidence, and honest about authority. The difference is leverage - more people and systems can now make better decisions without depending on one person.

Appendices

Reusable structures, review gates, companion use, terminology, and the completed fictional dossier index.

Appendix A - Deployment Dossier Templates

Each template states purpose, inputs, completion, and structure. Copy and version it; do not treat an empty template as evidence.

Use this common header for every artifact: ID/version/status; purpose; scope/cohort; owner/authority; inputs/provenance; facts/unknowns/assumptions; decisions; evidence/limitations; open gaps; next trigger; linked source/build/issue. The short examples below are fictional and incomplete by design; copy the structure and replace them with governed engagement evidence.

OA-01 Role and discovery

- **Purpose:** establish the role boundary, legitimate access, stakeholders, questions, and evidence confidence before delivery commitments harden.
- **Inputs:** initial request, responsibility charter, known authority/confidentiality/data constraints, stakeholder hypotheses.
- **Complete when:** the next discovery decision can proceed with purpose-limited access, named owners, escalation, and visible gaps.
- **Structure:** objective/outcome hypothesis; FDE core/shared/supporting/out-of-scope decisions; stakeholder map; discovery questions/method; access-data record; evidence log/confidence; assumptions; stop/escalation; exit.
- **Orchid example:** investigate evidence-supported service resolution for one pump family and region; safety approval, security/privacy decisions, commercial promises, and roadmap priority remain designated-owner work; production data access is not approved.

OA-02 Workflow and problem

- **Purpose:** model the actual actors, states, decisions, data, tools, handoffs, exceptions, incentives, waiting, and failure cost.
- **Inputs:** approved observation/interviews, records/artifacts, technical reproduction, operating evidence, and OA-01 boundaries.
- **Complete when:** representative users/owners validate the normal and consequential exceptional paths and the problem statement can change scope.
- **Structure:** service journey; swimlane/state/intent model; decision-evidence table; system/data inventory; exception topology; observed/reported/inferred/unknown labels; confidence/gaps; validation record; revised problem.
- **Orchid example:** equipment ID may be ambiguous, evidence provenance affects trust, inventory completion can be unknown after timeout, and qualified approval is a separate decision.

OA-03 Outcome contract

- **Purpose:** define outcome, guardrails, adoption, baseline, attribution, rights, non-goals, and exit before the implementation becomes the success definition.
- **Inputs:** validated workflow/problem, evidence definitions, owners, baseline feasibility, and decision context.
- **Complete when:** every measure has a population/formula/source/segment/window/owner and every evidence state changes a named decision.
- **Structure:** metric tree; metric specifications; baseline/confounders; segments; gaming tests; adoption diagnosis; attribution level; guardrail ladder; assumptions/constraints; decision rights; non-goals/rejection; entrance/exit/stop.
- **Orchid example:** reduce evidence-search burden within safety/data/latency/cost guardrails for the bounded cohort; a favorable aggregate cannot erase a low-connectivity or critical-equipment failure.

OA-04 Scope and delivery

- **Purpose:** select the least exposed representative path that produces decisive evidence.
- **Inputs:** OA-03, dependencies, authority, uncertainty, reversible/irreversible state, delivery horizon.
- **Complete when:** the slice crosses the consequential workflow, dependencies/experiments are owned, milestones are reversible where possible, and defer/reject/stop are explicit.
- **Structure:** scope cone; must-prove/must-build/defer/reject; dependency network; uncertainty experiments; option comparison; estimates/ranges; risk/debt; milestones; change budget/control; stop/exit.
- **Orchid example:** bounded evidence-assisted review for one pump family/region; automatic ordering rejected, voice deferred, unknown inventory completion proven through controlled tests before external writes.

OA-05 Design dossier

- **Purpose:** make structural, runtime, data, trust, failure, responsibility, interface, environment, AI, and control decisions reviewable.
- **Inputs:** approved scope, workflow contracts, customer environment constraints, threat/risk and specialist evidence.
- **Complete when:** every consequential relationship has semantics, owner, failure state, evidence, authority, and review trigger.
- **Structure:** context/container/boundary diagrams; responsibility/decision table; interface catalog; semantic data/time/intent/reconciliation; topology/environment matrix;

identity/network/secrets/config/isolation/capacity; AI decision/eval/control/fallback;
threat/control/evidence/approval.

- **Orchid example:** confirmed equipment gates evidence access; inventory intent reconciles unknown completion; model suggestion remains separate from policy, authorization, and qualified approval; region/tenant boundaries apply per request.

OA-06 Vertical slice

- **Purpose:** implement one small path that is complete across consequential boundaries and failure behavior.
- **Inputs:** OA-04/OA-05, synthetic fixtures, versioned contracts, environment/config record, debt authority.
- **Complete when:** a second reviewer can run, trace, fail, and explain the path from a clean checkout without secrets, paid services, customer data, or hidden author steps.
- **Structure:** user intent and coverage; repository/owner/test/config/signal/runbook map; runnable command; states/failures; contracts/adapters; config/feature controls; audit/telemetry; review record; doubles/non-proof; debt owner/exit.
- **Orchid example:** synthetic ticket to confirmed equipment, approved evidence, inventory state, bounded suggestion, deterministic policy, qualified review state, minimized audit/metric; ambiguous equipment and unknown completion remain blocked/reconcile.

OA-07 Verification

- **Purpose:** connect every material workflow risk and release claim to the cheapest credible evidence and honest disposition.
- **Inputs:** runnable slice, contracts/error taxonomy, outcome/guardrails, threats/controls, representative evidence plan.
- **Complete when:** every risk has criterion/method/data/version/segment/observation/limitation/owner/disposition and critical failures cannot hide in an aggregate.
- **Structure:** risk/requirement/control; criterion; method/evidence type; data/provenance/version/split/leakage; segment; expected/observed; uncertainty/limitation; owner; pass/fail/inconclusive/not-run/waiver; release effect.
- **Orchid example:** contract tests cover equipment binding and intent; critical-segment eval blocks a prohibited suggestion despite an aggregate pass; UAT observes a qualified technician using ambiguity, evidence, fallback, reconciliation, and support paths.

OA-08 Operations, release, and recovery

- **Purpose:** turn release claims into operable signals, versioned exposure, and demonstrated recovery.

- **Inputs:** OA-07 results/limitations, target promises/cohorts, release bundle, migration/data state, ownership/access.
- **Complete when:** operators can detect a failed promise, inspect next evidence, act through a tested runbook, identify the exact release, and recover representative state.
- **Structure:** promise/failure/signal/segment/objective/action; dashboards/alerts/runbooks/blind spots; support/capacity/cost; artifact/config/schema/migration/model/policy/evidence/cohort identity; feature controls; compatibility; rollback/roll-forward/isolate/restore/stop; restore/break-glass; rehearsal record.
- **Orchid example:** low-connectivity cohort remains visible; partial schema migration is labeled divergent and rolled forward; local 42 ms rehearsal is explicitly not a production RTO claim.

OA-09 Readiness

- **Purpose:** select go, conditional go, reduced scope, delay, or stop for an exact cohort under current evidence and authority.
- **Inputs:** OA-03 through OA-08, user/operator/support preparation, open risk/exception records.
- **Complete when:** every criterion is met/gap/exception/reduced/block and the decision names facts/unknowns, options, authority, cohort/window, triggers, communications, and recovery.
- **Structure:** criterion/evidence/status table; options/tradeoffs; recommendation; authority/signature; cohort/pattern/ramp; observation/critical cases; absolute/comparative gates; command/support/comms; widen/hold/stop/recover; conditions/expiry/review.
- **Orchid example:** unexecuted recovery access blocks wider/safety-relevant exposure; only the bounded internal non-safety cohort may proceed under its own evidence and authority.

OA-10 Stabilization and ownership

- **Purpose:** preserve truthful incident learning, deliver durable correction, diagnose adoption, and demonstrate customer ownership/access exit.
- **Inputs:** rollout/command plan, signals/recovery, current limitations, user/operator/support roles, access inventory.
- **Complete when:** stabilization criteria pass and customer users/operators demonstrate the whole task plus ten ownership areas without hidden FDE dependency.
- **Structure:** command/impact/timeline/decision/update; evidence/containment/restoration/correction/verification/residual risk; adoption-friction hypotheses/tests; task/accessibility/enablement evidence; knowledge/access/telemetry/runbook/release/recovery/support/decisions/open-risk/change acceptance; access revocation; exception/acceptance.
- **Orchid example:** retry amplification is corrected and monitored; technicians inspect evidence; customer operators diagnose, hold, release, recover, support, and decide; privileged FDE access is revoked or explicitly time-bounded.

OA-11 Product and portfolio

- **Purpose:** close bounded outcomes, classify customer-specific versus reusable learning, allocate attention, delegate, communicate, grow capability, and preserve resumable closure.
- **Inputs:** completed lifecycle dossier, outcome/adoption/guardrail evidence, decision history, customer/contract boundaries, multiple engagement records.
- **Complete when:** reuse decisions state
invariant/variance/isolation/cost/owner/validation/disconfirmation and portfolio/closure records state
consequence/evidence/authority/opportunity cost/next trigger.
- **Structure:** bounded outcome review; pattern ledger; twelve-artifact classification; product packet; propose/defer/reject; portfolio row/order rationale; delegation contract; multi-altitude memo; cadence/staffing; growth plan; source/build hashes; issue/risk/reuse state; next owners.
- **Orchid example:** equipment mappings remain local; intent/reconciliation module waits for a third context; Orchid ownership is closed; product lead owns the next reuse review; Phase 09/10/11 owners remain explicit.

Appendix B - Gate and Review Checklists

Checklists summarize chapter models; they do not replace evidence or authority.

For every item record met, gap, exception, reduced scope, block, or not applicable with reason; link current evidence/version; name owner and decision authority; state consequence and next trigger. A checked box without executed evidence is a gap. A reviewer may recommend; only the designated authority can accept an exception or exposure decision.

Discovery gate

- Legitimate purpose/access/data minimization and role boundary.
- Actual users/workflow/exceptions/handoffs validated.
- Evidence confidence, gaps, assumptions, escalation visible.

Scope/design gate

- Outcome/guardrails/rights/non-goals/exit defined.
- Representative vertical scope/dependencies/uncertainty/stop defined.
- Architecture/interfaces/data/environment/identity/AI/control evidence and owners complete.

Build/verification gate

- Clean synthetic path and consequential failures run.
- Contracts preserve semantics/intent/reconciliation.
- Risk maps to typed, segmented, versioned evidence and limitations.
- Security/accessibility/UAT/recovery gaps owned.

Operations/release gate

- Promises map to cohort/tail signals and actionable alerts/runbooks.
- Telemetry minimized; missing/stale evidence visible.
- Artifact/config/schema/migration/evidence/cohort bound.

- Recovery choice, restore, break-glass, and rehearsal executed.

Production readiness gate

- Every criterion is met/gap/exception/reduced/block.
- Facts/unknowns/authority/options explicit.
- Cohort/window/absolute and comparative gates/command/comms/support complete.
- Stop/recovery triggers precommitted.

Stabilization gate

- Command/impact/timeline/decisions/updates coherent.
- Containment/restoration and durable correction distinguished.
- Regression/monitoring/corrective ownership/residual risk complete.

Ownership gate

- Representative users complete whole task.
- Customer demonstrates knowledge/access/telemetry/runbook/release/recovery/support/decisions/open risk/change.
- FDE privilege revoked or formally bounded; acceptance signed.

Product/portfolio gate

- Invariant/variance/contexts/isolation/maintenance/disconfirmation explicit.
- One context does not become prevalence.
- Portfolio priority/delegation/memos/opportunity cost/review triggers evidence-based.

Gate decision record

Use this compact closeout after any checklist:

- gate/artifact version and review date;
- exact scope/cohort/environment;
- attendees, evidence owners, and decision authority;

- met criteria and linked evidence;
- gaps, consequence, affected segments, and owner;
- exceptions with authority, conditions, monitoring, expiry, and recovery;
- supported options and tradeoffs;
- decision/recommendation/dissent;
- immediate actions and communications;
- widen/hold/stop/recover or revisit triggers;
- next review and source/build/issue reference.

Fictional readiness example

Orchid's synthetic/local verification, bounded internal UAT, support schedule, and release record are current. Recovery access under the failed identity dependency is unexecuted. Disposition: gap, not met. Consequence: wider/safety-relevant cohort may lack an exercised recovery path. Recommendation: delay that cohort; a separately evidenced internal non-safety cohort may remain reduced scope. Release authority decides; security owns emergency-access conditions. Revisit only after executed current evidence is bound to the release record.

This example demonstrates checklist use. It is not a real launch or risk decision.

Appendix C - Provider-Neutral Companion Guide

Purpose

The companion demonstrates how the chapter decision models compose in one synthetic local path. Node.js built-ins are the default; there are no secrets, customer data, paid services, or provider accounts.

Commands

```
npm run test:companion  
npm run demo:companion  
npm run rehearse:companion
```

Module map

- contracts/reconciliation/intent: semantic data and safe external state;
- environment/authorization: deployment configuration and scoped identity;
- model/policy/evaluation: bounded suggestion, deterministic control, segments;
- control-evidence/verification: evidence completeness, authority, graders;
- vertical-slice/synthetic fixture: runnable ticket-to-review path;
- observability: signals, cohorts, privacy, diagnosis;
- release-recovery/readiness: artifact/migration/rehearsal and disposition;
- incident/ownership: stabilization and demonstrated handoff;
- product-leverage/portfolio: reuse and dossier closure.

Security and failure boundaries

- Configuration rejects embedded secret fields.
- Telemetry/audit use allowlists and exclude raw payload/prompt/document/secret/free text.
- Tenant/region/resource/operation/expiry are evaluated.
- Missing/ambiguous/unknown states remain visible.
- Model output never owns authorization or formal authority.

What it does not prove

Real adapters, identity, networking, tenancy, regional processing, performance, accessibility/UAT, provider-model behavior, production telemetry, restore/break-glass, customer authority, outcomes, or RTO/SLOs require representative evidence.

Extension rule

Replace injected boundaries behind the same consequential contracts; preserve tests and add representative integration evidence. Do not silently weaken semantics for provider convenience.

Appendix D - Glossary and Source Use

Core terms

- **deployment:** socio-technical path from scoped workflow to adopted ownership.
- **release:** controlled software/configuration change inside a deployment.
- **production outcome:** sustained measurable workflow value within guardrails.
- **guardrail:** evidence value is not achieved through unacceptable harm/risk/cost/degradation.
- **vertical slice:** smallest end-to-end path crossing representative constraints.
- **intent:** caller's semantic request identity, distinct from transport attempt.
- **unknown completion:** external acceptance/finality cannot be proven; reconcile before retry.
- **evaluation:** systematic behavior measurement against task-relevant criteria.
- **UAT:** representative task acceptance, not an informal demo.
- **stabilization:** bounded post-rollout correction and ownership confirmation.
- **handoff:** demonstrated transfer of operation/support/knowledge/decisions/access.
- **product leverage:** reusable capability or learning earned through bounded field evidence.

Evidence language

- **met** requires current executed evidence.
- **gap** includes missing/stale/inconclusive/failed evidence.
- **exception** requires designated authority, scope, consequence, conditions, monitoring, expiry.
- **confirmed** requires named evidence; probable and unknown remain labeled.
- Local/synthetic timing, tests, and examples never become production outcome claims.

Source policy

- Prefer official standards/specifications/documentation and first-party technical reports.
- Employer materials show observed practice, not universal role definition.
- Vendor/customer case outcomes remain attributed and are not independent verification.
- Version volatile sources with access date; teach durable concepts independently of current UI/API.
- Do not reproduce copyrighted standards; map selected IDs/scope/version/evidence/limits.
- Every manuscript claim ID resolves through `claims.json` to `sources.json`.

Appendix E - Completed Dossier Index

Manuscript evidence

- Chapters 01-19: canonical MDX, learning pack, state handoff, immediate QA.
- Final figures: F01.1 through F19.2, with canonical SVGs, web mirrors, captions, alt text, long descriptions, hashes, and Phase 10 QA records.
- Registries: publication, sources, claims, figures, errata.

Orchid dossier

- OA-01: responsibility charter/discovery plan.
- OA-02: current workflow/problem model.
- OA-03: outcome contract/decision rights.
- OA-04: safe scope/delivery plan.
- OA-05: architecture/interfaces/data/environment/AI/governance.
- OA-06: runnable vertical slice/repository/debt.
- OA-07: verification/evaluation/UAT/limitations.
- OA-08: signals/operations/release/recovery/rehearsal.
- OA-09: readiness/cohort/go-no-go.
- OA-10: incident/stabilization/adoption/ownership/access exit.
- OA-11: outcome/pattern/reuse/portfolio/growth/closure.

Executable evidence

- Companion module/test map: Appendix C.
- Commands: `test:companion`, `demo:companion`, `rehearse:companion`.
- Synthetic data only; production non-proof remains explicit.

Control state

- No FDE/customer risk authority is fabricated.
- No customer-specific data is included.

- Reuse: intent/reconciliation seam deferred for third context; Orchid mappings rejected.
- Ownership: fictional acceptance model requires ten demonstrated areas and revoked FDE privilege.

Provenance and next owners

Source/build hashes and exact GitHub issue state are recorded in the companion dossier index. Phase 09 whole-book audit and Phase 10 figures are complete; Phase 11 owns publication/PDF release.

Figure registry

All figures are original Komal synthesis. Orchid and satellite contexts are fictional/synthetic.

FIG-001 - F01.1 — The FDE integrates workflow, engineering, and product learning while formal authority remains explicitly assigned.

Long description: Boundary map placing the FDE between customer workflow, production engineering, and product learning, with formal authorities outside the shared delivery area. Reading order: Customer workflow: Observed work: Adoption evidence; Production engineering: Reliable system: Recovery evidence; Product learning: Reusable pattern: Disconfirmation; Formal authority: Security · privacy · safety: Named approvers; Adjacent roles: Product · account · strategy: Shared, not absorbed.

FIG-002 - F01.2 — Release is one event inside a larger evidence loop from discovery through ownership and field learning.

Long description: Circular lifecycle from discovery and outcome contract through build, verification, launch, stabilization, ownership, and product learning, with evidence returning to earlier decisions. Reading order: Discover: Workflow evidence; Contract: Outcome · guardrails; Scope: Safe vertical path; Build: Production slice; Verify: Decision evidence; Release: Controlled exposure; Stabilize: Restore · correct; Own: Customer capability; Learn: Bounded reuse.

FIG-003 - F02.1 — Impact, knowledge, access, influence, responsibility, and decision authority are recorded separately.

Long description: Stakeholder matrix showing sponsor, technician, regional owner, security owner, and FDE across impact, workflow knowledge, access, influence, responsibility, and authority. Reading order: Sponsor: Influence · business decision: Partial workflow knowledge; Technician: High impact · high knowledge: No regional risk authority; Regional owner: Data and operating boundary: Named regional decision; Security owner: Control and exception authority: Needs implementation evidence; FDE: Discovery · synthesis · delivery: Recommends and escalates.

FIG-004 - F02.2 — Confidence can rise from reported to production evidence without making any observation universal.

Long description: Five-step evidence ladder from reported to observed, measured, reproduced, and production evidence, with source, context, sample, confidence, contradiction, and limitation retained at every rung. Reading order: Reported: Stakeholder statement; Observed: Work seen in context; Measured: Defined population and window; Reproduced: Repeatable condition; Production: Representative live evidence.

FIG-005 - F03.1 — Observed work, waiting states, unofficial channels, decisions, and reopen loops stay visible.

Long description: Swimlane-style workflow from service request through dispatch, equipment identification, evidence, inspection, approval, inventory, resolution, record update, escalation, and reopen. Reading order: Request: Incoming symptom: Ticket created; Dispatch: Queue · wait: Technician assigned; Identify: Equipment match?: Personal notes; Evidence: Manual · site inspection: Missing or stale; Approve: Qualified safety decision: Escalate when absent; Resolve: Inventory · action: Update or reconcile; Close / reopen: Outcome evidence: Return on contradiction.

FIG-006 - F03.2 — Each exception names evidence, owner, response, and a return or terminal state.

Long description: Exception topology around the Orchid normal path, including identifier mismatch, missing manual, stale inventory, low connectivity, approval, API timeout, accessibility, support, cancellation, and reopen. Reading order: Data: Identifier mismatch: Missing manual; Dependency: Stale inventory: Timeout after write; Environment: Low connectivity: Inaccessible interface; Authority: Policy · approval: Safety escalation; Operations: Support gap: Cancellation · reopen.

FIG-007 - F04.1 — Metric branches show hypotheses and gaming paths, not guaranteed causality.

Long description: Metric tree with safe resolution outcome above workflow, adoption, system quality, and guardrail branches, plus gaming paths where closure improves while safe resolution worsens. Reading order: Contracted outcome: Safe resolution for the defined population; Leading workflow: Evidence found · decision time; Adoption: Representative task completion; System quality: Latency · failure · reconciliation; Guardrails: Unsafe exclusion · transfer · premature close.

FIG-008 - F04.2 — Recommend, approve, execute, and inform are separated for every consequential decision.

Long description: Decision-rights matrix for outcome, safety, privacy, security exception, pilot scope, release, go or no-go, incident command, and handoff, separating FDE recommendation from approval. Reading order: Recommend: FDE prepares options and evidence; Approve: Named domain authority; Execute: Delegated delivery owner; Inform: Affected users and owners; Escalate: Missing evidence or authority blocks exposure.

FIG-009 - F05.1 — Small means least exposed while still crossing the uncertainty-bearing boundary.

Long description: Scope cone narrowing a broad AI copilot request through outcome, consequence, uncertainty, dependency, representativeness, reversibility, and evidence yield to one bounded Orchid path. Reading order: Broad request: AI copilot for service; Outcome relevance: Safe resolution evidence; Consequence: Guardrails and authority; Dependencies: Representative boundary; Reversibility: Bounded exposure; Selected path: One region · cohort · equipment family.

FIG-010 - F05.2 — External owners and evidence gates govern sequencing more honestly than a calendar alone.

Long description: Dependency network linking ticketing, equipment registry, manuals, inventory, identity, network, approvals, support, and pilot users to vertical milestones and a critical uncertainty path. Reading order: Customer systems: Ticketing · registry: Inventory · manuals; Operating boundary: Identity · network · region: Support · pilot users; Authority gates: Safety · security · privacy: Owner evidence; Vertical milestones: Contract · slice · verification: Release decision; Critical uncertainty: Equipment binding: Unknown-write reconciliation.

FIG-011 - F06.1 — Actors, services, data, trust, region, operations, and external ownership appear in one context.

Long description: System context showing technicians, dispatch, approvers, support, Orchid Assist, ticketing, equipment registry, inventory, manuals, identity, telemetry, and audit within ownership and Region West boundaries. Reading order: Human actors: Technician · dispatch: Approver · support; Orchid Assist: Workflow · adapters: Policy · bounded model; Customer systems: Ticketing · registry · ERP: Manuals · identity; Operating evidence: Telemetry · audit: Support ownership; Boundary overlays: Region West · tenant: Trust · data · owner.

FIG-012 - F06.2 — Propagation, detection, containment, correction, reconciliation, and owner remain connected.

Long description: Failure and ownership matrix connecting Orchid failures to propagation, detection, containment or fallback, correction or reconciliation, and accountable owner, including configured versus operational state. Reading order: Failure: Identity · dependency · data: Configured ≠ operational; Propagation: User and workflow effect; Detection: Signal · evidence · blind spot; Containment: Fallback · isolate · stop; Correction: Reconcile · verify · restore; Owner: Named action and authority.

FIG-013 - F07.1 — Correlation, idempotent intent, status lookup, and reconciliation make uncertainty visible.

Long description: Sequence from ticket and equipment confirmation through manual and inventory reads, showing correlation, timeout, retry decision, idempotent intent, unknown state, status lookup, and reconciliation. Reading order: Confirm: Ticket + equipment: Correlation ID; Read evidence: Manual + inventory: Freshness bound; Create intent: Idempotency key: Pending state; Timeout: Do not guess success: Expose unknown; Status lookup: Completed · absent · conflict; Reconcile: One visible result: Owner on conflict.

FIG-014 - F07.2 — Meaning, provenance, quality, version, sensitivity, and exception behavior complete the field definition.

Long description: Data-contract anatomy for Orchid equipment evidence, contrasting a valid but ambiguous schema with semantic meaning, proposition owner, provenance, quality rule, version, sensitivity, and invalid or conflicting behavior. Reading order: Shape: Field · type · required; Meaning: Means / does not mean: Proposition owner; Evidence: Provenance · freshness: Quality rule; Change: Version · compatibility: Migration; Failure: Missing · invalid · conflict: Exception owner.

FIG-015 - F08.1 — Connectivity, tenancy, regional, capacity, and operating assumptions are visible before launch.

Long description: Topology from technician device through network and ingress to customer runtime, workflow services, adapters, stores, customer systems, control plane, telemetry, and support with region and tenant boundaries. Reading order: User edge: Technician device: Intermittent network; Ingress: Regional gateway: User policy; Customer runtime: Workflow · adapters · stores: Bounded workload identity; External systems: Ticketing · registry · ERP: Egress controls; Operations: Control plane · telemetry: Support and recovery owner.

FIG-016 - F08.2 — Human context informs policy but never becomes unrestricted service authority.

Long description: Identity flow from technician and approver sessions through gateway policy, bounded workload identity, downstream authorization, secret retrieval, audit, expiry and revocation, with denied region, tenant, and role paths. Reading order: Human session: Technician / approver; Gateway policy: Region · tenant · role; Workload identity: Short-lived · bounded; Resource auth: Explicit downstream action; Secrets: Certificate retrieval; Audit + revoke: Expiry · deny · trace.

FIG-017 - F09.1 — Every step upward adds evaluation, permission, monitoring, recovery, latency, cost, and authority needs.

Long description: Mechanism ladder from deterministic rule through retrieval, model suggestion, constrained action, and bounded agent, with Orchid stopping at evidence retrieval plus suggestion and qualified approval. Reading order: Rule: Deterministic; Retrieval: Bounded evidence; Suggestion: Model output · no authority; Constrained action: Policy + approval; Bounded agent: Tools · state · recovery.

FIG-018 - F09.2 — Aggregate quality cannot hide a critical segment or an error without a safe control.

Long description: AI risk and evaluation matrix mapping error type to workflow consequence, detectability, deterministic, human and system controls, evaluation segment and grader, and release or stop threshold. Reading order: Error: Wrong equipment · stale evidence: Unsafe omission; Consequence: Workflow and guardrail effect; Detectability: Before action? after effect?; Controls: Rule · human · system; Evaluation: Segment · grader · calibration; Disposition: Release · limit · stop.

FIG-019 - F10.1 — Contextual risk flows through objective, implementation, verification, ownership, and authority.

Long description: Chain from asset and threat through contextual risk, control objective, implementation, verification, versioned evidence, operating owner, and designated approval or exception, with failed verification returning to discovery. Reading order: Context: Asset · obligation · threat; Risk statement: Condition + consequence; Control objective: Required property; Implementation: Actual mechanism; Verification: Executed result; Decision: Owner · approve · exception.

FIG-020 - F10.2 — The FDE prepares and recommends; designated authorities approve, reject, or request evidence.

Long description: Authority map routing FDE and engineering evidence to domain, security, privacy, operations, accessibility, legal, and business owners, with missing evidence returning and absent authority blocking exposure. Reading order: FDE + engineering: Implement · verify · recommend; Domain owner: Workflow and safety decision; Security / privacy: Control and data decision; Operations / accessibility: Operability and task access; Legal / business: Obligation and outcome decision; Stop path: Required evidence or authority absent.

FIG-021 - F11.1 — The slice crosses contracts, evidence, policy, approval, audit, metrics, and controlled failure.

Long description: End-to-end trace from synthetic ticket and intent through environment control, equipment reconciliation, evidence store, inventory adapter, bounded model, qualified approval, deterministic policy, audit, metric, and response. Reading order: Request: Synthetic ticket · intent; Bind: Equipment contract: Block ambiguity; Retrieve: Approved evidence: Dependency fallback; Suggest: Bounded model: Versioned output; Approve: Qualified human: Policy blocks reject; Record: Audit · metric: Reconcile unknown; Respond: Reviewable result: Explicit limitation.

FIG-022 - F11.2 — Every runtime component connects to owner, contract, test, configuration, signal, failure, runbook, and debt.

Long description: Supportability matrix for orchestrator, equipment contract, evidence store, inventory adapter, model, policy, authorization, audit, metrics, and feature configuration across owner, test, signal, runbook, and debt. Reading order: Component: Runtime responsibility; Owner: Change + support; Contract: Version + compatibility; Proof: Test + current result; Operations: Config + signal + runbook; Debt: Open limitation + next evidence.

FIG-023 - F12.1 — Higher, more representative layers complement rather than replace lower, faster checks.

Long description: Verification stack from static review and unit tests through contracts, integration, end-to-end failure, AI evaluation, security and accessibility, performance, UAT, simulation, and controlled production monitoring. Reading order: Static + unit: Cheap · narrow; Contract + component: Boundary behavior; Integration + end to end: Failure · recovery; Eval + specialist checks: AI · security · accessibility; Performance + UAT: Representative task; Simulation + monitoring: Controlled exposure.

FIG-024 - F12.2 — Requirements flow to evidence, limitations, and a named release or waiver disposition.

Long description: Traceability map from outcomes, guardrails, risks, interface, data, AI and operating requirements through test, evaluation, UAT, simulation and rehearsal evidence to limitations and release disposition. Reading order: Claim inputs: Outcome · guardrail · risk; Requirements: Interface · data · AI · ops; Acceptance: Criterion + segment; Evidence: Test · eval · UAT · rehearsal; Limitation: Missing · non-proof · confidence; Disposition: Release · waive · limit · block.

FIG-025 - F13.1 — Correlation and version identity connect signals while privacy, cardinality, and retention bound collection.

Long description: Signal map connecting workflow promises to service, dependency, adoption, data, AI, policy, security, audit, support, capacity, cost, and outcome signals. Reading order: Workflow + adoption: Task state · cohort; Service + dependency: Latency · failure · fallback; Data + evidence: Binding · provenance · freshness; AI + policy: Bundle · review · block; Security + audit: Denied paths · trace; Support + capacity: Demand · cost · owner action.

FIG-026 - F13.2 — Evidence-first branching prevents an immediate and often-wrong code change.

Long description: Diagnostic tree for a slow or untrusted recommendation, branching from user symptom and cohort through dependency, connectivity, equipment binding, evidence freshness, model bundle, policy, authorization, UI, support, containment and owner action. Reading order: Symptom + cohort: Slow · absent · untrusted; Service path: Dependency · network · fallback; Evidence path: Equipment · provenance · freshness; Decision path: Model bundle · policy · approval; Workflow path: UI · task · support; Contain + own: Safe state · next evidence · owner.

FIG-027 - F14.1 — Artifact, configuration, migration, policy, approvals, compatibility, and recovery move together.

Long description: Release path from source and hashed artifact through configuration, schema and migration, feature controls, model and policy bundle, approval, compatibility and recovery record to test, canary, cohort, wider exposure or stop. Reading order: Source: Commit + review; Artifact: Hash + provenance; Change bundle: Config · schema · feature; Decision bundle: Model · policy · approval; Recovery proof: Compatibility · restore; Exposure: Test · cohort · widen / stop.

FIG-028 - F14.2 — Rollback, roll-forward, isolate, restore, and stop are selected by state and consequence.

Long description: Recovery decision tree beginning with user-visible state and containment, then branching on dependency health, reversibility, external data change, compatibility, restore evidence, and consequence before verification. Reading order: Visible state: What changed for users?; Containment: Can exposure safely stop?; Reversibility: Code/config/data compatibility; Dependency: Healthy · degraded · unknown; Recovery option: Rollback · forward · isolate · restore; Verify + record: User state · evidence · owner.

FIG-029 - F15.1 — Named authority chooses go, conditional go, reduced scope, delay, or stop from explicit dispositions.

Long description: Readiness gate where outcome, scope, design, implementation, verification, operations, recovery, ownership, and risk evidence receive met, gap, exception, reduced-scope, or block dispositions. Reading order: Evidence packet: Outcome · design · build; Proof packet: Verification · operations; Ownership packet: Recovery · support · risk; Disposition: Met · gap · accepted exception; Authority: Named decision owner; Decision: Go · conditional · reduce · delay · stop.

FIG-030 - F15.2 — Pattern selection balances representativeness, blast radius, reversibility, support, and stop needs.

Long description: Comparison of internal, shadow, canary, cohort, regional, and full rollout across user effect, evidence, comparability, representativeness, blast radius, reversibility, duration, support and recovery. Reading order: Internal: Low user effect: Weak representativeness; Shadow: Comparable outputs: No direct action; Canary: Small live exposure: Fast stop; Cohort: Defined population: Representative task; Regional: Boundary evidence: Higher coordination; Full: Broad effect: Highest proof and recovery burden.

FIG-031 - F16.1 — Impact, diagnosis, recovery, communications, and correction share one confidence-tagged timeline.

Long description: Parallel incident timeline with impact and command, diagnosis and evidence, containment and recovery, communication and support, and correction and verification tracks, showing restoration begins stabilization. Reading order: Detect + declare: Impact · command; Diagnose: Evidence · confidence; Contain: Limit exposure; Restore: User-visible state; Communicate: Cadence · support; Correct: Cause · verification; Stabilize: Residual risk · owner.

FIG-032 - F16.2 — A durable incident response updates design, tests, signals, runbooks, ownership, and product patterns.

Long description: Learning loop from user impact and detection through timeline, contributing workflow, system and organizational conditions, containment, restoration, corrective action, verification, owner transfer and updated system artifacts. Reading order: Impact: User and guardrail effect; Timeline: Facts · decisions; Conditions: Workflow · system · organization; Restore: Contain · recover; Correct: Durable action; Verify: Evidence · monitoring; Transfer: Owner · residual risk; Update: Design · test · signal · runbook.

FIG-033 - F17.1 — Repair follows a tested friction hypothesis, not a judgment that users resist change.

Long description: Adoption diagnostic map branching non-use, bypass, dropout and support signals into capability, workflow fit, trust, access, performance, incentive, policy, accessibility, support and knowledge. Reading order: Capability: Can it do the task?; Workflow fit: Right state and handoff?; Trust + evidence: Can users verify?; Access + performance: Can they reach it in time?; Policy + incentive: Is use permitted and rational?; Accessibility + support: Can representative users operate it?.

FIG-034 - F17.2 — Customer authority accepts ten operating areas and hidden FDE dependency is removed.

Long description: Ownership-transfer model across knowledge, access, telemetry, runbook, release, recovery, support, decisions, open risk, and change path, each with customer owner and demonstrated evidence. Reading order: Knowledge + access: Named owner · least privilege; Telemetry + runbook: Detect · diagnose · act; Release + recovery: Repeat · restore · verify; Support + decisions: Escalation · authority; Risk + change path: Open limits · maintenance; Absence test: FDE access revoked or bounded.

FIG-035 - F18.1 — Isolation, variance, maintenance, compatibility, ownership, validation, and disconfirmation gate every step.

Long description: Reuse ladder from isolated customer-specific material through configurable pattern, stable shared contract, operational playbook, and product proposal, with evidence gates at every transition. Reading order: Isolate: Customer code · data · rule; Configure: Adapter · template · mapping; Share: Stable contract · library; Operationalize: Test · eval · playbook; Propose: Strategic product gap; Gate every step: Evidence · owner · disconfirmation.

FIG-036 - F18.2 — The packet preserves evidence, variance, cost, uncertainty, ownership, and a next validation decision.

Long description: Field-to-product packet from problem and bounded evidence through workaround, invariant and variance, proposal and reuse type, isolation, impact, uncertainty, maintenance cost, owner, validation and decision. Reading order: Problem: Workflow + bounded evidence; Current workaround: Customer-specific state; Pattern: Invariant + variance; Proposal: Reuse type + isolation; Economics: Impact · maintenance · compatibility; Decision: Owner · next validation · disconfirm.

FIG-037 - F19.1 — No summed risk score replaces consequence, evidence quality, timing, reversibility, and ownership.

Long description: Portfolio heatmap with fictional deployments across outcome, guardrail, adoption, consequence, reversibility, evidence gap, dependency age, decision latency, readiness, leverage, owner, confidence and review trigger. Reading order: Orchid Assist: Stabilize · ownership evidence; Northstar migration: Cutover dependency · reversible; Civic documents: Authority gap · stop protected; Voice dispatch: Latency consequence · limited cohort; Review dimensions: Evidence · time · leverage · owner; Decision: Attend · delegate · stop · reopen.

FIG-038 - F19.2 — Growth follows demonstrated outcome ownership, boundary judgment, recovery, leverage, delegation, and system capacity.

Long description: Capability path from bounded slice through end-to-end deployment, recovery and ownership, repeated product leverage, delegation and portfolio decisions, and system capacity, with required evidence at each step. Reading order: Bounded slice:

Artifact · test · boundary; Deployment: Outcome · guardrail · adoption; Operations: Recovery · ownership; Leverage: Repeated context · product packet; Leadership: Delegation · portfolio decision; System capacity: Teaching · review · next evidence.

Sources

SRC-001 - OpenAI. Forward Deployed Engineer (FDE), Seattle. OpenAI.

<https://openai.com/careers/forward-deployed-engineer-%28fde%29-seattle-seattle/> (accessed 2026-08-16).

SRC-002 - OpenAI. Forward Deployed Engineer (FDE), San Francisco. OpenAI.

<https://openai.com/careers/forward-deployed-engineer-%28fde%29-sf-san-francisco/> (accessed 2026-08-16).

SRC-003 - Palantir. Forward Deployed Software Engineer. Palantir.

<https://jobs.lever.co/palantir/dab396d4-2f14-4796-aac0-0d82883dccf0> (accessed 2026-08-16).

SRC-004 - OpenAI. OpenAI launches the Deployment Company. OpenAI.

<https://openai.com/index/openai-launches-the-deployment-company/> (accessed 2026-08-16).

SRC-005 - OpenAI. Introducing OpenAI Presence. OpenAI.

<https://openai.com/index/introducing-openai-presence/> (accessed 2026-08-16).

SRC-006 - UK Government Digital Service. What a service is. GOV.UK.

<https://www.gov.uk/service-manual/service-assessments/what-a-service-is> (accessed 2026-08-16).

SRC-007 - Scott Rose, Oliver Borchert, Stu Mitchell, Sean Connelly. Zero Trust Architecture. National

Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/207/final> (accessed 2026-08-16).

SRC-008 - National Institute of Standards and Technology. NIST Privacy Framework Version 1.0. National Institute of Standards and Technology.

<https://csrc.nist.gov/pubs/cswp/10/nist-privacy-framework-version-10/final> (accessed 2026-08-16).

SRC-009 - UK Government Digital Service. Understand users and their needs. GOV.UK.

<https://www.gov.uk/service-manual/service-standard/point-1-understand-user-needs> (accessed 2026-08-16).

SRC-010 - UK Government Digital Service. Start by learning user needs. GOV.UK.

<https://www.gov.uk/service-manual/user-research/start-by-learning-user-needs> (accessed 2026-08-16).

SRC-011 - UK Government Digital Service. Introduction to designing government services. GOV.UK.

<https://www.gov.uk/service-manual/design/introduction-designing-government-services> (accessed 2026-08-16).

SRC-012 - UK Government Digital Service. Map a user's whole problem. GOV.UK.

<https://www.gov.uk/service-manual/design/map-a-users-whole-problem> (accessed 2026-08-16).

SRC-013 - Object Management Group. Business Process Model and Notation Version 2.0.2. Object

Management Group. <https://www.omg.org/spec/BPMN> (accessed 2026-08-16).

SRC-014 - Roy Fielding, Mark Nottingham, Julian Reschke. RFC 9110: HTTP Semantics. Internet

Engineering Task Force. <https://www.rfc-editor.org/rfc/rfc9110.html> (accessed 2026-08-16).

SRC-015 - Malcolm Featonby. Making retries safe with idempotent APIs. Amazon Web Services. <https://aws.amazon.com/builders-library/making-retries-safe-with-idempotent-APIs/> (accessed 2026-08-16).

SRC-016 - UK Government Digital Service. Using data to improve your service: an introduction. GOV.UK. <https://www.gov.uk/service-manual/measuring-success/using-data-to-improve-your-service-an-introduction> (accessed 2026-08-16).

SRC-017 - UK Government Digital Service. Measuring service benefits. GOV.UK. <https://www.gov.uk/service-manual/measuring-success/measuring-service-benefits> (accessed 2026-08-16).

SRC-018 - UK Government Digital Service. Measuring user satisfaction. GOV.UK. <https://www.gov.uk/service-manual/measuring-success/measuring-user-satisfaction> (accessed 2026-08-16).

SRC-019 - Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Richard Murphy. Service Level Objectives. Google Site Reliability Engineering. <https://sre.google/sre-book/service-level-objectives/> (accessed 2026-08-16).

SRC-020 - OpenAI. Morgan Stanley uses AI evals to shape the future of financial services. OpenAI. <https://openai.com/index/morgan-stanley/> (accessed 2026-08-16).

SRC-021 - Amazon Web Services. AWS Well-Architected Framework. Amazon Web Services. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html> (accessed 2026-08-16).

SRC-022 - Microsoft. What is the Azure Well-Architected Framework?. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/well-architected/what-is-well-architected-framework> (accessed 2026-08-16).

SRC-023 - Simon Brown. The C4 model for visualising software architecture. C4 model. <https://c4model.com/> (accessed 2026-08-16).

SRC-024 - Microsoft. Architecture design principles. Microsoft Learn. <https://learn.microsoft.com/en-ie/azure/architecture/guide/design-principles/> (accessed 2026-08-16).

SRC-025 - Cloudflare. Cloudflare outage on February 20, 2026. Cloudflare. <https://blog.cloudflare.com/cloudflare-outage-february-20-2026/> (accessed 2026-08-16).

SRC-026 - Cloudflare. Deep dive into Cloudflare's September 12 dashboard and API outage. Cloudflare. <https://blog.cloudflare.com/deep-dive-into-cloudflares-sept-12-dashboard-and-api-outage/> (accessed 2026-08-16).

SRC-027 - OpenAPI Initiative. OpenAPI Specification 3.2.0. OpenAPI Initiative. <https://spec.openapis.org/oas/v3.2.0.html> (accessed 2026-08-16).

SRC-028 - AsyncAPI Initiative. AsyncAPI Specification 3.0.0. AsyncAPI Initiative. <https://www.asyncapi.com/docs/reference/specification/v3.0.0> (accessed 2026-08-16).

SRC-029 - JSON Schema project. JSON Schema Draft 2020-12. JSON Schema. <https://json-schema.org/draft/2020-12> (accessed 2026-08-16).

SRC-030 - International Organization for Standardization, International Electrotechnical Commission. ISO/IEC 25012:2008 Data quality model. ISO. <https://www.iso.org/standard/35736.html> (accessed 2026-08-16).

SRC-031 - Tom Preston-Werner. Semantic Versioning 2.0.0. Semantic Versioning. <https://semver.org/> (accessed 2026-08-16).

SRC-032 - Torsten Lodderstedt, John Bradley, Andrey Labunets, Daniel Fett. Best Current Practice for OAuth 2.0 Security. Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc9700/> (accessed 2026-08-16).

SRC-033 - Cloudflare. Cloudflare incident on October 30, 2023. Cloudflare. <https://blog.cloudflare.com/cloudflare-incident-on-october-30-2023/> (accessed 2026-08-16).

SRC-034 - Elham Tabassi. Artificial Intelligence Risk Management Framework 1.0. National Institute of Standards and Technology. <https://www.nist.gov/publications/artificial-intelligence-risk-management-framework-ai-rmf-10> (accessed 2026-08-16).

SRC-035 - National Institute of Standards and Technology. Artificial Intelligence Risk Management Framework: Generative AI Profile. National Institute of Standards and Technology. <https://nvlpubs.nist.gov/nistpubs/ai/NIST.AI.600-1.pdf> (accessed 2026-08-16).

SRC-036 - OpenAI. Evals. OpenAI Developers. <https://developers.openai.com/api/docs/guides/evals> (accessed 2026-08-16).

SRC-037 - Anthropic. Demystifying evals for AI agents. Anthropic. <https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents> (accessed 2026-08-16).

SRC-038 - Anthropic. Building effective agents. Anthropic. <https://www.anthropic.com/engineering/building-effective-agents> (accessed 2026-08-16).

SRC-039 - Google. Production ML systems. Google for Developers. <https://developers.google.com/machine-learning/crash-course/production-ml-systems> (accessed 2026-08-16).

SRC-040 - OWASP GenAI Security Project. OWASP Top 10 for LLM Applications 2026. OWASP. <https://genai.owasp.org/resource/owasp-genai-llm-top-10-2026/> (accessed 2026-08-16).

SRC-041 - OpenAI. Deployment simulation: Stress testing models in deployment-like contexts. OpenAI. <https://openai.com/index/deployment-simulation/> (accessed 2026-08-16).

SRC-042 - National Institute of Standards and Technology. Cybersecurity Framework 2.0. National Institute of Standards and Technology. <https://www.nist.gov/cyberframework> (accessed 2026-08-16).

SRC-043 - National Institute of Standards and Technology. Secure Software Development Framework Version 1.1. National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/218/final> (accessed 2026-08-16).

SRC-044 - OWASP Foundation. Application Security Verification Standard. OWASP Foundation. <https://owasp.org/www-project-application-security-verification-standard/> (accessed 2026-08-16).

SRC-045 - International Organization for Standardization, International Electrotechnical Commission. ISO/IEC 25010:2023 Systems and software Quality Requirements and Evaluation. International Organization for Standardization. <https://www.iso.org/standard/78176.html> (accessed 2026-08-16).

SRC-046 - World Wide Web Consortium. Web Content Accessibility Guidelines 2.2. World Wide Web Consortium. <https://www.w3.org/TR/WCAG22/> (accessed 2026-08-16).

SRC-047 - Amazon Web Services. AWS Well-Architected Framework. Amazon Web Services. <https://docs.aws.amazon.com/wellarchitected/latest/framework/welcome.html> (accessed 2026-08-16).

SRC-048 - Microsoft. Architecture design principles. Microsoft Learn. <https://learn.microsoft.com/en-ie/azure/architecture/guide/design-principles/> (accessed 2026-08-16).

SRC-049 - Dinanath Khatri, David Glasser, Release Engineering team. Release Engineering. Google Site Reliability Engineering. <https://sre.google/sre-book/release-engineering/> (accessed 2026-08-16).

SRC-050 - Google. Monitoring ML systems. Google for Developers. <https://developers.google.com/machine-learning/crash-course/production-ml-systems/monitoring> (accessed 2026-08-16).

SRC-051 - OpenAI. Agent evals. OpenAI Developers. <https://developers.openai.com/api/docs/guides/agent-evals> (accessed 2026-08-16).

SRC-052 - OpenAI. Safety best practices. OpenAI Developers. <https://developers.openai.com/api/docs/guides/safety-best-practices> (accessed 2026-08-16).

SRC-053 - UK Government Digital Service. Set up and manage user support. GOV.UK. <https://www.gov.uk/service-manual/helping-people-to-use-your-service/set-up-and-manage-user-support> (accessed 2026-08-16).

SRC-054 - Betsy Beyer, Chris Jones, Jennifer Petoff, Niall Richard Murphy. Service Level Objectives. Google Site Reliability Engineering. <https://sre.google/sre-book/service-level-objectives/> (accessed 2026-08-16).

SRC-055 - OpenTelemetry Authors. Signals. OpenTelemetry. <https://opentelemetry.io/docs/concepts/signals/> (accessed 2026-08-16).

SRC-056 - National Institute of Standards and Technology. Incident Response Recommendations and Considerations for Cybersecurity Risk Management. National Institute of Standards and Technology. <https://csrc.nist.gov/pubs/sp/800/61/r3/final> (accessed 2026-08-16).

SRC-057 - Google Site Reliability Engineering. Canarying Releases. Google Site Reliability Workbook. <https://sre.google/workbook/canarying-releases/> (accessed 2026-08-16).

SRC-058 - Microsoft. Recommendations for safe deployment practices. Microsoft Learn. <https://learn.microsoft.com/en-us/azure/well-architected/operational-excellence/safe-deployments> (accessed 2026-08-16).

SRC-059 - Microsoft. Prepare for cloud operations. Microsoft Learn. <https://learn.microsoft.com/en-gb/azure/cloud-adoption-framework/manage/ready> (accessed 2026-08-16).

SRC-060 - Google Site Reliability Engineering. Reliable Product Launches at Scale. Google Site Reliability Engineering. <https://sre.google/sre-book/reliable-product-launches/> (accessed 2026-08-16).

SRC-061 - Google Site Reliability Engineering. Postmortem Culture: Learning from Failure. Google Site Reliability Engineering. <https://sre.google/sre-book/postmortem-culture/> (accessed 2026-08-16).

Edition record

Version: 1.0.0

Published: 2026-08-16

Canonical URL: <https://komalnakrani.com/books/forward-deployed-engineering/>

Recorded errata: 0

Chapters: 19

Figures: 38

Sources: 61